

基于场景随机规划与滚动优化的微网购电调控策略

摘要

本文研究含光伏与储能的微网外购电计划问题。针对负荷、光伏预测和电价信息逐步丰富的四个任务，建立一套以 10 min 为步长的统一母线能量平衡模型，并用确定性混合整数规划、两阶段随机规划和随机模型预测控制依次扩展。模型严格区分功率与时段电量、144 个流量时段与 145 个库存状态，并以信息发布时间约束预测和决策，避免使用未来实际数据。

针对问题一，将全天负荷、光伏预测与分时电价视为已知，以购电费最小为目标建立储能调度 MILP。计算得到全天购电费为 35126.95 元，较不使用储能的可行基线降低 26.898%；计划购电量为 59482.70 kWh，期初和期末储能能量均为 6000 kWh。LP 与 MILP 费用一致，最终采用具有严格充放电互斥约束的 MILP 结果。

针对问题二，在每日 0:00 仅使用此前历史，比较加权历史均值、傅里叶-ARIMA 和随机森林三类预测器，并以 1 月 21-31 日滚动费用固定选择随机森林及 1.2 倍终端价值。最近 28 个历史日的负荷-光伏成对样本外残差按整日联合抽样 256 次，形成保持日内及跨变量相关性的场景。以购电和名义电池轨迹为第一阶段、紧急购电与富余处置为第二阶段，连续回放 2 月至 12 月 334 天。主方案实际购电费为 1447.93 万元，紧急购电量为 262.35 MWh；相对同一预测器的点预测方案费用降低 7.69%，相对同初始库存的旧机制对照降低 13.653%。

针对问题三，构造 0:00、6:00、12:00、18:00 四次更新的随机滚动模型，以历史样本外误差校准不同提前量的偏差与尺度，并用联合日块生成剩余时域场景。风险中性的条件分箱尺度方案费用为 1662.77 万元、紧急购电费为 91.25 万元；四次预报相对仅 0:00 新预报累计节省 41.21 万元。CVaR 方案把日紧急费用 $CVaR_{95}$ 由 16718.04 元降至 13088.86 元，但增加平均现金费用。

针对问题四，以最近 28 个历史日构造波动价格与净负荷场景，用 1 月历史验证冻结价格基线、日内残差修正和风险参数。仅制定日前计划时，价量边际独立的主方案全年实际费用为 1520.11 万元；允许日内调整时，采用一组前瞻分支的主方案费用为 1499.72 万元，比前者降低 20.39 万元。后一差额同时包含官方光伏预报信息与调整权限，不能只归因于更新频率。全部主方案均按真实电价和连续库存回放，第四问完成 10248 次求解独立复核，最大数值误差为 1.87×10^{-9} 。

关键词：微网调度；储能；场景随机规划；滚动时域优化；波动电价；CVaR

1 问题重述

某小区微网由外部电网、光伏发电、用户负荷与一套电池储能系统组成。储能容量为 12000 kWh，安全运行区间为 1200–10800 kWh，最大充、放电功率均为 5000 kW，2025 年 1 月 1 日 0:00 储能量为 6000 kWh。题目要求在满足负荷和储能约束的条件下，制定外购电与充放电计划，并依次考虑日前不确定性、日内光伏预报更新以及实时波动电价。

四问并非相互独立。问题一给出固定日的全部输入，重点是验证储能跨时段搬移电量的经济性；问题二要求每日 0:00 在信息不完全条件下做全年计划，并承担 5 倍电价的紧急购电；问题三允许在 6:00、12:00 和 18:00 利用新光伏预报调整尚未执行的计划；问题四进一步使外网电价随时间变化。故本文先建立公共物理模型，再逐层增加信息与结算结构。

2 问题分析

2.1 问题一分析

已知固定日的电价、负荷和光伏预测后，决策只在时间上耦合。低价时段或光伏富余时充电，高价时段放电，可以转移外购电时点；但若不设置日末库存约束，模型会无偿消耗初始电量。问题一适合用 MILP 表达能量平衡、库存递推、功率上限及充放电互斥，并用 LP 松弛作计算对照。

2.2 问题二分析

附件 2 给出的是事后实际曲线，不能在当天 0:00 直接作为已知量输入，否则高价紧急购电机制失去意义。本文用目标日以前的完整历史日形成点预测和联合残差场景；0:00 计划在全部场景间共享，实际偏差由紧急购电与富余处置补救。评价时再用当天实际曲线回放，并将日末储能连续传至下一日。

2.3 问题三分析

日内新预报的价值来自信息变好，但计划调整又有不对称结算成本。应严格区分预报发布时间、目标时刻和预测提前量；已经执行的时段不可修改。本文以四个更新时刻重复求解剩余时域的两阶段随机模型，只执行到下一次更新，并用 CVaR 刻画紧急费用上尾风险。

2.4 问题四分析

波动电价使储能同时承担能量套利和预测风险缓冲。若未来电价在决策时不可知，应与净负荷共同构造条件场景；把全年实际电价提前输入只能作为完美信息下界。小样

本下经验价量依赖可能估计过强，因此问题四还需校准依赖收缩强度；日内模型则应在当前时刻前瞻下一次预报更新，使当前留存电量体现后续条件调整价值。

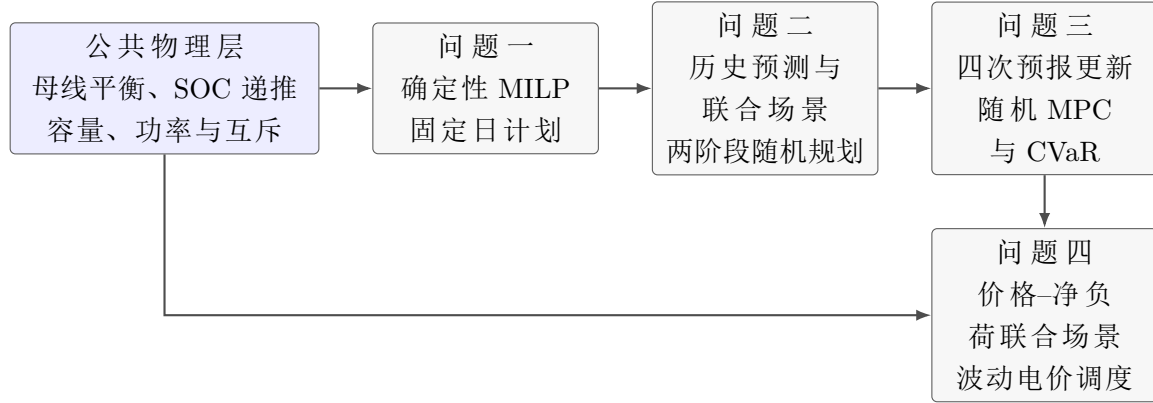


图 1 四问共用物理层及信息递进关系

3 模型假设

- A1.** 将附件中 00:10 对应数据视为区间 $[0:00, 0:10)$ 的平均功率，第 144 个值对应 $[23:50, 24:00)$ ；若官方澄清为区间起点，所有序列整体一致修正。
- A2.** 充电效率与放电效率均取 0.9，充放电量均在交流母线侧计量；总往返效率解释为 0.9 的口径留作敏感性比较。
- A3.** 不允许向外网售电；光伏或已购电配额过剩时允许形成无收益的富余电量。
- A4.** 问题一日初、日末储能量均为 6000 kWh。问题二至四跨日连续传递，不进行每日重置。
- A5.** 问题二在每日 0:00 只能使用目标日以前的数据。日前购电和名义电池轨迹固定，日内偏差仅由紧急购电和富余处置补救。
- A6.** 问题三主结算按最终执行购电量相对 0:00 原计划一次结算；下调电量按 0.5 倍电价退回。下调另加罚的解释作为备选口径。
- A7.** 题面没有给出并网购电上限、售电价、固定调整费和电池退化费，主模型不人为增加这些参数。

4 符号说明

一天划分为 $T = 144$ 个时段， $\Delta t = 1/6$ h。主要符号见表1。

表 1 主要符号及单位

符号	含义	单位
L_t, G_t	第 t 时段负荷电量、光伏电量	kWh
p_t	第 t 时段外购电价	元/kWh
q_t, r_t, e_t	日前计划、调整后正常购电、紧急购电量	kWh
c_t, d_t	交流母线侧充电量、放电量	kWh
s_t	第 t 时段开始时的电池内部储能量	kWh
u_t	未被负荷或电池利用的富余电量	kWh
η_c, η_d	充、放电效率，主模型均取 0.9	无量纲
Ω, π_ω	场景集合、场景概率	—

5 数据处理与公共物理模型

5.1 数据结构与时间对齐

附件 1 含 144 个日内点；附件 2 和附件 4 分别含 365 天 $\times$ 144 点的实际负荷、光伏和实时电价；附件 3 含 365 天每天 4 次、每次未来 24 小时的光伏预报，共 35040 个预报值。程序检查未发现缺失值、非有限值或负的负荷、光伏和电价值。全年负荷范围为 1995.72–7978.88 kW，实际光伏范围为 0–10216.20 kW，波动电价范围为 0.0076–1.7936 元/kWh。

附件给出功率，进入平衡式前统一转为时段电量：

$$L_t = P_t^L \Delta t, \quad G_t = P_t^G \Delta t, \quad \Delta t = \frac{1}{6} \text{ h}. \quad (5.1)$$

最大充、放电功率 5000 kW 对应单时段电量上界 $\bar{B} = 833.333$ kWh。状态 s_1 对应 0:00， s_{145} 对应 24:00。

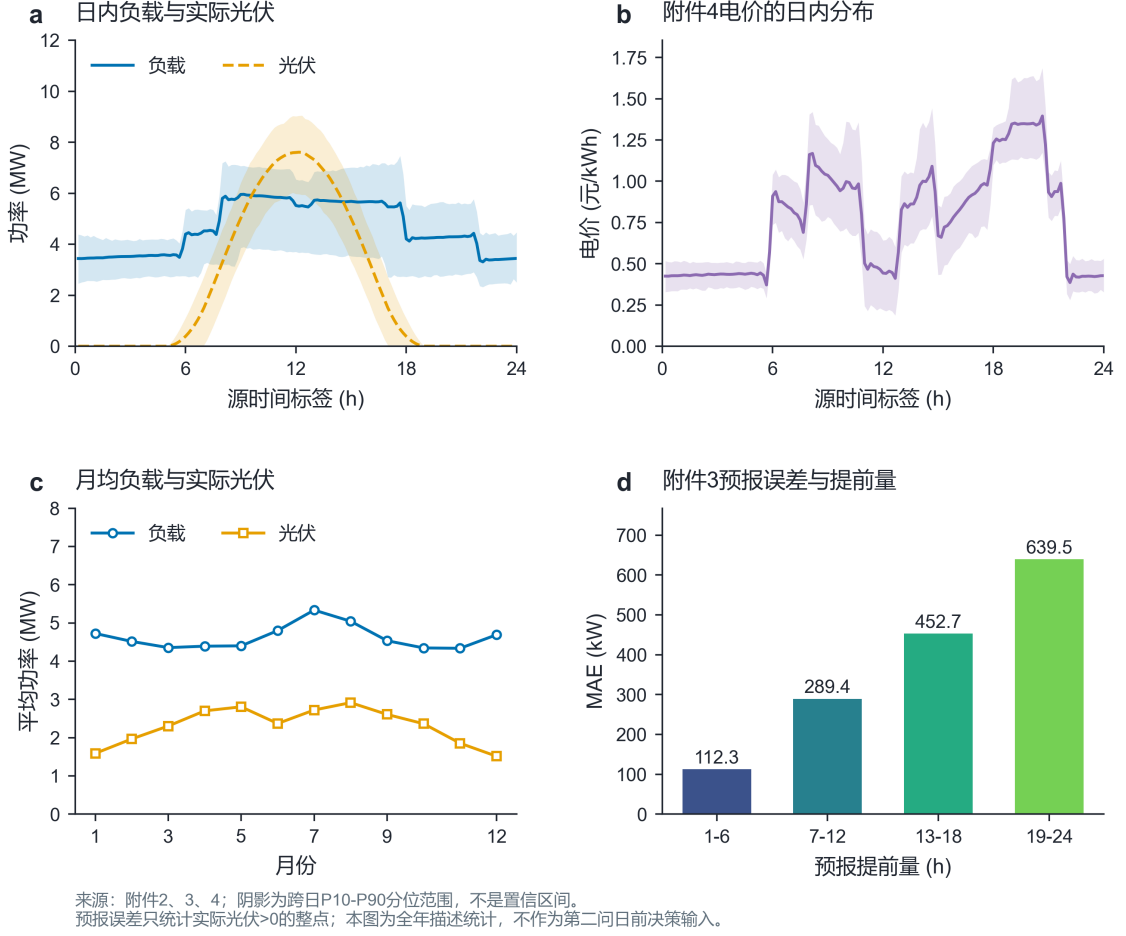


图 2 全年输入数据概览。阴影为同一源时间标签跨日的 P10–P90 范围；附件 3 误差仅在实际光伏大于 0 的目标整点统计。

5.2 公共约束

对任一时段，调整后正常购电、紧急购电、光伏和电池放电共同满足负荷及充电需求：

$$r_t + e_t + G_t + d_t = L_t + c_t + u_t. \quad (5.2)$$

其中 q_t 是 0:00 原计划， r_t 是最终执行的正常购电量，二者不是两股同时进入母线的电量。电池状态满足

$$s_{t+1} = s_t + \eta_c c_t - \frac{d_t}{\eta_d}, \quad (5.3)$$

$$1200 \leq s_t \leq 10800, \quad t = 1, \dots, 145, \quad (5.4)$$

$$0 \leq c_t \leq \bar{B} z_t, \quad 0 \leq d_t \leq \bar{B}(1 - z_t), \quad z_t \in \{0, 1\}. \quad (5.5)$$

问题二至四按 $s_{i+1,1} = s_{i,145}$ 跨日衔接。以上约束与光伏–储能微网的分层能量管理结构一致 [2]，构成四问共同的物理层；后续只改变可用信息、目标函数和结算规则。

6 问题一模型、求解与结果

6.1 确定性混合整数规划

问题一令 $r_t = q_t$ 、 $e_t = 0$ ，富余量退化为弃光 $0 \leq u_t \leq G_t$ 。目标为

$$\min F_1 = \sum_{t=1}^{144} p_t q_t, \quad (6.1)$$

并满足式(5.2)–(5.3)及 $s_1 = s_{145} = 6000$ kWh。二进制变量 z_t 严格排除同时充放电。另删除 z_t 求解 LP 松弛并计算同时充放电诊断量，用于验证整数约束是否影响结果。

6.2 调度结果

MILP 与 LP 均得到全天购电费 35126.948589 元，计划购电量 59482.698998 kWh，期初与期末储能量均为 6000 kWh。与不使用储能的可行基线 48052.046591 元相比，费用下降 26.898%。由于 MILP 能够直接保证物理互斥，后文采用 MILP 调度。

表 2 问题一指定时段计划购电量

时段	购电量/kWh	时段	购电量/kWh
10:00–10:10	0.000	12:00–12:10	480.412
14:00–14:10	0.000	16:00–16:10	445.432
18:00–18:10	531.894	20:00–20:10	0.000

图3表明，储能在低价时段和光伏富余时充电，在高价时段或净负荷较高时放电；状态始终处于 1.2–10.8 MWh 安全区间，并在日末回到 6 MWh。图中功率由每个 10 min 时段电量换算，未进行平滑。

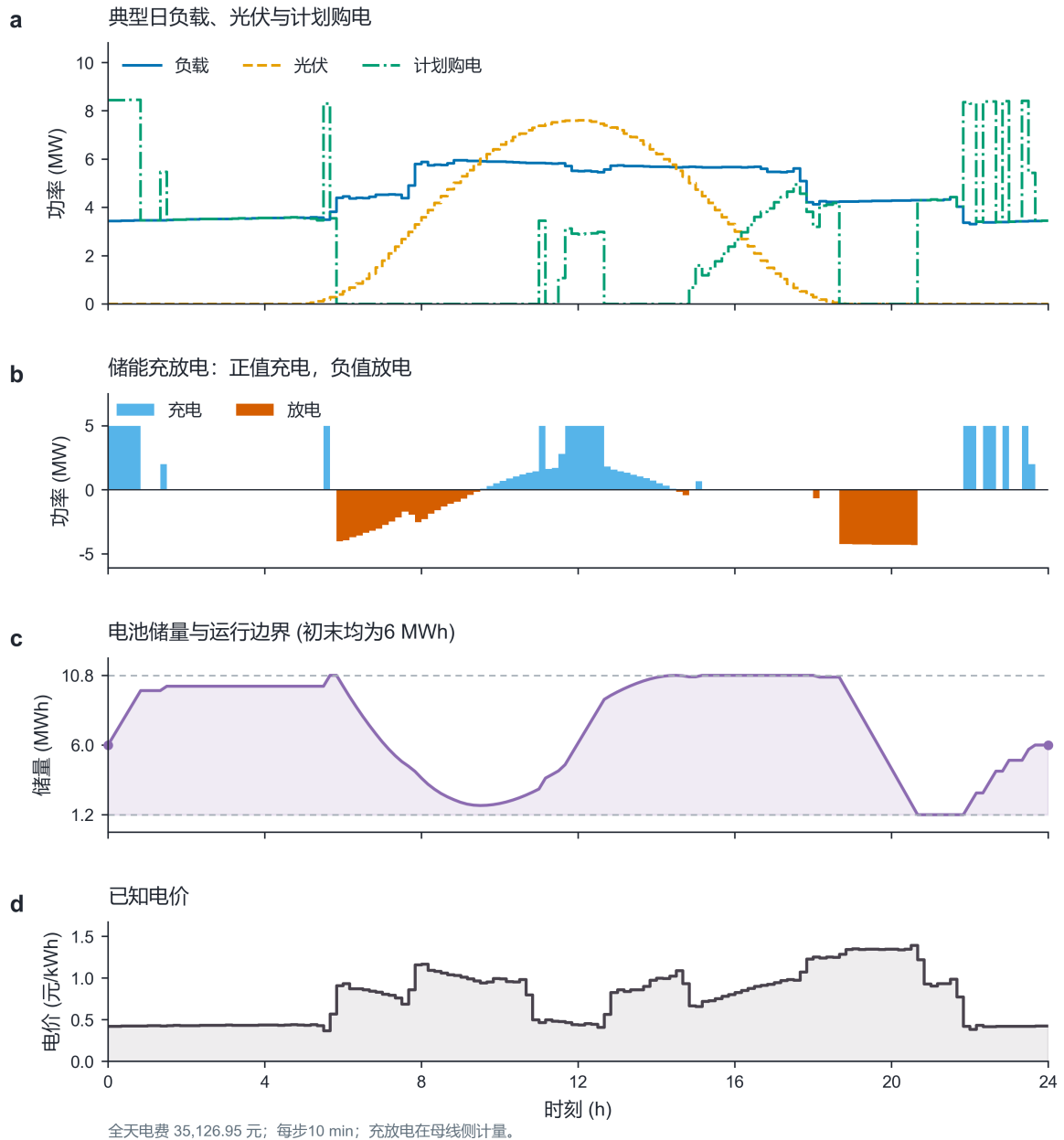


图 3 问题一典型日购电、充放电、储能状态与电价

7 问题二模型、求解与结果

7.1 因果预测、验证选择与联合场景

候选预测器包括 28 日加权历史均值 (P1)、傅里叶项加 ARIMA 误差 (P2) 以及分别预测负荷和光伏的随机森林 (P3)。三者均只使用目标日前的完整历史，缺少滞后量时显式回退到 P1。以 1 月 21–31 日滚动实际费用为主指标后，固定选择 P3 及 1.2 倍终端价值。正式期 P3 负荷和光伏 MAE 分别为 132.65 kW 和 157.03 kW；配置一经验证选定，便不利用 2 月至 12 月结果倒选。

对每个历史日保存负荷与光伏的整日样本外残差对，不独立打乱 144 个时段，也不拆开两类变量。目标日从最近 28 个残差日有放回联合抽样 256 次，重复日合并后的概率等于其抽中频率：

$$L_{i,t}^\omega = \max\{0, \hat{L}_{i,t} + \varepsilon_{j(\omega),t}^L\}, \quad G_{i,t}^\omega = \max\{0, \hat{G}_{i,t} + \varepsilon_{j(\omega),t}^G\}. \quad (7.1)$$

该构造借鉴概率轨迹与滚动回放的思想 [1]，保留日内连续性及同一时刻负荷-光伏误差的配对关系；所有预测与场景只由目标日以前的数据产生。256 次抽取允许重复，重复历史日被压缩为一个场景并按出现频数赋权，故并不等价于 256 个独立观测。还应注意：在固定名义电池轨迹且补救费用逐时可加的结构下，只要各时段净负荷边际分布不变，跨时段相关本身不改变期望费用；整日抽样的主要价值是保持可解释的轨迹与同刻度量间依赖，并为后续风险模型预留一致接口。

7.2 两阶段随机规划

第一阶段在 0:00 确定 $\mathbf{x}_i = (\mathbf{q}_i, \mathbf{c}_i, \mathbf{d}_i, \mathbf{s}_i, \mathbf{z}_i)$ ，并在全部场景间共享。第二阶段仅包含场景相关的紧急购电 $e_{i,t}^\omega$ 与富余量 $u_{i,t}^\omega$ 。日模型为

$$\min_{\mathbf{x}_i} \sum_t p_t q_{i,t} + \sum_{\omega \in \Omega_i} \pi_\omega \sum_t 5p_t e_{i,t}^\omega + \Phi_i(s_{i,145}), \quad (7.2)$$

并对每个场景满足式(5.2)。终端价值取 $\Phi_i(s) = -mvs$ ，其中

$$v = \min_t (p_t) / \eta_c = 0.412555556 \text{ 元/电池内部 kWh}, \quad (7.3)$$

倍率 m 由 1 月验证选为 1.2，故目标中的单位库存价值为 $mv = 0.495066667$ 元/内部 kWh。该项只近似跨日库存价值，不计入实际账单。1 月以预先指定的 P1 策略连续热身，1 月 1 日使用附件 1 日型先验；2 月 1 日储能量为 1200 kWh。2 月至 12 月逐日连续回放，12 月 31 日末约束为 6000 kWh。

取得计划后，用当天实际曲线计算

$$e_{i,t}^{\text{real}} = [L_{i,t}^{\text{real}} + c_{i,t} - q_{i,t} - G_{i,t}^{\text{real}} - d_{i,t}]^+, \quad (7.4)$$

实际费用为 $F_i^{\text{real}} = \sum_t p_t q_{i,t} + \sum_t 5p_t e_{i,t}^{\text{real}}$ 。因此全年过程是滚动求解 334 个日内两阶段随机 MILP，并非 334 阶段随机规划。

7.3 全年结果与对照

表 3 问题二 334 天方法与机制对照

方法	实际总费/万元	紧急购电费/万元	紧急购电量/MWh
P3 随机森林 + 联合场景 (主方案)	1447.93	98.62	262.35
P3 随机森林 + 点预测	1568.55	335.79	858.50
P2 傅里叶-ARIMA+ 联合 场景	1668.65	130.65	344.29
P1 加权历史 + 联合场景	1677.80	124.81	329.83
旧机制同初始库存对照	1676.87	121.67	320.32

主方案实际总费为 1447.93 万元，其中正常购电 1349.31 万元、紧急购电 98.62 万元，另有 3073.18 MWh 未利用电量。相对同一 P3 预测器的点预测方案，主方案费用降低 7.69%，紧急购电量由 858.50 MWh 降至 262.35 MWh，说明在 5 倍紧急电价下显式处理预测误差具有经济作用。相对同初始库存的旧机制对照，费用降低 13.653%；该差异同时包含预测器、抽样和终端价值选择，不能归因于单一算法。

全年 P1、P2、P3 的负荷 MAE 分别为 739.97、705.80 和 132.65 kW，光伏 MAE 分别为 177.81、225.01 和 157.03 kW。预测误差和调度费用在本次候选中方向大体一致，但模型选择只由事前固定的 1 月验证完成，不能利用全年结果倒选参数。

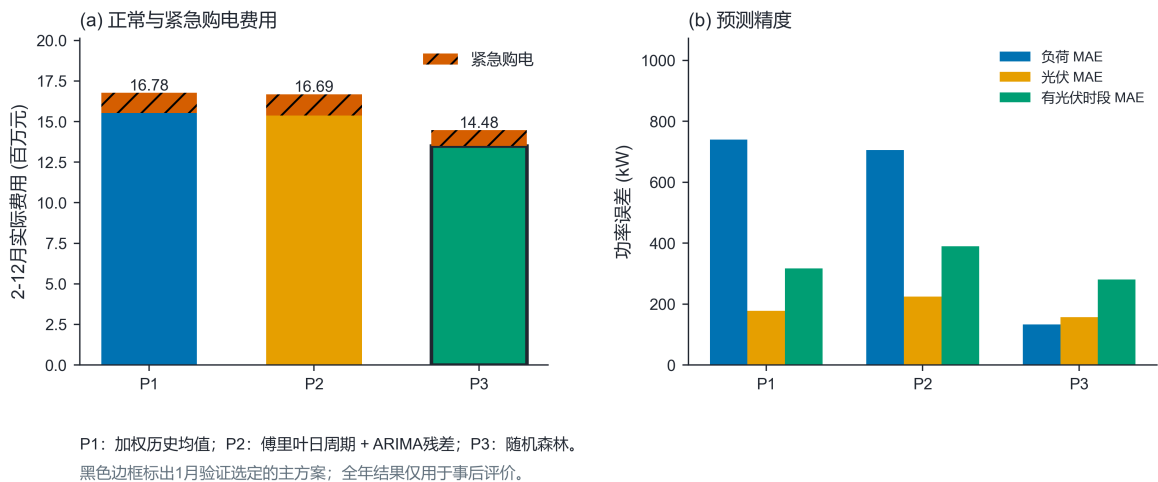
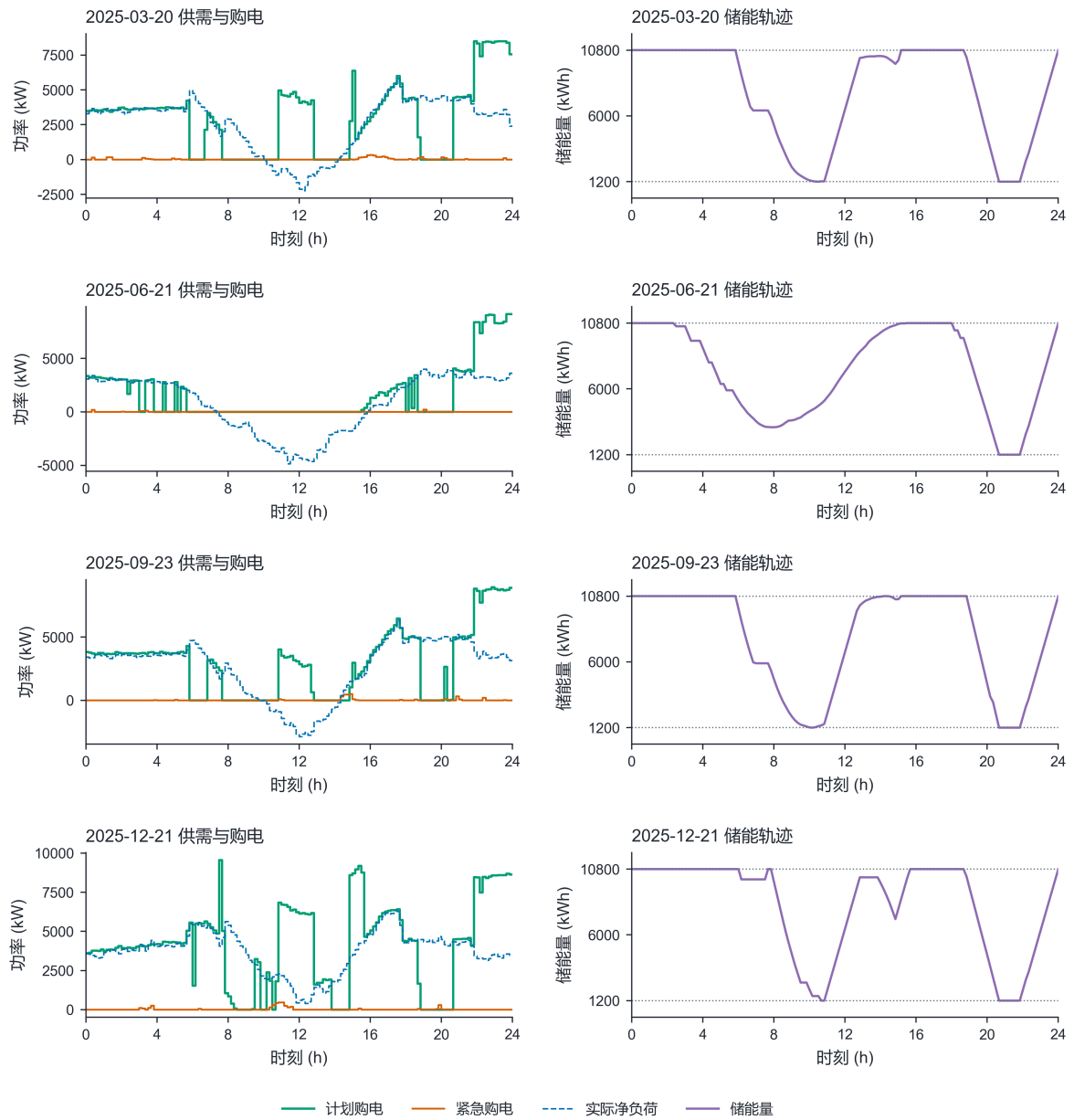


图 4 问题二预测方法与调度费用对照。各候选均按相同因果验证流程选定配置。

四个指定日期结果见表4。日初、日末库存均为 10800 kWh，反映线性终端价值倾向于保留电量；紧急购电的时段聚集程度存在明显差异，说明全年总量之外仍需检查季节与日内短缺分布。

表 4 问题二指定日期汇总

日期	计划购电/kWh	实际总费/元	紧急购电/kWh	日末库存/kWh
3 月 20 日	71527.72	45984.39	656.32	10800
6 月 21 日	40689.25	23403.55	132.98	10800
9 月 23 日	72187.02	47282.23	476.44	10800
12 月 21 日	98700.07	65366.39	559.62	10800



购电曲线由10分钟电量乘6转换为平均功率；虚线净负荷包含当天实际值，仅用于事后展示。

图 5 问题二四个指定日期的计划购电、充放电及储能状态

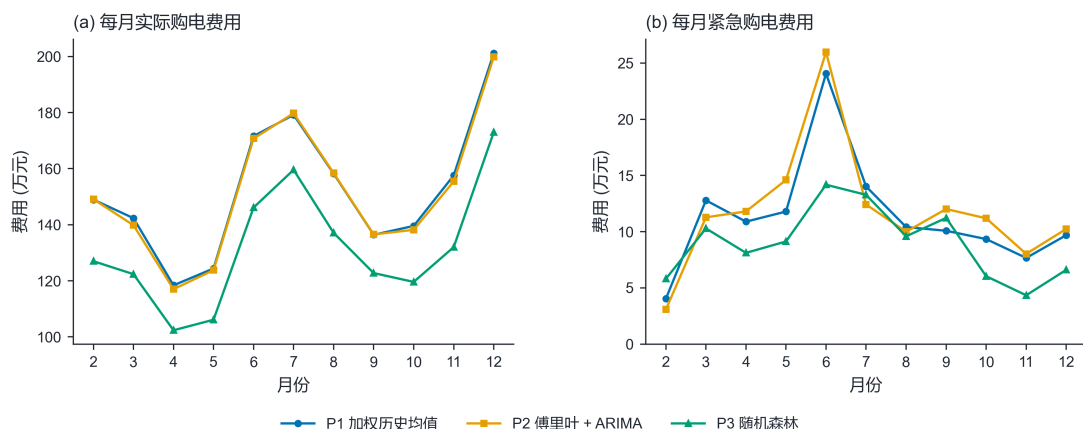


图 6 问题二逐月实际费用、紧急购电量与预测误差。月度分解用于检查总体改进是否由少数月份驱动。

7.4 敏感性与计算复核

主配置的终端倍率减 20% 后费用变化为 -0.205% ；残差幅度减、增 20% 时分别变化 $+0.138\%$ 和 $+0.325\%$ 。另两个正式期抽样种子相对主方案变化 $+0.005\%$ 和 -0.022% 。这些局部试验说明抽样扰动较小，但终端倍率的 1 月选择并不保证全年最优，故不利用全年敏感性倒选主参数。

程序完成 28 项测试，并对 25 个实验、4837 个日模型或回放记录独立复算；最大数值残差为 3.73×10^{-9} ，低于 10^{-5} 阈值。检查覆盖日期与时间索引、能量守恒、SOC 边界、充放电互斥、费用重算、模板读回、防未来信息泄漏及 Benders 小样例等价性。主结果采用整体 MILP。

需要指出，主方案有 333 天日末储能量贴近 10800 kWh 上限，表明 1.2 倍线性终端价值较强。该现象不破坏物理可行性，但限制了跨日运行解释；后续应比较 48 h 滚动时域或更精细的库存价值函数。

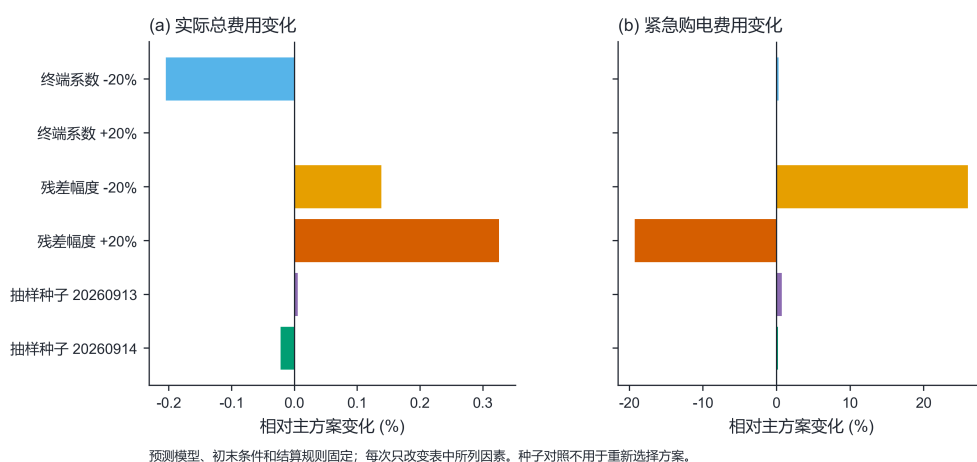


图 7 问题二终端价值、残差尺度与抽样种子的稳定性检验

8 问题三模型、求解与结果

8.1 滚动信息时间线

令更新时刻 $k \in \{0, 6, 12, 18\}$ h, $m(k) = 6k + 1$ 。在 k 时刻仅优化尚未执行的时段 $\mathcal{T}_k = \{m(k), \dots, 144\}$, 并只执行到下一次更新。信息集包含当前真实 SOC、已实现的负荷与光伏、已发布预报、固定电价和 0:00 原计划, 不包含未来实际值或之后才发布的预报。

对发布时间 a 、当前时刻 τ 和目标时刻 v , 预测提前量 $h = v - a$, 预报年龄 $A = \tau - a$, 距目标时间 $H = v - \tau$, 满足 $h = A + H$ 。使用旧预报时不能把 H 误当作该预报原始提前量。附件 3 的小时节点按线性函数在 10 min 区间积分, 形成与公共模型一致的 kWh 输入。

8.2 提前量校准、联合场景与可信度指标

主方案以过去 28 个完整日估计分发布时间、6 h 提前量箱的偏差与尺度, 并用 3 日收缩稳定小样本估计。候选还包括统一尺度和指数尺度

$$\sigma_g(h) = \sigma_{g,0} \exp(\beta h), \quad \beta \geq 0, \quad (8.1)$$

以及归一化展示指标

$$R_g(h) = \frac{1}{1 + \{\sigma_g(h)/s_{\text{ref}}\}^2}. \quad (8.2)$$

R 只反映条件误差尺度并可反解场景离散度, 不是预测正确概率, 也不改变预测中心或场景概率。场景按同一历史日联合抽取负荷与光伏标准化残差块, 每次 64 个抽样; 各更新时刻根据当前已发布预报和真实 SOC 重建剩余时域场景。

8.3 调整结算与 CVaR 随机 MPC

令 q_t 为 0:00 原计划, $r_{t|k}$ 为更新后的正常购电量。主结算为

$$g_t(q, r) = p_t q + 1.5p_t [r - q]^+ - 0.5p_t [q - r]^+ = p_t r + 0.5p_t |r - q|. \quad (8.3)$$

它表示相对原计划的上调和下调均承担 50% 偏差成本。定义剩余时域场景紧急费用 $C_k^\omega = \sum_{t \in \mathcal{T}_k} 5p_t e_{t|k}^\omega$, 风险增强目标为

$$\min \sum_{t \in \mathcal{T}_k} g_t(q_t, r_{t|k}) + \mathbb{E}[C_k] + \lambda \text{CVaR}_\alpha(C_k) + \gamma_k b_k^{\text{end}}, \quad (8.4)$$

其中

$$\text{CVaR}_\alpha(C_k) = \min_{\zeta} \left\{ \zeta + \frac{1}{1 - \alpha} \sum_{\omega} \pi_{\omega} [C_k^\omega - \zeta]^+ \right\}. \quad (8.5)$$

CVaR 表示剩余紧急费用的上尾风险 [5]; 风险惩罚和日末软目标只参与决策, 不计入实际账单。每天在四个更新点重算电池计划, 区间内固定执行到下一节点; 全年末统一回到 6000 kWh。

8.4 消融、预测检验与经济结果

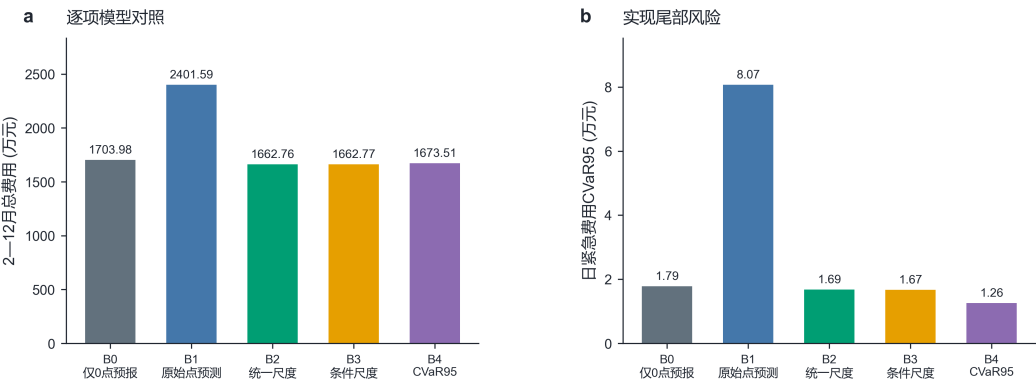
表5按相同 334 天、相同跨日衔接给出主要消融。B0 仍保留每日四次求解，但只让 0:00 预报成为新信息；B1 使用官方点预测；B2、B3 依次加入联合场景和提前量条件分箱尺度；B4 在 B3 上增加 CVaR。由此避免把控制机会与新信息价值混为一谈。

表 5 问题三 B0-B4 主要对照

方案	总费/万元	紧急费/万元	日紧急费 CVaR95/元
B0 仅 0:00 新预报	1703.98	126.22	17898.31
B1 官方点预测	2401.59	1196.73	80723.79
B2 统一尺度联合场景	1662.76	91.56	16854.47
B3 条件分箱尺度（主方案）	1662.77	91.25	16718.04
B4 CVaR90, $\lambda = 0.1$	1671.75	65.99	13088.86

B3 实际总费为 16627713.37 元，其中紧急购电费 912512.24 元、紧急购电量 238.47 MWh。B2 费用比 B3 低 92.03 元，表明条件尺度在本年度并未稳定降低平均费用；但 B3 的紧急费和经验 CVaR95 略低，故不能把该模块宣称为全面优于统一尺度。B4 将日紧急费 CVaR95 降至 13088.86 元、紧急费降至 659931.40 元，同时使总费增加 89781.36 元，体现平均费用与尾部风险之间的权衡。

光伏正值样本的合并 CRPS 为 184.54 kW，90% 经验区间覆盖率为 80.67%。覆盖率低于标称值，说明有限日块及非负截断后的区间仍偏窄； R 与 σ 的一一映射不能提供概率安全保证。



B2→B3只改变误差尺度；B3→B4只增加风险项（ $\alpha=0.95$, $\lambda=0.1$ ）。B0仍每日优化4次，仅屏蔽日内新预报。

图 8 问题三 B0-B4 费用与风险对照

月度分解见图9。风险增强方案并非在每个月都占优，其作用主要体现为压低高紧

急费用月份的上尾；因此选择 λ 应视运营方风险偏好而定，不能把 CVaR 惩罚作为无成本改进。

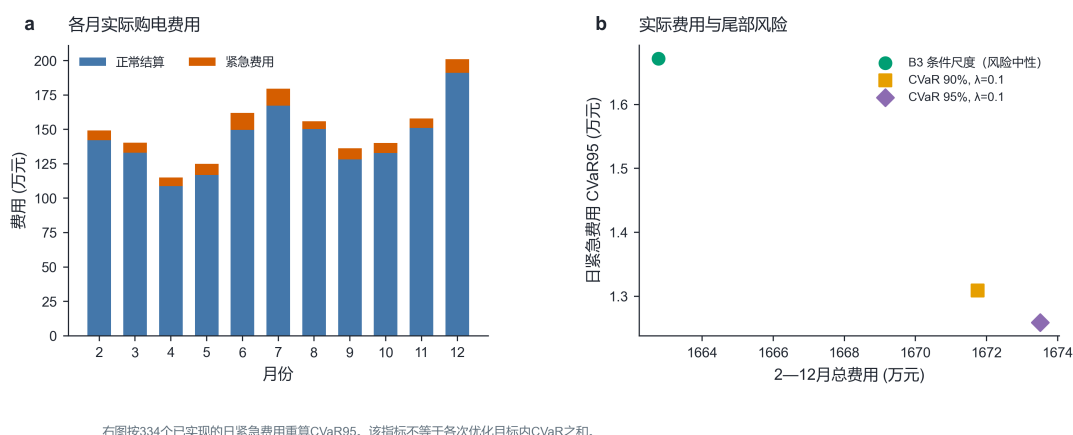


图 9 问题三月度费用、紧急购电与风险中性/风险增强方案对照

8.5 预报更新时间价值

全四次新预报相对仅 0:00 新预报累计节省 412063.77 元。已有预报时刻的 8 种组合中，事后最低费用组合为 0:00、6:00 和 12:00，共节省 415281.22 元；加入 18:00 预报的边际节省为 -3217.45 元。配对 7 日移动块 Bootstrap 给出的 18:00 边际日节省 95% 探索性区间为 $[-21.41, 0.78]$ 元，不能据此断言该时刻必然有正值。附件未提供 3:00、9:00、15:00 和 21:00 的新预报，重算旧预报不能冒充额外气象信息。

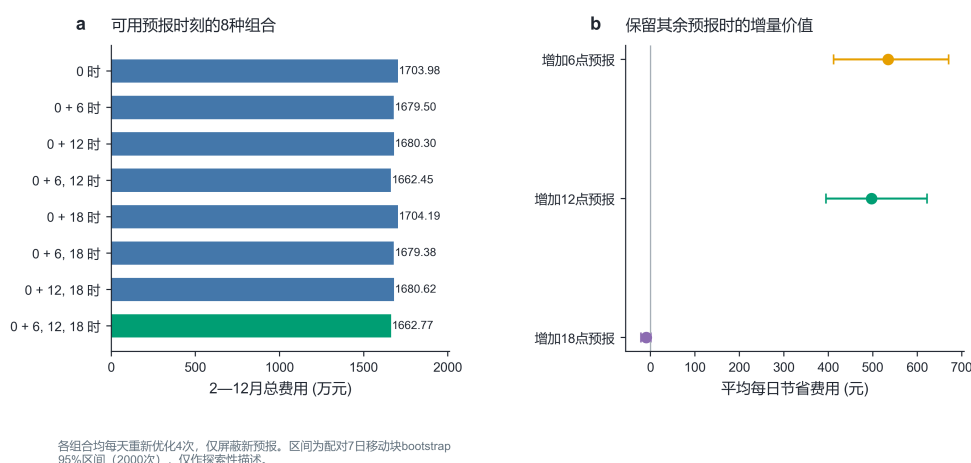


图 10 问题三已有预报时刻的增量信息价值

主方案共完成 1336 次外层滚动、334 天执行回放；全部对照共含 21 组 334 日实验、28056 次正式滚动求解。物理约束、信息时序、跨日连续、结算重算和概率尺度均经独立审计，最大残差为 7.55×10^{-7} ，20 项自动测试通过。

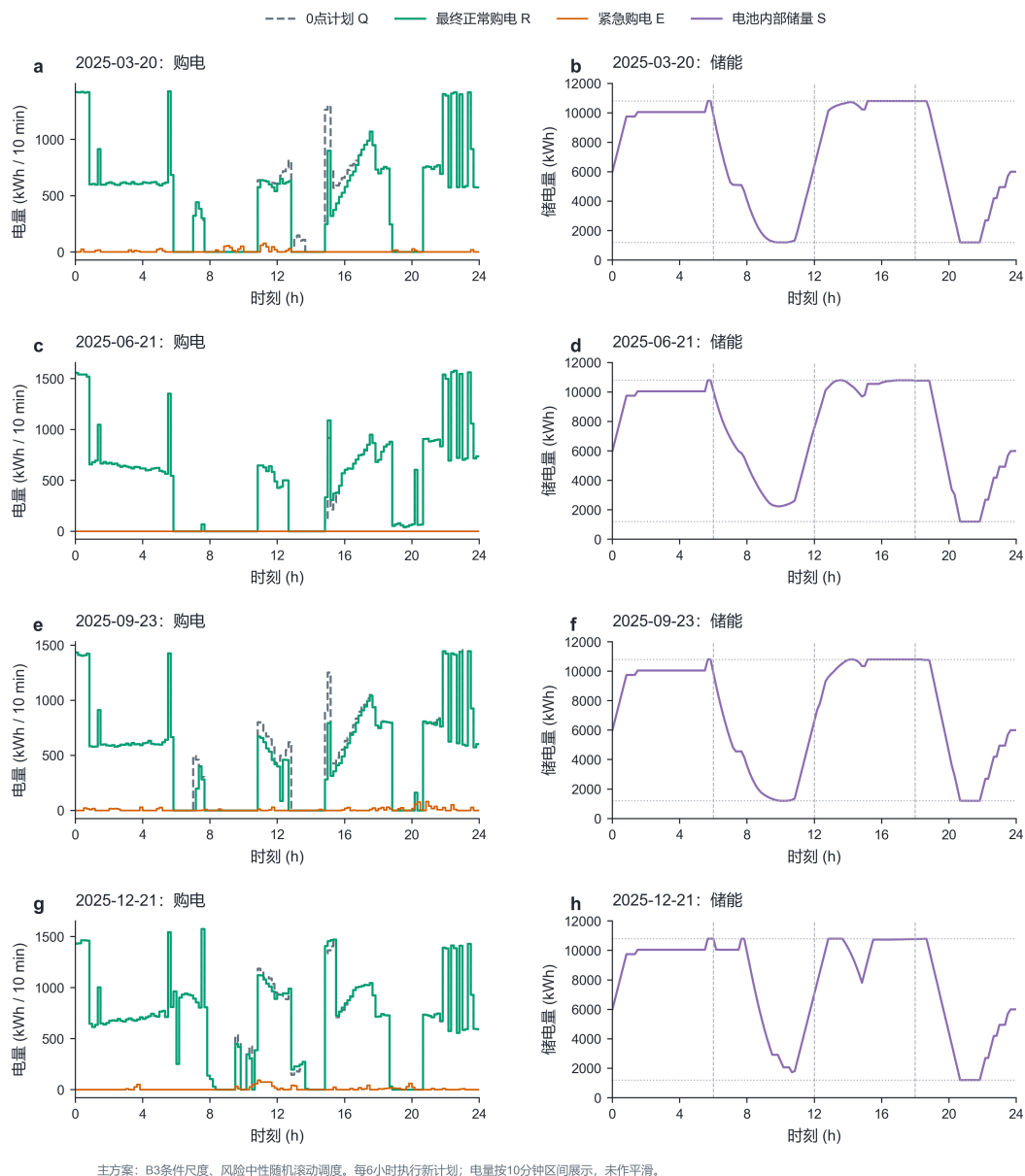


图 11 问题三四个指定日期的正常购电、调整、紧急购电与库存轨迹

表 6 问题三指定日期执行汇总

日期	原计划/kWh	最终正常购电/kWh	紧急购电/kWh	实际总费/元
3 月 20 日	73655.48	69660.81	814.18	46675.88
6 月 21 日	71728.33	72348.82	0.00	44955.16
9 月 23 日	72870.32	68194.27	956.60	47898.00
12 月 21 日	95608.86	95643.20	1180.13	65911.55

图12把概率预测的尖锐性和校准性分开报告。90% 区间覆盖不足说明模型不能把

可信度 R 解释成安全概率；模型给出的只是由历史误差尺度驱动的条件场景。因此本文同时报告平均账单、紧急费用和尾部风险，而不只给单一费用指标。

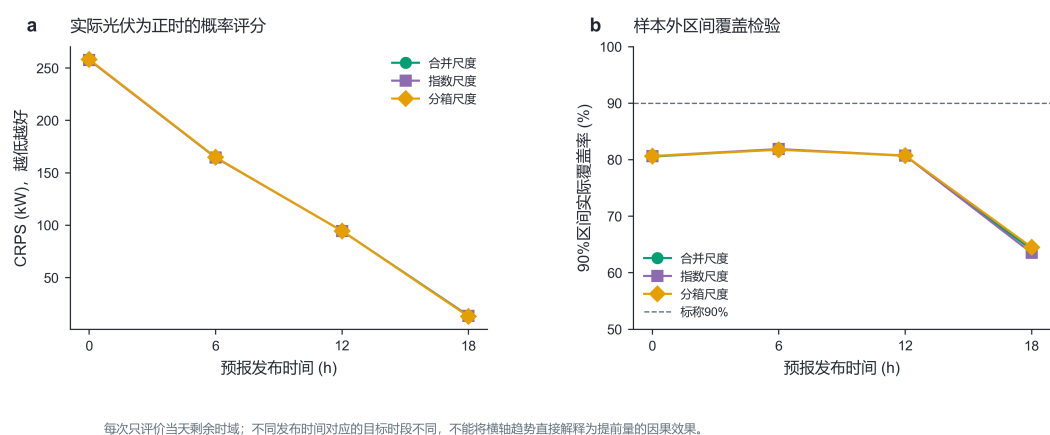


图 12 问题三概率预测样本外检验：CRPS、区间覆盖率与区间宽度

图13给出提前量尺度和可信度 R 的对应关系。随着提前量变化，尺度并非恒定；但有限历史样本会使分箱估计波动，所以本文使用收缩，并将统一尺度作为必要对照。

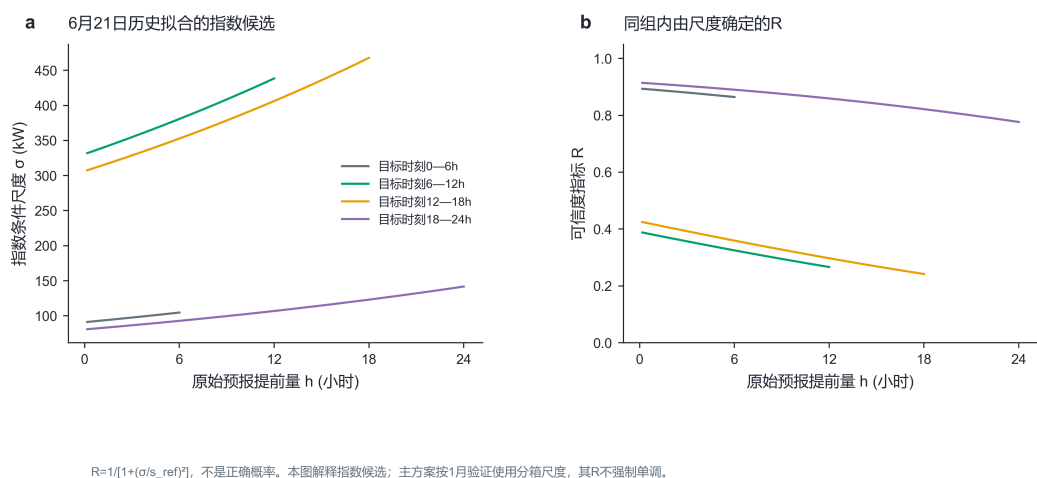
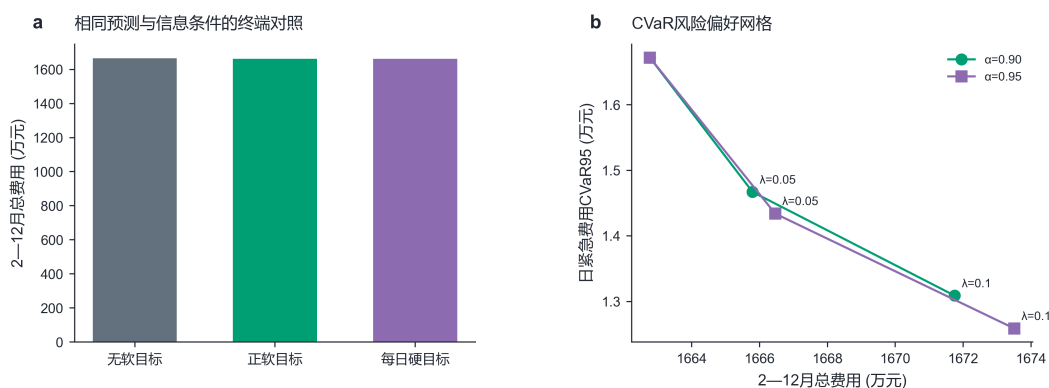


图 13 问题三提前量误差尺度、分箱收缩与可信度指标解释

终端条件和风险参数的敏感性见图14。图中现金费用均由真实执行轨迹重算，终端价值和 CVaR 惩罚仅改变决策，不被混入实际账单，从而保持不同参数之间可比。



所有方案跨日连续并在年末统一6000 kWh。软目标仅进入优化目标，不计入实际电费； $\lambda=0$ 对应B3。

图 14 问题三终端条件与 CVaR 风险参数敏感性

9 问题四：波动电价下的调控策略

9.1 信息结构、价格预测与统一评价

问题四沿用式(5.2)–(5.3)，但所有正常购电、调整与紧急购电均按交付时段的真实波动价结算。预测价格仅服务决策，绝不替代真实账单。研究期为 2025 年 2 月 1 日至 12 月 31 日，共 334 天；1 月只用于热身和固定参数，正式期后不再倒选。

记净负荷 $N_{i,t} = L_{i,t} - G_{i,t}$ 。价格基线在加权均值、14 日均值和 28 日均值中用 1 月验证选择 28 日均值；在日内更新点，以节点前已实现的对数价格残差按 $\alpha = 0.2$ 、衰减率 $0.1h^{-1}$ 修正未来基线。最近至多 28 个完整历史日的净负荷残差和价格对数残差按同一历史日配对，构成因果场景。指数变换保证价格为正，净负荷则允许为负，从而保留光伏富余。

电价预测与信息更新

2025年2—12月评价；右侧按已执行时段汇总，不重复计算滚动预测的重叠部分。

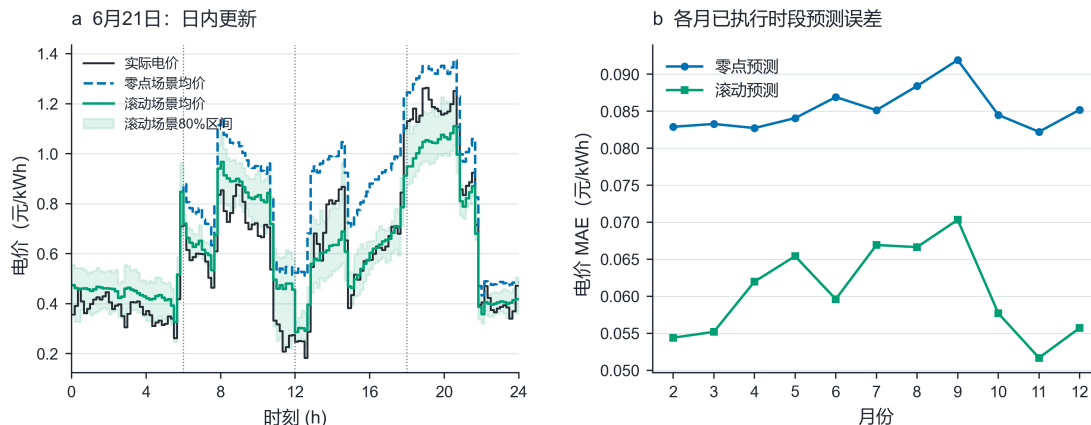


图 15 问题四价格预测、日内残差修正和信息更新时间线

仅制定日前计划的实际账单为

$$F_{4,2}^{\text{real}} = \sum_t p_{i,t}^{\text{real}} (q_{i,t} + 5e_{i,t}^{\text{real}}). \quad (9.1)$$

允许日内调整时，令 $b(q, r) = r + 0.5|r - q|$ ，则

$$F_{4,3}^{\text{real}} = \sum_t p_{i,t}^{\text{real}} \{b(q_{i,t}, r_{i,t}^{\text{exec}}) + 5e_{i,t}^{\text{real}}\}. \quad (9.2)$$

两类策略均按同一日期、同一真实样本和连续库存回放。1 月热身产生的 2 月 1 日初始库存约为 6000 kWh，非人为重置；12 月 31 日末统一约束回到 6000 kWh。

9.2 仅日前决策：价量依赖收缩模型

小样本下直接使用经验价格—净负荷配对分布，可能把偶然依赖误当作稳定规律。引入全局收缩系数 $\rho \in [0, 1]$ ：

$$\hat{\mathbb{P}}_t^\rho = \rho \hat{\mathbb{P}}_t^{\text{pair}} + (1 - \rho)(\hat{\mathbb{P}}_t^p \otimes \hat{\mathbb{P}}_t^N), \quad (9.3)$$

相应的价格加权缺电函数为

$$H_t^\rho(A) = \sum_\omega \pi_\omega \{\rho p_t^\omega + (1 - \rho)\bar{p}_t\} (N_t^\omega - A)^+. \quad (9.4)$$

$\rho = 1$ 保留同日配对， $\rho = 0$ 保留各自边际分布但在期望费用中独立组合。1 月历史验证选择 $\rho = 0$ ；这不是声称价格与净负荷真实独立，而是说明现有样本不足以支持依赖项带来稳定收益。终端价值倍率同样在 1 月固定为 $\gamma = 0.5$ 。

表7给出严格样本外对照。主方案费用为 1520.11 万元，比点预测策略减少 124.60 万元（7.576%）。 $\rho = 0$ 比 $\rho = 1$ 少 6433.06 元，但 7 日移动块 Bootstrap 的 95% 区间为 $[-689.82, 13358.52]$ 元，跨过 0，故不宣称独立收缩具有确定优势。

表 7 问题四仅日前决策的 334 天对照

方案	实际总费/万元	紧急费/万元	紧急购电/MWh
点预测	1644.71	350.15	—
成对场景， $\rho = 1$	1520.75	100.67	—
边际独立收缩， $\rho = 0$ （主方案）	1520.11	104.05	256.56
主方案但 $\gamma = 0$	1522.22	—	—

主方案正常购电费 14160543.12 元，未利用电量 3091.10 MWh，充、放电量分别为 6900.89 MWh 和 5589.72 MWh。全年有 71 个日末库存落在 1200 kWh 下界，无日末落在 10800 kWh 上界，说明波动价格下电池更多被当作风险缓冲和套利资源，而非被终端价值长期推至满电。

9.3 允许日内调整：下一节点条件前瞻

在更新点 $k \in \{0, 6, 12, 18\}$ ，模型只展开下一更新点 κ 。用届时可观测的光伏预报修订量和节点前已实现价格偏差构成信号，并将场景划成 $K \in \{1, 2, 3\}$ 组。分叉前正常购电、充放电和库存对所有场景共享；各分支必须从同一桥接库存 S_b 出发：

$$S_b^1 = \dots = S_b^K, \quad (9.5)$$

分叉后动作才允许随信息组变化。这样，下一阶段继续运行的价值能够影响当前留存电量，同时不使用下一节点以后才会知道真实负荷、价格或最优费用。 $K = 1$ 是无条件分组基准；1 月验证选择 $K = 1$ 为提交主方案，尽管 $K = 2, 3$ 在正式期事后费用更低，也不追溯改选。

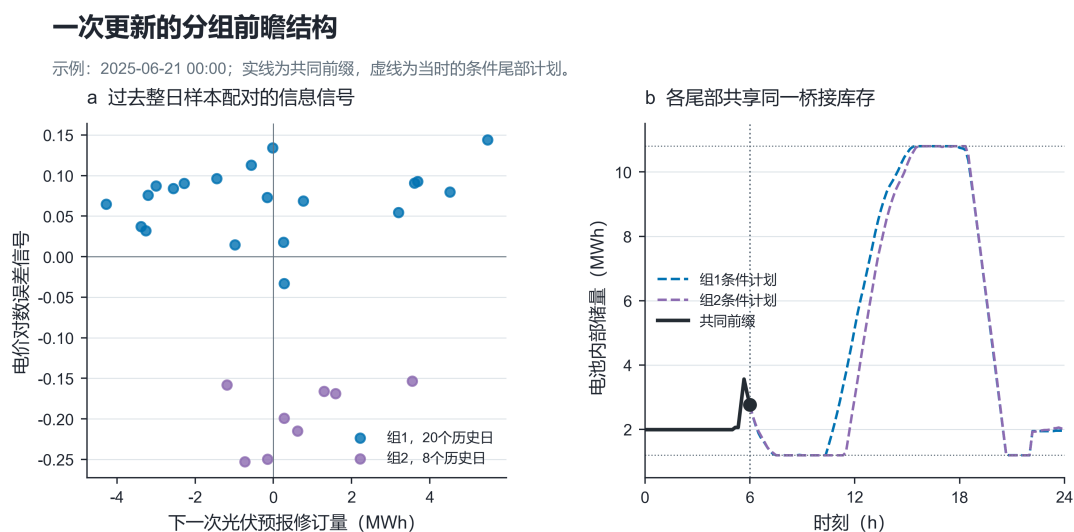


图 16 问题四条件分组信号、非预见性与桥接库存示意

表8显示，主方案全年实际费用 14997167.51 元，其中原计划费 14400074.38 元、调整净费用 -102744.03 元、紧急购电费 699837.16 元；紧急购电 176.24 MWh，上调 145.26 MWh、下调 749.15 MWh。相对同机制点预测策略节省 1060565.04 元（6.605%）。价格残差修正比不修正少 1507.00 元，但 Bootstrap 区间 $[-905.59, 3938.87]$ 元仍跨 0。

表 8 问题四允许日内调整的 334 天对照

方案	实际总费/万元	紧急费/万元	相对主方案差额/元
点预测	1605.77	275.31	1060565.04
不做价格残差修正	1499.87	—	1507.00
$K = 1$ (1 月验证所选主方案)	1499.72	69.98	0
$K = 2$	1499.48	—	-2350.94
$K = 3$	1498.90	—	-8205.12
主方案但 $\gamma = 0$	1501.90	—	21848.34

$K = 2$ 相对 $K = 1$ 的 7 日移动块 Bootstrap 区间为 $[-2110.17, 6774.51]$ 元； $K = 3$ 为 $[168.62, 16792.18]$ 元。后者在正式期显示正节省，但 $K = 2, 3$ 均未通过事前 1 月验证，因此只能作为样本外敏感性证据，不能替代冻结的主方案。1002 个可形成下一节点的更新中， $K = 2$ 成功分成两组 884 次， $K = 3$ 成功分成三组 616 次，其余自动退化；18:00 后没有下一次更新，因而不构造虚假分支。

全年实际购电费对照

评价期为2月1日至12月31日；1月已冻结主方案，全年对照结果不用于倒选参数。

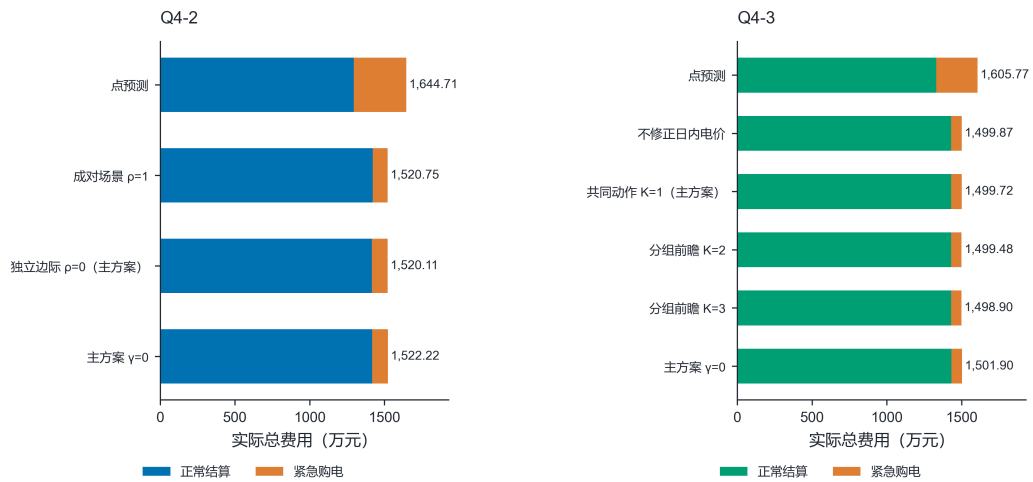


图 17 问题四各方案全年真实账单对照；主方案由 1 月验证预先冻结

月度费用与紧急购电

两条主方案均从1月1日6000 kWh开始热身，正式评价持续继承真实库存。

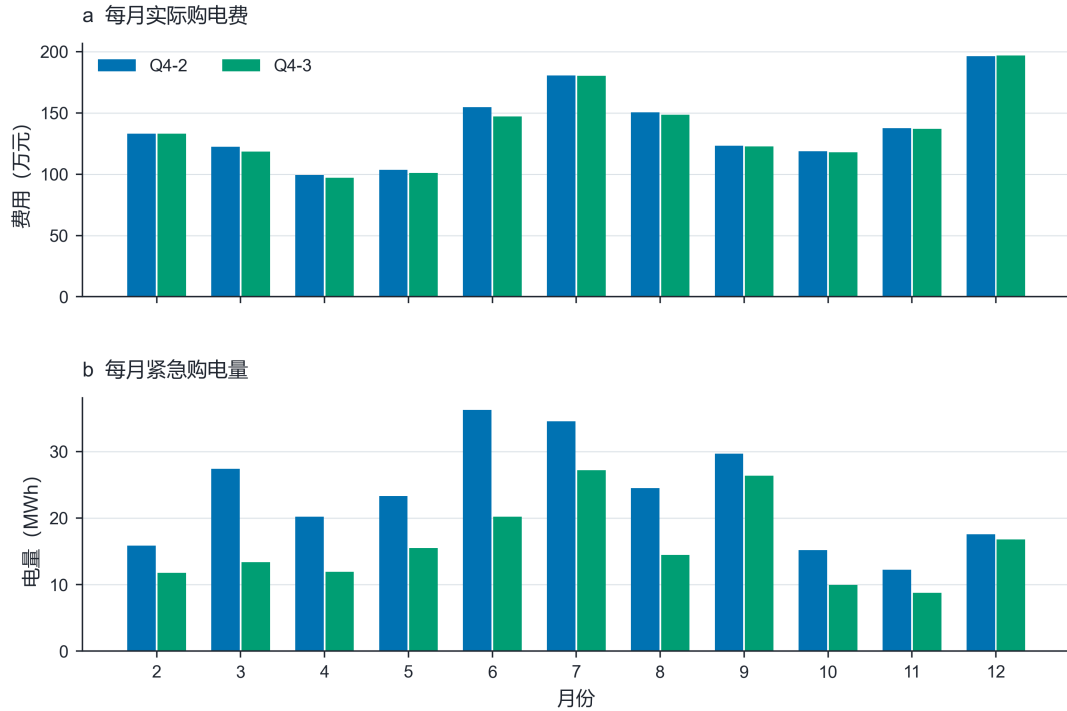


图 18 问题四主方案逐月总费、紧急购电费及紧急购电量

9.4 两种权限的比较与指定日期

允许日内调整的主方案比仅日前决策主方案少 203918.18 元，即 1.341%。这是一项“制度包”比较：后者同时获得官方光伏预报更新和调整正常购电的权限，不能把差额全部归因为预报更新频率。四个指定日期见表9；库存跨日连续，因此 6 月 21 日的日初库存并非整值 6000 kWh。

表 9 问题四指定日期实际执行汇总

权限	日期	正常购电/kWh	紧急购电/kWh	实际总费/元
仅日前	3 月 20 日	71444.89	635.62	46935.03
仅日前	6 月 21 日	39044.28	145.32	19815.66
仅日前	9 月 23 日	72473.21	454.94	49460.39
仅日前	12 月 21 日	98632.66	551.06	76913.47
日内调整	3 月 20 日	68591.13	965.91	48349.16
日内调整	6 月 21 日	36993.31	138.25	19065.54
日内调整	9 月 23 日	73107.88	127.93	49748.49
日内调整	12 月 21 日	97959.73	470.69	76490.46

Q4-2: 题目指定四日调度

左列为交流母线侧平均功率，右列为电池内部储量；所有曲线保留原始10分钟分辨率。

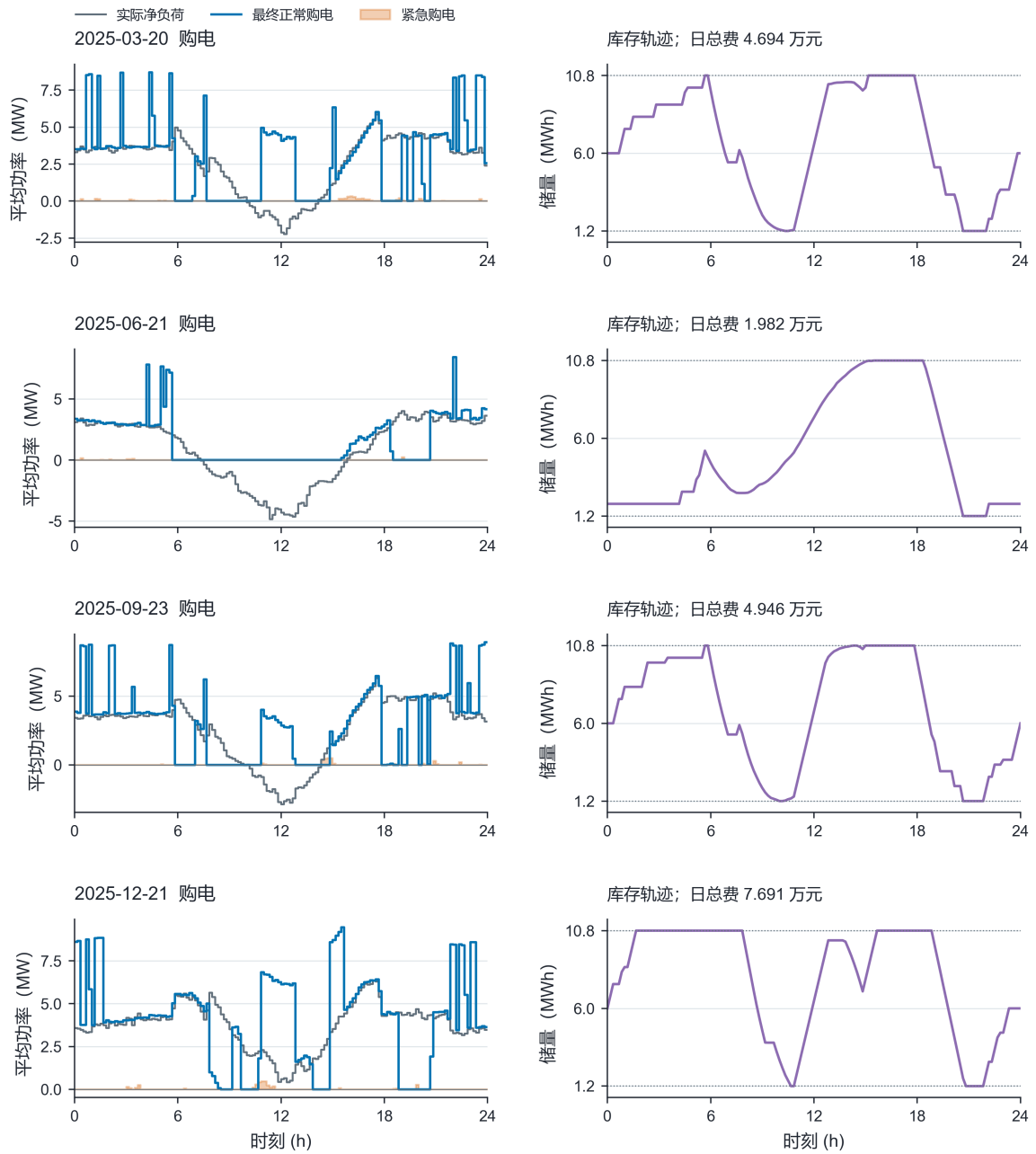


图 19 问题四仅日前决策主方案在四个指定日期的购电与储能轨迹

Q4-3: 题目指定四日调度

左列为交流母线侧平均功率，右列为电池内部储量；所有曲线保留原始10分钟分辨率。

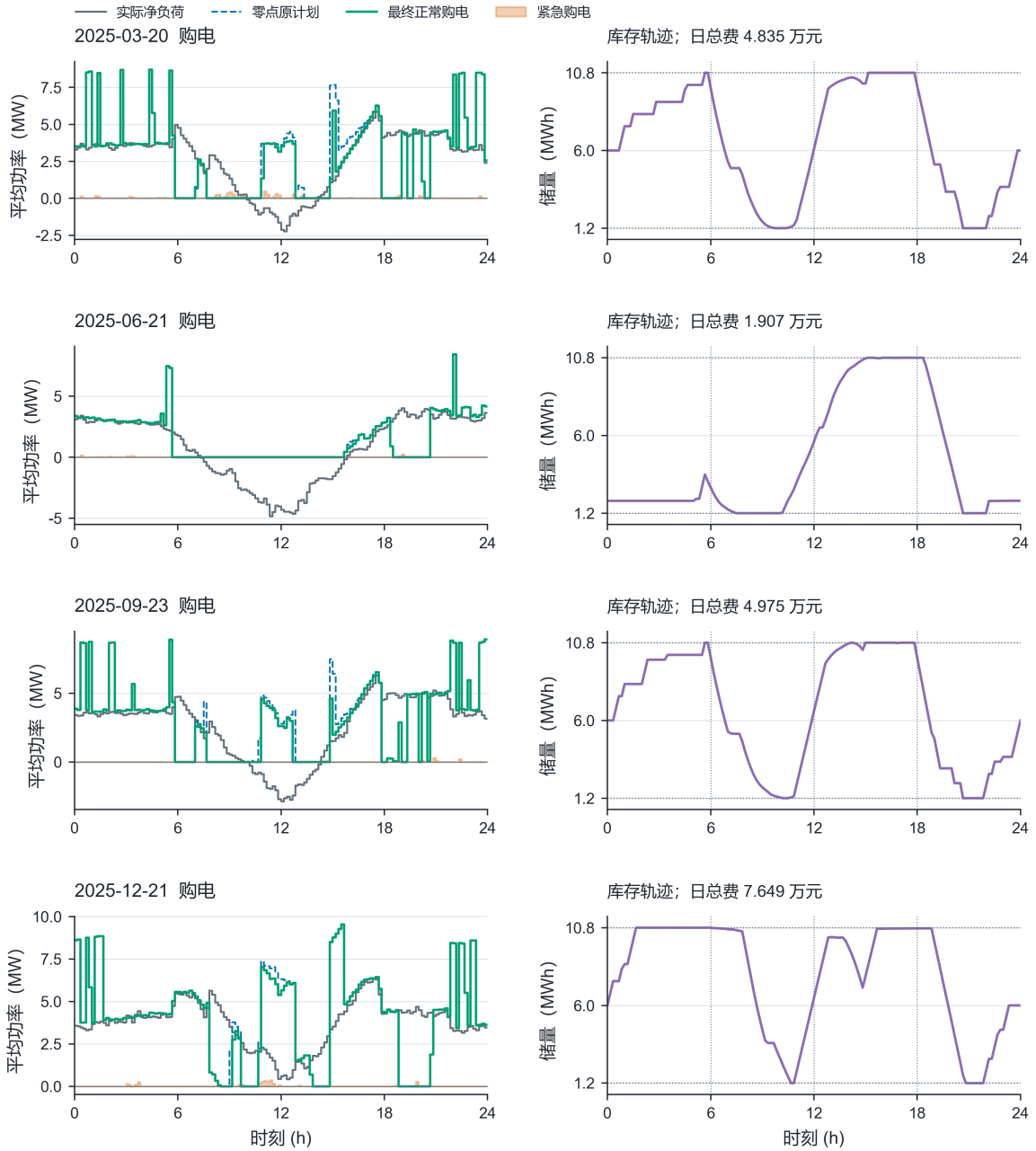


图 20 问题四允许日内调整主方案在四个指定日期的购电与储能轨迹

9.5 参数冻结、数值审计与可解释性

图21并列展示 1 月参数选择和正式期样本外结果，避免把事后最优误写成可执行选择。价格预测方面，仅日前模型的 MAE、RMSE、CRPS 和 80% 区间覆盖率分别为 0.085214、0.109005、0.056824 元/kWh 和 79.26%；日内修正后分别为 0.060565、0.080700、0.041726 元/kWh 和 78.57%。点指标改善并未自动转化为统计显著的费用改善，说明预

测检验和决策检验必须同时报告。

参数选择与样本外检验

上排和左下仅使用1月21—31日账单；右下展示2—12月结果，不改变2月前已冻结的选择。

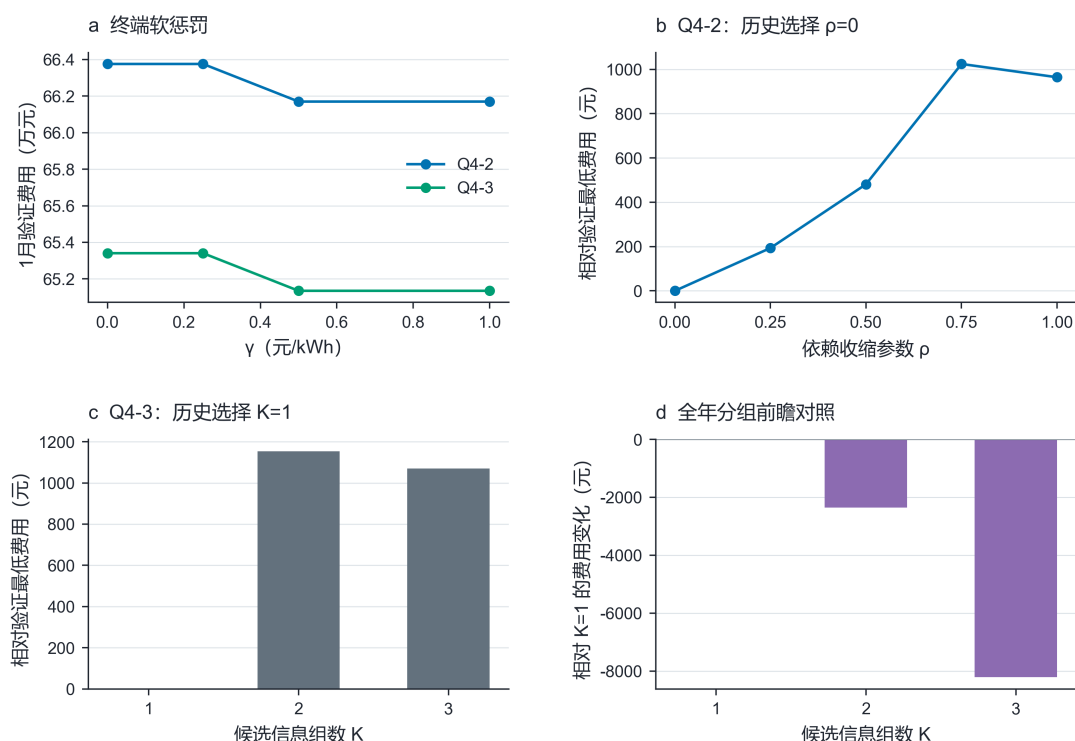


图 21 问题四参数选择、样本外差额及块 Bootstrap 区间

第四问 18 项机制测试全部通过。独立审计覆盖 29 组计算、3912 个策略日、10248 次求解，能量守恒、SOC、结算和信息时序的最大绝对误差为 1.863×10^{-9} ，阈值为 2×10^{-5} ；另抽取 32 个真实日进行 LP 与 MILP 比较，最大目标差 7.276×10^{-12} 。在正电价、允许无收益富余处置且紧急购电无上限的条件下，可将 LP 中的同时充放电净化而保持库存路径并不增费，故大规模主计算采用 LP 并在输出端净化，并非接受不物理的松弛解。结果工作簿只读复核 163722 个数值单元，最大读回差 8.731×10^{-11} 。

10 统一求解流程与策略落地

10.1 从预测到真实账单的双层流程

四问共享“决策模型”和“执行评价”两层。决策层只读当时可用的信息，输出正常购电和电池计划；评价层才读入当天真实负荷、光伏与价格，逐 10 min 回放并计算紧急购电。若将评价层的真实量送回同日更早的决策，就会产生不可执行的信息泄漏。本文以时间戳断言和扰动测试同时检查这一边界。

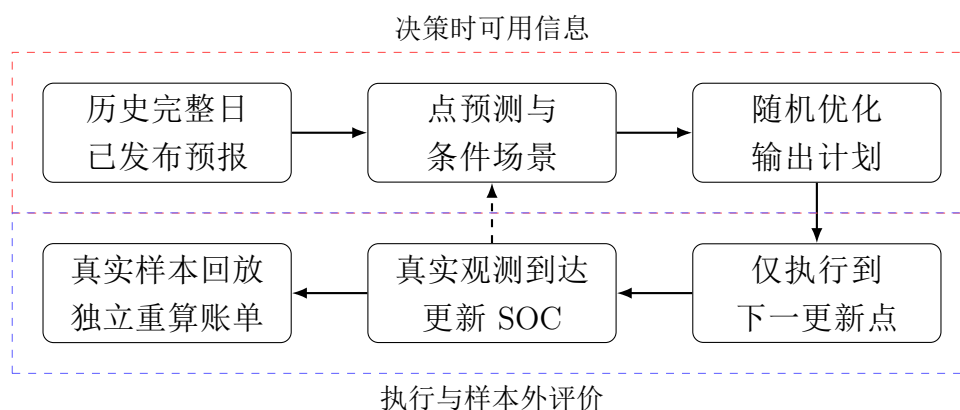


图 22 统一的因果决策、滚动执行和真实账单复核流程

对问题二，每个目标日执行以下步骤：首先用目标日前历史更新候选预测器；其次从最近 28 个样本外残差日抽取并压缩场景；再次求解共享日前变量的两阶段模型；最后以当天真实值计算补救量，保存日末库存并传给下一日。问题三与问题四的日内调整版本在一天内重复“更新信息集—重建剩余场景—求解—执行一个区间”，已经执行的决策冻结不变。该滚动实现比一次性建立全年多阶段场景树更易审计，也避免场景数随更新次数指数增长。

10.2 参数选择与防止事后挑选

所有影响主策略的离散选择先在 1 月验证期冻结，2 月至 12 月仅做一次连续样本外评价。表10区分物理参数、题面结算参数和数据驱动参数；前两类不参与调优，后一类才由历史验证选择。正式期中表现更好的备选仍保留在消融表中，但不会替换主方案。

表 10 关键参数来源、固定值与作用

类别	参数	主值	依据及作用
物理	$\Delta t, E_{\min}, E_{\max}$	1/6 h, 1200, 10800 kWh	题面给定，统一控制时间与库存边界
物理	$\eta_c, \eta_d, \bar{P}$	0.9, 0.9, 5000 kW	主假设与题面功率上限
结算	紧急购电倍率	5	题面给定，进入真实现金账单
结算	调整系数	上调 1.5、下调 0.5	以原计划为基准一次结算
问题二	预测器、场景数	随机森林、256 抽取	1 月滚动费用验证；重复场景压缩赋权
问题二	终端倍率 m	1.2	近似次日库存价值
问题三	历史窗、场景数	28 日、64 抽取	校准提前量误差并控制计算规模
问题三	风险参数	$\alpha = 0.9, \lambda = 0.1$	作为风险增强备选，不替代风险中性主方案
问题四	价格基线	28 日均值	1 月历史验证优于备选基线
问题四	修正与终端参数	$\alpha_p = 0.2$, 衰减 0.1, $\gamma = 0.5$	冻结日内修正速度与日末库存价值
问题四	ρ, K	0, 1	1 月验证选择； $K = 2, 3$ 仅作敏感性

这种“验证一次、正式期锁定”的设计牺牲了事后最优数字，却使方案在真实运行时可复现。尤其是第四问 $K = 3$ 的正式期费用较低，但若根据这一结果改选，就等价于让未来 334 天参与模型选择；本文明确拒绝这一做法。

10.3 算法规模、输出和异常处理

求解器输入全部使用 kWh 和元，外部展示需要功率时再除以 Δt 。每次求解后先检查状态方程、母线平衡、变量上下界和互斥或净化条件，再把执行区间写入轨迹。日末执行以下守卫：库存不得因四舍五入漂移，下一日直接读取原浮点值；最后一天施加 6000 kWh 终端条件；缺少历史滞后或场景无法分组时回退到预先规定的简单模型，而非读取未来补齐。

表 11 正式计算规模与独立复核证据

任务	正式日数	求解或记录规模	主要复核结论
问题一	1	LP 与 MILP 各 1 次	目标一致, MILP 严格互斥
问题二	334	25 组实验、4837 条日模型/回放记录	28 项测试通过, 最大残差 3.73×10^{-9}
问题三	334	21 组实验、28056 次滚动求解	20 项测试通过, 最大残差 7.55×10^{-7}
问题四	334	29 组、3912 策略日、10248 次求解	18 项测试通过, 最大误差 1.863×10^{-9}

最终输出分为三层：论文只保留关键模型、主结果和足以解释结论的图表；结果工作簿保存题目要求的指定时段、日期汇总与完整轨迹；附录和支撑材料保存全部源程序、配置、测试与审计记录。图表由正式 CSV 或工作簿自动生成，未手工改写数据点。这样既保持正文可读，也能从任一汇总数追溯到 10 min 执行记录。

10.4 跨问题结果的可比口径

表12汇总各问主结果，但不同问题的价格、信息和结算权限不同，绝不能横向把总费差额直接解释为某个算法的收益。有效比较只发生在同一问题内部、使用相同正式日期与评价账单的对照之间。

表 12 四问主方案结果及正确比较口径

问题	主方案	总费/万元	紧急量/MWh	可支持的结论
一	确定性 MILP	3.51	0	相对无储能同日基线下降 26.898%
二	随机森林 + 联合场景	1447.93	262.35	相对同预测器点方案下降 7.690%
三	条件尺度随机 MPC	1662.77	238.47	四次新预报相对仅 0:00 新预报节省 41.21 万元
四（日 前）	边际独立收缩	1520.11	256.56	相对波动价点方案下降 7.576%
四（日 内）	一组条件前瞻	1499.72	176.24	相对同机制点方案下降 6.605%

问题二与问题三使用题面固定电价，问题四使用真实波动价；问题三和问题四日内

版本允许调整正常购电，而问题二不允许。因此，跨行比较只能用于描述任务递进，不能据此判断哪个模型“绝对更优”。这一限制与样本外显著性区间一起构成本文结论边界。

11 模型检验与评价

11.1 模型检验

四问结果均通过统一的一致性检查：每个自然日含 144 个 10 min 时段和 145 个 SOC 状态；功率进入能量平衡前乘 $1/6$ ；充电增加库存时乘 η_c ，放电减少库存时除以 η_d ；SOC 与充放电功率不越界；跨日 SOC 按原浮点值连续传递；正常费用、调整费用与 5 倍紧急费用均由执行轨迹独立重算一致；扰动未来实际数据或尚未发布预报不改变当前决策。问题三另查提前量、发布时间、CVaR 离散分位质量和可信度反演；问题四另查对数价格残差、依赖收缩、条件分组、桥接库存、LP 净化和真实价格账单。

11.2 模型优点

本文用同一组物理约束贯穿四问，变量含义和量纲一致；按发布时间构建信息集，区分可执行策略与完美信息下界；联合残差日块保留可解释的时序轨迹和跨变量依赖；主体为线性或混合整数线性结构，便于解释、复现和审计。第二问通过事前验证选择预测器，第三问把提前量校准、概率场景、预报更新和尾部风险拆成可单独检验的模块，第四问则把价量依赖估计与下一节点前瞻设置端点和退化对照，使增量价值来源可追溯。

11.3 局限与改进

问题二固定 0:00 的名义电池轨迹，未利用日内已实现误差反馈；较强线性终端价值使 333 个正式日日末 SOC 贴近上限；随机森林也没有天气和节假日等外生变量。问题三 90% 区间覆盖率仅 80.67%，且条件分箱尺度的平均费未优于统一尺度，说明有限日块仍需更长样本检验。问题四中依赖收缩和价格修正的费用差额区间均跨 0， $K = 3$ 虽在正式期较优却未通过事前验证，说明一年样本不足以支持精细价量结构的稳定选择。后续可采用 48 h 滚动时域、非线性库存价值、外生天气与日历变量，并用跨年度或保持季节性的块自助法检验。

11.4 主要结论

问题一验证储能在已知日型下的跨时段套利能力；问题二表明，因果预测、联合残差场景和储能计划能显著减少高倍紧急购电，但收益不可拆分归因给单一模块；问题三表明 6:00 与 12:00 的新预报具有主要增量价值，CVaR 可以用较高平均费用换取较低尾部紧急成本。问题四表明场景方案相对点预测显著减少高价缺电，但价量依赖收缩和

日内价格修正的细小增益尚无稳定统计证据；允许日内更新的冻结主方案比仅日前主方案少 1.341%，该差额应解释为预报信息和调整权限的合成价值。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、论文结构整理、LaTeX 排版辅助和代码审阅等，详细使用情况见支撑材料。

参考文献

- [1] Lemos-Vinasco J, Schledorn A, Pourmousavi S A, et al. Economic evaluation of stochastic home energy management systems in a realistic rolling horizon setting[EB/OL]. arXiv:2203.08639, 2022.
- [2] Elkazaz M, Sumner M, Thomas D. Real-time energy management for a small scale PV-battery microgrid: Modeling, design, and experimental verification[J]. Energies, 2019, 12(14): 2712.
- [3] Schulz B, El Ayari M, Lerch S, et al. Post-processing numerical weather prediction ensembles for probabilistic solar irradiance forecasting[J]. Solar Energy, 2021, 220: 1016–1031.
- [4] Gneiting T, Lerch S, Schulz B. Probabilistic solar forecasting: Benchmarks, post-processing, verification[J]. Solar Energy, 2023, 252: 72–80.
- [5] Rockafellar R T, Uryasev S. Optimization of conditional value-at-risk[J]. Journal of Risk, 2000, 2(3): 21–41.
- [6] Elmachtoub A N, Grigas P. Smart “Predict, then Optimize”[J]. Management Science, 2022, 68(1): 9–26.
- [7] Alghumayjan S, Han J, Zheng N, et al. Energy storage arbitrage in two-settlement markets: A transformer-based approach[EB/OL]. arXiv:2404.17683, 2024.
- [8] Li N, Uckun C, Constantinescu E M, et al. Flexible operation of batteries in power system scheduling with renewable energy[J]. IEEE Transactions on Sustainable Energy, 2016, 7(2): 685–696.

A 支撑材料清单与复现说明

电子支撑材料按问题分为四组：问题一、二的预测与优化程序，问题三的滚动预测与 CVaR 程序，问题四的价格预测、场景调度、独立审计与工作簿校验程序，以及单独的“AI 工具使用详情.pdf”。原始赛题附件不重复分发。正式结果还包括完整 10 min 轨迹、每日与月度汇总、指定日期表、参数选择记录、测试日志和结果工作簿；配置在正式期开始前冻结。

运行环境为 Python 3，主要依赖 NumPy、pandas、SciPy、scikit-learn、statsmodels、openpyxl 和 Matplotlib。各组入口分别为 `run_q1.py`、`run_v2.py`、`run_q3.py` 和 `run_all_q4.py`；审计程序读取固定配置与正式结果，独立重算约束和账单。以下附录直接编入实际建模所用完整源文件，便于在论文之外的支撑材料丢失时仍可检查算法。

B 问题一源程序

`microgrid_core.py`

```
1 """ 微网通用计算模块。所有调度流量为 kWh，功率上限为交流母线侧 kW。
2
3 函数不解释 Excel 时间标签、不推断初末条件、不选择预测模型。
4 第一问首尾相等由 solve_q1 显式实现；固定计划回放不重置跨日状态。
5 """
6 from dataclasses import asdict, dataclass
7 from pathlib import Path
8 import csv
9 from datetime import date
10 import math
11 import time
12
13 import numpy as np
14 from scipy.optimize import Bounds, LinearConstraint, linprog, milp
15 from scipy.sparse import lil_matrix
16
17
18 @dataclass(frozen=True)
19 class Battery:
20     eta_charge: float
21     eta_discharge: float
22     min_kwh: float = 1200.0
23     max_kwh: float = 10800.0
24     max_power_kw: float = 5000.0
25     step_hours: float = 1/6
26
27     def __post_init__(self):
28         if not all(math.isfinite(v) for v in asdict(self).values()):
29             raise ValueError("Battery parameters must be finite")
30         if not 0 < self.eta_charge <= 1 or not 0 < self.eta_discharge <= 1:
31             raise ValueError("Charge and discharge efficiencies must be in (0, 1]")
32         if not 0 <= self.min_kwh < self.max_kwh:
33             raise ValueError("Invalid battery capacity bounds")
34         if self.max_power_kw <= 0 or self.step_hours <= 0:
35             raise ValueError("Power and time step must be positive")
36
37     @property
38     def limit_kwh(self):
39         return self.max_power_kw * self.step_hours
40
41     def validate_initial(self, value):
42         # 跨日独立重算可产生 1e-10 级边界误差，允许数值容差，不重置或截断储量。
```

```

43         if not math.isfinite(value) or not self.min_kwh-1e-7 <= value <= self.max_kwh+1e-7:
44             raise ValueError("Initial stored energy is outside the permitted range")
45
46
47 def vector(values, name, length=None, nonnegative=True):
48     a = np.asarray(values, dtype=float)
49     if a.ndim != 1 or len(a) == 0 or (length is not None and len(a) != length):
50         raise ValueError(f"{name}: expected a nonempty vector of length {length}")
51     if not np.isfinite(a).all() or (nonnegative and (a < 0).any()):
52         raise ValueError(f"{name}: invalid numeric values")
53     return a
54
55
56 def dispatch_checks(charge, discharge, state, battery, initial):
57     """ 独立按物理公式计算残差, 输出单位为 kWh。 """
58     c = vector(charge, "charge", nonnegative=False)
59     d = vector(discharge, "discharge", len(c), nonnegative=False)
60     s = vector(state, "state", len(c)+1)
61     return {
62         "nonnegative_dispatch_violation_kwh": float(max(0, -c.min(), -d.min())),
63         "state_transition_residual_kwh":
64             float(np.abs(np.diff(s)-battery.eta_charge*c+d/battery.eta_discharge).max()),
65         "state_bound_violation_kwh": float(max(0, battery.min_kwh-s.min(), s.max()-battery.max_kwh)),
66         "power_bound_violation_kwh": float(max(0, c.max()-battery.limit_kwh, d.max()-battery.limit_kwh)),
67         "simultaneous_charge_discharge_kwh": float(np.minimum(np.maximum(c,0), np.maximum(d,0)).sum()),
68         "initial_state_difference_kwh": float(abs(s[0]-initial)),
69     }
70
71 def require_checks(checks, tolerance=1e-5):
72     failed = {k:v for k,v in checks.items() if not math.isfinite(v) or v > tolerance}
73     if failed:
74         raise RuntimeError(f"Independent verification failed: {failed}")
75
76
77 def solve_q1(price, load_kwh, pv_kwh, *, battery, initial_kwh,
78             method="milp", time_limit_s=60., mip_rel_gap=1e-9):
79     """ 求解第一问, 首尾储量均为 initial_kwh; LP 同时充放电另行标记。
80
81 失败或超时不会作为已验证最优结果返回。输入的时间/日期含义由调用方提供。
82 """
83     p = vector(price, "price")
84     n = len(p)
85     load, pv = vector(load_kwh, "load", n), vector(pv_kwh, "pv", n)
86     if (p <= 0).any():
87         raise ValueError("This implementation requires strictly positive prices")
88     if method not in ("lp", "milp"):
89         raise ValueError("method must be lp or milp")
90     if not math.isfinite(time_limit_s) or time_limit_s <= 0 or not 0 <= mip_rel_gap < 1:
91         raise ValueError("Invalid solver limits")
92     battery.validate_initial(initial_kwh)
93     integer = method == "milp"
94     size = (6 if integer else 5)*n+1
95     q,c,d,w = [np.arange(k*n,(k+1)*n) for k in range(4)]
96     s,z = np.arange(4*n,5*n+1), np.arange(5*n+1,6*n+1)
97     objective = np.zeros(size)
98     objective[q] = p
99     low, high = np.zeros(size), np.full(size,np.inf)
100     high[c] = high[d] = battery.limit_kwh
101     high[w] = pv
102     low[s], high[s] = battery.min_kwh,battery.max_kwh
103     low[s[0]] = high[s[0]] = initial_kwh
104     low[s[-1]] = high[s[-1]] = initial_kwh
105     eq = lil_matrix((2*n,size))
106     rhs = np.r_[load-pv,np.zeros(n)]
107     for t in range(n):
108         eq[t,q[t]],eq[t,c[t]],eq[t,d[t]],eq[t,w[t]] = 1,-1,1,-1
109         eq[n+t,s[t+1]],eq[n+t,s[t]] = 1,-1
110         eq[n+t,c[t]],eq[n+t,d[t]] = -battery.eta_charge,1/battery.eta_discharge
111     started = time.perf_counter()
112     if integer:
113         high[z] = 1
114         integrality = np.zeros(size)

```

```

115     integrality[z] = 1
116     ub = lil_matrix((2*n,size))
117     for t in range(n):
118         ub[t,c[t]],ub[t,z[t]]=1,-battery.limit_kwh
119         ub[n+t,d[t]],ub[n+t,z[t]]=1,battery.limit_kwh
120     result = milp(objective,integrality=integrality,bounds=Bounds(low,high),
121                  constraints=[LinearConstraint(eq.tocsr(),rhs,rhs),
122                               LinearConstraint(ub.tocsr(),-np.inf,np.r_[np.zeros(n),np.full(n,battery.limit_kwh)
123                                     ↪ h)]),
124                  options={"time_limit":time_limit_s,"mip_rel_gap":mip_rel_gap})
125 else:
126     result = linprog(objective,A_eq=eq.tocsr(),b_eq=rhs,bounds=list(zip(low,high)),
127                     method="highs",options={"time_limit":time_limit_s})
128 runtime = time.perf_counter()-started
129 if not result.success:
130     raise RuntimeError(f"{method} not solved to requested tolerance; status={result.status}: {result.message}")
131 Q,C,D,W,S = (result.x[i] for i in (q,c,d,w,s))
132 # 浮点求解器可能返回极小负数, 保留原值并按容差检查, 不修改调度来通过检查。
133 negative = float(max(0,-result.x.min()))
134 if negative > 1e-5:
135     raise RuntimeError("Solver returned negative energy outside tolerance")
136 checks = {
137     "bus_balance_residual_kwh":float(np.abs(Q+pv+D-load-C-W).max()),
138     "state_transition_residual_kwh":float(np.abs(np.diff(S)-battery.eta_charge*C+D/battery.eta_discharge).max())
139     ↪ ),
140     "state_bound_violation_kwh":float(max(0,battery.min_kwh-S.min(),S.max()-battery.max_kwh)),
141     "power_bound_violation_kwh":float(max(0,C.max()-battery.limit_kwh,D.max()-battery.limit_kwh)),
142     "pv_spill_bound_violation_kwh":float(max(0,(W-pv).max(),-W.min())),
143     "nonnegative_violation_kwh":negative,
144     "initial_state_difference_kwh":float(abs(S[0]-initial_kwh)),
145     "terminal_state_difference_kwh":float(abs(S[-1]-initial_kwh)),
146     "objective_difference_yuan":float(abs(p@Q-result.fun)),
147 }
148 require_checks(checks)
149 simultaneous = float(np.minimum(np.maximum(C,0),np.maximum(D,0)).sum())
150 if integer:
151     require_checks({"simultaneous_charge_discharge_kwh":simultaneous})
152 return {
153     "method":method,"status":int(result.status),"message":str(result.message),
154     "runtime_s":runtime,"mip_gap":float(result.mip_gap) if integer else None,
155     "cost_yuan":float(p@Q),"purchase_kwh":float(Q.sum()),
156     "simultaneous_charge_discharge_kwh":simultaneous,
157     "physically_feasible":simultaneous <= 1e-5,"checks":checks,
158     "plan":{name:values.tolist() for name,values in zip(("Q","C","D","W","S"),(Q,C,D,W,S))},
159 }
160
161 def simulate_fixed_plan(price, load_kwh, pv_kwh, purchase_kwh, charge_kwh, discharge_kwh,
162                        *, battery, initial_kwh, emergency_multiplier):
163     """ 第二问固定电池计划的逐时回放; 实际曲线仅用于结算, 不改变日前计划。
164     164 总富余可能来自光伏、计划购电或电池, 不能全部称作已付费浪费。
165     165 调用方必须把返回的终值作为下一日初值; 本函数没有每日重置逻辑。
166     """
167     p = vector(price,"price")
168     n = len(p)
169     load,pv = vector(load_kwh,"load",n),vector(pv_kwh,"pv",n)
170     q,c,d = [vector(v,name,n,nonnegative=False) for v,name in zip(
171         (purchase_kwh,charge_kwh,discharge_kwh),("Q","C","D"))]
172     require_checks({"negative_plan_violation_kwh":float(max(0,-q.min(),-c.min(),-d.min()))})
173     battery.validate_initial(initial_kwh)
174     if (p <= 0).any() or not math.isfinite(emergency_multiplier) or emergency_multiplier <= 0:
175         raise ValueError("Prices and emergency multiplier must be positive")
176     s = np.r_[initial_kwh,initial_kwh+np.cumsum(battery.eta_charge*c-d/battery.eta_discharge)]
177     deficit = load+c-q-pv-d
178     e,u = np.maximum(deficit,0),np.maximum(-deficit,0)
179     checks = dispatch_checks(c,d,s,battery,initial_kwh)
180     checks["bus_balance_residual_kwh"] = float(np.abs(q+e+pv+d-load-c-u).max())
181     require_checks(checks)
182     planned,emergency = float(p@q),float(emergency_multiplier*p@e)
183     return {"policy":"fixed_day_ahead_battery","planned_cost_yuan":planned,
184           "emergency_cost_yuan":emergency,"cash_cost_yuan":planned+emergency,
185           "terminal_kwh":float(s[-1]),"checks":checks,

```

```

186         "S":s.tolist(),"E":e.tolist(),"U_total_unused":u.tolist())
187
188
189 def load_baseline_csv(path, *, step_hours):
190     """ 读取已核查派生 CSV, 严格保留原始记录次序, 不推断区间起点。"""
191     with Path(path).open(encoding="utf-8-sig",newline="") as stream:
192         rows = list(csv.DictReader(stream))
193         if len(rows) != 144 or [int(r["point_index"]) for r in rows] != list(range(1,145)):
194             raise ValueError("Baseline must contain ordered point_index 1..144")
195         if not math.isfinite(step_hours) or step_hours <= 0:
196             raise ValueError("Invalid conversion time step")
197         p = vector([r["price_yuan_per_kwh"] for r in rows],"price",144)
198         load = vector([r["load_kw"] for r in rows],"load_kw",144)*step_hours
199         pv = vector([r["pv_forecast_kw"] for r in rows],"pv_kw",144)*step_hours
200         return {"point_index":[int(r["point_index"]) for r in rows],
201                 "source_time_label":[r["time_label"] for r in rows],
202                 "price":p,"load_kwh":load,"pv_kwh":pv}
203
204
205 def history_before_date(path, decision_date):
206     """ 按源记录所属日期严格截取日前历史, 尚不解释 10 分钟标签。
207
208     返回值可供后续预测模块使用; 不进行补值、标准化或全年参数估计。
209     """
210     cutoff = date.fromisoformat(decision_date)
211     rows = []
212     keys = set()
213     with Path(path).open(encoding="utf-8-sig",newline="") as stream:
214         for row in csv.DictReader(stream):
215             day = date.fromisoformat(row["date"])
216             if day >= cutoff:
217                 continue
218             index = int(row["point_index"])
219             key = (day,index)
220             if not 1 <= index <= 144 or key in keys:
221                 raise ValueError("Invalid or duplicate historical date/point key")
222             keys.add(key)
223             values = vector([row["load_kw"],row["pv_actual_kw"]],"historical power",2)
224             rows.append(**row,"point_index":index,"load_kw":float(values[0]),"pv_actual_kw":float(values[1]))
225     rows.sort(key=lambda r:(r["date"],r["point_index"]))
226     return rows

```

prepare_data.py

```

1 """ 只整理和检查 C 题输入; 不假定区间起止, 不预测、不优化、不填写模板。"""
2 from __future__ import annotations
3
4 import argparse
5 import csv
6 import hashlib
7 import json
8 import math
9 import shutil
10 from datetime import date, datetime, time, timedelta
11 from pathlib import Path
12
13 import openpyxl
14
15 ROOT = Path(__file__).resolve().parents[1]
16 EXPECTED_LABELS = [f"{i // 6:02d}:{i % 6 * 10:02d}" for i in range(1, 144)] + ["24:00"]
17 EXPECTED_DATES = [(date(2025, 1, 1) + timedelta(days=i)).isoformat() for i in range(365)]
18 INPUT_FILES = ["C 题.pdf", *[f"附件/附件 {i}.xlsx" for i in range(1, 5)],
19               *[f"附件/附件 5/{n}.xlsx" for n in ("result1", "result2", "result3", "result4-2", "result4-3")]]
20
21
22 def sha256(path):
23     return hashlib.sha256(path.read_bytes()).hexdigest()
24
25
26 def snapshot(source):
27     """ 保存准确副本; 已有副本如与来源不同, 停止并提示人工处理。"""

```

```

28     source = source.resolve()
29     for relative in INPUT_FILES:
30         src, dst = source / relative, ROOT / "data/raw" / relative
31         if not src.is_file():
32             raise FileNotFoundError(f" 缺少输入: {src}")
33         if dst.exists() and sha256(src) != sha256(dst):
34             raise ValueError(f" 来源与已有副本不同, 请先核实版本: {relative}")
35     for relative in INPUT_FILES:
36         src, dst = source / relative, ROOT / "data/raw" / relative
37         dst.parent.mkdir(parents=True, exist_ok=True)
38         if not dst.exists():
39             shutil.copy2(src, dst)
40     manifest = {"source_directory": str(source), "files": [
41         {"file": p, "sha256": sha256(ROOT / "data/raw" / p)} for p in INPUT_FILES]}
42     (ROOT / "data/raw/manifest.json").write_text(json.dumps(manifest, ensure_ascii=False, indent=2),
43     ↵ encoding="utf-8")
44
45 def verify_snapshot():
46     manifest_path = ROOT / "data/raw/manifest.json"
47     if not manifest_path.exists():
48         raise FileNotFoundError(" 首次运行请传入 --source, 保存输入副本及来源记录。")
49     manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
50     for item in manifest["files"]:
51         if sha256(ROOT / "data/raw" / item["file"]) != item["sha256"]:
52             raise ValueError(f" 原始副本已改变: {item['file']}; 请先核实。")
53
54
55 def rows(filename, sheet):
56     wb = openpyxl.load_workbook(ROOT / "data/raw/附件" / filename, read_only=True, data_only=True)
57     try:
58         return list(wb[sheet].values)
59     finally:
60         wb.close()
61
62
63 def clock_label(value):
64     if isinstance(value, (datetime, time)):
65         return value.strftime("%H:%M")
66     value = str(value).strip()
67     if value in ("0:00+1", "00:00+1", "24:00"):
68         return "24:00"
69     h, m = value.split(":")
70     return f"{int(h):02d}:{int(m):02d}"
71
72
73 def date_label(value):
74     if isinstance(value, (datetime, date)):
75         return value.strftime("%Y-%m-%d")
76     return datetime.strptime(str(value).strip(), "%Y-%m-%d").date().isoformat()
77
78
79 def number(value, location):
80     if isinstance(value, bool) or not isinstance(value, (int, float)) or not math.isfinite(value):
81         raise ValueError(f" 数值缺失或不是有限数: {location}: {value!r}")
82     return value
83
84
85 def require(condition, message):
86     if not condition:
87         raise ValueError(message)
88
89
90 def write_csv(name, headers, data):
91     with (ROOT / "data/processed" / name).open("w", encoding="utf-8-sig", newline="") as f:
92         writer = csv.writer(f)
93         writer.writerow(headers)
94         writer.writerows(data)
95
96
97 def read_daily(filename, sheet):
98     table = rows(filename, sheet)
99     require(len(table) == 366, f"{filename}/{sheet} 应有 365 个日期")

```

```

100 labels = [clock_label(v) for v in table[0][1:]]
101 require(labels == EXPECTED_LABELS, f"{filename}/{sheet} 时间标签不符合预期")
102 dates = [date_label(r[0]) for r in table[1:]]
103 require(dates == EXPECTED_DATES, f"{filename}/{sheet} 日期不连续或存在重复")
104 data = [[number(v, f"{filename}/{sheet} 行 {ridx} 列 {cidx}")
105          for cidx, v in enumerate(r[1:], 2)] for ridx, r in enumerate(table[1:], 2)]
106 require(all(len(r) == 144 for r in data), f"{filename}/{sheet} 每天应有 144 个数值")
107 return data
108
109
110 def main():
111     parser = argparse.ArgumentParser(description=__doc__)
112     parser.add_argument("--source", type=Path, help="桌面 C 题 文件夹; 首次运行需要")
113     args = parser.parse_args()
114     for folder in ("data/raw", "data/processed", "reports", "outputs/figures", "outputs/results"):
115         (ROOT / folder).mkdir(parents=True, exist_ok=True)
116     if args.source:
117         snapshot(args.source)
118     verify_snapshot()
119
120     baseline = rows("附件 1.xlsx", "Sheet1")
121     require(len(baseline) == 145, "附件 1 应有 144 条数据")
122     require([clock_label(r[0]) for r in baseline[1:]] == EXPECTED_LABELS, "附件 1 时间标签异常")
123     base = [[i, clock_label(r[0]), *number(v, f"附件 1 行 {i+1}") for v in r[1:]]
124             for i, r in enumerate(baseline[1:], 1)]
125     require(all(len(r) == 5 for r in base), "附件 1 字段数异常")
126
127     load = read_daily("附件 2.xlsx", "小区负载")
128     pv = read_daily("附件 2.xlsx", "光伏发电实际功率")
129     prices = read_daily("附件 4.xlsx", "Sheet1")
130     actual = [[d, i + 1, t, load[j][i], pv[j][i]]
131              for j, d in enumerate(EXPECTED_DATES) for i, t in enumerate(EXPECTED_LABELS)]
132     price_points = [[d, i + 1, t, prices[j][i]]
133                    for j, d in enumerate(EXPECTED_DATES) for i, t in enumerate(EXPECTED_LABELS)]
134
135     source_forecast = rows("附件 3.xlsx", "Sheet1")
136     require(len(source_forecast) == 1461, "附件 3 应有 1460 批预报")
137     require(list(source_forecast[0][2:]) == [f"预报 {i} 小时" for i in range(1, 25)], "附件 3 预报列异常")
138     forecasts, seen, current_date, filled = [], set(), None, 0
139     for ridx, row in enumerate(source_forecast[1:], 2):
140         if row[0] not in (None, ""):
141             current_date = date_label(row[0])
142         else:
143             filled += 1
144             require(current_date is not None, f"附件 3 第 {ridx} 行日期无法确定")
145             issued_clock = clock_label(row[1])
146             require(issued_clock in ("00:00", "06:00", "12:00", "18:00"), "预报发布时间异常")
147             key = (current_date, issued_clock)
148             require(key not in seen, f"重复预报批次: {key}")
149             seen.add(key)
150             issued = datetime.fromisoformat(f"{current_date}T{issued_clock}:00")
151             require(len(row[2:]) == 24, f"附件 3 第 {ridx} 行不是 24 个预报值")
152             for lead, value in enumerate(row[2:], 1):
153                 forecasts.append([issued.isoformat(), lead, (issued + timedelta(hours=lead)).isoformat(),
154                                number(value, f"附件 3 行 {ridx} 预报 {lead} 小时")])
155     require(seen == {(d, t) for d in EXPECTED_DATES for t in ("00:00", "06:00", "12:00", "18:00")},
156            "预报日期或发布批次不完整")
157
158     # 所有输入检查成功后再输出; 不做时段偏移、插值或未来数据切分。
159     write_csv("baseline_points.csv", ["point_index", "time_label", "price_yuan_per_kwh", "load_kw",
160                                     ↵ "pv_forecast_kw"], base)
161     write_csv("actual_points.csv", ["date", "point_index", "time_label", "load_kw", "pv_actual_kw"], actual)
162     write_csv("price_points.csv", ["date", "point_index", "time_label", "price_yuan_per_kwh"], price_points)
163     write_csv("pv_forecasts.csv", ["issued_at", "lead_hours", "valid_at", "pv_forecast_kw"], forecasts)
164
165     template_notes = []
166     for name in ("result1", "result2", "result3", "result4-2", "result4-3"):
167         wb = openpyxl.load_workbook(ROOT / f"data/raw/附件/附件 5/{name}.xlsx", read_only=True, data_only=True)
168         try:
169             sheet = wb["计划购电量"]
170             first, last = (sheet["A2"].value, sheet["A145"].value) if name == "result1" else (sheet["B1"].value,
171             ↵ sheet["E01"].value)
172             template_notes.append({"file": name + ".xlsx", "first_interval": first, "last_interval": last})

```

```

171         finally:
172             wb.close()
173
174     summary = {
175         "baseline_points": len(base), "actual_points": len(actual), "price_points": len(price_points),
176         "forecast_batches": len(seen), "forecast_points": len(forecasts),
177         "forecast_date_labels_filled": filled,
178         "date_start": EXPECTED_DATES[0], "date_end": EXPECTED_DATES[-1],
179         "checks": ["原始副本SHA-256一致", "全年日期连续且无重复", "每日时间标签一致", "数值均有限且无缺失",
180             ↪ "预报批次完整且无重复"],
181         "template_labels": template_notes,
182         "ranges": {
183             "load_kw": [min(r[3] for r in actual), max(r[3] for r in actual)],
184             "pv_actual_kw": [min(r[4] for r in actual), max(r[4] for r in actual)],
185             "price_yuan_per_kwh": [min(r[3] for r in price_points), max(r[3] for r in price_points)],
186             "unresolved": ["输入时间点如何对应10分钟区间", "模板首尾时间标签", "充放电效率与接口口径",
187                 ↪ "调减购电款是否退还及多次调整结算", "未来电价何时可知", "跨日及期末储电要求"]}
188     (ROOT / "reports/data-audit.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2),
189         ↪ encoding="utf-8")
190
191     lines = ["# C 题数据检查结果", "", " 只读检查并保留原件副本; 尚未预测、优化或填写结果模板。", "",
192         "| 检查对象 | 结果 |", "|---|---|",
193         f"| 附件 1 | {len(base)} 个时间点, 3 列数值完整 |",
194         f"| 附件 2 | 365 天 × 144 个时间点 = {len(actual)} 行, 负载和实际光伏完整 |",
195         f"| 附件 3 | {len(seen)} 批 × 24 小时 = {len(forecasts)} 个预报值 |",
196         f"| 附件 4 | {len(price_points)} 个电价数值完整 |",
197         "| 日期、时间和重复记录 | 检查通过 |", "| 原始副本 | 与来源的 SHA-256 一致 |", "",
198         " 上述检查不代表数据已通过物理合理性审查, 也不代表建模口径已确定。", "",
199         "## 数据处理说明", "",
200         f"- 附件 3 的 {filled} 个空白日期标签按同一天的第一条记录补齐, 不是填补预测数值。",
201         "- 00:10 到 24:00 暂时保留为原始采样标签。没有自动决定它们代表前 10 分钟还是后 10 分钟。",
202         "- 预报保留发布时间 issued_at、提前量 lead_hours 和目标时刻 valid_at。决策时只能使用 issued_at
203         ↪ 不晚于当前时刻的预报。",
204         "- 12 月 31 日发布的跨年预报保留; 没有为其虚构 2026 年实际值。",
205         "- 没有进行插值、功率转电量、训练集划分或模型求解。", "",
206         "## 结果模板疑点", "", "| 文件 | 首段标签 | 末段标签 |", "|---|---|---|"]
207     lines += [f"| {r['file']} | {r['first_interval']} | {r['last_interval']} |" for r in template_notes]
208     lines += ["", " 这些标签与题面当天 00:00 到 24:00 的要求存在疑似错位。已保留原样, 需要核实后再生成最终填表程序。", "",
209         "## 与建模手确认", "", ] + [f"- {x}。" for x in summary["unresolved"]]
210     (ROOT / "reports/数据检查.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
211     print(json.dumps({k: summary[k] for k in ("baseline_points", "actual_points", "forecast_batches",
212         ↪ "forecast_points", "checks")}, ensure_ascii=False, indent=2))
213     print(" 输出位置: ", ROOT)
214
215
216
217 if __name__ == "__main__":
218     main()

```

run_q1.py

```

1 """ 可复现第一问诊断入口, 输出原始点序调度, 不导出正式比赛模板。"""
2 from dataclasses import asdict
3 from datetime import datetime, timezone
4 from pathlib import Path
5 import argparse
6 import csv
7 import hashlib
8 import json
9 import platform
10 import traceback
11 import uuid
12
13 import scipy
14
15 from microgrid_core import Battery, load_baseline_csv, solve_q1
16
17 ROOT = Path(__file__).resolve().parents[1]
18
19
20 def write_json(path, value):
21     path.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8")
22

```

```

23
24 def main():
25     parser = argparse.ArgumentParser(description=__doc__)
26     parser.add_argument("--config", type=Path, default=R00T/"configs/q1_diagnostic.json")
27     args = parser.parse_args()
28     config = json.loads(args.config.read_text(encoding="utf-8"))
29     if config["run_status"] != "diagnostic_pending_model_confirmation" or config["source_time_interpretation"] !=
↪ "record_order_only":
30         raise ValueError("This entry point currently supports diagnostic record-order runs only")
31     if config["power_integration"] != "piecewise_constant_over_10_minutes":
32         raise ValueError("Unsupported power integration rule")
33     methods = config["methods"]
34     if not methods or len(set(methods)) != len(methods) or any(m not in ("lp", "milp") for m in methods):
35         raise ValueError("Invalid methods configuration")
36     run_id = "q1_" + datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S") + "_" + uuid.uuid4().hex[:8]
37     out = R00T/"outputs/runs"/run_id
38     out.mkdir(parents=True, exist_ok=False)
39     source = R00T/"data/processed/baseline_points.csv"
40     try:
41         battery = Battery(eta_charge=config["eta_charge"], eta_discharge=config["eta_discharge"])
42         metadata = {"run_id": run_id, "config": config, "battery": asdict(battery),
43                     "python": platform.python_version(), "scipy": scipy.__version__,
44                     "input_path": str(source), "input_sha256": hashlib.sha256(source.read_bytes()).hexdigest(),
45                     "code_sha256": {p.name: hashlib.sha256(p.read_bytes()).hexdigest() for p in
↪ (Path(__file__), R00T/"src/microgrid_core.py")}}
46         write_json(out/"config.json", metadata)
47         data = load_baseline_csv(source, step_hours=battery.step_hours)
48         results = {}
49         for method in methods:
50             result = solve_q1(data["price"], data["load_kwh"], data["pv_kwh"], battery=battery,
51                               initial_kwh=config["initial_kwh"], method=method,
52                               time_limit_s=config["solver_time_limit_s"], mip_rel_gap=config["mip_rel_gap"])
53             results[method] = result
54             write_json(out/(method+".json"), result)
55             with (out/(method+"_plan_by_source_point.csv")).open("w", encoding="utf-8-sig", newline="") as stream:
56                 writer = csv.writer(stream)
57                 writer.writerow(["point_index", "source_time_label", "Q_kwh", "C_bus_kwh", "D_bus_kwh", "W_pv_kwh", "S_st"]
↪ art_kwh, "S_end_kwh", "cost_yuan"])
58                 a = result["plan"]
59                 for t in range(144):
60                     writer.writerow([t+1, data["source_time_label"][t], a["Q"][t], a["C"][t], a["D"][t], a["W"][t], a["S"]
↪ ] [t], a["S"][t+1], float(data["price"][t]*a["Q"][t])])
61             summary = {"status": "completed_diagnostic", "run_id": run_id,
62                       "scope": " 按源记录次序试算，时间区间未定稿：未填写 result1.xlsx。",
63                       "results": {m: {k: v for k, v in r.items() if k != "plan"} for m, r in results.items()}}
64             if "lp" in results and "milp" in results:
65                 difference = results["milp"]["cost_yuan"] - results["lp"]["cost_yuan"]
66                 if difference < -1e-5:
67                     raise RuntimeError("MILP cost is below the LP lower bound")
68                 summary["milp_minus_lp_yuan"] = difference
69             write_json(out/"summary.json", summary)
70             (out/"run.log").write_text("Completed diagnostic. See config.json, summary.json and
↪ plans.\n", encoding="utf-8")
71             print(json.dumps({"output_directory": str(out), "cost_yuan": {m: r["cost_yuan"] for m, r in
↪ results.items()}}, ensure_ascii=False, indent=2))
72     except Exception as exc:
73         write_json(out/"failure.json", {"status": "failed", "error": str(exc)})
74         (out/"run.log").write_text(traceback.format_exc(), encoding="utf-8")
75         raise
76
77
78 if __name__ == "__main__":
79     main()

```

reporting.py

```

1 """ 由完整 10 分钟轨迹生成论文表格、模板填充值与独立核对材料。"""
2 from pathlib import Path
3 import json
4
5 import numpy as np

```

```

6
7 from forecasting import interval_label, clock_text
8
9
10 def emergency_segments(energy, threshold=1e-6):
11     e=np.asarray(energy, dtype=float)
12     if not np.isfinite(threshold) or threshold<0:
13         raise ValueError("Emergency reporting threshold must be finite and nonnegative")
14     if e.shape!=(144,) or not np.isfinite(e).all() or (e<0).any():
15         raise ValueError("Expected 144 nonnegative emergency energy values")
16     mask=e>threshold
17     segments=[]
18     t=0
19     while t<144:
20         if not mask[t]:
21             t+=1; continue
22         start=t
23         while t<144 and mask[t]:
24             t+=1
25         segments.append({"start_index":start, "end_index_exclusive":t,
26                         "interval":clock_text(start*10)+"-"+clock_text(t*10),
27                         "energy_kwh":float(e[start:t].sum())})
28     return segments, float(e[~mask].sum())
29
30
31 def four_hour_rows(charge, discharge, state):
32     c,d,s=np.asarray(charge), np.asarray(discharge), np.asarray(state)
33     if c.shape!=(144,) or d.shape!=(144,) or s.shape!=(145,):
34         raise ValueError("Four-hour aggregation requires 144 intervals and 145 states")
35     return [[clock_text(j*240)+"-"+clock_text((j+1)*240), float(c[j*24:(j+1)*24].sum()),
36             float(d[j*24:(j+1)*24].sum()), "00:00" if j==0 else "24:00" if j==1 else None,
37             float(s[0]) if j==0 else float(s[-1]) if j==1 else None] for j in range(6)]
38
39
40 def build_payload(run_path, dataset, config):
41     run=Path(run_path)
42     q1=json.loads((run/"q1.json").read_text(encoding="utf-8"))["milp"]
43     data=np.load(run/"main_weighted28/trajectories.npz")
44     ids=data["day_indices"].tolist()
45     if ids!=list(range(31,365)):
46         raise ValueError("Formal template payload requires all 334 dates")
47     dates=[dataset.dates[day].isoformat() for day in ids]
48     plan_rows=[]; battery_rows=[]; emergency_rows=[]; daily=[]; segments_by_day={}
49     excluded=0.
50     for i, day in enumerate(ids):
51         q,c,d,s,e,u=[data[key][i] for key in ("Q", "C", "D", "S", "E", "U")]
52         planned_cost=float(dataset.prices[q])
53         emergency_cost=float(5*dataset.prices[e])
54         total_cost=planned_cost+emergency_cost
55         plan_rows.append([dates[i], *q.tolist(), float(q.sum()), total_cost])
56         for j, row in enumerate(four_hour_rows(c,d,s)):
57             battery_rows.append([dates[i] if j==0 else None, *row])
58             segments, omitted=emergency_segments(e, config["emergency_reporting_threshold_kwh"])
59             excluded+=omitted; segments_by_day[dates[i]]=segments
60             for j, segment in enumerate(segments):
61                 emergency_rows.append([dates[i] if j==0 else None, segment["interval"], segment["energy_kwh"]])
62             daily.append({"date":dates[i], "planned_purchase_kwh":float(q.sum()), "emergency_purchase_kwh":float(e.sum()),
63                         "planned_plus_emergency_kwh":float(q.sum()+e.sum()), "planned_cost_yuan":planned_cost,
64                         "emergency_cost_yuan":emergency_cost, "actual_total_cost_yuan":total_cost,
65                         "initial_kwh":float(s[0]), "terminal_kwh":float(s[-1]), "total_unused_kwh":float(u.sum())})
66     if excluded>1e-4:
67         raise ValueError(f"Emergency reporting threshold suppresses material energy: {excluded} kWh")
68     selected=[]
69     for date_text in ("2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"):
70         i=dates.index(date_text)
71         selected.append({"daily[i], "purchases_at_requested_intervals": [[interval_label(hour*6), float(data["Q"][i,h]
72             ↳ our*6)]] for hour in (10,12,14,16,18,20)],
73                         "battery_four_hour":four_hour_rows(data["C"][i], data["D"][i], data["S"][i]),
74                         "emergency_segments":segments_by_day[date_text]})
75     a=q1["plan"]
76     payload={"q1":{"purchases": [[interval_label(t), a["Q"][t] for t in range(144)],
77                                "battery":four_hour_rows(a["C"], a["D"], a["S"]),
78                                "cost_yuan":q1["cost_yuan"], "purchase_kwh":q1["purchase_kwh"],

```

```

78         "requested_intervals":[[interval_label(hour*6),a["Q"][hour*6]] for hour in (10,12,14,16,18,20)]},
79         "q2":{"headers":[" 日期\\时间",*[interval_label(t) for t in range(144)]," 全天购电量"," 全天购电费"],
80         "purchases":plan_rows,"battery":battery_rows,"emergency":emergency_rows,
81         "daily":daily,"selected_dates":selected,"emergency_omitted_below_threshold_kwh":excluded},
82         "notes":{"time_mapping":" 源标签为区间终点; 输出模板改为 00:00-00:10 至 23:50-24:00, 点序和数值不循环平移。",
83         "q2_total_columns":["计划购电量表EP列为计划Q的日合计; EQ列为计划费用加实际紧急购电费用。紧急电量在单独工作表。
84         ↪ "],
85         "energy_units":" 各电量 kWh, 费用元。C、D 为交流母线侧电量, S 为电池内部储量。",
86         "sources":"C 题题面、附件 1、附件 2、附件 5 及前两问修订模型。",
87         "method":" 零点固定全天电池计划; 28 日加权历史预测、最近历史样本外配对残差经验场景: 一月闲置、年末 6000。"}}
88     return payload

```

academic_figures.py

```

1  """ 从已保存的真实结果生成科研图表。仅变更展示, 不运行预测、求解或调参。"""
2  from pathlib import Path
3  from datetime import date
4  import argparse
5  import hashlib
6  import json
7
8  import numpy as np
9  import matplotlib
10 matplotlib.use('Agg')
11 import matplotlib.pyplot as plt
12 from matplotlib.backends.backend_pdf import PdfPages
13 from matplotlib.patches import Patch
14
15 from forecasting import read_dataset
16 from scientific_style import (COLORS as C, METHOD_COLORS, STYLE_VERSION,
17                             apply_style, step, time_axis, panel, note, save_figure)
18
19 ROOT = Path(__file__).resolve().parents[1]
20 METHODS = ['main_weighted28', 'comparison_mean7', 'comparison_point', 'comparison_no_storage']
21 METHOD_LABELS = ['加权历史 + 场景规划', '7 日均值 + 场景规划',
22                '加权历史 + 单一预测', '同场景 + 无储能']
23
24
25 def plot_q1(ds, result):
26     a = result['plan']
27     fig, axes = plt.subplots(4, 1, figsize=(7.2, 7.8), sharex=True,
28                             gridspec_kw={'height_ratios': [1.5, 1, 1, .8]})
29     fig.subplots_adjust(left=.12, right=.97, top=.94, bottom=.09, hspace=.48)
30     panel(axes[0], 'a', '典型日负载、光伏与计划购电')
31     for values, color, label, linestyle in (
32         (ds.baseline_load_kw/1000, C['load'], '负载', '-'),
33         (ds.baseline_pv_kw/1000, C['pv'], '光伏', '--'),
34         (np.asarray(a['Q'])*.006, C['purchase'], '计划购电', '-.')):
35         step(axes[0], values, color=color, label=label, ls=linestyle)
36     axes[0].set_ylabel('功率 (MW)')
37     axes[0].set_ylim(0, max(ds.baseline_load_kw.max()/1000,
38                             ds.baseline_pv_kw.max()/1000, max(a['Q'])*.006)*1.28)
39     axes[0].legend(ncol=3, loc='upper left')
40     panel(axes[1], 'b', '储能充放电: 正值充电, 负值放电')
41     t = np.arange(144)/6
42     axes[1].bar(t, np.asarray(a['C'])*.006, width=1/6, align='edge',
43                 color=C['charge'], linewidth=0, label='充电')
44     axes[1].bar(t, -np.asarray(a['D'])*.006, width=1/6, align='edge',
45                 color=C['discharge'], linewidth=0, label='放电')
46     axes[1].axhline(0, color=C['ink'], lw=.6)
47     axes[1].set_ylabel='功率 (MW)', ylim=(-6.1, 7.5), yticks=[-5, 0, 5])
48     axes[1].legend(ncol=2, loc='upper left', borderaxespad=.1)
49     panel(axes[2], 'c', '电池储量与运行边界 (初末均为 6 MWh)')
50     storage = np.asarray(a['S'])/1000
51     axes[2].fill_between(np.arange(145)/6, 1.2, storage, color=C['state'], alpha=.13)
52     axes[2].plot(np.arange(145)/6, storage, color=C['state'], lw=1.4)
53     for value in [1.2, 10.8]:
54         axes[2].axhline(value, color=C['rule'], lw=.75, ls=(0,(4,3)))
55     axes[2].plot([0,24], storage[0,-1], 'o', ms=3.5, color=C['state'], clip_on=False)
56     axes[2].set_ylabel='储量 (MWh)', ylim=(0,12), yticks=[1.2,6,10.8])
57     panel(axes[3], 'd', '已知电价')

```

```

58     step(axes[3], ds.prices, color=C['price'], lw=1.4)
59     axes[3].fill_between(np.arange(145)/6, 0, np.r_[ds.prices,ds.prices[-1]],
60                          step='post', color=C['price'], alpha=.09)
61     axes[3].set(ylabel='电价 (元/kWh)', ylim=(0,float(ds.prices.max())*1.2))
62     for ax in axes:
63         time_axis(ax, label=ax is axes[-1])
64     note(fig, f'全天电费 {result["cost_yuan"]:.2f} 元; 每步 10 min; 充放电在母线侧计量。')
65     return fig
66
67
68 def plot_methods(by_name):
69     planned = np.array([by_name[n]['total_planned_cost_yuan'] for n in METHODS])/1e4
70     emergency = np.array([by_name[n]['total_emergency_cost_yuan'] for n in METHODS])/1e4
71     energy = np.array([by_name[n]['total_emergency_purchase_kwh'] for n in METHODS])/1000
72     totals = planned+emergency
73     fig, axes = plt.subplots(2,1,figsize=(7.2,6.0))
74     fig.subplots_adjust(left=.26,right=.96,top=.91,bottom=.17,hspace=.62)
75     y=np.arange(4)
76     panel(axes[0], 'a', '实际购电费及其构成')
77     axes[0].axhspan(-.43, .43,color=C['purchase'],alpha=.07,lw=0)
78     axes[0].barh(y,planned,height=.56,color=C['planned_cost'],label='计划购电费')
79     axes[0].barh(y,emergency,left=planned,height=.56,color=C['emergency'],label='紧急购电费')
80     for i,total in enumerate(totals):
81         axes[0].text(total+28,i,f'{total:.2f}',va='center',fontsize=8,
82                      fontweight='bold' if i==0 else 'normal')
83     axes[0].legend(loc='upper left',ncol=2,borderaxespad=.3)
84     axes[0].set(xlim=(0,totals.max()*1.19),xlabel='334 天实际购电费 (万元)')
85     panel(axes[1], 'b', '实际紧急购电量')
86     axes[1].axhspan(-.43, .43,color=C['purchase'],alpha=.07,lw=0)
87     axes[1].barh(y,energy,height=.56,color=METHOD_COLORS)
88     for i,total in enumerate(energy):
89         axes[1].text(total+32,i,f'{total:.2f}',va='center',fontsize=8)
90     axes[1].set(xlim=(0,energy.max()*1.19),xlabel='334 天紧急购电量 (MWh)')
91     for ax in axes:
92         ax.set_yticks(y,METHOD_LABELS);ax.invert_yaxis()
93         ax.get_yticklabels()[0].set_fontweight('bold')
94         ax.spines['left'].set_visible(False);ax.tick_params(axis='y',length=0)
95     axes[0].set_ylim(3.65,-1.15)
96     base = by_name['main_weighted28']['total_cash_cost_yuan']
97     dp=(1-base/by_name['comparison_point']['total_cash_cost_yuan'])*100
98     dn=(1-base/by_name['comparison_no_storage']['total_cash_cost_yuan'])*100
99     fig.text(.26,.025,f'主方案费用较单一预测低 {dp:.2f}%, 较无储能低 {dn:.2f}%。 \n'
100             '期间: 2025 年 2-12 月; 所有对照使用相同日期和初末储量条件。',fontsize=7.2,color=C['muted'])
101     return fig
102
103
104 def plot_selected(ds, arrays, payload):
105     fig=plt.figure(figsize=(7.2,7.3))
106     outer=fig.add_gridspec(2,2,left=.11,right=.97,top=.93,bottom=.105,wspace=.30,hspace=.49)
107     dates=['2025-03-20','2025-06-21','2025-09-23','2025-12-21']
108     indices=[ds.dates.index(date.fromisoformat(d))-31 for d in dates]
109     max_q=float(arrays['Q'][indices].max())*.006
110     max_e=float(arrays['E'][indices].max())*6
111     totals={item['date']:item for item in payload['q2']['selected_dates']}
112     for letter, day, idx, grid in zip('abcd',dates,indices,outer):
113         inner=grid.subgridspec(2,1,height_ratios=[2.2,1],hspace=.12)
114         top=fig.add_subplot(inner[0]);bottom=fig.add_subplot(inner[1],sharex=top)
115         panel(top,letter,day)
116         step(top,arrays['Q'][idx]*.006,color=C['purchase'])
117         top.fill_between(np.arange(145)/6,0,np.r_[arrays['Q'][idx],arrays['Q'][idx,-1]]*.006,
118                        step='post',color=C['purchase'],alpha=.10)
119         top.set(ylabel='计划购电 (MW)',ylim=(0,max_q*1.23))
120         top.tick_params(labelbottom=False)
121         top.text(.03,.91,f'应急电量 {totals[day]["emergency_purchase_kwh"]:.1f} kWh',
122                transform=top.transAxes,fontsize=7.5,va='top')
123         step(bottom,arrays['E'][idx]*6,color=C['emergency'])
124         bottom.fill_between(np.arange(145)/6,0,np.r_[arrays['E'][idx],arrays['E'][idx,-1]]*6,
125                           step='post',color=C['emergency'],alpha=.24)
126         bottom.set(ylabel='应急 (kW)',ylim=(0,max_e*1.18))
127         if arrays['E'][idx].max()<1e-6:
128             bottom.text(.5,.45,'无紧急购电',transform=bottom.transAxes,ha='center',fontsize=8)
129         time_axis(top,label=False,ticks=6);time_axis(bottom,ticks=6)
130     note(fig, '四天使用相同的计划功率刻度和相同的应急功率刻度; 上下两类面板的单位不同。')

```

```

131     return fig
132
133
134 def plot_sensitivity(by_name):
135     base=by_name['main_weighted28']['total_cash_cost_yuan']
136     fig,axes=plt.subplots(1,2,figsize=(7.2,3.65),sharey=True)
137     fig.subplots_adjust(left=.12,right=.965,top=.85,bottom=.23,wspace=.27)
138     cases=[('terminal','终端价值系数',C['state'],'o'),('residual','历史残差幅度',C['pv'],'s')]
139     for ax,letter,(key,title,color,marker) in zip(axes,'ab',cases):
140         panel(ax,letter,title)
141         vals=[by_name[f'sensitivity_{key}_0_8']['total_cash_cost_yuan'],base,
142               by_name[f'sensitivity_{key}_1_2']['total_cash_cost_yuan']]
143         change=(np.asarray(vals)/base-1)*100
144         ax.axhline(0,color=C['rule'],ls=(0,(4,3)),lw=.8)
145         ax.plot([.8,1,1.2],change,ls='--',marker=marker,color=color,mfc='white',mew=1.3,ms=5)
146         for x,y in zip([.8,1,1.2],change):
147             ax.annotate(f'{y:+.3f}%',(x,y),xytext=(0,9),textcoords='offset points',
148                        ha='center',fontsize=8)
149         ax.set(xlim=(.75,1.25),ylim=(-.3,2.2),xticks=[.8,1,1.2],xlabel='参数相对基准倍数')
150     axes[0].set_ylabel('334 天实际费用变化 (%)')
151     note(fig,'每个点均重新回放 334 天; 两面板使用相同纵轴。虚线只连接已计算的三个点。',y=.055)
152     return fig
153
154
155 def plot_annual(ds, arrays):
156     fig,axes=plt.subplots(2,1,figsize=(7.2,7.5))
157     fig.subplots_adjust(left=.12,right=.90,top=.93,bottom=.11,hspace=.38)
158     dates=ds.dates[31:]
159     rows=[i for i,d in enumerate(dates) if d.day==1]
160     labels=[f'{dates[i].month:02d} 月' for i in rows]
161     for ax,letter,key,title,cmap in zip(axes,'ab',['Q','E'],
162                                         ['计划购电的日内与季节分布','紧急购电的日内与季节分布'],['viridis','magma_r']):
163         panel(ax,letter,title)
164         values=arrays[key]*.006
165         image=ax.imshow(values,origin='upper',aspect='auto',interpolation='none',
166                        extent=(0,24,len(dates)-.5,-.5),vmin=0,vmax=float(values.max()),cmap=cmap)
167         ax.set(yticks=rows,yticklabels=labels,ylabel='2025 年')
168         time_axis(ax,ticks=4)
169         bar=fig.colorbar(image,ax=ax,pad=.025,fraction=.035,aspect=25)
170         bar.set_label('功率 (MW)');bar.outline.set_linewidth(.5)
171     note(fig,'每格为一天的一个 10 min 区间; 两幅色标独立, 均从 0 起, 无平滑、无极值截断。')
172     return fig
173
174
175 def plot_data_overview(project):
176     p=json.loads((project/'reports/plot_payload.json').read_text(encoding='utf-8'))
177     s=json.loads((project/'reports/dataset_profile.json').read_text(encoding='utf-8'))
178     fig,axes=plt.subplots(2,2,figsize=(7.2,5.9))
179     fig.subplots_adjust(left=.12,right=.97,top=.92,bottom=.14,wspace=.33,hspace=.53)
180     h=np.asarray(p['hours'])
181     panel(axes[0,0],'a','日内负载与实际光伏')
182     for key,label,color,ls in [('load','负载',C['load'],'-'),('pv','光伏',C['pv'],'--')]:
183         q=p['profiles'][key]
184         axes[0,0].fill_between(h,np.asarray(q['p10'])/1000,np.asarray(q['p90'])/1000,color=color,alpha=.17,lw=0)
185         axes[0,0].plot(h,np.asarray(q['mean'])/1000,color=color,label=label,ls=ls)
186     axes[0,0].set_ylabel='功率 (MW)',ylim=(0,12));axes[0,0].legend(loc='upper left',ncol=2)
187     panel(axes[0,1],'b','附件 4 电价的日内分布')
188     q=p['profiles']['price']
189     axes[0,1].fill_between(h,q['p10'],q['p90'],color=C['state'],alpha=.20,lw=0)
190     axes[0,1].plot(h,q['mean'],color=C['state'])
191     axes[0,1].set_ylabel='电价 (元/kWh)',ylim=(0,float(max(q['p90']))*1.10))
192     for ax in axes[0]:
193         time_axis(ax,ticks=6);ax.set_xlabel('源时间标签 (h)')
194     panel(axes[1,0],'c','月均负载与实际光伏')
195     months=[r['month'] for r in p['monthly']]
196     for field,label,color,marker in [('load_mean_kw','负载',C['load'],'o'),('pv_mean_kw','光伏',C['pv'],'s')]:
197         axes[1,0].plot(months,np.array([r[field] for r in p['monthly']])/1000,
198                        marker=marker,color=color,label=label,ms=3.5,mfc='white',mew=1)
199     axes[1,0].set(xlabel='月份',ylabel='平均功率 (MW)',xticks=[1,3,5,7,9,12],ylim=(0,8))
200     axes[1,0].legend(loc='upper left',ncol=2)
201     panel(axes[1,1],'d','附件 3 预报误差与提前量')
202     bands=s['forecast_alignment']['lead_bands']
203     keys=['1-6','7-12','13-18','19-24'];values=np.array([bands[k]['mae_kw'] for k in keys])

```

```

204     bars=axes[1,1].bar(keys,values,color=plt.colormaps['viridis']([.25,.43,.61,.79]),width=.62)
205     for bar,value in zip(bars,values):
206         axes[1,1].text(bar.get_x()+bar.get_width()/2,value+15,f'{value:.1f}',ha='center',fontsize=8)
207     axes[1,1].set(xlabel='预报提前量 (h)',ylabel='MAE (kW)',ylim=(0,float(values.max())*1.20))
208     note(fig,'来源: 附件 2、3、4; 阴影为跨日 P10-P90 分位范围, 不是置信区间。\\n'
209           '预报误差只统计实际光伏>0 的整点; 本图为全年描述统计, 不作为第二问日前决策输入。')
210     return fig
211
212
213 CAPTIONS={
214     '第一问调度': '第一问典型日调度。a为负载、光伏和计划购电功率, b为母线侧充放电功率, c为电池储量及1.2/10.8 MWh边界, d为已知电价。
    ↳ 区间功率采用真实10 min阶梯; 电池储量为区间边界值的连线。',
215     '第二问方法对照': '相同334天和初末储量条件下的方法对照。a分解计划与紧急购电费用, b比较紧急购电量。浅色带及加粗标签标识预先指定的主方案。
    ↳ 主方案相对7日均值场景方案费用仅降低约0.184%, 不支持宣称显著优势; 费用更低也不意味着所有对照下应急电量均更少。',
216     '第二问指定日期': '四个指定日期的购电轨迹。每组上图为计划功率(MW), 下图为应急功率(kW); 同类面板共用尺度,
    ↳ 避免为突出微小波动而逐日改变纵轴。日累计应急电量由原轨迹求和, 单位kWh。',
217     '第二问灵敏度': '终端价值及历史残差幅度的单因素±20%灵敏度。每个结果均连续回放334天。两面板使用相同纵轴; 虚线为阅读辅助,
    ↳ 不意味着已计算连续参数空间或已证明普遍稳健。',
218     '第二问全年调度热力图': '主方案334×144个时段的计划及应急功率。两图分别用各自完整数据范围映射颜色, 色条均明确标出MW;
    ↳ 没有分位数截断或平滑。可用于观察时段分布, 不能据颜色差异直接比较两个独立色标下的绝对功率。',
219     'C题数据概览': '全年附件描述统计。日内阴影为同一源时间标签跨日P10-P90范围, 实线/虚线为均值。附件3预报MAE使用实际光伏为正的目标整点。
    ↳ 附件4电价展示和全年描述统计不参与本版第二问的日前决策。',
220 }
221
222
223 def build_figures(run, project, output, *, include_overview=True, dpi=450):
224     run,project,output=Path(run),Path(project),Path(output)
225     ds=read_dataset(project/'data/processed')
226     summaries=json.loads((run/'experiment_summaries.json').read_text(encoding='utf-8'))
227     by_name={s['name']:s for s in summaries}
228     if len(by_name)!=8:
229         raise ValueError('All eight completed experiments are required')
230     audit=json.loads((run/'independent_audit.json').read_text(encoding='utf-8'))
231     if audit['status']!='passed' or len(audit['experiments'])!=8 or 'workbooks' not in audit:
232         raise ValueError('Only independently audited results may be plotted')
233     metadata=json.loads((run/'config.json').read_text(encoding='utf-8'))
234     if ds.sources!=metadata['input_sha256']:
235         raise ValueError('Processed input does not match the audited run')
236     q1=json.loads((run/'q1.json').read_text(encoding='utf-8'))['milp']
237     payload=json.loads((run/'workbook_payload.json').read_text(encoding='utf-8'))
238     with np.load(run/'main_weighted28/trajectories.npz') as stored:
239         arrays={k:stored[k].copy() for k in ['Q','C','D','S','E','U','day_indices']}
240     if not np.array_equal(arrays['day_indices'],np.arange(31,365)):
241         raise ValueError('Unexpected trajectory date alignment')
242     np.testing.assert_allclose(arrays['E'].sum(),by_name['main_weighted28']['total_emergency_purchase_kwh'],rtol=1e-10)
243     for s in summaries:
244         np.testing.assert_allclose(s['total_planned_cost_yuan']+s['total_emergency_cost_yuan'],s['total_cash_cost_yuan'],rtol=1e-12)
245     inputs=[run/n for n in ['experiment_summaries.json','q1.json','workbook_payload.json',
246                           'independent_audit.json','config.json','main_weighted28/trajectories.npz']]
247     if include_overview:
248         inputs += [project/'reports'/n for n in ['plot_payload.json','dataset_profile.json']]
249     before={str(p):hashlib.sha256(p.read_bytes()).hexdigest() for p in inputs}
250     apply_style();output.mkdir(parents=True,exist_ok=True)
251     jobs=[('第一问调度',lambda:plot_q1(ds,q1)),
252          ('第二问方法对照',lambda:plot_methods(by_name)),
253          ('第二问指定日期',lambda:plot_selected(ds,arrays,payload)),
254          ('第二问灵敏度',lambda:plot_sensitivity(by_name)),
255          ('第二问全年调度热力图',lambda:plot_annual(ds,arrays))]
256     if include_overview:
257         jobs.append(('C 题数据概览',lambda:plot_data_overview(project)))
258     records=[]
259     with PdfPages(output/'科研图表合集.pdf',metadata={'Title':'C 题科研图表合集','Author':'C 题建模团队',
260              'Subject':'已审计结果的可复现科研可视化','Keywords':STYLE_VERSION}) as pdf:
261         for name,build in jobs:
262             records.append(save_figure(build(),output,name,pdf=pdf,dpi=dpi))
263     for path,digest in before.items():
264         if hashlib.sha256(Path(path).read_bytes()).hexdigest()!=digest:
265             raise AssertionError('A figure input was modified')
266     lines=['# 图表说明与使用','','f'视觉版本: {STYLE_VERSION}。所有图均直接使用已保存数据; 本次未重新拟合、求解或调整模型参数。','','',
267           'PNG 为 450 dpi; SVG 保留可编辑文字; PDF 合集保留矢量坐标、文字与曲线, 热力图数据层为嵌入图像。',
268           '本组按约 183 mm 双栏宽度设计, 缩小到单栏时请重排布局及字号, 不宜直接缩小到一半。','']

```

```

269     for i,(name,_) in enumerate(jobs,1):
270         lines.extend([f'## 图 {i} {name}', '', CAPTIONS[name], ''])
271     (output/'图表说明.md').write_text('\n'.join(lines), encoding='utf-8')
272     details={'style_version': STYLE_VERSION, 'run_id': run.name, 'figure_count': len(records),
273             'inputs_unchanged': True, 'input_sha256': before, 'figures': records,
274             'note': '程序边界与缺字检查通过; 视觉审查结果另行记录。'}
275     (output/'figure_manifest.json').write_text(json.dumps(details, ensure_ascii=False, indent=2) + '\n', encoding='utf-8')
276     return details
277
278
279 def main():
280     parser=argparse.ArgumentParser(description=__doc__)
281     parser.add_argument('run', type=Path)
282     parser.add_argument('--project-root', type=Path, default=ROOT)
283     parser.add_argument('--output', type=Path)
284     parser.add_argument('--skip-overview', action='store_true')
285     args=parser.parse_args()
286     output=args.output or args.run/'deliverables/figures'
287     result=build_figures(args.run, args.project_root, output, include_overview=not args.skip_overview)
288     print(json.dumps({'figure_count': result['figure_count'], 'output': str(output), 'inputs_unchanged': True}, ensure_ascii=False))
289
290
291 if __name__=='__main__':
292     main()

```

audit_results.py

```

1  """ 独立审核完整轨迹与导出文件; 不调用规划或运行回放函数。 """
2  from pathlib import Path
3  from datetime import date, datetime
4  import argparse
5  import csv
6  import hashlib
7  import json
8
9  import numpy as np
10 import openpyxl
11
12 from forecasting import read_dataset, interval_label, clock_text
13
14 ROOT=Path(__file__).resolve().parents[1]
15
16
17 def close(name, value, tolerance=1e-5):
18     error=float(np.max(np.abs(value)))
19     if not np.isfinite(error) or error>tolerance:
20         raise AssertionError(f"{name}: {error}")
21     return error
22
23
24 def audit_arrays(run, dataset, config):
25     summaries=json.loads((run/"experiment_summaries.json").read_text(encoding="utf-8"))
26     results={}
27     for summary in summaries:
28         name=summary["name"]
29         folder=run/name
30         with np.load(folder/"trajectories.npz") as a:
31             ids=a["day_indices"]
32             if not np.array_equal(ids, np.arange(31, 365)):
33                 raise AssertionError(f"{name}: incomplete dates")
34             q, c, d, s, e, u=(a[k] for k in ("Q", "C", "D", "S", "E", "U"))
35             for x in (q, c, d, e, u):
36                 if x.shape!=(334, 144) or not np.isfinite(x).all() or x.min() < -1e-5:
37                     raise AssertionError(f"{name}: invalid trajectory")
38             if s.shape!=(334, 145) or not np.isfinite(s).all():
39                 raise AssertionError("Invalid state shape")
40             checks={
41                 "bus_balance_kwh": close("bus", q+e+dataset.pv_kw[ids]/6+d-dataset.load_kw[ids]/6-c-u),

```

```

42         "battery_state_kwh":close("state",np.diff(s,axis=1)-config["eta_charge"]*c+d/config["eta_discharge"]
↪     ]),
43         "cross_day_kwh":close("cross day",s[1:,0]-s[:-1,-1]),
44         "initial_kwh":close("initial",s[0,0]-6000),
45         "year_end_kwh":close("year end",s[-1,-1]-6000),
46         "storage_bounds_kwh":close("bounds",max(0,1200-s.min()),s.max()-10800)),
47         "power_kwh":close("power",max(0,c.max()-5000/6,d.max()-5000/6)),
48         "simultaneous_kwh":close("simultaneous",np.minimum(np.maximum(c,0),np.maximum(d,0))),
49         "emergency_rule_kwh":close("E",e-np.maximum(dataset.load_kw[ids]/6+c-q-dataset.pv_kw[ids]/6-d,0)),
50         "unused_rule_kwh":close("U",u-np.maximum(q+dataset.pv_kw[ids]/6+d-dataset.load_kw[ids]/6-c,0)),
51     }
52     planned=q@dataset.prices
53     emergency=e@(5*dataset.prices)
54     cash=planned+emergency
55     checks["annual_cash_reconciliation_yuan"]=close("annual
↪     cash",cash.sum()-summary["total_cash_cost_yuan"],1e-4)
56     with (folder/"daily_results.csv").open(encoding="utf-8-sig",newline="") as stream:
57         records=list(csv.DictReader(stream))
58         if [r["date"] for r in records]!=dataset.dates[i].isoformat() for i in ids]:
59             raise AssertionError("Daily results dates are not aligned")
60         checks["daily_cash_reconciliation_yuan"]=close("daily cash",cash-np.array([float(r["cash_cost_yuan"])
↪         for r in records]))
61         info=json.loads((folder/"information_audit.json").read_text(encoding="utf-8"))
62         if len(info)!=334:
63             raise AssertionError("Missing information audit records")
64         for audit,day in zip(info,ids):
65             origin=dataset.dates[day].isoformat()
66             if audit["origin"]!=origin or audit["training_end"]>=origin or (audit["residual_end"] is not None
↪             and audit["residual_end"]>=origin):
67                 raise AssertionError("Information boundary violated")
68         results[name]={"days":334,"checks":checks,"cash_recomputed_yuan":float(cash.sum())}
69     return results
70
71
72 def audit_workbooks(run,dataset):
73     q1=json.loads((run/"q1.json").read_text(encoding="utf-8"))["milp"]["plan"]
74     checks={}
75     w=openpyxl.load_workbook(run/"deliverables/result1.xlsx",read_only=True,data_only=True)
76     if w.sheetnames!="计划购电量","充放电量":
77         raise AssertionError("Q1 template sheets changed unexpectedly")
78     rows=list(w["计划购电量"].iter_rows(min_row=2,max_row=145,max_col=2,values_only=True))
79     if [r[0] for r in rows]!=interval_label(i) for i in range(144)):
80         raise AssertionError("Q1 export time mapping incorrect")
81     checks["q1_purchase_kwh"]=close("q1 Q",np.array([r[1] for r in rows])-q1["Q"])
82     rows=list(w["充放电量"].iter_rows(min_row=2,max_row=7,max_col=5,values_only=True))
83     checks["q1_charge_blocks_kwh"]=close("q1 C blocks",np.array([r[1] for r in
↪     rows])-np.array(q1["C"]).reshape(6,24).sum(axis=1))
84     checks["q1_discharge_blocks_kwh"]=close("q1 D blocks",np.array([r[2] for r in
↪     rows])-np.array(q1["D"]).reshape(6,24).sum(axis=1))
85     close("q1 start",rows[0][4]-6000);close("q1 end",rows[1][4]-6000)
86     w.close()
87     a=np.load(run/"main_weighted28/trajectories.npz")
88     dates=[dataset.dates[i] for i in a["day_indices"]]
89     w=openpyxl.load_workbook(run/"deliverables/result2.xlsx",read_only=True,data_only=True)
90     if w.sheetnames!="计划购电量","充放电量","紧急购电量":
91         raise AssertionError("Q2 template sheets changed unexpectedly")
92     p=w["计划购电量"]
93     if [p.cell(1,t+2).value for t in range(144)]!=interval_label(t) for t in range(144)):
94         raise AssertionError("Q2 time header mismatch")
95     rows=list(p.iter_rows(min_row=2,max_row=335,max_col=147,values_only=True))
96     if any(not isinstance(r[0],datetime) or r[0].date()!=day for r,day in zip(rows,dates)):
97         raise AssertionError("Q2 dates are not typed/aligned")
98     numbers=np.array([r[1:] for r in rows],dtype=float)
99     checks["q2_purchase_matrix_kwh"]=close("q2 Q",numbers[:,:144]-a["Q"])
100     checks["q2_daily_purchase_sums_kwh"]=close("q2 sums",numbers[:,:144]-a["Q"].sum(axis=1))
101     checks["q2_daily_cash_yuan"]=close("q2 cash",numbers[:,:145]-(a["Q"]@dataset.prices+a["E"]@(5*dataset.prices)))
102     rows=list(w["充放电量"].iter_rows(min_row=2,max_row=2005,max_col=6,values_only=True))
103     if len(rows)!=334*6:
104         raise AssertionError("Q2 battery row count")
105     for i,day in enumerate(dates):
106         block=rows[i*6:(i+1)*6]
107         if not isinstance(block[0][0],datetime) or block[0][0].date()!=day:
108             raise AssertionError("Battery block date mismatch")

```

```

109         close("block C",np.array([r[2] for r in block])-a["C"][i].reshape(6,24).sum(axis=1))
110         close("block D",np.array([r[3] for r in block])-a["D"][i].reshape(6,24).sum(axis=1))
111         close("block S start",block[0][5]-a["S"][i,0]);close("block S end",block[1][5]-a["S"][i,-1])
112 rows=[r for r in w["紧急购电量"].iter_rows(min_row=2,max_col=3,values_only=True) if any(x is not None for x in
↪ r)]
113 totals={day:0. for day in dates};current=None;last_end=-1
114 for row in rows:
115     if row[0] is not None:
116         if not isinstance(row[0],datetime):
117             raise AssertionError("Emergency dates must be typed")
118         current=row[0].date();last_end=-1
119         if current not in totals or not isinstance(row[1],str) or not isinstance(row[2],(int,float)):
120             raise AssertionError("Invalid emergency row")
121         lo,hi=row[1].split("-")
122         start=int(lo[:2])*6+int(lo[3:])/10
123         end=int(hi[:2])*6+int(hi[3:])/10
124         if not 0<=start<end<=144 or start<last_end:
125             raise AssertionError("Emergency intervals overlap or cross day incorrectly")
126         close("event sum",row[2]-a["E"][dates.index(current),start:end].sum())
127         totals[current]+=row[2];last_end=end
128 checks["emergency_export_daily_kwh"]=close("E export",np.array([totals[day] for day in
↪ dates])-a["E"].sum(axis=1),1e-4)
129 for sheet in w:
130     if any(cell.data_type=="e" for row in sheet for cell in row):
131         raise AssertionError("Formula error in exported workbook")
132 w.close();a.close()
133 checks["exported_dates"]=334;checks["emergency_segments"]=len(rows)
134 return checks
135
136
137 def main():
138     parser=argparse.ArgumentParser()
139     parser.add_argument("run",type=Path)
140     parser.add_argument("--workbooks",action="store_true")
141     args=parser.parse_args()
142     dataset=read_dataset(ROOT/"data/processed")
143     metadata=json.loads((args.run/"config.json").read_text(encoding="utf-8"))
144     if dataset.sources!=metadata["input_sha256"]:
145         raise AssertionError("Processed input hashes changed")
146     manifest=json.loads((ROOT/"data/raw/manifest.json").read_text(encoding="utf-8"))
147     source=Path(manifest["source_directory"])
148     for item in manifest["files"]:
149         for path in (ROOT/"data/raw"/item["file"],source/item["file"]):
150             if hashlib.sha256(path.read_bytes()).hexdigest()!=item["sha256"]:
151                 raise AssertionError(f"Source or snapshot changed: {path}")
152     report={"raw_files_unchanged":len(manifest["files"]), "processed_hashes_match":True,
153           "experiments":audit_arrays(args.run,dataset,metadata["config"])}
154     if args.workbooks:
155         report["workbooks"]=audit_workbooks(args.run,dataset)
156     report["status"]="passed"
157     (args.run/"independent_audit.json").write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding="utf-8")
158     print(json.dumps({"status":"passed", "experiments":len(report["experiments"]), "workbooks_checked":args.workbooks
↪ },ensure_ascii=False))
159
160
161 if __name__=="__main__":
162     main()

```

C 问题二源程序

forecasting.py

```

1 """ 只使用目标日前已完成日期的预测与历史样本外残差场景。
2
3 候选模型是预先定义的历史均值/加权均值，不使用全年数据调参。
4 枚举最近历史联合残差，形成离散经验分布，不额外随机重抽样。
5 """
6 from dataclasses import dataclass

```

```

7 from datetime import date, timedelta
8 from pathlib import Path
9 import csv
10 import hashlib
11
12 import numpy as np
13
14
15 def clock_text(minute):
16     if not isinstance(minute,int) or not 0<=minute<=1440:
17         raise ValueError("Minute must be an integer from 0 to 1440")
18     return f"{minute//60:02d}:{minute%60:02d}"
19
20
21 def interval_label(index):
22     if not isinstance(index,int) or not 0<=index<144:
23         raise ValueError("Interval index must be 0..143")
24     return clock_text(index*10)+"-"+clock_text((index+1)*10)
25
26
27 @dataclass
28 class Dataset:
29     dates: list
30     load_kw: np.ndarray
31     pv_kw: np.ndarray
32     prices: np.ndarray
33     baseline_load_kw: np.ndarray
34     baseline_pv_kw: np.ndarray
35     sources: dict
36
37
38 def read_dataset(processed):
39     processed=Path(processed)
40     actual_path=processed/"actual_points.csv"
41     baseline_path=processed/"baseline_points.csv"
42     with actual_path.open(encoding="utf-8-sig",newline="") as stream:
43         rows=list(csv.DictReader(stream))
44     expected=[date(2025,1,1)+timedelta(days=i) for i in range(365)]
45     if len(rows)!=365*144:
46         raise ValueError("Expected 365 complete days, each with 144 records")
47     load,pv=np.empty((365,144)),np.empty((365,144))
48     for row_id,row in enumerate(rows):
49         day,t=divmod(row_id,144)
50         if row["date"]!=expected[day].isoformat() or int(row["point_index"])!=t+1 or
51             ↪ row["time_label"]!=clock_text((t+1)*10):
52             raise ValueError(f"Unexpected date/time/point mapping at input row {row_id+2}")
53         load[day,t],pv[day,t]=float(row["load_kw"]),float(row["pv_actual_kw"])
54     with baseline_path.open(encoding="utf-8-sig",newline="") as stream:
55         baseline=list(csv.DictReader(stream))
56     if len(baseline)!=144:
57         raise ValueError("Invalid baseline length")
58     for t,row in enumerate(baseline):
59         if int(row["point_index"])!=t+1 or row["time_label"]!=clock_text((t+1)*10):
60             raise ValueError("Baseline time mapping differs from actual data")
61     p,bl,bg=[np.array([float(row[field]) for row in baseline]) for field in
62             ↪ ("price_yuan_per_kwh","load_kw","pv_forecast_kw")]
63     for arr in (load,pv,p,bl,bg):
64         if not np.isfinite(arr).all() or (arr<0).any():
65             raise ValueError("Nonfinite or negative input")
66     if (p<=0).any():
67         raise ValueError("Strictly positive electricity prices are required")
68     return Dataset(expected,load,pv,p,bl,bg,{x.name:hashlib.sha256(x.read_bytes()).hexdigest() for x in
69             ↪ (actual_path,baseline_path)})
70
71
72 def point_forecast(history_load, history_pv, history_dates, target_date, model):
73     """ 历史只能包含 target_date 之前的完整日。模型参数预先指定。 """
74     load,pv=np.asarray(history_load,dtype=float),np.asarray(history_pv,dtype=float)
75     if load.ndim!=2 or load.shape!=pv.shape or len(load)!=len(history_dates) or len(load)==0:
76         raise ValueError("Invalid history shape")
77     if any(d>=target_date for d in history_dates) or any(b-a!=timedelta(days=1) for a,b in
78             ↪ zip(history_dates,history_dates[1:])):
79         raise ValueError("History must contain only consecutive completed dates before the forecast origin")

```

```

76     if history_dates[-1]!=target_date-timedelta(days=1):
77         raise ValueError("Latest completed historical date is missing")
78     if not np.isfinite(load).all() or not np.isfinite(pv).all() or (load<0).any() or (pv<0).any():
79         raise ValueError("Invalid historical power")
80     if model=="mean7":
81         window=min(7,len(load)); weights=np.ones(window)
82     elif model=="weighted28":
83         window=min(28,len(load))
84         ages=np.arange(window,0,-1)
85         weights=np.exp(-np.log(2)*ages/7)
86         weights*=np.array([2. if d.weekday()==target_date.weekday() else 1. for d in history_dates[-window:]])
87     else:
88         raise ValueError("Unknown forecast model")
89     weights/=weights.sum()
90     return weights@load[-window:],weights@pv[-window:],{
91         "model":model,"training_start":history_dates[-window].isoformat(),
92         "training_end":history_dates[-1].isoformat(),"origin":target_date.isoformat(),
93         "history_days":window,"weights":weights.tolist()
94     }
95
96 def causal_forecast_cache(dataset, model, minimum_history_days=7):
97     """ 离线重建每个历史日当时能得到的预测; 调用时只传入其历史前缀。"""
98     if minimum_history_days<1 or minimum_history_days>len(dataset.dates):
99         raise ValueError("Invalid historical warmup")
100     forecast_load=np.full_like(dataset.load_kw,np.nan,dtype=float)
101     forecast_pv=np.full_like(dataset.pv_kw,np.nan,dtype=float)
102     metadata={}
103     for day in range(minimum_history_days,len(dataset.dates)):
104         l,g,info=point_forecast(dataset.load_kw[:day],dataset.pv_kw[:day],dataset.dates[:day],dataset.dates[day],mo
↵ del)
105         forecast_load[day],forecast_pv[day]=l,g
106         metadata[day]=info
107     return {"model":model,"load_kw":forecast_load,"pv_kw":forecast_pv,"metadata":metadata,
108           "minimum_history_days":minimum_history_days}
109
110
111 def make_scenarios(dataset, cache, day, *, residual_days=28, residual_scale=1., deterministic=False):
112     """ 整日负荷/光伏残差配对; 截取 day 之前的样本外残差, 转换为 kWh。
113
114 PV 支撑范围只由最近 28 个已观测日确定, 完全未观测到发电的时段置零。
115 这不预知日出时间, 也不能保证未来不会出现新的发电时段。
116 """
117     if day<cache["minimum_history_days"]+1 or day>len(dataset.dates):
118         raise ValueError("Insufficient historical out-of-sample residuals")
119     if not isinstance(residual_days,int) or residual_days<1 or not np.isfinite(residual_scale) or residual_scale<0:
120         raise ValueError("Invalid residual configuration")
121     point_load=cache["load_kw"][day]
122     point_pv=cache["pv_kw"][day]
123     begin=max(cache["minimum_history_days"],day-residual_days)
124     ids=np.arange(begin,day)
125     if deterministic:
126         raw_load=point_load[None,:]
127         raw_pv=point_pv[None,:]
128     else:
129         # 只从已完成的历史日期减去其当时可用预测, 不使用目标日误差。
130         residual_load=dataset.load_kw[ids]-cache["load_kw"][ids]
131         residual_pv=dataset.pv_kw[ids]-cache["pv_kw"][ids]
132         raw_load=point_load+residual_scale*residual_load
133         raw_pv=point_pv+residual_scale*residual_pv
134     support=dataset.pv_kw[max(0,day-28):day].max(axis=0)>0
135     scenario_load=np.maximum(raw_load,0)/6
136     scenario_pv=np.where(support,np.maximum(raw_pv,0),0)/6
137     if not np.isfinite(scenario_load).all() or not np.isfinite(scenario_pv).all():
138         raise ValueError("Nonfinite forecast/scenario; inspect warmup and history")
139     count=len(scenario_load)
140     audit={**cache["metadata"][day],"scenario_count":count,
141           "residual_start":None if deterministic else dataset.dates[begin].isoformat(),
142           "residual_end":None if deterministic else dataset.dates[day-1].isoformat(),
143           "scenario_construction":"point_forecast" if deterministic else
↵ "enumerated_paired_daily_out_of_sample_residuals",
144           "residual_scale":residual_scale,"negative_load_values_clipped":int((raw_load<0).sum()),
145           "negative_pv_values_clipped":int((raw_pv<0).sum()),
146           "pv_values_masked_outside_recent_support":int(((raw_pv>0)&~support).sum())}

```

```

147     return scenario_load, scenario_pv, np.full(count, 1/count), audit
148
149
150 def no_storage_purchase(load_scenarios, pv_scenarios, probabilities, quantile=.8):
151     net=load_scenarios-pv_scenarios
152     if not 0<quantile<1:
153         raise ValueError("Quantile must be in (0,1)")
154     order=np.argsort(net,axis=0)
155     sorted_net=np.take_along_axis(net,order,axis=0)
156     cdf=np.cumsum(np.asarray(probabilities)[order],axis=0)
157     index=np.argmax(cdf>=quantile-1e-12,axis=0)
158     return np.maximum(sorted_net[index,np.arange(net.shape[1])],0)

```

forecasting_v2.py

```

1 """PDF §3.2-3.3: 因果预测、配对整日残差、有放回抽样。
2
3 P2 使用日周期傅里叶回归 + ARIMA(1,0,0) 残差; P3 使用随机森林。
4 所有拟合函数只接收已完成日期, 禁止把目标日实际值作为特征。
5 """
6 from datetime import timedelta
7 import warnings
8 import numpy as np
9 from sklearn.ensemble import RandomForestRegressor
10 from statsmodels.tsa.arima.model import ARIMA
11 from forecasting import point_forecast
12
13 MODELS = ('P1_weighted28', 'P2_fourier_arima', 'P3_random_forest')
14
15
16 def fourier(t, harmonics=8):
17     t = np.asarray(t)
18     angle = 2 * np.pi * t[:, None] * np.arange(1, harmonics + 1) / 144
19     return np.c_[np.ones(len(t)), np.sin(angle), np.cos(angle)]
20
21
22 def arima_forecast(history):
23     """ 先估计确定日周期, 再对残差拟合低阶平稳 ARIMA; 不做全样本调参。"""
24     y = np.asarray(history[-28:], dtype=float).ravel()
25     design = fourier(np.arange(len(y)))
26     coef = np.linalg.lstsq(design, y, rcond=None)[0]
27     residual = y - design @ coef
28     if np.std(residual) < 1e-8:
29         phi, correction, diagnostics = 0., np.zeros(144), []
30     else:
31         with warnings.catch_warnings(record=True) as captured:
32             warnings.simplefilter('always')
33             fit = ARIMA(residual, order=(1, 0, 0), trend='n').fit(method='burg')
34             phi = float(fit.arparams[0])
35             correction = np.asarray(fit.forecast(144))
36             diagnostics = [str(w.message) for w in captured]
37     prediction = fourier(np.arange(len(y), len(y) + 144)) @ coef + correction
38     return np.maximum(prediction, 0), {'ar1': phi, 'harmonics': 8,
39         'fit': 'Fourier OLS then ARIMA(1,0,0), Burg estimation', 'warnings': diagnostics}
40
41
42 def tree_features(history, day, target_date):
43     """day 可等于 len(history); 只索引 day-1 及更早的整日曲线。"""
44     if day < 7 or day > len(history):
45         raise ValueError('Tree features require seven completed days')
46     previous = history[day-1]
47     period = np.arange(144)
48     clock = fourier(period, 3)[: , 1:]
49     calendar = np.tile([np.sin(2*np.pi*target_date.weekday()/7),
50         np.cos(2*np.pi*target_date.weekday()/7),
51         np.sin(2*np.pi*(target_date.month-1)/12),
52         np.cos(2*np.pi*(target_date.month-1)/12)], (144, 1))
53     return np.c_[clock, calendar, previous, history[day-7], history[day-7:day].mean(axis=0),
54         np.roll(previous, 1), np.roll(previous, -1)]
55
56

```

```

57 def tree_forecast(history, history_dates, target_date, seed):
58     end = len(history)
59     begin = max(7, end-28)
60     x = np.vstack([tree_features(history, j, history_dates[j]) for j in range(begin, end)])
61     y = history[begin:end].ravel()
62     reg = RandomForestRegressor(n_estimators=64, max_depth=10, min_samples_leaf=8,
63                               max_features=1., random_state=seed, n_jobs=1)
64     reg.fit(x, y)
65     return np.maximum(reg.predict(tree_features(history, end, target_date)), 0)
66
67
68 def predict(history_load, history_pv, history_dates, target_date, model, prior_load, prior_pv, seed):
69     n = len(history_dates)
70     if model not in MODELS:
71         raise ValueError('Unknown candidate')
72     if n:
73         if history_dates[-1] != target_date-timedelta(days=1) or any(d >= target_date for d in history_dates):
74             raise ValueError('Future observations or missing latest completed day')
75         if any(b-a != timedelta(days=1) for a, b in zip(history_dates, history_dates[1:])):
76             raise ValueError('History must be consecutive')
77     info = {'model': model, 'origin': target_date.isoformat(),
78           'training_end': history_dates[-1].isoformat() if n else None,
79           'training_start': history_dates[max(0, n-28)].isoformat() if n else None,
80           'history_days': min(n, 28)}
81     if n == 0:
82         info['implementation'] = 'attachment1_prior_PDF_3.9.4'
83         return np.asarray(prior_load).copy(), np.asarray(prior_pv).copy(), info
84     if model == MODELS[0] or n < (7 if model == MODELS[1] else 8):
85         l, g, meta = point_forecast(history_load, history_pv, history_dates, target_date, 'weighted28')
86         info['implementation'] = 'weighted28' if model == MODELS[0] else 'short_history_weighted28_fallback'
87         info['weights'] = meta['weights']
88         return l, g, info
89     if model == MODELS[1]:
90         l, il = arima_forecast(history_load)
91         g, ig = arima_forecast(history_pv)
92         info.update(implementation='Fourier_plus_ARIMA_errors', load_fit=il, pv_fit=ig)
93     else:
94         l = tree_forecast(history_load, history_dates, target_date, seed)
95         g = tree_forecast(history_pv, history_dates, target_date, seed)
96         # 最早训练行还引用其前 7 天，因此记录的原始观测边界再向前 7 天。
97         info['training_start'] = history_dates[max(0, n-35)].isoformat()
98         info.update(implementation='separate_random_forests', n_estimators=64,
99                   max_depth=10, min_samples_leaf=8, training_target_days=min(n-7, 28))
100     if not np.isfinite(l).all() or not np.isfinite(g).all():
101         raise ValueError('Nonfinite forecast')
102     return l, g, info
103
104
105 def build_cache(dataset, model, seed=20260912, stop=365, progress=None):
106     l, g = np.full_like(dataset.load_kw, np.nan), np.full_like(dataset.pv_kw, np.nan)
107     metadata = {}
108     for day in range(stop):
109         l[day], g[day], metadata[day] = predict(dataset.load_kw[:day], dataset.pv_kw[:day],
110         dataset.dates[:day], dataset.dates[day], model, dataset.baseline_load_kw,
111         dataset.baseline_pv_kw, seed)
112         if progress and (day == 0 or (day+1) % 60 == 0 or day+1 == stop):
113             progress(f'{model}: forecasts {day+1}/{stop}')
114     return {'model': model, 'load_kw': l, 'pv_kw': g, 'metadata': metadata}
115
116
117 def scenarios(dataset, cache, day, *, seed=20260912, draws=256, residual_days=28,
118             residual_scale=1., mode='bootstrap'):
119     """ 同一抽样日索引同时作用于 144 点负荷和 144 点光伏。
120
121     重复抽到同一天时合并变量块，概率为抽中次数/draws，与未压缩模型严格等价。
122     仅按式 (25) 截断负值，不叠加历史支撑范围遮罩。
123     """
124     if mode not in ('bootstrap', 'enumerate', 'point'):
125         raise ValueError('Unknown scenario rule')
126     if day < 0 or day >= len(dataset.dates) or draws < 1 or residual_days < 1 or residual_scale < 0:
127         raise ValueError('Invalid scenario configuration')
128     pl, pg = cache['load_kw'][day], cache['pv_kw'][day]
129     audit = {'**cache['metadata'][day], 'scenario_method': mode, 'seed': seed,

```

```

130         'draw_count': draws if mode == 'bootstrap' and day else 0,
131         'residual_scale': residual_scale}
132     if day == 0 or mode == 'point':
133         ids, counts = np.array([], dtype=int), np.array([1])
134         raw_l, raw_g = pl[None, :], pg[None, :]
135         prob = np.array([1.])
136     else:
137         pool = np.arange(max(0, day-residual_days), day)
138         if mode == 'bootstrap':
139             rng = np.random.default_rng(np.random.SeedSequence([seed, day]))
140             ids, counts = np.unique(rng.choice(pool, size=draws, replace=True), return_counts=True)
141         else:
142             ids, counts = pool, np.ones(len(pool), dtype=int)
143             raw_l = pl + residual_scale*(dataset.load_kw[ids]-cache['load_kw'][ids])
144             raw_g = pg + residual_scale*(dataset.pv_kw[ids]-cache['pv_kw'][ids])
145             prob = counts/counts.sum()
146         audit.update(residual_dates=[dataset.dates[j].isoformat() for j in ids],
147                     residual_counts=counts.tolist(), scenario_count=len(prob),
148                     negative_load_clipped=int((raw_l < 0).sum()), negative_pv_clipped=int((raw_g < 0).sum()))
149     l, g = np.maximum(raw_l, 0)/6, np.maximum(raw_g, 0)/6
150     if not np.isfinite(l).all() or not np.isfinite(g).all():
151         raise ValueError('Forecast cache is incomplete')
152     return l, g, prob, audit

```

microgrid_core.py

```

1 """ 微网通用计算模块。所有调度流量为 kWh，功率上限为交流母线侧 kW。
2
3 函数不解释 Excel 时间标签、不推断初末条件、不选择预测模型。
4 第一问首尾相等由 solve_q1 显式实现；固定计划回放不重置跨日状态。
5 """
6 from dataclasses import asdict, dataclass
7 from pathlib import Path
8 import csv
9 from datetime import date
10 import math
11 import time
12
13 import numpy as np
14 from scipy.optimize import Bounds, LinearConstraint, linprog, milp
15 from scipy.sparse import lil_matrix
16
17
18 @dataclass(frozen=True)
19 class Battery:
20     eta_charge: float
21     eta_discharge: float
22     min_kwh: float = 1200.0
23     max_kwh: float = 10800.0
24     max_power_kw: float = 5000.0
25     step_hours: float = 1/6
26
27     def __post_init__(self):
28         if not all(math.isfinite(v) for v in asdict(self).values()):
29             raise ValueError("Battery parameters must be finite")
30         if not 0 < self.eta_charge <= 1 or not 0 < self.eta_discharge <= 1:
31             raise ValueError("Charge and discharge efficiencies must be in (0, 1]")
32         if not 0 <= self.min_kwh < self.max_kwh:
33             raise ValueError("Invalid battery capacity bounds")
34         if self.max_power_kw <= 0 or self.step_hours <= 0:
35             raise ValueError("Power and time step must be positive")
36
37     @property
38     def limit_kwh(self):
39         return self.max_power_kw * self.step_hours
40
41     def validate_initial(self, value):
42         # 跨日独立重算可产生 1e-10 级边界误差，允许数值容差，不重置或截断储量。
43         if not math.isfinite(value) or not self.min_kwh-1e-7 <= value <= self.max_kwh+1e-7:
44             raise ValueError("Initial stored energy is outside the permitted range")
45

```

```

46
47 def vector(values, name, length=None, nonnegative=True):
48     a = np.asarray(values, dtype=float)
49     if a.ndim != 1 or len(a) == 0 or (length is not None and len(a) != length):
50         raise ValueError(f"{name}: expected a nonempty vector of length {length}")
51     if not np.isfinite(a).all() or (nonnegative and (a < 0).any()):
52         raise ValueError(f"{name}: invalid numeric values")
53     return a
54
55
56 def dispatch_checks(charge, discharge, state, battery, initial):
57     """ 独立按物理公式计算残差, 输出单位为 kWh。"""
58     c = vector(charge, "charge", nonnegative=False)
59     d = vector(discharge, "discharge", len(c), nonnegative=False)
60     s = vector(state, "state", len(c)+1)
61     return {
62         "nonnegative_dispatch_violation_kwh": float(max(0, -c.min(), -d.min())),
63         "state_transition_residual_kwh":
64             float(np.abs(np.diff(s)-battery.eta_charge*c+d/battery.eta_discharge).max()),
65         "state_bound_violation_kwh": float(max(0, battery.min_kwh-s.min(), s.max()-battery.max_kwh)),
66         "power_bound_violation_kwh": float(max(0, c.max()-battery.limit_kwh, d.max()-battery.limit_kwh)),
67         "simultaneous_charge_discharge_kwh": float(np.minimum(np.maximum(c,0), np.maximum(d,0)).sum()),
68         "initial_state_difference_kwh": float(abs(s[0]-initial)),
69     }
70
71 def require_checks(checks, tolerance=1e-5):
72     failed = {k:v for k,v in checks.items() if not math.isfinite(v) or v > tolerance}
73     if failed:
74         raise RuntimeError(f"Independent verification failed: {failed}")
75
76
77 def solve_q1(price, load_kwh, pv_kwh, *, battery, initial_kwh,
78             method="milp", time_limit_s=60., mip_rel_gap=1e-9):
79     """ 求解第一问, 首尾储量均为 initial_kwh; LP 同时充放电另行标记。
80
81 失败或超时不会作为已验证最优结果返回。输入的日期/时间含义由调用方提供。
82 """
83     p = vector(price, "price")
84     n = len(p)
85     load, pv = vector(load_kwh, "load", n), vector(pv_kwh, "pv", n)
86     if (p <= 0).any():
87         raise ValueError("This implementation requires strictly positive prices")
88     if method not in ("lp", "milp"):
89         raise ValueError("method must be lp or milp")
90     if not math.isfinite(time_limit_s) or time_limit_s <= 0 or not 0 <= mip_rel_gap < 1:
91         raise ValueError("Invalid solver limits")
92     battery.validate_initial(initial_kwh)
93     integer = method == "milp"
94     size = (6 if integer else 5)*n+1
95     q,c,d,w = [np.arange(k*n,(k+1)*n) for k in range(4)]
96     s,z = np.arange(4*n,5*n+1), np.arange(5*n+1,6*n+1)
97     objective = np.zeros(size)
98     objective[q] = p
99     low, high = np.zeros(size), np.full(size,np.inf)
100     high[c] = high[d] = battery.limit_kwh
101     high[w] = pv
102     low[s], high[s] = battery.min_kwh,battery.max_kwh
103     low[s[0]] = high[s[0]] = initial_kwh
104     low[s[-1]] = high[s[-1]] = initial_kwh
105     eq = lil_matrix((2*n,size))
106     rhs = np.r_[load-pv,np.zeros(n)]
107     for t in range(n):
108         eq[t,q[t]],eq[t,c[t]],eq[t,d[t]],eq[t,w[t]]=1,-1,1,-1
109         eq[n+t,s[t+1]],eq[n+t,s[t]]=1,-1
110         eq[n+t,c[t]],eq[n+t,d[t]]=-battery.eta_charge,1/battery.eta_discharge
111     started = time.perf_counter()
112     if integer:
113         high[z] = 1
114         integrality = np.zeros(size)
115         integrality[z] = 1
116         ub = lil_matrix((2*n,size))
117         for t in range(n):

```

```

118         ub[t,c[t]],ub[t,z[t]]=1,-battery.limit_kwh
119         ub[n+t,d[t]],ub[n+t,z[t]]=1,battery.limit_kwh
120     result = milp(objective,integrality=integrality,bounds=Bounds(low,high),
121                  constraints=[LinearConstraint(eq.tocsr(),rhs,rhs),
122                               LinearConstraint(ub.tocsr(),-np.inf,np.r_[np.zeros(n),np.full(n,battery.limit_kwh)
123                               ↪ h)]),
124                  options={"time_limit":time_limit_s,"mip_rel_gap":mip_rel_gap})
125 else:
126     result = linprog(objective,A_eq=eq.tocsr(),b_eq=rhs,bounds=list(zip(low,high)),
127                     method="highs",options={"time_limit":time_limit_s})
128 runtime = time.perf_counter()-started
129 if not result.success:
130     raise RuntimeError(f"{method} not solved to requested tolerance; status={result.status}: {result.message}")
131 Q,C,D,W,S = (result.x[i] for i in (q,c,d,w,s))
132 # 浮点求解器可能返回极小负数, 保留原值并按容差检查, 不修改调度来通过检查。
133 negative = float(max(0,-result.x.min()))
134 if negative > 1e-5:
135     raise RuntimeError("Solver returned negative energy outside tolerance")
136 checks = {
137     "bus_balance_residual_kwh":float(np.abs(Q+pv+D-load-C-W).max()),
138     "state_transition_residual_kwh":float(np.abs(np.diff(S)-battery.eta_charge*C+D/battery.eta_discharge).max())
139     ↪ ),
140     "state_bound_violation_kwh":float(max(0,battery.min_kwh-S.min(),S.max()-battery.max_kwh)),
141     "power_bound_violation_kwh":float(max(0,C.max()-battery.limit_kwh,D.max()-battery.limit_kwh)),
142     "pv_spill_bound_violation_kwh":float(max(0,(W-pv).max(),-W.min())),
143     "nonnegative_violation_kwh":negative,
144     "initial_state_difference_kwh":float(abs(S[0]-initial_kwh)),
145     "terminal_state_difference_kwh":float(abs(S[-1]-initial_kwh)),
146     "objective_difference_yuan":float(abs(p@Q-result.fun)),
147 }
148 require_checks(checks)
149 simultaneous = float(np.minimum(np.maximum(C,0),np.maximum(D,0)).sum())
150 if integer:
151     require_checks({"simultaneous_charge_discharge_kwh":simultaneous})
152 return {
153     "method":method,"status":int(result.status),"message":str(result.message),
154     "runtime_s":runtime,"mip_gap":float(result.mip_gap) if integer else None,
155     "cost_yuan":float(p@Q),"purchase_kwh":float(Q.sum()),
156     "simultaneous_charge_discharge_kwh":simultaneous,
157     "physically_feasible":simultaneous <= 1e-5,"checks":checks,
158     "plan":{name:values.tolist() for name,values in zip(("Q","C","D","W","S"),(Q,C,D,W,S))},
159 }
160
161 def simulate_fixed_plan(price, load_kwh, pv_kwh, purchase_kwh, charge_kwh, discharge_kwh,
162                        *, battery, initial_kwh, emergency_multiplier):
163     """ 第二问固定电池计划的逐时回放; 实际曲线仅用于结算, 不改变日前计划。
164     总富余可能来自光伏、计划购电或电池, 不能全部称作已付费浪费。
165     调用方必须把返回的终值作为下一日初值; 本函数没有每日重置逻辑。
166     """
167     p = vector(price,"price")
168     n = len(p)
169     load,pv = vector(load_kwh,"load",n),vector(pv_kwh,"pv",n)
170     q,c,d = [vector(v,name,n,nonnegative=False) for v,name in zip(
171         (purchase_kwh,charge_kwh,discharge_kwh),("Q","C","D"))]
172     require_checks({"negative_plan_violation_kwh":float(max(0,-q.min(),-c.min(),-d.min()))})
173     battery.validate_initial(initial_kwh)
174     if (p <= 0).any() or not math.isfinite(emergency_multiplier) or emergency_multiplier <= 0:
175         raise ValueError("Prices and emergency multiplier must be positive")
176     s = np.r_[initial_kwh,initial_kwh+np.cumsum(battery.eta_charge*c-d/battery.eta_discharge)]
177     deficit = load+c-q-pv-d
178     e,u = np.maximum(deficit,0),np.maximum(-deficit,0)
179     checks = dispatch_checks(c,d,s,battery,initial_kwh)
180     checks["bus_balance_residual_kwh"] = float(np.abs(q+e+pv+d-load-c-u).max())
181     require_checks(checks)
182     planned,emergency = float(p@q),float(emergency_multiplier*p@e)
183     return {"policy":"fixed_day_ahead_battery","planned_cost_yuan":planned,
184           "emergency_cost_yuan":emergency,"cash_cost_yuan":planned+emergency,
185           "terminal_kwh":float(s[-1]),"checks":checks,
186           "S":s.tolist(),"E":e.tolist(),"U_total_unused":u.tolist()}
187
188

```

```

189 def load_baseline_csv(path, *, step_hours):
190     """ 读取已核查派生 CSV, 严格保留原始记录次序, 不推断区间起点。"""
191     with Path(path).open(encoding="utf-8-sig",newline="") as stream:
192         rows = list(csv.DictReader(stream))
193         if len(rows) != 144 or [int(r["point_index"]) for r in rows] != list(range(1,145)):
194             raise ValueError("Baseline must contain ordered point_index 1..144")
195         if not math.isfinite(step_hours) or step_hours <= 0:
196             raise ValueError("Invalid conversion time step")
197         p = vector([r["price_yuan_per_kwh"] for r in rows], "price", 144)
198         load = vector([r["load_kw"] for r in rows], "load_kw", 144)*step_hours
199         pv = vector([r["pv_forecast_kw"] for r in rows], "pv_kw", 144)*step_hours
200         return {"point_index": [int(r["point_index"]) for r in rows],
201                 "source_time_label": [r["time_label"] for r in rows],
202                 "price": p, "load_kwh": load, "pv_kwh": pv}
203
204
205 def history_before_date(path, decision_date):
206     """ 按源记录所属日期严格截取日前历史, 尚不解释 10 分钟标签。
207
208     返回值可供后续预测模块使用; 不进行补值、标准化或全年参数估计。
209     """
210     cutoff = date.fromisoformat(decision_date)
211     rows = []
212     keys = set()
213     with Path(path).open(encoding="utf-8-sig",newline="") as stream:
214         for row in csv.DictReader(stream):
215             day = date.fromisoformat(row["date"])
216             if day >= cutoff:
217                 continue
218             index = int(row["point_index"])
219             key = (day, index)
220             if not 1 <= index <= 144 or key in keys:
221                 raise ValueError("Invalid or duplicate historical date/point key")
222             keys.add(key)
223             values = vector([row["load_kw"], row["pv_actual_kw"]], "historical power", 2)
224             rows.append({**row, "point_index": index, "load_kw": float(values[0]), "pv_actual_kw": float(values[1])})
225     rows.sort(key=lambda r: (r["date"], r["point_index"]))
226     return rows

```

stochastic_dispatch.py

```

1 """ 第二问固定日前充放电策略: 整体 MILP 与多割 Benders, 共用显式输入。
2
3 场景生成不在本模块内, 调用者负责提供无未来泄漏的负荷/光伏场景。
4 terminal_kwh 和 terminal_cost_per_kwh 必须明确传入, 不暗定日末规则。
5 """
6 import math
7 import time
8
9 import numpy as np
10 from scipy.optimize import Bounds, LinearConstraint, milp
11 from scipy.sparse import lil_matrix
12
13 from microgrid_core import vector, require_checks, simulate_fixed_plan
14
15
16 def indices(n):
17     return (np.arange(n), np.arange(n, 2*n), np.arange(2*n, 3*n),
18             np.arange(3*n, 4*n+1), np.arange(4*n+1, 5*n+1))
19
20
21 def solve_q2_fixed(price, load_scenarios_kwh, pv_scenarios_kwh, probabilities, *,
22                   battery, initial_kwh, terminal_kwh, terminal_cost_per_kwh,
23                   method="extensive", emergency_multiplier=5., time_limit_s=60.,
24                   max_iterations=200, absolute_gap=1e-6, relative_gap=1e-8):
25     """ 返回期望现金费、含终端项的目标值、计划、逐场景回放及界。
26
27     terminal_cost_per_kwh 是  $\Phi(s)$  系数 *s 的带符号系数 (元/内部 kWh);
28     储能具有正价值 v 时传入 -v。它是规划项, 不计入实际电费。
29     terminal_kwh=None 表示不额外固定日末储量, 否则施加该等式。
30     """

```

```

31 p=vector(price,"price")
32 n=len(p)
33 load=np.asarray(load_scenarios_kwh,dtype=float)
34 pv=np.asarray(pv_scenarios_kwh,dtype=float)
35 if load.ndim != 2 or load.shape[0] == 0 or load.shape[1] != n or pv.shape != load.shape:
36     raise ValueError("Scenario inputs must both have shape (positive scenario count, period count)")
37 if not np.isfinite(load).all() or not np.isfinite(pv).all() or (load<0).any() or (pv<0).any():
38     raise ValueError("Scenario load/PV must be finite nonnegative energy")
39 k=load.shape[0]
40 prob=vector(probabilities,"probabilities",k)
41 if (prob<=0).any() or abs(float(prob.sum())-1)>1e-10:
42     raise ValueError("Scenario probabilities must be strictly positive and sum to 1; no silent normalization")
43 if (p<=0).any() or not math.isfinite(emergency_multiplier) or emergency_multiplier<=0:
44     raise ValueError("Positive prices and emergency multiplier are required")
45 if not math.isfinite(terminal_cost_per_kwh):
46     raise ValueError("A finite signed terminal coefficient is required")
47 if method not in ("extensive","benders"):
48     raise ValueError("method must be extensive or benders")
49 if not math.isfinite(time_limit_s) or time_limit_s<=0 or not isinstance(max_iterations,int) or max_iterations<1:
50     raise ValueError("Invalid solve limits")
51 if not math.isfinite(absolute_gap) or absolute_gap<=0 or not math.isfinite(relative_gap) or not
↪ 0<=relative_gap<1:
52     raise ValueError("Invalid gap tolerance")
53 battery.validate_initial(initial_kwh)
54 if terminal_kwh is not None:
55     battery.validate_initial(terminal_kwh)
56 net=load-pv
57 q,c,d,s,z=indices(n)
58 base=5*n+1
59 started=time.perf_counter()
60 deadline=started+time_limit_s
61
62 def build_common(size):
63     objective=np.zeros(size)
64     objective[q]=p
65     objective[s[-1]]=terminal_cost_per_kwh
66     low,high=np.zeros(size),np.full(size,np.inf)
67     low[s],high[s]=battery.min_kwh,battery.max_kwh
68     low[s[0]]=high[s[0]]=initial_kwh
69     if terminal_kwh is not None:
70         low[s[-1]]=high[s[-1]]=terminal_kwh
71     high[z]=1
72     integ=np.zeros(size)
73     integ[z]=1
74     a=lil_matrix((3*n,size))
75     for t in range(n):
76         a[t,s[t+1]],a[t,s[t]],a[t,c[t]],a[t,d[t]]=1,-1,-battery.eta_charge,1/battery.eta_discharge
77         a[n+t,c[t]],a[n+t,z[t]]=1,-battery.limit_kwh
78         a[2*n+t,d[t]],a[2*n+t,z[t]]=1,battery.limit_kwh
79     constraints=[LinearConstraint(a.tocsr(),np.r_[np.zeros(n),np.full(2*n,-np.inf)],
80                                     np.r_[np.zeros(2*n),np.full(n,battery.limit_kwh)])]
81     return objective,Bounds(low,high),integ,constraints
82
83 def solve(objective,bounds,integ,constraints):
84     remaining=deadline-time.perf_counter()
85     if remaining<=0:
86         raise RuntimeError("Total solve time limit reached; no certified result returned")
87     r=milp(objective,integrality=integ,bounds=bounds,constraints=constraints,
88           options={"time_limit":remaining,"mip_rel_gap":1e-10})
89     if not r.success:
90         raise RuntimeError(f"MILP status={r.status}: {r.message}")
91     return r
92
93 def assess(x):
94     cash=float(p@x[q])+float(prob@np.maximum(net-x[q]+x[c]-x[d],0)*(emergency_multiplier*p))
95     return cash+terminal_cost_per_kwh*float(x[s[-1]])
96
97 history=[]
98 cut_count=0
99 if method=="extensive":
100     size=base+2*k*n
101     objective,bounds,integ,constraints=build_common(size)
102     objective[base:base+k*n]=(prob[:,None]*emergency_multiplier*p).ravel()

```

```

103     a=lil_matrix((k*n,size))
104     for omega in range(k):
105         for t in range(n):
106             row=omega*n+t
107             a[row,q[t]],a[row,c[t]],a[row,d[t]]=1,-1,1
108             a[row,base+row],a[row,base+k*n+row]=1,-1
109     constraints.append(LinearConstraint(a.tocsr(),net.ravel(),net.ravel()))
110     r=solve(objective,bounds,integ,constraints)
111     best=r.x
112     lower=float(r.mip_dual_bound)
113     upper=assess(best)
114     require_checks({"objective_recompute_difference_yuan":abs(upper-float(r.fun))})
115     history=[{"iteration":1,"lower_bound_yuan":lower,"upper_bound_yuan":upper,"cuts":0}]
116 else:
117     size=base+k
118     objective,bounds,integ,constraints=build_common(size)
119     objective[base:]=prob
120     upper=np.inf
121     lower=-np.inf
122     best=None
123     for iteration in range(1,max_iterations+1):
124         r=solve(objective,bounds,integ,constraints)
125         lower=max(lower,float(r.mip_dual_bound))
126         value=assess(r.x)
127         if value<upper:
128             upper,best=value,r.x.copy()
129         if upper<lower-1e-5:
130             raise RuntimeError("Benders upper bound is below its lower bound")
131         history.append({"iteration":iteration,"lower_bound_yuan":lower,"upper_bound_yuan":upper,"cuts":cut_count,
132             ↪ t})
132         if upper-lower<=absolute_gap+relative_gap*abs(upper):
133             break
134         deficit=net-r.x[q]+r.x[c]-r.x[d]
135         recourse=np.maximum(deficit,0)@(emergency_multiplier*p)
136         cut_rows,cut_rhs=[],[]
137         for omega in range(k):
138             if recourse[omega]-r.x[base+omega]<=absolute_gap/10:
139                 continue
140             lam=np.where(deficit[omega]>0,emergency_multiplier*p,0.)
141             row=np.zeros(size)
142             row[q],row[c],row[d]=lam,-lam,lam
143             row[base+omega]=1
144             cut_rows.append(row)
145             cut_rhs.append(float(lam@net[omega]))
146         if not cut_rows:
147             raise RuntimeError("Benders stalled before the requested bound tolerance")
148         constraints.append(LinearConstraint(np.asarray(cut_rows),np.asarray(cut_rhs),np.inf))
149         cut_count+=len(cut_rows)
150     else:
151         raise RuntimeError("Benders iteration limit reached before convergence")
152 if upper-lower>absolute_gap+relative_gap*abs(upper):
153     raise RuntimeError("Requested total objective gap was not attained")
154 # 回放模块重建状态和逐时能量平衡，不复用优化矩阵。
155 replays=[simulate_fixed_plan(p,load[i],pv[i],best[q],best[c],best[d],battery=battery,
156     ↪ initial_kwh=initial_kwh,emergency_multiplier=emergency_multiplier) for i in
157     ↪ range(k)]
158 reconstructed=np.asarray(replays[0][:"S"])
159 checks={"planned_vs_reconstructed_state_kwh":float(np.abs(best[s]-reconstructed).max())}
160 if terminal_kwh is not None:
161     checks["terminal_state_difference_kwh"]=abs(float(reconstructed[-1])-terminal_kwh)
162     expected_cash=float(prob@np.array([r["cash_cost_yuan"] for r in replays]))
163     terminal_cost=float(terminal_cost_per_kwh*reconstructed[-1])
164     checks["objective_recompute_difference_yuan"]=abs(expected_cash+terminal_cost-upper)
165     require_checks(checks)
166 return {"method":method,"policy":"fixed_day_ahead_battery","status":"solved_to_requested_tolerance",
167     ↪ "objective_yuan":upper,"expected_cash_cost_yuan":expected_cash,"terminal_cost_yuan":terminal_cost,
168     ↪ "lower_bound_yuan":lower,"absolute_gap_yuan":max(0,upper-lower),"runtime_s":time.perf_counter()-started,
169     ↪ "iterations":len(history),"cuts":cut_count,"bounds_history":history,"checks":checks,
170     ↪ "plan":{name:best[idx].tolist() for name,idx in zip(("Q","C","D","S","Z"),(q,c,d,s,z))},
171     ↪ "scenario_replays":replays}

```

prepare_local_inputs.py

```

1 """Copy the four existing Q1/Q2 input files locally after checking their hashes."""
2 from pathlib import Path
3 import argparse
4 import hashlib
5 import json
6 import shutil
7
8
9 def main():
10     root = Path(__file__).resolve().parents[1]
11     parser = argparse.ArgumentParser(description=__doc__)
12     parser.add_argument('--raw-dir', type=Path, default=root.parents[1] / '03_ 计算工程/data/raw/附件')
13     args = parser.parse_args()
14     expected = json.loads((root / 'configs/raw_inputs_sha256.json').read_text(encoding='utf-8'))
15     jobs = []
16     for relative, digest in expected.items():
17         source = args.raw_dir / relative
18         target = root / 'data/raw/附件' / relative
19         if not source.is_file():
20             raise FileNotFoundError(f'Missing input: {source}; use --raw-dir to specify the attachment directory')
21         if hashlib.sha256(source.read_bytes()).hexdigest() != digest:
22             raise ValueError(f'Input hash differs from the validated dataset: {source}')
23         if target.exists() and hashlib.sha256(target.read_bytes()).hexdigest() != digest:
24             raise ValueError(f'Refusing to overwrite a different local file: {target}')
25         jobs.append((source, target, digest))
26     for source, target, digest in jobs:
27         target.parent.mkdir(parents=True, exist_ok=True)
28         if not target.exists():
29             shutil.copy2(source, target)
30         assert hashlib.sha256(target.read_bytes()).hexdigest() == digest
31     print(f'Prepared {len(jobs)} local input files; raw inputs are ignored by Git.')
32
33
34 if __name__ == '__main__':
35     main()

```

prepare_v2.py

```

1 from pathlib import Path
2 import sys
3 import csv
4 import json
5 import numpy as np
6 from forecasting import read_dataset, interval_label
7 from reporting_v2 import build_payload
8 from run_v2 import write_json, write_csv
9
10
11 def main(run):
12     root = Path(__file__).resolve().parents[1]
13     dataset = read_dataset(root / 'data/processed')
14     config = json.loads((run / 'run_manifest.json').read_text(encoding='utf-8'))['config']
15     selection = json.loads((run / 'selection.json').read_text(encoding='utf-8'))
16     summaries = json.loads((run / 'experiment_summaries.json').read_text(encoding='utf-8'))
17     note = f"version2: 零点固定Q/C/D; 1月验证选择{selection['chosen_model']}; 256次整日配残差抽样; 1月连续预热; 年末储量6000。"
18     payload = build_payload(run, dataset, config, selection['main_experiment'], note)
19     write_json(run / 'workbook_payload.json', payload)
20     out = run / 'deliverables'
21     out.mkdir(exist_ok=True)
22     write_csv(out / '第二问每日费用与电量.csv', payload['q2']['daily'])
23     write_json(out / '指定日期结果.json', payload['q2']['selected_dates'])
24     # NpzFile 按键访问会重新解压整块数组; 导出逐点表前只加载一次。
25     with np.load(run / selection['main_experiment'] / 'trajectories.npz') as saved:
26         a = {key: saved[key] for key in saved.files}
27         trajectory = []
28         for i, day in enumerate(a['day_indices']):
29             for t in range(144):
30                 trajectory.append({'date': dataset.dates[day].isoformat(), 'interval': interval_label(t),
31                                     'price_yuan_per_kwh': float(dataset.prices[t]), 'load_kw': float(dataset.load_kw[day, t]),
32                                     'pv_kw': float(dataset.pv_kw[day, t]), **{k + '_kwh': float(a[k][i, t]) for k in ('Q', 'C', 'D', 'E', 'U')},
33                                     'S_start_kwh': float(a['S'][i, t]), 'S_end_kwh': float(a['S'][i, t+1])})

```

```

34 write_csv(out/'第二问完整 10 分钟轨迹.csv',trajectory)
35 fields = ['name','model','total_cash_cost_yuan','total_planned_cost_yuan','total_emergency_cost_yuan',
36           'total_emergency_purchase_kwh','total_unused_kwh','initial_kwh','terminal_kwh','day_end_at_min_count']
37 write_csv(out/'第二问全部方案对比.csv',[{k:s[k] for k in fields} for s in summaries])
38 write_csv(out/'一月验证与选择.csv',[{model:s['model'],'terminal_multiplier':s['terminal_multiplier'],
39   'cash_cost_yuan':s['total_cash_cost_yuan'],**{k:v for k,v in s['forecast_metrics'].items() if
   ↳ k!='daylight_definition'}} for s in selection['validation']])
40 write_csv(out/'全年预测精度.csv',[{model:s['model'],**{k:v for k,v in s['forecast_metrics'].items() if
   ↳ k!='daylight_definition'}}
41   for s in summaries if s['name'].startswith('candidate_')])
42 lines = ['# 题目指定时段与指定日期结果','', '电量单位为 kWh, 费用单位为元。所有时段采用区间起点 00:00 至末端 24:00。','',
43   '## 第一问','', ' | 时段 | 计划购电量 |','|---|---:|']
44 lines += [f'| {t} | {v:.6f} |' for t,v in payload['q1']['requested_intervals']]
45 lines += ['',f" 全天购电量 {payload['q1']['purchase_kwh']:.6f}, 费用 {payload['q1']['cost_yuan']:.6f}。",'']
46 for item in payload['q2']['selected_dates']:
47   lines += [f"## {item['date']}","", ' | 时段 | 计划购电量 |','|---|---:|']
48   lines += [f'| {t} | {v:.6f} |' for t,v in item['purchases_at_requested_intervals']]
49   lines += ['',f" 四小时时段 | 充电量 | 放电量 |','|---|---:|---:|']
50   lines += [f'| {r[0]} | {r[1]:.6f} | {r[2]:.6f} |' for r in item['battery_four_hour']]
51   lines += ['',f" 初始储量 {item['initial_kwh']:.6f}, 末端储量 {item['terminal_kwh']:.6f}, 实际费用
   ↳ {item['actual_total_cost_yuan']:.6f}。",'',
52     '| 紧急购电区间 | 电量 |','|---|---:|']
53   lines += [f'| {x['interval']} | {x['energy_kwh']:.6f} |' for x in item['emergency_segments']]
54   lines += ['']
55 (out/'题目指定时段与日期表格.md').write_text('\n'.join(lines),encoding='utf-8')
56 print(json.dumps({'main':selection['main_experiment'],'dates':len(payload['q2']['purchases']),
57   'emergency_segments':len(payload['q2']['emergency'])},ensure_ascii=False))
58
59
60 if __name__ == '__main__':
61   main(Path(sys.argv[1]).resolve())

```

precompute_v2.py

```

1 """ 并行预计算互相独立的后段实验；仅把完整结果原子发布给主程序恢复使用。
2
3 避免两个进程同时写同一实验：工作目录与主运行分开；主目录已经存在时
4 保留独立工作副本，不写入该目录。数值输入和函数与主入口完全相同。
5 """
6 from pathlib import Path
7 import sys,json,hashlib
8 import numpy as np
9 from run_v2 import experiment, load_cache, ROOT
10 from forecasting import read_dataset
11 from forecasting_v2 import MODELS
12 from microgrid_core import Battery
13
14
15 def main(run, group):
16   manifest=json.loads((run/'run_manifest.json').read_text(encoding='utf-8'))
17   for name, expected in manifest['code'].items():
18     if hashlib.sha256((ROOT/'src'/name).read_bytes()).hexdigest()!=expected:
19       raise ValueError('Parallel worker numeric code does not match running snapshot')
20   config=manifest['config']
21   selected=json.loads((run/'selection.json').read_text(encoding='utf-8'))
22   data=read_dataset(ROOT/'data/processed')
23   if data.sources!=manifest['inputs']:raise ValueError('Input mismatch')
24   model=selected['chosen_model'];mult=selected['chosen_terminal_multiplier']
25   initial=selected['formal_common_initial_kwh']
26   cache=load_cache(data,model,config,run/'forecasts',365)
27   stage=(ROOT/'outputs'/(run.name+'_parallel')/group).resolve()
28   days=list(range(31,365))
29   if group=='seed_eta':
30     jobs=[('seed_20260913',{ 'seed':20260913}),('seed_20260914',{ 'seed':20260914})]
31   elif group=='sensitivity':
32     jobs=[('residual_0_8',{ 'residual_scale':.8}),('residual_1_2',{ 'residual_scale':1.2}),
33     ('terminal_0_8',{ 'terminal_multiplier':mult*.8}),('terminal_1_2',{ 'terminal_multiplier':mult*1.2})]
34   else:raise ValueError('Unknown bounded worker group')
35   workspace=ROOT.parent.resolve()
36
37   def publish(name):

```

```

38     source=(stage/name).resolve();target=(run/name).resolve()
39     if not source.is_relative_to(workspace) or not target.is_relative_to(workspace):
40         raise ValueError('Move must stay inside version2 workspace')
41     if target.exists():
42         print(f'{name}: primary already exists; retained separate worker copy',flush=True)
43         return
44     try:source.rename(target)
45     except FileExistsError:
46         print(f'{name}: primary started concurrently; worker copy retained',flush=True)
47         return
48     print(f'{name}: completed worker result published',flush=True)
49
50 for name, kw in jobs:
51     if (run/name/'summary.json').exists():continue
52     args={'terminal_multiplier':mult,**kw}
53     experiment(data,cache,config,stage/name,days,initial,final_target=6000.,**args)
54     publish(name)
55 if group=='seed_eta':
56     alternate=Battery(np.sqrt(.9),np.sqrt(.9))
57     p1=load_cache(data,MODELS[0],config,run/'forecasts',365)
58     warm=experiment(data,p1,config,stage/'warmup_roundtrip90',range(31),6000.,battery=alternate)
59     publish('warmup_roundtrip90')
60     experiment(data,cache,config,stage/'roundtrip90',days,warm['terminal_kwh'],
61               battery=alternate,terminal_multiplier=mult,final_target=6000.)
62     publish('roundtrip90')
63
64
65 if __name__=='__main__':main(Path(sys.argv[1]).resolve(),sys.argv[2])

```

run_v2.py

```

1 """ 前两问 version2: PDF 候选比较、1 月预热与选择、334 天封闭回测。
2
3 运行: python src/run_v2.py --run v2_20260912 [--pilot]
4 同名完整实验可恢复: 输入、配置或数值源码变化时拒绝混用旧结果。
5 """
6 from dataclasses import asdict
7 from datetime import date
8 from pathlib import Path
9 import argparse
10 import csv
11 import hashlib
12 import json
13 import platform
14 import shutil
15 import time
16 import traceback
17 import numpy as np
18 import scipy
19 import sklearn
20 import statsmodels
21 from forecasting import read_dataset, no_storage_purchase, make_scenarios
22 from forecasting_v2 import MODELS, build_cache, scenarios
23 from microgrid_core import Battery, solve_q1, simulate_fixed_plan, require_checks
24 from stochastic_dispatch import solve_q2_fixed
25
26 ROOT = Path(__file__).resolve().parents[1]
27
28
29 def validate_config(config):
30     fixed = {'time_interpretation': 'source_label_is_interval_end',
31             'january_policy': 'common_predeclared_P1_continuous_dispatch_from_Jan1',
32             'january_first_day_forecast': 'attachment1_prior_as_per_PDF_3.9.4',
33             'initial_january_kwh': 6000, 'year_end_kwh': 6000,
34             'selection_dates': ['2025-01-21','2025-01-31'], 'selection_terminal_kwh': 6000,
35             'selection_primary_metric': 'actual_cash_cost_yuan',
36             'terminal_rule': 'Phi(s)=-multiplier*min(price)/eta_charge*s',
37             'output_start': '2025-02-01', 'output_end': '2025-12-31',
38             'candidates': list(MODELS), 'main_solver': 'extensive'}
39     for k, v in fixed.items():
40         if config.get(k) != v:

```

```

41         raise ValueError(f'Unsupported modeling rule: {k}; update implementation explicitly')
42     Battery(config['eta_charge'], config['eta_discharge'])
43     for key in ('bootstrap_draws', 'residual_days', 'seed'):
44         if not isinstance(config[key], int) or config[key] < 1:
45             raise ValueError(f'Invalid {key}')
46     if not config['terminal_multipliers'] or any(not np.isfinite(x) or x < 0 for x in
↳ config['terminal_multipliers']):
47         raise ValueError('Invalid terminal candidates')
48
49
50 def write_json(path, obj):
51     Path(path).write_text(json.dumps(obj, ensure_ascii=False, indent=2, allow_nan=False), encoding='utf-8')
52
53
54 def write_csv(path, rows):
55     if not rows:
56         return
57     with Path(path).open('w', encoding='utf-8-sig', newline='') as stream:
58         writer = csv.DictWriter(stream, fieldnames=list(rows[0]))
59         writer.writeheader()
60         writer.writerows(rows)
61
62
63 def forecast_metrics(dataset, cache, days):
64     el = cache['load_kw'][days]-dataset.load_kw[days]
65     eg = cache['pv_kw'][days]-dataset.pv_kw[days]
66     active = dataset.pv_kw[days] > 0
67     return {'load_mae_kw': float(abs(el).mean()), 'load_rmse_kw': float(np.sqrt(np.mean(el**2))),
68           'pv_mae_kw': float(abs(eg).mean()), 'pv_rmse_kw': float(np.sqrt(np.mean(eg**2))),
69           'pv_daylight_mae_kw': float(abs(eg[active]).mean()),
70           'pv_daylight_rmse_kw': float(np.sqrt(np.mean(eg[active]**2))),
71           'daylight_definition': 'actual PV > 0, used only in retrospective scoring'}
72
73
74 def experiment(dataset, cache, config, folder, days, initial, *, terminal_multiplier=1.,
75               residual_scale=1., seed=None, mode='bootstrap', battery=None,
76               final_target=None, no_storage=False):
77     folder = Path(folder)
78     battery = battery or Battery(config['eta_charge'], config['eta_discharge'])
79     seed = config['seed'] if seed is None else seed
80     coefficient = -terminal_multiplier*float(dataset.prices.min())/battery.eta_charge
81     specification = {'model': cache['model'], 'days': list(days), 'initial_kwh': initial,
82                     'terminal_multiplier': terminal_multiplier, 'terminal_coefficient': coefficient,
83                     'residual_scale': residual_scale, 'seed': seed, 'mode': mode,
84                     'no_storage': no_storage, 'battery': asdict(battery), 'final_target': final_target}
85     if (folder/'summary.json').exists():
86         if json.loads((folder/'specification.json').read_text(encoding='utf-8')) != specification:
87             raise ValueError(f'Resume specification mismatch: {folder}')
88         return json.loads((folder/'summary.json').read_text(encoding='utf-8'))
89     folder.mkdir(parents=True, exist_ok=True)
90     write_json(folder/'specification.json', specification)
91     state = initial
92     rows, audits = [], []
93     trajectories = {k: [] for k in ('Q', 'C', 'D', 'S', 'E', 'U')}
94     maxima = {}
95     start = time.perf_counter()
96     with (folder/'progress.jsonl').open('w', encoding='utf-8') as log:
97         for count, day in enumerate(days, 1):
98             if mode == 'v1_enumerate':
99                 l, g, prob, info = make_scenarios(dataset, {'**cache', 'minimum_history_days': 7}, day,
100                 residual_days=config['residual_days'], residual_scale=residual_scale)
101             else:
102                 l, g, prob, info = scenarios(dataset, cache, day, seed=seed,
103                 draws=config['bootstrap_draws'], residual_days=config['residual_days'],
104                 residual_scale=residual_scale, mode=mode)
105             terminal = final_target if count == len(days) else None
106             if no_storage:
107                 q = no_storage_purchase(l, g, prob)
108                 c, d = np.zeros(144), np.zeros(144)
109                 plan_s = np.full(145, state)
110                 gap, runtime = 0., 0.
111                 objective = float(dataset.prices@q + prob@np.maximum(1-g-q, 0)@(5*dataset.prices)) +
↳ coefficient*state

```

```

112         checks = {}
113     else:
114         result = solve_q2_fixed(dataset.prices, l, g, prob, battery=battery,
115                                 initial_kwh=state, terminal_kwh=terminal, terminal_cost_per_kwh=coefficient,
116                                 method=config['main_solver'], time_limit_s=config['time_limit_s'],
117                                 absolute_gap=config['absolute_gap_yuan'], relative_gap=config['relative_gap'])
118         q, c, d, plan_s = [np.asarray(result['plan'][k]) for k in ('Q', 'C', 'D', 'S')]
119         gap, runtime = result['absolute_gap_yuan'], result['runtime_s']
120         objective, checks = result['objective_yuan'], dict(result['checks'])
121     actual = simulate_fixed_plan(dataset.prices, dataset.load_kw[day]/6, dataset.pv_kw[day]/6,
122                                 q, c, d, battery=battery, initial_kwh=state, emergency_multiplier=5.)
123     s, e, u = [np.asarray(actual[k]) for k in ('S', 'E', 'U_total_unused')]
124     checks.update(actual['checks'])
125     checks['plan_state_reconstruction_kwh'] = float(np.max(abs(s-plan_s)))
126     if trajectories['S']:
127         checks['cross_day_kwh'] = float(abs(s[0]-trajectories['S'][-1][-1]))
128     if terminal is not None and not no_storage:
129         checks['final_target_kwh'] = float(abs(s[-1]-terminal))
130     require_checks(checks)
131     for k, v in checks.items():
132         maxima[k] = max(maxima.get(k, 0.), v)
133     row = {'date': dataset.dates[day].isoformat(),
134           'planned_cost_yuan': actual['planned_cost_yuan'],
135           'emergency_cost_yuan': actual['emergency_cost_yuan'],
136           'cash_cost_yuan': actual['cash_cost_yuan'], 'planned_purchase_kwh': float(q.sum()),
137           'emergency_purchase_kwh': float(e.sum()), 'total_unused_kwh': float(u.sum()),
138           'charge_kwh': float(c.sum()), 'discharge_kwh': float(d.sum()),
139           'initial_kwh': float(s[0]), 'terminal_kwh': float(s[-1]),
140           'solver_gap_yuan': gap, 'solver_runtime_s': runtime,
141           'planning_objective_yuan': objective, 'scenario_count': len(prob)}
142     rows.append(row)
143     audits.append(info)
144     for k, v in zip(trajectories, (q, c, d, s, e, u)):
145         trajectories[k].append(v)
146     state = float(s[-1])
147     log.write(json.dumps(row, ensure_ascii=False)+'\n')
148     log.flush()
149     if count == 1 or count % 60 == 0 or count == len(days):
150         print(f'{folder.name}: {count}/{len(days)}, elapsed {time.perf_counter()-start:.1f}s', flush=True)
151     np.savez_compressed(folder/'trajectories.npz', day_indices=np.array(days),
152                        **{k: np.asarray(v) for k, v in trajectories.items()})
153     write_csv(folder/'daily_results.csv', rows)
154     write_json(folder/'information_audit.json', audits)
155     summary = {'name': folder.name, 'model': cache['model'], 'days': len(days),
156              'initial_kwh': initial, 'terminal_kwh': state,
157              'total_cash_cost_yuan': sum(r['cash_cost_yuan'] for r in rows),
158              'total_planned_cost_yuan': sum(r['planned_cost_yuan'] for r in rows),
159              'total_emergency_cost_yuan': sum(r['emergency_cost_yuan'] for r in rows),
160              'total_emergency_purchase_kwh': sum(r['emergency_purchase_kwh'] for r in rows),
161              'total_unused_kwh': sum(r['total_unused_kwh'] for r in rows),
162              'day_end_at_min_count': sum(abs(r['terminal_kwh']-battery.min_kwh) < 1e-5 for r in rows),
163              'max_solver_gap_yuan': max(r['solver_gap_yuan'] for r in rows),
164              'checks_maxima': maxima, 'forecast_metrics': forecast_metrics(dataset, cache, list(days)),
165              'runtime_s': time.perf_counter()-start}
166     write_json(folder/'summary.json', summary)
167     return summary
168
169
170 def load_cache(dataset, model, config, folder, stop):
171     file = folder/f'{model}.npz'
172     if file.exists():
173         arr = np.load(file)
174         return {'model': model, 'load_kw': arr['load_kw'], 'pv_kw': arr['pv_kw'],
175               'metadata': {int(k): v for k, v in
176                           ↪ json.loads(file.with_suffix('.json').read_text(encoding='utf-8')).items()}}
177     result = build_cache(dataset, model, config['seed'], stop=stop, progress=lambda s: print(s, flush=True))
178     np.savez_compressed(file, load_kw=result['load_kw'], pv_kw=result['pv_kw'])
179     write_json(file.with_suffix('.json'), result['metadata'])
180     return result
181
182 def main():
183     parser = argparse.ArgumentParser(description=__doc__)

```

```

184 parser.add_argument('--run', default='v2_20260912')
185 parser.add_argument('--pilot', action='store_true')
186 args = parser.parse_args()
187 config = json.loads((ROOT/'configs/q1_q2_v2.json').read_text(encoding='utf-8'))
188 validate_config(config)
189 dataset = read_dataset(ROOT/'data/processed')
190 out = ROOT/'outputs'/args.run
191 out.mkdir(parents=True, exist_ok=True)
192 numeric_files = ['run_v2.py', 'forecasting_v2.py', 'forecasting.py', 'microgrid_core.py',
↪ 'stochastic_dispatch.py']
193 manifest = {'config': config, 'pilot': args.pilot, 'inputs': dataset.sources,
194             'code': {n: hashlib.sha256((ROOT/'src'/n).read_bytes()).hexdigest() for n in numeric_files}}
195 if (out/'run_manifest.json').exists():
196     if json.loads(out/'run_manifest.json').read_text(encoding='utf-8') != manifest:
197         raise ValueError('Numeric source/config/input changed: create a new run id')
198 else:
199     write_json(out/'run_manifest.json', manifest)
200     (out/'code_snapshot').mkdir(exist_ok=True)
201     for n in numeric_files:
202         shutil.copy2(ROOT/'src'/n, out/'code_snapshot'/n)
203 write_json(out/'environment.json', {'python': platform.python_version(), 'numpy': np.__version__,
204                                     'scipy': scipy.__version__, 'sklearn': sklearn.__version__, 'statsmodels': statsmodels.__version__})
205 battery = Battery(config['eta_charge'], config['eta_discharge'])
206 q1 = {m: solve_q1(dataset.prices, dataset.baseline_load_kw/6, dataset.baseline_pv_kw/6,
207                  battery=battery, initial_kwh=6000, method=m) for m in ('lp', 'milp')}
208 q1['roundtrip90_sensitivity'] = solve_q1(dataset.prices, dataset.baseline_load_kw/6,
209                                         dataset.baseline_pv_kw/6, battery=Battery(np.sqrt(.9), np.sqrt(.9)), initial_kwh=6000, method='milp')
210 write_json(out/'q1.json', q1)
211 (out/'forecasts').mkdir(exist_ok=True)
212 caches = {m: load_cache(dataset, m, config, out/'forecasts', 34 if args.pilot else 365) for m in MODELS}
213 warm = experiment(dataset, caches[MODELS[0]], config, out/'warmup', range(31), 6000.)
214 warm_s = np.load(out/'warmup/trajectories.npz')['S']
215 validation = []
216 for model in MODELS:
217     for mult in config['terminal_multipliers']:
218         summary = experiment(dataset, caches[model], config,
219                             out/'validation'/f'{model}_terminal_{mult}', range(20, 31), float(warm_s[19, -1]),
220                             terminal_multiplier=mult, final_target=6000.)
221         validation.append({'model': model, 'terminal_multiplier': mult, **summary})
222 chosen = min(validation, key=lambda r: r['total_cash_cost_yuan'])
223 model, mult = chosen['model'], chosen['terminal_multiplier']
224 selection = {'chosen_model': model, 'chosen_terminal_multiplier': mult,
225             'chosen_validation_cost_yuan': chosen['total_cash_cost_yuan'],
226             'selection_information_cutoff': '2025-01-31T24:00',
227             'selection_start': '2025-01-21', 'selection_end': '2025-01-31',
228             'validation_common_initial_kwh': float(warm_s[19, -1]), 'validation_common_terminal_kwh': 6000.,
229             'formal_common_initial_kwh': warm['terminal_kwh'], 'validation': validation,
230             'main_experiment': f'candidate_{model}',
231             'scope': 'historical validation among preregistered candidates, not a global optimum guarantee'}
232 write_json(out/'selection.json', selection)
233 print('JANUARY SELECTION '+json.dumps({k: selection[k] for k in
↪ ('chosen_model', 'chosen_terminal_multiplier', 'formal_common_initial_kwh')}), ensure_ascii=False, flush=True)
234 days = list(range(31, 34 if args.pilot else 365))
235 final = None if args.pilot else 6000.
236 initial = warm['terminal_kwh']
237 specifications = []
238 for m in MODELS:
239     best = min((r for r in validation if r['model'] == m), key=lambda r: r['total_cash_cost_yuan'])
240     specifications.append((f'candidate_{m}', m, {'terminal_multiplier': best['terminal_multiplier']}))
241 specifications += [(('point_selected', model, {'mode': 'point', 'terminal_multiplier': mult}),
242                    ('no_storage_selected', model, {'no_storage': True, 'terminal_multiplier': mult}),
243                    ('enumerate_selected', model, {'mode': 'enumerate', 'terminal_multiplier': mult}),
244                    ('v1_mechanism_matched_initial', MODELS[0], {'mode': 'v1_enumerate', 'terminal_multiplier': 1.}),
245                    ('terminal_0_8', model, {'terminal_multiplier': mult*.8}),
246                    ('terminal_1_2', model, {'terminal_multiplier': mult*1.2}),
247                    ('residual_0_8', model, {'terminal_multiplier': mult, 'residual_scale': .8}),
248                    ('residual_1_2', model, {'terminal_multiplier': mult, 'residual_scale': 1.2}),
249                    ('seed_20260913', model, {'terminal_multiplier': mult, 'seed': 20260913}),
250                    ('seed_20260914', model, {'terminal_multiplier': mult, 'seed': 20260914})]
251 summaries = []
252 for name, m, kwargs in specifications:
253     # 无储能基线不消耗起始库存，也不要求补足无法充入的年度储量。
254     # 与储能策略的库存差额须单列，禁止隐含为同一终端条件。

```

```

255         end = None if kwargs.get('no_storage') else final
256         summary = experiment(dataset, caches[m], config, out/name, days, initial, final_target=end, **kwargs)
257         summaries.append(summary)
258         write_json(out/'experiment_summaries.json', summaries)
259         alternate = Battery(np.sqrt(.9), np.sqrt(.9))
260         aw = experiment(dataset, caches[MODELS[0]], config, out/'warmup_roundtrip90', range(31), 6000.,
↪         battery=alternate)
261         summaries.append(experiment(dataset, caches[model], config, out/'roundtrip90', days, aw['terminal_kwh'],
262         battery=alternate, terminal_multiplier=mult, final_target=final))
263         write_json(out/'experiment_summaries.json', summaries)
264         write_json(out/'completion.json', {'status': 'numerical_results_complete', 'formal_days': len(days),
265         'experiments': len(summaries), 'validation_experiments': len(validation), 'pilot': args.pilot,
266         'exports_verified': False})
267         print('COMPLETE '+str(out), flush=True)
268
269
270 if __name__ == '__main__':
271     try:
272         main()
273     except Exception:
274         traceback.print_exc()
275         raise

```

run_pipeline.py

```

1 """ 前两问完整基线：因果预测、场景规划、连续回放及可追踪数值输出。
2
3 本程序输出用于 Excel 导出及论文图表的结果包，Excel 由独立模板导出器生成。
4 """
5 from dataclasses import asdict
6 from datetime import datetime, timezone, date
7 from pathlib import Path
8 import argparse
9 import csv
10 import hashlib
11 import json
12 import platform
13 import time
14 import traceback
15 import uuid
16
17 import numpy as np
18 import scipy
19
20 from forecasting import read_dataset, causal_forecast_cache, make_scenarios, no_storage_purchase, interval_label
21 from microgrid_core import Battery, solve_q1, simulate_fixed_plan, require_checks
22 from stochastic_dispatch import solve_q2_fixed
23
24 ROOT=Path(__file__).resolve().parents[1]
25
26
27 def validate_config(config):
28     expected={"time_interpretation":"source_label_is_interval_end","power_integration":"piecewise_constant_10_minut_
↪     es",
29             "january_battery_policy":"idle_no_self_discharge","q2_policy":"fixed_day_ahead_battery",
30             "terminal_value_rule":"minus_min_daily_price_divided_by_charge_efficiency_times_stored_kwh",
31             "main_forecast":"weighted28",
32             "scenario_method":"enumerate_recent_paired_out_of_sample_daily_residuals_equal_probability",
33             "template_time_labels":"correct_to_00_00_through_24_00_in_output_copies"}
34     if any(config[k]!=v for k,v in expected.items()):
35         raise ValueError("An unsupported modeling rule was requested; do not silently ignore it")
36     if config["forecast_candidates"]!=["mean7","weighted28"]:
37         raise ValueError("This baseline entry point has two predefined forecast candidates")
38     if config["initial_february_kwh"]!=6000:
39         raise ValueError("The chosen January idle policy implies February initial storage 6000")
40     if config["output_start"]!="2025-02-01" or config["output_end"]!="2025-12-31":
41         raise ValueError("This full-year entry point expects the contest output date range")
42
43
44 def write_json(path,data):
45     Path(path).write_text(json.dumps(data,ensure_ascii=False,indent=2,allow_nan=False),encoding="utf-8")

```

```

46
47
48 def write_csv(path,rows):
49     with Path(path).open("w",encoding="utf-8-sig",newline="") as stream:
50         if rows:
51             writer=csv.DictWriter(stream,fieldnames=list(rows[0]))
52             writer.writeheader();writer.writerows(rows)
53
54
55 def scenario_experiment(dataset,cache,config,battery,days,*,name,mode="stochastic",
56                        terminal_multiplier=1.,residual_scale=1.,residual_days=28,folder):
57     folder.mkdir(parents=True,exist_ok=False)
58     state=config["initial_february_kwh"]
59     minimum_price=float(dataset.prices.min())
60     terminal_coefficient=-terminal_multiplier*minimum_price/battery.eta_charge
61     records=[]
62     audits=[]
63     trajectories={key:[] for key in ("Q","C","D","S","E","U")}
64     maximum_checks={}
65     started=time.perf_counter()
66     experiment_config={"name":name,"model":cache["model"],"mode":mode,
67                       "terminal_coefficient_yuan_per_internal_kwh":terminal_coefficient,
68                       "residual_days":residual_days,"residual_scale":residual_scale,
69                       "start":dataset.dates[0].isoformat(),"end":dataset.dates[days[-1]].isoformat(),
70                       "initial_kwh":state,"year_end_kwh":config["year_end_kwh"],
71                       "date_count":len(days),"main_solver":config["main_solver"]}
72     write_json(folder/"config.json",experiment_config)
73     log=(folder/"run.log").open("w",encoding="utf-8")
74     try:
75         for count,day in enumerate(days,1):
76             day_date=dataset.dates[day]
77             L,G,prob,audit=make_scenarios(dataset,cache,day,residual_days=residual_days,
78                                         residual_scale=residual_scale,deterministic=mode=="point")
79             terminal=config["year_end_kwh"] if day_date==date(2025,12,31) else None
80             if mode=="no_storage":
81                 q=no_storage_purchase(L,G,prob)
82                 c,d=np.zeros(144),np.zeros(144)
83                 plan_state=np.full(145,state)
84                 solver={"runtime_s":0.,"absolute_gap_yuan":0.,"iterations":0,"checks":{}},
85                 "objective_yuan":float(dataset.prices@q+prob@(np.maximum(L-G-q,0)*(5*dataset.prices))+termi
86                 ↪ nal_coefficient*state)}
87             else:
88                 solver=solve_q2_fixed(dataset.prices,L,G,prob,battery=battery,initial_kwh=state,
89                                     terminal_kwh=terminal,terminal_cost_per_kwh=terminal_coefficient,
90                                     method=config["main_solver"],time_limit_s=config["solver_time_limit_s"],
91                                     absolute_gap=config["solver_absolute_gap_yuan"],relative_gap=config["solver_re
92                                     ↪ lative_gap"])
93                 q,c,d,plan_state=[np.asarray(solver[key]) for key in ("Q","C","D","S")]
94                 actual=simulate_fixed_plan(dataset.prices,dataset.load_kw[day]/6,dataset.pv_kw[day]/6,q,c,d,
95                                     battery=battery,initial_kwh=state,emergency_multiplier=5)
96                 s=np.asarray(actual["S"])
97                 checks={**actual["checks"],**solver["checks"],"plan_vs_actual_state_kwh":float(np.abs(s-plan_state).max)
98                 ↪ ()}
99                 if trajectories["S"]:
100                     checks["cross_day_difference_kwh"]=float(abs(s[0]-trajectories["S"][-1][-1]))
101                 if terminal is not None:
102                     checks["year_end_difference_kwh"]=float(abs(s[-1]-terminal))
103                 require_checks(checks)
104                 for key,value in checks.items():
105                     maximum_checks[key]=max(maximum_checks.get(key,0),value)
106                 record={"date":day_date.isoformat(),"planned_cost_yuan":actual["planned_cost_yuan"],
107                       "emergency_cost_yuan":actual["emergency_cost_yuan"],"cash_cost_yuan":actual["cash_cost_yuan"],
108                       "planned_purchase_kwh":float(q.sum()),"emergency_purchase_kwh":float(np.sum(actual["E"])),
109                       "total_unused_kwh":float(np.sum(actual["U_total_unused"])),
110                       "charge_kwh":float(c.sum()),"discharge_kwh":float(d.sum()),
111                       "initial_kwh":float(s[0]),"terminal_kwh":float(s[-1]),
112                       "load_mae_kw":float(np.abs(cache["load_kw"][day]-dataset.load_kw[day]).mean()),
113                       "pv_mae_kw":float(np.abs(cache["pv_kw"][day]-dataset.pv_kw[day]).mean()),
114                       "solver_runtime_s":solver["runtime_s"],"solver_absolute_gap_yuan":solver["absolute_gap_yuan"],
115                       "scenario_count":len(prob),"planning_objective_yuan":solver["objective_yuan"]}
116                 records.append(record);audits.append(audit)
117                 for key,value in zip(trajectories,(q,c,d,s,actual["E"],actual["U_total_unused"])):
118                     trajectories[key].append(value)

```

```

116         state=actual["terminal_kwh"]
117         log.write(json.dumps({"date":record["date"],"cash_cost_yuan":record["cash_cost_yuan"],
118                               "state_end_kwh":state,"maximum_residual":max(checks.values())},ensure_ascii=False,
                               ↪ ))+"\n")
119     log.flush()
120     if count==1 or count%30==0 or count==len(days):
121         print(f"{name}: {count}/{len(days)} days; last={day_date};
122               ↪ elapsed={time.perf_counter()-started:.1f}s",flush=True)
123     arrays={key:np.asarray(value) for key,value in trajectories.items()}
124     arrays["day_indices"]=np.array(days)
125     np.savez_compressed(folder/"trajectories.npz",**arrays)
126     write_csv(folder/"daily_results.csv",records)
127     write_json(folder/"information_audit.json",audits)
128     prediction_load=cache["load_kw"][days]-dataset.load_kw[days]
129     prediction_pv=cache["pv_kw"][days]-dataset.pv_kw[days]
130     active=dataset.pv_kw[days]>0
131     summary={"name":name,"days":len(days),"start":records[0]["date"],"end":records[-1]["date"],
132             "total_cash_cost_yuan":sum(r["cash_cost_yuan"] for r in records),
133             "total_planned_cost_yuan":sum(r["planned_cost_yuan"] for r in records),
134             "total_emergency_cost_yuan":sum(r["emergency_cost_yuan"] for r in records),
135             "total_planned_purchase_kwh":sum(r["planned_purchase_kwh"] for r in records),
136             "total_emergency_purchase_kwh":sum(r["emergency_purchase_kwh"] for r in records),
137             "total_unused_kwh":sum(r["total_unused_kwh"] for r in records),
138             "initial_kwh":records[0]["initial_kwh"],"terminal_kwh":state,
139             "forecast_load_mae_kw":float(np.abs(prediction_load).mean()),
140             "forecast_load_rmse_kw":float(np.sqrt((prediction_load**2).mean())),
141             "forecast_pv_mae_kw":float(np.abs(prediction_pv).mean()),
142             "forecast_pv_rmse_kw":float(np.sqrt((prediction_pv**2).mean())),
143             "forecast_pv_active_mae_kw":float(np.abs(prediction_pv[active]).mean()),
144             "runtime_s":time.perf_counter()-started,"checks_maxima":maximum_checks,
145             "max_solver_absolute_gap_yuan":max(r["solver_absolute_gap_yuan"] for r in records)}
146     write_json(folder/"summary.json",summary)
147     return summary
148 except Exception as exc:
149     write_json(folder/"failure.json",{"date":dataset.dates[day].isoformat(),"error":str(exc),"completed_days":1
150           ↪ en(records)})
151     write_csv(folder/"partial_daily_results.csv",records)
152     log.write(traceback.format_exc())
153     raise
154 finally:
155     log.close()
156
157 def main():
158     parser=argparse.ArgumentParser(description=__doc__)
159     parser.add_argument("--config",type=Path,default=ROOT/"configs/q1_q2_baseline.json")
160     parser.add_argument("--pilot-days",type=int,default=0)
161     parser.add_argument("--sensitivity",action="store_true")
162     args=parser.parse_args()
163     config=json.loads(args.config.read_text(encoding="utf-8"))
164     validate_config(config)
165     if args.pilot_days<0:
166         raise ValueError("pilot-days must be nonnegative")
167     dataset=read_dataset(ROOT/"data/processed")
168     battery=Battery(config["eta_charge"],config["eta_discharge"])
169     cache={m:causal_forecast_cache(dataset,m,config["minimum_history_days"]) for m in config["forecast_candidates"]}
170     days=list(range(31,365))
171     if args.pilot_days:
172         days=days[:args.pilot_days]
173     run_id=("pilot" if args.pilot_days else
174           ↪ "full")+ "_" +datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S")+"_"+uuid.uuid4().hex[:8]
175     out=ROOT/"outputs/runs"/run_id
176     out.mkdir(parents=True,exist_ok=False)
177     metadata={"run_id":run_id,"config":config,"battery":asdict(battery),"python":platform.python_version(),
178             "scipy":scipy.__version__,"input_sha256":dataset.sources,
179             "code_sha256":{p.name:hashlib.sha256(p.read_bytes()).hexdigest() for p in (ROOT/"src").glob("*.py")}}
180     write_json(out/"config.json",metadata)
181     write_json(out/"time_mapping.json",[{ "point_index":t+1,"source_time_label":interval_label(t).split("-")[1],
182           ↪ "interval":interval_label(t),"minute_start":t*10,"minute_end":(t+1)*10} for
183           ↪ t in range(144)])
184
185 try:
186     q1={m:solve_q1(dataset.prices,dataset.baseline_load_kw/6,dataset.baseline_pv_kw/6,battery=battery,
187           ↪ initial_kwh=6000,method=m) for m in ("lp","milp")}

```

```

184     write_json(out/"q1.json",q1)
185     np.savez_compressed(out/"forecasts.npz",**{f"{m}_{kind}":c[kind+"_kw"] for m,c in cache.items() for kind in
↪ ("load","pv")})
186     specifications=[("main_weighted28","weighted28","stochastic",1.,1.),
187                     ("comparison_mean7","mean7","stochastic",1.,1.),
188                     ("comparison_point","weighted28","point",1.,1.),
189                     ("comparison_no_storage","weighted28","no_storage",1.,1.))
190     if args.sensitivity:
191         specifications.extend([("sensitivity_terminal_0_8","weighted28","stochastic",.8,1.),
192                                ("sensitivity_terminal_1_2","weighted28","stochastic",1.2,1.),
193                                ("sensitivity_residual_0_8","weighted28","stochastic",1.,.8),
194                                ("sensitivity_residual_1_2","weighted28","stochastic",1.,1.2)])
195     summaries=[]
196     for name,model,mode,terminal_multiplier,residual_scale in specifications:
197         summaries.append(scenario_experiment(dataset,cache[model],config,battery,days,name=name,mode=mode,
198                                             terminal_multiplier=terminal_multiplier*config["terminal_value_mul_
↪ tiplier"],
199                                             residual_scale=residual_scale*config["residual_scale"],
200                                             residual_days=config["residual_days"],folder=out/name))
201     write_json(out/"experiment_summaries.json",summaries)
202     write_json(out/"completion.json",{"status":"numerical_pipeline_completed","date_count":len(days),
203                                     "experiments":len(summaries),"formal_workbooks_exported":False})
204     print(json.dumps({"output_directory":str(out),"experiments":[{"name":s["name"],"cost":s["total_cash_cost_yu_
↪ an"]} for s in summaries]},ensure_ascii=False,indent=2))
205 except Exception as exc:
206     write_json(out/"failure.json",{"error":str(exc),"traceback":traceback.format_exc()})
207     raise
208
209
210 if __name__=="__main__":
211     main()

```

audit_v2.py

```

1  """ 从保存结果按公式独立复算，不调用优化矩阵或场景生成函数。"""
2  from pathlib import Path
3  from datetime import date
4  import sys
5  import csv
6  import json
7  import numpy as np
8  from forecasting import read_dataset
9  from run_v2 import write_json
10
11
12 def audit(run):
13     root = Path(__file__).resolve().parents[1]
14     dataset = read_dataset(root/'data/processed')
15     config = json.loads((run/'run_manifest.json').read_text(encoding='utf-8'))['config']
16     selection = json.loads((run/'selection.json').read_text(encoding='utf-8'))
17     records = []
18     for folder in sorted(p.parent for p in run.rglob('summary.json')):
19         spec = json.loads((folder/'specification.json').read_text(encoding='utf-8'))
20         summary = json.loads((folder/'summary.json').read_text(encoding='utf-8'))
21         info = json.loads((folder/'information_audit.json').read_text(encoding='utf-8'))
22         a = np.load(folder/'trajectories.npz')
23         days = a['day_indices'].tolist()
24         if days != spec['days']:
25             raise ValueError('Saved day indices differ')
26         n = len(days)
27         q,c,d,s,e,u = [a[k] for k in ('Q','C','D','S','E','U')]
28         if any(x.shape != (n,144) for x in (q,c,d,e,u)) or s.shape != (n,145):
29             raise ValueError(f'Bad shape: {folder}')
30         if not all(np.isfinite(x).all() for x in (q,c,d,s,e,u)):
31             raise ValueError('Nonfinite results')
32         b = spec['battery']
33         net = dataset.load_kw[days]/6 + c - q - dataset.pv_kw[days]/6 - d
34         checks = {
35             'flow_nonnegative_kwh': max(0., -min(x.min() for x in (q,c,d,e,u))),
36             'state_equation_kwh': float(np.max(abs(np.diff(s,axis=1)-b['eta_charge']*c+d/b['eta_discharge']))),
37             'energy_balance_kwh': float(np.max(abs(q+e+dataset.pv_kw[days]/6+d-dataset.load_kw[days]/6-c-u))),

```

```

38     'emergency_replay_kwh': float(np.max(abs(e-np.maximum(net,0)))),
39     'unused_replay_kwh': float(np.max(abs(u-np.maximum(-net,0)))),
40     'simultaneous_per_day_kwh': float(np.max(np.minimum(np.maximum(c,0),np.maximum(d,0)).sum(axis=1))),
41     'state_bounds_kwh': max(0.,b['min_kwh']-s.min(),s.max()-b['max_kwh']),
42     'power_bounds_kwh': max(0.,c.max()-b['max_power_kw']/6,d.max()-b['max_power_kw']/6),
43     'cross_day_kwh': float(np.max(abs(s[1:,0]-s[:-1,-1]))) if n>1 else 0.,
44     'initial_kwh': float(abs(s[0,0]-spec['initial_kwh'])),
45     'cash_total_yuan': float(abs((q+5*e)@dataset.prices @ np.ones(n)-summary['total_cash_cost_yuan'])))
46 if spec['final_target'] is not None and not spec['no_storage']:
47     checks['final_kwh'] = float(abs(s[-1,-1]-spec['final_target']))
48 with np.load(run/'forecasts'/f"{spec['model']}.npz") as saved_forecasts:
49     forecasts = {key:saved_forecasts[key] for key in saved_forecasts.files}
50 with (folder/'daily_results.csv').open(encoding='utf-8-sig',newline='') as stream:
51     daily = list(csv.DictReader(stream))
52 objective_max = 0.
53 for i, (day, meta) in enumerate(zip(days,info)):
54     origin = dataset.dates[day]
55     if meta['origin'] != origin.isoformat() or (meta['training_end'] and
56         ↪ date.fromisoformat(meta['training_end']) >= origin):
57         raise ValueError('Training information leak')
58     if meta.get('residual_end') and date.fromisoformat(meta['residual_end']) >= origin:
59         raise ValueError('V1 residual information leak')
60     if meta.get('residual_dates'):
61         ids = [(date.fromisoformat(x)-dataset.dates[0]).days for x in meta['residual_dates']]
62         if any(j >= day for j in ids):
63             raise ValueError('Bootstrap information leak')
64         counts = np.asarray(meta['residual_counts'])
65         if spec['mode'] == 'bootstrap' and counts.sum() != config['bootstrap_draws']:
66             raise ValueError('Bootstrap count mismatch')
67         if spec['mode'] == 'bootstrap':
68             pool = np.arange(max(0,day-config['residual_days']),day)
69             sampled = np.random.default_rng(np.random.SeedSequence([spec['seed'],day])).choice(pool,config[
70                 ↪ 'bootstrap_draws'],replace=True)
71             expected_ids = np.unique(sampled, return_counts=True)
72             if not np.array_equal(ids, expected_ids) or not np.array_equal(counts, expected_counts):
73                 raise ValueError('Bootstrap seed reconstruction failed')
74         elif spec['mode'] == 'v1_enumerate':
75             ids = list(range(max(7,day-config['residual_days']),day))
76             counts = np.ones(len(ids))
77         else:
78             ids, counts = [], np.ones(1)
79         pl, pg = forecasts['load_kw'][day], forecasts['pv_kw'][day]
80         if ids:
81             l = np.maximum(pl+spec['residual_scale']*(dataset.load_kw[ids]-forecasts['load_kw'][ids]),0)/6
82             g = np.maximum(pg+spec['residual_scale']*(dataset.pv_kw[ids]-forecasts['pv_kw'][ids]),0)/6
83             if spec['mode'] == 'v1_enumerate':
84                 support = dataset.pv_kw[max(0,day-28):day].max(axis=0)>0
85                 g[:,~support] = 0
86         else:
87             l, g = np.maximum(pl[None,:],0)/6,np.maximum(pg[None,:],0)/6
88         expected = float(dataset.prices@q[i] +
89             ↪ (counts/counts.sum())@(np.maximum(l+c[i]-q[i]-g-d[i],0)@(5*dataset.prices))
90             + spec['terminal_coefficient']*s[i,-1])
91         objective_max = max(objective_max,abs(expected-float(daily[i]['planning_objective_yuan'])))
92     checks['scenario_objective_reconstruction_yuan'] = objective_max
93 if any(not np.isfinite(v) or v > 1e-5 for v in checks.values()):
94     raise ValueError(f'{folder.name}: {checks}')
95 records.append({'name':str(folder.relative_to(run)), 'days':n, 'checks':checks})
96 chosen = min(selection['validation'], key=lambda r:r['total_cash_cost_yuan'])
97 if chosen['model'] != selection['chosen_model'] or chosen['terminal_multiplier'] !=
98     ↪ selection['chosen_terminal_multiplier']:
99     raise ValueError('Selection does not match January validation')
100 report = {'audited_experiments':len(records), 'audited_days':sum(r['days'] for r in records),
101     'maximum_numerical_residual':max(v for r in records for v in r['checks'].values()),
102     'selection_from_january_only':True, 'records':records}
103 write_json(run/'independent_audit.json',report)
104 print(json.dumps({k:v for k,v in report.items() if k!='records'},ensure_ascii=False,indent=2))
105
106
107 if __name__ == '__main__':
108     audit(Path(sys.argv[1]).resolve())

```

finalize_v2.py

```
1 """ 检查交付前置条件, 保存最终源码及展示检查记录。"""
2 from pathlib import Path
3 import sys,json,hashlib,shutil
4 from run_v2 import write_json
5
6
7 def main(run):
8     root=Path(__file__).resolve().parents[1]
9     complete=json.loads((run/'completion.json').read_text(encoding='utf-8'))
10    audit=json.loads((run/'independent_audit.json').read_text(encoding='utf-8'))
11    workbook=json.loads((run/'workbook_qa/readonly_export_check.json').read_text(encoding='utf-8'))
12    figures=json.loads((run/'figure_checks.json').read_text(encoding='utf-8'))
13    if complete['formal_days']!=334 or complete['experiments']!=14 or audit['audited_experiments']!=25 or
14    ↪ audit['audited_days']!=4837:
15        raise ValueError('Not all preregistered experiments were audited')
16    if len(figures)!=4 or any(x['text_bounds']!= 'passed' or x['glyph_check']!= 'passed' for x in figures):
17        raise ValueError('Figure checks incomplete')
18    if any(not row['all_values_verified'] for key,row in workbook.items() if key.startswith('result')):
19        raise ValueError('Workbook checks incomplete')
20    for file in (run/'deliverables').glob('*.inspect.ndjson'):
21        target=run/'workbook_qa'/file.name
22        if target.exists():raise ValueError('Unexpected existing inspection target')
23        # 两端路径固定于本次 workspace 运行目录; 只移动工具产生的单个 QA 文件。
24        if not file.resolve().is_relative_to(root.parent.resolve()) or not
25        ↪ target.resolve().is_relative_to(root.parent.resolve()):
26            raise ValueError('QA move outside workspace')
27        file.rename(target)
28    for folder in ('src','tests','configs'):
29        destination=run/('final_'+folder)
30        destination.mkdir(exist_ok=True)
31        for source in (root/folder).iterdir():
32            if source.is_file():shutil.copy2(source,destination/source.name)
33    hashes={str(p.relative_to(root)):hashlib.sha256(p.read_bytes()).hexdigest()
34            for part in ('src','tests','configs') for p in (root/part).iterdir() if p.is_file()}
35    write_json(run/'final_code_sha256.json',hashes)
36    write_json(run/'presentation_review.json',{
37        'workbook_sheets_reviewed':['result1/计划购电量','result1/充放电量','result2/计划购电量','result2/充放电量','resu
38        ↪ lt2/紧急购电量'],
39        'extra_views':['first/last intervals','year-end totals','method/time notes'],
40        'scientific_pngs_reviewed':[x['name'] for x in figures],
41        'result':'legible; no clipped content or overlapping legends; negative zero cleaned in Excel only',
42        'export_revision':'result2 Excel zeros below 1e-9 normalized, all exported values checked within 1e-6',
43        'performance_fix':'trajectory/forecast NPZ arrays loaded once during export/audit, no mathematical change',
44        'parallel_execution':'worker terminal_1_2 also finished separately after primary started; separate duplicate
45        ↪ not used',
46        'main_solver_failures':{}}
47    complete['exports_verified']=True
48    complete['independent_audit_verified']=True
49    complete['figures_visually_reviewed']=True
50    write_json(run/'completion.json',complete)
51    print(json.dumps(complete,ensure_ascii=False,indent=2))
52
53 if __name__=='__main__':main(Path(sys.argv[1]).resolve())
```

report_v2.py

```
1 """ 以已经完成的数值输出生成可核查的模型落实说明和科研图表。"""
2 from pathlib import Path
3 import sys,json,csv
4 import numpy as np
5 from run_v2 import write_json
6 from forecasting import read_dataset
7 from scientific_style import apply_style, save_figure, step, time_axis, COLORS
8 import matplotlib.pyplot as plt
9
10 LABELS = {'P1_weighted28':'P1 加权历史均值','P2_fourier_arma':'P2 傅里叶 + ARIMA','P3_random_forest':'P3 随机森林'}
11
```

```

12
13 def daily_rows(folder):
14     with (folder/'daily_results.csv').open(encoding='utf-8-sig',newline='') as f:
15         return list(csv.DictReader(f))
16
17
18 def make_plots(run, selection, summaries, out):
19     apply_style()
20     main = summaries[selection['main_experiment']]
21     candidates = [summaries['candidate_'+m] for m in LABELS]
22     fig, axs = plt.subplots(1,2,figsize=(8.5,3.7))
23     fig.subplots_adjust(left=.09,right=.98,bottom=.26,top=.88,wspace=.3)
24     x = np.arange(3)
25     planned = np.array([s['total_planned_cost_yuan'] for s in candidates])/1e6
26     emergency = np.array([s['total_emergency_cost_yuan'] for s in candidates])/1e6
27     colors = ['#0072B2', '#E69F00', '#009E73']
28     bars = axs[0].bar(x,planned,color=colors,width=.58)
29     axs[0].bar(x,emergency,bottom=planned,color=COLORS['emergency'],width=.58,hatch='/',label='紧急购电')
30     for i,s in enumerate(candidates):
31         axs[0].text(i,planned[i]+emergency[i]+.15,f'{planned[i]+emergency[i]:.2f}',ha='center',fontsize=8)
32         if s['model']==selection['chosen_model']:
33             bars[i].set_edgecolor('#202630');bars[i].set_linewidth(1.8)
34     axs[0].set_xticks(x,['P1','P2','P3']);axs[0].set_ylabel('2-12 月实际费用 (百万元)')
35     axs[0].set_title('(a) 正常与紧急购电费用',loc='left');axs[0].legend(loc='upper right')
36     axs[0].set_ylim(0,max(planned+emergency)*1.2)
37     width=.24
38     for j,(key,label,col) in enumerate([('load_mae_kw','负荷 MAE','#0072B2'),('pv_mae_kw','光伏 MAE','#E69F00'),
39                                         ('pv_daylight_mae_kw','有光伏时段 MAE','#009E73')]):
40         axs[1].bar(x+(j-1)*width,[s['forecast_metrics'][key] for s in candidates],width,label=label,color=col)
41     axs[1].set_xticks(x,['P1','P2','P3']);axs[1].set_ylabel('功率误差 (kW)')
42     axs[1].set_title('(b) 预测精度',loc='left');axs[1].legend(loc='upper right',fontsize=7)
43     axs[1].set_ylim(0,max(s['forecast_metrics']['load_mae_kw'] for s in candidates)*1.45)
44     fig.text(.09,.10,'P1: 加权历史均值; P2: 傅里叶日周期 + ARIMA 残差; P3: 随机森林.',fontsize=8)
45     fig.text(.09,.045,'黑色边框标出 1 月验证选定的主方案; 全年结果仅用于事后评价。',fontsize=8,color=COLORS['muted'])
46     checks=[save_figure(fig,out,'01_ 预测方法与调度费用')]
47
48     fig,axs=plt.subplots(1,2,figsize=(8.5,3.5))
49     fig.subplots_adjust(left=.09,right=.98,bottom=.23,top=.88,wspace=.30)
50     for i,s in enumerate(candidates):
51         rows=daily_rows(run/s['name'])
52         for ax,key in zip(axs,['cash_cost_yuan','emergency_cost_yuan']):
53             monthly=[sum(float(r[key]) for r in rows if int(r['date'][:5:7])==m)/1e4 for m in range(2,13)]
54             ax.plot(range(2,13),monthly,color=colors[i],marker=['o','s','^'][i],markersize=3,label=LABELS[s['model']]
55                     ↪  ])
56             ax.set_xticks(range(2,13));ax.set_xlabel('月份');ax.set_ylabel('费用 (万元)')
57     axs[0].set_title('(a) 每月实际购电费用',loc='left');axs[1].set_title('(b) 每月紧急购电费用',loc='left')
58     handles,labels=axs[0].get_legend_handles_labels()
59     fig.legend(handles,labels,loc='lower center',bbox_to_anchor=(.5,.015),ncol=3,fontsize=8)
60     checks.append(save_figure(fig,out,'02_ 逐月费用与紧急购电'))
61
62     a=np.load(run/selection['main_experiment']/'trajectories.npz')
63     data=read_dataset(Path(__file__).resolve().parents[1]/'data/processed')
64     fig,axs=plt.subplots(4,2,figsize=(8.5,9.1))
65     fig.subplots_adjust(left=.10,right=.98,bottom=.12,top=.95,hspace=.66,wspace=.28)
66     for i,dt in enumerate(['2025-03-20','2025-06-21','2025-09-23','2025-12-21']):
67         day=next(j for j,d in enumerate(data.dates) if d.isoformat()==dt)
68         j=list(a['day_indices']).index(day)
69         step(axs[i,0],a['Q'][j]*6,color=COLORS['purchase'],label='计划购电')
70         step(axs[i,0],a['E'][j]*6,color=COLORS['emergency'],label='紧急购电',linewidth=1.)
71         step(axs[i,0],data.load_kw[day]-data.pv_kw[day],color=COLORS['load'],label='实际净负荷',linestyle='--',linewidth
72             ↪  idth=.85)
73         axs[i,0].set_ylabel('功率 (kW)');axs[i,0].set_title(dt+' 供需与购电',loc='left')
74         axs[i,1].plot(np.arange(145)/6,a['S'][j],color=COLORS['state'],label='储能')
75         axs[i,1].axhline(1200,color=COLORS['muted'],linestyle=':',linewidth=.7)
76         axs[i,1].axhline(10800,color=COLORS['muted'],linestyle=':',linewidth=.7)
77         axs[i,1].set_ylim(0,12000);axs[i,1].set_yticks([1200,6000,10800]);axs[i,1].set_ylabel('储能 (kWh)')
78         axs[i,1].set_title(dt+' 储能轨迹',loc='left')
79         for ax in axs[i]:time_axis(ax)
80     h,l=axs[0,0].get_legend_handles_labels();h2,l2=axs[0,1].get_legend_handles_labels()
81     fig.legend(h+h2,l+l2,loc='lower center',bbox_to_anchor=(.5,.025),ncol=4)
82     fig.text(.10,.008,'购电曲线由 10 分钟电量乘 6 转换为平均功率; 虚线净负荷包含当天实际值, 仅用于事后展示。',fontsize=7)
83     checks.append(save_figure(fig,out,'03_ 指定日期调度与储能'))

```

```

83
84 names=['terminal_0_8','terminal_1_2','residual_0_8','residual_1_2','seed_20260913','seed_20260914']
85 labels=['终端系数 -20%','终端系数 +20%','残差幅度 -20%','残差幅度 +20%','抽样种子 20260913','抽样种子 20260914']
86 fig,axs=plt.subplots(1,2,figsize=(8.5,3.8),sharey=True)
87 fig.subplots_adjust(left=.22,right=.97,bottom=.19,top=.88,wspace=.28)
88 for ax,key,title in zip(axs,['total_cash_cost_yuan','total_emergency_cost_yuan'],['(a) 实际总费用变化','(b)
↳ 紧急购电费用变化']):
89     values=[(summaries[n][key]/main[key]-1)*100 for n in names]
90     ax.barh(np.arange(6),values,color=['#56B4E9','#0072B2','#E69F00','#D55E00','#8C6BB1','#009E73'])
91     ax.axvline(0,color=COLORS['ink'],linewidth=.7);ax.set_yticks(np.arange(6),labels)
92     ax.set_xlabel('相对主方案变化 (%)');ax.set_title(title,loc='left')
93     axs[0].invert_yaxis()
94     fig.text(.22,.04,'预测模型、初末条件和结算规则固定；每次只改变表中所列因素。种子对照不用于重新选择方案。',fontsize=7)
95     checks.append(save_figure(fig,out,'04_ 参数与抽样稳定性'))
96     write_json(run/'figure_checks.json',checks)
97
98
99 def main(run):
100     selection=json.loads((run/'selection.json').read_text(encoding='utf-8'))
101     summaries={s['name']:s for s in json.loads((run/'experiment_summaries.json').read_text(encoding='utf-8'))}
102     audit=json.loads((run/'independent_audit.json').read_text(encoding='utf-8'))
103     q1=json.loads((run/'q1.json').read_text(encoding='utf-8'))
104     main=summaries[selection['main_experiment']]
105     with np.load(run/selection['main_experiment']/'trajectories.npz') as saved:
106         day_end_at_max=int(np.sum(np.abs(saved['S'][:,-1]-10800)<1e-5))
107     matched=summaries['v1_mechanism_matched_initial']
108     out=run/'deliverables'
109     out.mkdir(exist_ok=True)
110     make_plots(run,selection,summaries,out/'科研图表')
111     rows=['# 前两问 version2: 模型落实与求解报告','',
112          '本版以 model-q1-q2-revised.pdf 为模型依据，重点补齐第二问。原工程和原结果保留。第三问尚未纳入本次 version2。','',
113          '## 结果与适用范围','',
114          f" 第一问 MILP 费用为 **{q1['milp']['cost_yuan']:.6f} 元 **；LP 费用为 {q1['lp']['cost_yuan']:.6f} 元。",
115          f" 第二问采用1月验证选定的 **{LABELS[selection['chosen_model']]**，终端价值倍率为
116          ↳ **{selection['chosen_terminal_multiplier']}**.",
117          f"2025年2月1日至12月31日实际费用为 **{main['total_cash_cost_yuan']:.2f} 元**，其中正常购电
118          ↳ {main['total_planned_cost_yuan']:.2f} 元、紧急购电 {main['total_emergency_cost_yuan']:.2f} 元。",
119          f"紧急购电量 {main['total_emergency_purchase_kwh']:.2f} kWh，总未利用电量 {main['total_unused_kwh']:.2f} kWh。
120          ↳ ",
121          '这是给定信息、候选模型、场景和固定计划策略下的历史回溯结果。MILP 达到数值最优容差，不等于真实未知分布下的全局最优策略。','',
122          '## 模型与代码对应','',
123          '| PDF 内容 | version2 落实 | 代码 |','|---|---|---|',
124          '| S1.2、式 (3)-(5) 单位与效率 | kW 乘 1/6 得到 kWh；主口径  $\eta_c=\eta_d=0.9$  | forecasting_v2.py、microgrid_core.py |',
125          '| S2、式 (6)-(15) 第一问 | 复用 MILP 约束、光伏弃电上限、首尾 6000 及充放电互斥 | microgrid_core.solve_q1 |',
126          '| S2.6 LP 对照 | 同数据解 LP 并核合同时充放电；不预先假定等价 | microgrid_core.solve_q1 |',
127          '| S1.3 效率敏感性 |  $\eta_c=\eta_d=0.9$ ，前两问均重置 | run_v2.py |',
128          '| S3.2 P1/P2/P3 | 加权历史均值；傅里叶项 +ARIMA 误差；随机森林，分别预测负荷和光伏 | forecasting_v2.predict |',
129          '| S3.2.4 因果选择 | 1 月 21-31 日滚动起点评价，主指标为实际费用，同时报告 MAE/RMSE | run_v2.py |',
130          '| S3.3、式 (24)-(25) 场景 | 最近 28 日整日配对样本外残差，有放回抽样 256 次，负值截零 | forecasting_v2.scenarios |',
131          '| S3.4 第一阶段共享 | Q/C/D/S/Z 在 0 点固定，全部场景共同使用；E/U 随场景变化 | stochastic_dispatch.solve_q2_fixed |',
132          '| S3.8、式 (27) 终端价值 | 选用 PDF 允许的线性价值， $\Phi(s)=-m \cdot \min(p)/\eta_c \cdot s$  | run_v2.experiment |',
133          '| S3.9、式 (28)-(38) 物理约束 | 145 个状态点、容量/功率边界、互斥、跨日衔接；年末 6000 | stochastic_dispatch.py |',
134          '| S3.9.4 一月预热 | 1 月 1 日 6000，1 月固定使用预先指定的 P1 连续调度；首日采用 PDF 允许的附件 1 先验 | run_v2.py |',
135          '| S3.10 分解结构 | 主计算使用整体MILP；原Benders实现及等价性测试保留 | stochastic_dispatch.py、test_dispatch.py |',
136          '| S3.11、式 (46)-(48) 真实结算 | 用附件2实际值重置E/U，现金费用只含pq+5pE | microgrid_core.simulate_fixed_plan
137          ↳ |','',
138          'PDF允许的48小时时域与线性终端价值是替代方案，本版选择后者；S3.13实时电池响应是扩展，本版按已确定的零点固定计划实现。
139          ↳ 不会把这些可选分支写成已完成实验。','',
140          '## 清晰的求解顺序','',
141          '1. 读取附件 1 的日内电价与固定日型、附件 2 的 365 天实际数据；核查每一天 144 点、顺序与非负性。',
142          '2. 求解第一问 MILP、LP 以及效率口径对照。',
143          '3. 对每个预测日，只传入当天之前的完整日数据拟合预测；历史残差也由当时可得到的预测计算。',
144          '4. 1 月 1 日用附件 1 固定日型作先验，之后使用历史数据；按 P1 预先指定策略连续运行整月，产生 2 月 1 日真实衔接储量。',
145          '5. 从同一个1月20日未末储量出发，比较3种预测×3个终端倍率，共9个验证分支；各分支1月31日未末固定6000，
146          ↳ 避免验证费用受不同剩余库存影响。验证分支不重写1月实际预热轨迹。',
147          '6. 2月1日0点按1月验证费用固定主方案。在每个正式日构造场景、求解一个共享日前计划、用实际数据回放，并把未末储量带给次日。',
148          '7. 输出完整 10 分钟轨迹、334 天表格、指定日期结果；独立复算物理约束、每个场景目标、现金费用、概率和时间信息边界。','',
149          '## 预测与抽样的具体选择','',
150          '- P1: 最多 28 个历史日，时间权重半衰期 7 天，同星期权重乘 2；沿用经过验证的实现。',
151          '- P2: 最多28天的10分钟数据，8阶日周期傅里叶回归后对残差拟合ARIMA(1,0,0)，使用Burg估计。它对应PDF列出的
152          ↳ “傅里叶项+ARIMA误差”分支；本版没有另行运行144季节周期的SARIMA。',
153          '- P3: 两套随机森林分别预测负荷和光伏，64棵树、深度10、最小叶8。特征包括前1天、前7天同点、前7天均值、前1天相邻点、时刻、
154          ↳ 星期及月份。训练目标最多28天，滞后特征所需原始数据最多再向前7天。没有引入题目未提供的天气。',

```

```

147         '- 最早无历史的一天用附件 1 先验; P2 不足 7 日、P3 不足 8 日时显式回退 P1, 记录在逐日信息日志中。',
148         '- 每次抽到的日索引同时决定该日144点负荷残差与144点光伏残差。重复索引合并后概率为抽中次数/256, 数学上等价于保留256个等权场景;
↳ 不把不同分钟或负荷/光伏各自打乱。',
149         '- 除式 (25) 规定的负值截零, 本版场景不额外加入历史零光伏支撑遮罩。',
150         '- 28天、256次、傅里叶阶数和树参数是实现时预先固定的配置, 并非PDF给出的唯一常数, 也未宣称已全面调优。残差幅度、
↳ 终端系数与随机种子有单独对照。', '',
151         '## 一月验证', '',
152         '| 预测方法 | 终端倍率 | 验证实际费用/元 | 负荷 MAE/kW | 光伏 MAE/kW |', '|---|---|---|---|---|']
153 for s in selection['validation']:
154     rows.append(f"| {LABELS[s['model']] | {s['terminal_multiplier']} | {s['total_cash_cost_yuan']:.2f} |
↳ {s['forecast_metrics']['load_mae_kw']:.2f} | {s['forecast_metrics']['pv_mae_kw']:.2f} |")
155 rows += [' ', '1 月只有 11 个验证日, 季节代表性有限。下表的全年结果只作样本外评价, 不能据此追溯改变 1 月选择。', '',
156         '## 全年候选评价', '',
157         '| 方法 | 实际费用/元 | 紧急购电费/元 | 负荷 MAE/kW | 光伏 MAE/kW | 有光伏时段 MAE/kW |',
158         '|---|---|---|---|---|---|']
159 for m in LABELS:
160     s=summaries['candidate_'+m];f=s['forecast_metrics']
161     rows.append(f"| {LABELS[m]} | {s['total_cash_cost_yuan']:.2f} | {s['total_emergency_cost_yuan']:.2f} |
↳ {f['load_mae_kw']:.2f} | {f['pv_mae_kw']:.2f} | {f['pv_daylight_mae_kw']:.2f} |")
162 annual_best=min((summaries['candidate_'+m] for m in LABELS),key=lambda s:s['total_cash_cost_yuan'])
163 rows += [' ', f"事后全年费用最低的候选为{LABELS[annual_best['model']]}.
↳ 主结果仍使用1月选定的{LABELS[selection['chosen_model']]}. 各候选的终端倍率也分别在1月选定;
↳ 因此这张表比较的是经相同验证流程确定的完整方案。", '',
164         '## 控制变量对照与敏感性', '',
165         '| 设置 | 实际费用/元 | 相对主方案 | 紧急购电费/元 |', '|---|---|---|---|']
166 comparisons = [('point_selected', '只用点预测'), ('enumerate_selected', '配对残差枚举'),
167                ('terminal_0_8', '终端倍率在主方案基础上减 20%'), ('terminal_1_2', '终端倍率在主方案基础上增 20%'),
168                ('residual_0_8', '残差幅度减 20%'), ('residual_1_2', '残差幅度增 20%'),
169                ('seed_20260913', '抽样种子 20260913'), ('seed_20260914', '抽样种子 20260914'),
170                ('roundtrip90', '往返效率 90% 口径, 另行预热')]
171 for name,label in comparisons:
172     s=summaries[name]
173     rows.append(f"| {label} | {s['total_cash_cost_yuan']:.2f} |
↳ {(s['total_cash_cost_yuan']/main['total_cash_cost_yuan']-1)*100:+.3f}% |
↳ {s['total_emergency_cost_yuan']:.2f} |")
174 rows += [' ',
175         '## 与旧版的关系及对照', '',
176         f"旧版原主结果为16,766,455.69元, 2月1日初始储量为6000kWh。本版一月连续预热后起点为{main['initial_kwh']:.6f}kWh,
↳ 两者不能直接作为控制其他因素的单变量比较。",
177         f"为此, 另用旧版P1预测、最近样本外残差枚举及终端倍率1, 在本版同一初始储量和相同年末条件下重算, 费用为
↳ **{matched['total_cash_cost_yuan']:.2f}元**.",
178         f"本版主方案相对该对照的实际费用变化为 **{main['total_cash_cost_yuan']-matched['total_cash_cost_yuan']:+.2f}元
↳ ((main['total_cash_cost_yuan']/matched['total_cash_cost_yuan']-1)*100:+.3f)%**。这反映预测、
↳ 抽样和终端选择的组合效果, 不单独归因于任何一种算法。",
179         '随机种子对照保留同一预测模型、终端系数和 1 月预热后起点, 只改变正式期的场景抽样; 枚举对照保留选定预测, 用于识别额外抽样误差。',
180         '无储能对照保持库存不动, 无法满足从本版1200恢复到6000的年度储量条件; 其费用只用于辅助观察, 不作为严格同终端条件的节省率证据。
↳ 效率敏感性则使用替代效率重新预热, 起点差异在结果表中列出。', '',
181         '## 必须如实说明的边界', '',
182         '1. 时间标签: 本版保留此前采用的区间终点解释, 原附件点序不循环平移, 输出区间为00:00-00:10至23:50-24:00。
↳ PDF51.1初始描述按模板位置对应, 模板标签向后一个时段。这里是明确保留的解释差异, 论文的时间约定也必须同步;
↳ 不能声称仅靠作图已经证实标签含义。',
183         '2. 富余电量: 式 (28) 中的 U 可包含免费光伏, 因此报告称“总未利用电量”, 不把全部 U 都称为已经付费的外网浪费。',
184         f"3. 终端价值: 主方案有{main['day_end_at_min_count']}个日末在1200kWh附近, 另有{day_end_at_max}个日末在10800kWh附近。
↳ 线性终端项使该参数下的储量偏向上限, 并非已经消除日未效应。将选定系数降低20%的事后费用反而下降约0.205%,
↳ 说明1月选择不保证全年参数最优。主结果仍保留预先选定参数, 不能用全年敏感性结果倒选。",
185         '4. 1月1日附件1日型先验是PDF53.9.4允许的初始化信息。它作为题目已提供的固定输入使用, 未用附件2未来日期重新计算该先验。
↳ 正式期模型只使用历史实际数据。',
186         '5. 固定计划下, 电池当天不能响应真实误差; 紧急购电可能被用于完成事先承诺的充电。这来自PDF主模型的策略限制,
↳ 不能在代码中暗改为日内自适应。',
187         '6. 只使用 1 年数据且初始验证集中在 1 月, 结果不构成跨年份稳定性保证。3 个随机种子仅检验抽样扰动, 不是统计置信区间。', '',
188         '## 数值验证与交付', '',
189         f"独立复算覆盖{audit['audited_experiments']}个实验、{audit['audited_days']}个日模型/回放记录, 包含预热、
↳ 验证和14组正式结果; 最大数值残差为{audit['maximum_numerical_residual']:.3g}, 按相应检查分别以kWh或元计, 均低于1e-5。
↳ ",
190         '28项测试包括Benders与整体模型的小样例等价性、储能损耗、跨日衔接、未来数据扰动、ARIMA周期样例、残差配对和压缩前后优化等价性。',
191         f"第一问替代效率口径(ηc=ηd=v0.9)的费用为{q1['roundtrip90_sensitivity']['cost_yuan']:.6f}元。第一问主结果与旧版一致,
↳ 说明复用的主模型未被本次预测模块改动。",
192         'result1.xlsx、result2.xlsx沿用题目模板的工作表结构。第二问购电表的EP列为计划Q合计, EQ列为pQ+5pE实际费用;
↳ 紧急购电量在独立工作表。规划用的终端价值不算作真实电费。',
193         '第二问 Excel 中绝对值小于 1e-9 的浮点噪声置零, 避免出现“-0.00”; 原始求解轨迹未据此截断, 逐值导出误差仍须小于 1e-6。',
194         '完整10分钟CSV、全部方案对比、1月验证、预测精度、指定日期表格和图表放在结果目录; 场景抽样索引、种子、源码快照、
↳ 输入哈希和独立审计记录放在复核目录。', '' ]

```

```

195     (out/'前两问 version2 模型落实与求解报告.md').write_text('\n'.join(rows),encoding='utf-8')
196     print('REPORT AND FIGURES COMPLETE')
197
198
199 if __name__=='__main__':main(Path(sys.argv[1]).resolve())

```

reporting.py

```

1 """ 由完整 10 分钟轨迹生成论文表格、模板填充值与独立核对材料。"""
2 from pathlib import Path
3 import json
4
5 import numpy as np
6
7 from forecasting import interval_label, clock_text
8
9
10 def emergency_segments(energy,threshold=1e-6):
11     e=np.asarray(energy,dtype=float)
12     if not np.isfinite(threshold) or threshold<0:
13         raise ValueError("Emergency reporting threshold must be finite and nonnegative")
14     if e.shape!=(144,) or not np.isfinite(e).all() or (e<0).any():
15         raise ValueError("Expected 144 nonnegative emergency energy values")
16     mask=e>threshold
17     segments=[]
18     t=0
19     while t<144:
20         if not mask[t]:
21             t+=1;continue
22         start=t
23         while t<144 and mask[t]:
24             t+=1
25         segments.append({"start_index":start,"end_index_exclusive":t,
26                         "interval":clock_text(start*10)+"-"+clock_text(t*10),
27                         "energy_kwh":float(e[start:t].sum())})
28     return segments,float(e[~mask].sum())
29
30
31 def four_hour_rows(charge,discharge,state):
32     c,d,s=np.asarray(charge),np.asarray(discharge),np.asarray(state)
33     if c.shape!=(144,) or d.shape!=(144,) or s.shape!=(145,):
34         raise ValueError("Four-hour aggregation requires 144 intervals and 145 states")
35     return [[clock_text(j*240)+"-"+clock_text((j+1)*240),float(c[j*24:(j+1)*24].sum()),
36             float(d[j*24:(j+1)*24].sum()),"00:00" if j==0 else "24:00" if j==1 else None,
37             float(s[0]) if j==0 else float(s[-1]) if j==1 else None] for j in range(6)]
38
39
40 def build_payload(run_path,dataset,config):
41     run=Path(run_path)
42     q1=json.loads((run/"q1.json").read_text(encoding="utf-8"))["milp"]
43     data=np.load(run/"main_weighted28/trajectories.npz")
44     ids=data["day_indices"].tolist()
45     if ids!=list(range(31,365)):
46         raise ValueError("Formal template payload requires all 334 dates")
47     dates=[dataset.dates[day].isoformat() for day in ids]
48     plan_rows=[];battery_rows=[];emergency_rows=[];daily=[];segments_by_day={}
49     excluded=0.
50     for i,day in enumerate(ids):
51         q,c,d,s,e,u=[data[key][i] for key in ("Q","C","D","S","E","U")]
52         planned_cost=float(dataset.prices[q])
53         emergency_cost=float(5*dataset.prices@e)
54         total_cost=planned_cost+emergency_cost
55         plan_rows.append([dates[i],*q.tolist(),float(q.sum()),total_cost])
56         for j,row in enumerate(four_hour_rows(c,d,s)):
57             battery_rows.append([dates[i] if j==0 else None,*row])
58             segments,omitted=emergency_segments(e,config["emergency_reporting_threshold_kwh"])
59             excluded+=omitted;segments_by_day[dates[i]]=segments
60         for j,segment in enumerate(segments):
61             emergency_rows.append([dates[i] if j==0 else None,segment["interval"],segment["energy_kwh"]])
62         daily.append({"date":dates[i],"planned_purchase_kwh":float(q.sum()),"emergency_purchase_kwh":float(e.sum()),
63                     "planned_plus_emergency_kwh":float(q.sum()+e.sum()),"planned_cost_yuan":planned_cost,

```

```

64         "emergency_cost_yuan":emergency_cost,"actual_total_cost_yuan":total_cost,
65         "initial_kwh":float(s[0]),"terminal_kwh":float(s[-1]),"total_unused_kwh":float(u.sum()))
66     if excluded>1e-4:
67         raise ValueError(f"Emergency reporting threshold suppresses material energy: {excluded} kWh")
68     selected=[]
69     for date_text in ("2025-03-20","2025-06-21","2025-09-23","2025-12-21"):
70         i=dates.index(date_text)
71         selected.append(**daily[i],"purchases_at_requested_intervals":[[interval_label(hour*6),float(data["Q"][i,h]
72             ↳ our*6)]] for hour in (10,12,14,16,18,20)],
73             "battery_four_hour":four_hour_rows(data["C"][i],data["D"][i],data["S"][i]),
74             "emergency_segments":segments_by_day[date_text]))
75     a=q1["plan"]
76     payload={"q1":{"purchases":[[interval_label(t),a["Q"][t]] for t in range(144)],
77         "battery":four_hour_rows(a["C"],a["D"],a["S"]),
78         "cost_yuan":q1["cost_yuan"],"purchase_kwh":q1["purchase_kwh"],
79         "requested_intervals":[[interval_label(hour*6),a["Q"][hour*6]] for hour in (10,12,14,16,18,20)]},
80         "q2":{"headers":["日期\\时间",*[interval_label(t) for t in range(144)],"全天购电量","全天购电费"],
81         "purchases":plan_rows,"battery":battery_rows,"emergency":emergency_rows,
82         "daily":daily,"selected_dates":selected,"emergency_omitted_below_threshold_kwh":excluded},
83         "notes":{"time_mapping":"源标签为区间终点; 输出模板改为 00:00-00:10 至 23:50-24:00, 点序和数值不循环平移。",
84             "q2_total_columns":"计划购电量表EP列为计划Q的日合计; EQ列为计划费用加实际紧急购电费用。紧急电量在单独工作表。
85             ↳ ",
86             "energy_units":"各电量 kWh, 费用元。C、D 为交流母线侧电量, S 为电池内部储量。",
87             "sources":"C 题题面、附件 1、附件 2、附件 5 及前两问修订模型。",
88             "method":"零点固定全天电池计划; 28 日加权历史预测、最近历史样本外配对残差经验场景; 一月闲置、年末 6000。"}}
89     return payload

```

reporting_v2.py

```

1  """ 由完整 10 分钟轨迹生成论文表格、模板填充值与独立核对材料。"""
2  from pathlib import Path
3  import json
4
5  import numpy as np
6
7  from forecasting import interval_label, clock_text
8
9
10 def emergency_segments(energy,threshold=1e-6):
11     e=np.asarray(energy,dtype=float)
12     if not np.isfinite(threshold) or threshold<0:
13         raise ValueError("Emergency reporting threshold must be finite and nonnegative")
14     if e.shape!=(144,) or not np.isfinite(e).all() or (e<0).any():
15         raise ValueError("Expected 144 nonnegative emergency energy values")
16     mask=e>threshold
17     segments=[]
18     t=0
19     while t<144:
20         if not mask[t]:
21             t+=1;continue
22         start=t
23         while t<144 and mask[t]:
24             t+=1
25         segments.append({"start_index":start,"end_index_exclusive":t,
26             "interval":clock_text(start*10)+"-"+clock_text(t*10),
27             "energy_kwh":float(e[start:t].sum())})
28     return segments,float(e[~mask].sum())
29
30
31 def four_hour_rows(charge,discharge,state):
32     c,d,s=np.asarray(charge),np.asarray(discharge),np.asarray(state)
33     if c.shape!=(144,) or d.shape!=(144,) or s.shape!=(145,):
34         raise ValueError("Four-hour aggregation requires 144 intervals and 145 states")
35     return [[clock_text(j*240)+"-"+clock_text((j+1)*240),float(c[j*24:(j+1)*24].sum()),
36         float(d[j*24:(j+1)*24].sum()),"00:00" if j==0 else "24:00" if j==1 else None,
37         float(s[0]) if j==0 else float(s[-1]) if j==1 else None] for j in range(6)]
38
39
40 def build_payload(run_path,dataset,config,main_folder,method_note):
41     run=Path(run_path)
42     q1=json.loads((run/"q1.json").read_text(encoding="utf-8"))["milp"]

```

```

43 with np.load(run/main_folder/"trajectories.npz") as saved:
44     data={key:saved[key] for key in saved.files}
45     ids=data["day_indices"].tolist()
46     if ids!=list(range(31,365)):
47         raise ValueError("Formal template payload requires all 334 dates")
48     dates=[dataset.dates[day].isoformat() for day in ids]
49     plan_rows=[];battery_rows=[];emergency_rows=[];daily=[];segments_by_day={}
50     excluded=0.
51     for i,day in enumerate(ids):
52         q,c,d,s,e,u=[data[key][i] for key in ("Q","C","D","S","E","U")]
53         planned_cost=float(dataset.prices@q)
54         emergency_cost=float(5*dataset.prices@e)
55         total_cost=planned_cost+emergency_cost
56         plan_rows.append([dates[i],*q.tolist(),float(q.sum()),total_cost])
57         for j,row in enumerate(four_hour_rows(c,d,s)):
58             battery_rows.append([dates[i] if j==0 else None,*row])
59             segments,omitted=emergency_segments(e,config["emergency_reporting_threshold_kwh"])
60             excluded+=omitted;segments_by_day[dates[i]]=segments
61             for j,segment in enumerate(segments):
62                 emergency_rows.append([dates[i] if j==0 else None,segment["interval"],segment["energy_kwh"]])
63             daily.append({"date":dates[i],"planned_purchase_kwh":float(q.sum()),"emergency_purchase_kwh":float(e.sum()),
64                 "planned_plus_emergency_kwh":float(q.sum()+e.sum()),"planned_cost_yuan":planned_cost,
65                 "emergency_cost_yuan":emergency_cost,"actual_total_cost_yuan":total_cost,
66                 "initial_kwh":float(s[0]),"terminal_kwh":float(s[-1]),"total_unused_kwh":float(u.sum())})
67     if excluded>1e-4:
68         raise ValueError(f"Emergency reporting threshold suppresses material energy: {excluded} kWh")
69     selected=[]
70     for date_text in ("2025-03-20","2025-06-21","2025-09-23","2025-12-21"):
71         i=dates.index(date_text)
72         selected.append(**daily[i],"purchases_at_requested_intervals":[[interval_label(hour*6),float(data["Q"][i,h]
73             ↳ our*6))] for hour in (10,12,14,16,18,20)],
74             "battery_four_hour":four_hour_rows(data["C"][i],data["D"][i],data["S"][i]),
75             "emergency_segments":segments_by_day[date_text])
76     a=q1["plan"]
77     payload={"q1":{"purchases":[[interval_label(t),a["Q"][t] for t in range(144)],
78         "battery":four_hour_rows(a["C"],a["D"],a["S"]),
79         "cost_yuan":q1["cost_yuan"],"purchase_kwh":q1["purchase_kwh"],
80         "requested_intervals":[[interval_label(hour*6),a["Q"][hour*6] for hour in (10,12,14,16,18,20)]},
81         "q2":{"headers":[" 日期\\时间",*[interval_label(t) for t in range(144)]," 全天购电量"," 全天购电费"],
82             "purchases":plan_rows,"battery":battery_rows,"emergency":emergency_rows,
83             "daily":daily,"selected_dates":selected,"emergency_omitted_below_threshold_kwh":excluded},
84         "notes":{"time_mapping":" 按源标签为区间终点解释: 输出区间为 00:00-00:10 至 23:50-24:00, 点序和数值不循环平移。",
85             "q2_total_columns":"计划购电量表EP列为计划Q的日合计; EQ列为计划费用加实际紧急购电费用。紧急电量在单独工作表。
86             ↳ ",
87             "energy_units":" 各电量 kWh, 费用元。C、D 为交流母线侧电量, S 为电池内部储量。",
88             "sources":"C 题题面、附件 1、附件 2、附件 5 及前两问修订模型。",
89             "method":method_note}}
90     return payload

```

scientific_style.py

```

1 """ 科研图表的统一视觉约定: 颜色表达变量身份, 标记和线型提供第二种区分。"""
2 from pathlib import Path
3 import hashlib
4 import json
5 import warnings
6 import textwrap
7
8 import numpy as np
9 import matplotlib
10 matplotlib.use('Agg')
11 import matplotlib.pyplot as plt
12 from matplotlib import font_manager
13 from matplotlib.text import Text
14
15 STYLE_VERSION = 'academic_v2_20260912'
16 COLORS = {
17     'load': '#0072B2', 'pv': '#E69F00', 'purchase': '#009E73',
18     'charge': '#56B4E9', 'discharge': '#D55E00', 'state': '#8C6BB1',
19     'price': '#48A24C', 'planned_cost': '#4477AA', 'emergency': '#D55E00',
20     'ink': '#202630', 'muted': '#62717D', 'rule': '#A6AFB7', 'light': '#FFF3F6',

```

```

21 }
22 METHOD_COLORS = ['#009E73', '#56B4E9', '#D55E00', '#8C6BB1']
23
24
25 def apply_style():
26     for name in ('arial.ttf', 'msyh.ttc'):
27         file = Path('C:/Windows/Fonts') / name
28         if file.exists():
29             font_manager.fontManager.addfont(str(file))
30 plt.rcParams.update({
31     'font.family': ['Arial', 'Microsoft YaHei', 'DejaVu Sans'],
32     'font.size': 9, 'axes.labelsize': 9, 'axes.titlesize': 9.5,
33     'xtick.labelsize': 8, 'ytick.labelsize': 8, 'legend.fontsize': 8,
34     'axes.unicode_minus': False, 'axes.spines.top': False,
35     'axes.spines.right': False, 'axes.linewidth': .7, 'axes.edgecolor': COLORS['ink'],
36     'axes.labelcolor': COLORS['ink'], 'text.color': COLORS['ink'],
37     'xtick.color': COLORS['ink'], 'ytick.color': COLORS['ink'],
38     'xtick.direction': 'out', 'ytick.direction': 'out',
39     'xtick.major.width': .7, 'ytick.major.width': .7,
40     'xtick.major.size': 3, 'ytick.major.size': 3, 'axes.grid': False,
41     'legend.frameon': False, 'legend.handlelength': 2.2,
42     'lines.linewidth': 1.25, 'figure.facecolor': 'white', 'axes.facecolor': 'white',
43     'savefig.facecolor': 'white', 'savefig.dpi': 450,
44     'svg.fonttype': 'none', 'pdf.fonttype': 42, 'ps.fonttype': 42,
45     'svg.hashsalt': STYLE_VERSION,
46 })
47
48
49 def step(ax, values, **kwargs):
50     """144 个区间值映射到 0—24 时，保留真实阶梯，不插值平滑。"""
51     values = np.asarray(values, dtype=float)
52     if values.shape != (144,) or not np.isfinite(values).all():
53         raise ValueError('A daily step series must have 144 finite values')
54     return ax.step(np.arange(145)/6, np.r_[values, values[-1]], where='post', **kwargs)
55
56
57 def time_axis(ax, *, label=True, ticks=4):
58     ax.set_xlim(0, 24)
59     ax.set_xticks(np.arange(0, 25, ticks))
60     if label:
61         ax.set_xlabel('时刻 (h)')
62     ax.margins(x=0)
63
64
65 def panel(ax, letter, title):
66     ax.set_title(title, loc='left', pad=9, fontweight='normal')
67     ax.text(-.105, 1.04, letter, transform=ax.transAxes, fontsize=11,
68           fontweight='bold', va='bottom', ha='left')
69
70
71 def note(fig, text, *, y=.025):
72     text = '\n'.join(textwrap.fill(line, width=60, break_long_words=True) for line in text.splitlines())
73     fig.text(.12, y, text, fontsize=7.2, color=COLORS['muted'], va='bottom')
74
75
76 def save_figure(fig, folder, name, *, pdf=None, dpi=450):
77     folder = Path(folder)
78     folder.mkdir(parents=True, exist_ok=True)
79     # 保存时记录缺字及布局警告，避免仅凭程序未报错便交付。
80     with warnings.catch_warnings(record=True) as caught:
81         warnings.simplefilter('always')
82         fig.canvas.draw()
83         renderer = fig.canvas.get_renderer()
84         outside = []
85         for obj in fig.findobj(match=Text):
86             if not obj.get_visible() or not obj.get_text() or obj.get_clip_on():
87                 continue
88             box = obj.get_window_extent(renderer)
89             if box.width and box.height and (box.x0 < -1 or box.y0 < -1 or
90             box.x1 > fig.bbox.width+1 or box.y1 > fig.bbox.height+1):
91                 outside.append(obj.get_text())
92         if outside:
93             raise ValueError(f'{name}: text extends outside figure: {outside}')

```

```

94     files = []
95     for suffix in ('png', 'svg'):
96         target = folder/f'{name}.{suffix}'
97         fig.savefig(target, dpi=dpi, facecolor='white')
98         files.append({'file': target.name, 'bytes': target.stat().st_size,
99                     'sha256': hashlib.sha256(target.read_bytes()).hexdigest()})
100     if pdf is not None:
101         pdf.savefig(fig, dpi=dpi)
102     problems = sorted({str(w.message) for w in caught
103                       if 'Glyph' in str(w.message) or 'layout' in str(w.message).lower()})
104     if problems:
105         raise ValueError(f'{name}: rendering warnings: {problems}')
106     result = {'name': name, 'width_mm': round(fig.get_figwidth()*25.4, 1),
107             'height_mm': round(fig.get_figheight()*25.4, 1), 'png_dpi': dpi,
108             'axes': len(fig.axes), 'text_bounds': 'passed', 'glyph_check': 'passed',
109             'files': files}
110     plt.close(fig)
111     return result

```

verify_workbooks_v2.py

```

1  """ 只读核对实际导出的每个日期和数值；不会写入 Excel。 """
2  from pathlib import Path
3  import sys, json
4  import numpy as np
5  from openpyxl import load_workbook
6
7
8  def verify(run):
9      p = json.loads((run/'workbook_payload.json').read_text(encoding='utf-8'))
10     checks = {}
11     for number, names in [(1,['计划购电量', '充放电量']), (2,['计划购电量', '充放电量', '紧急购电量'])]:
12         w = load_workbook(run/f'deliverables/result{number}.xlsx', data_only=True)
13         if w.sheetnames != names:
14             raise ValueError('Template sheet topology changed')
15         if number == 1:
16             blocks = [('计划购电量', p['q1']['purchases']), ('充放电量', p['q1']['battery'])]
17         else:
18             blocks = [('计划购电量', p['q2']['purchases']), ('充放电量', p['q2']['battery']), ('紧急购电量', p['q2']['emerge']
19             ↪ ncy)]]
20         for sheet, expected in blocks:
21             actual = w[sheet].iter_rows(min_row=2, max_row=len(expected)+1, max_col=len(expected[0]), values_only=True)
22             for index, (row, target) in enumerate(zip(actual, expected)):
23                 for col, (a, e) in enumerate(zip(row, target)):
24                     if number == 2 and col == 0 and e is not None:
25                         assert a.date().isoformat() == e, (sheet, index, col)
26                     elif isinstance(e, (int, float)):
27                         assert a is not None and abs(a-e) < 1e-6, (sheet, index, col, a, e)
28                     else:
29                         assert a == e, (sheet, index, col, a, e)
30             checks[f'result{number}/{sheet}'] = {'rows': len(expected), 'all_values_verified': True}
31     for sheet in w:
32         for row in sheet:
33             assert all(c.data_type != 'e' for c in row)
34     if number == 2:
35         assert w['计划购电量']['B1'].value == '00:00-00:10'
36         assert w['计划购电量']['E01'].value == '23:50-24:00'
37         assert 'version2' in w['计划购电量']['A338'].value
38         checks['total_cash_cost_yuan'] = sum(w['计划购电量'].cell(i, 147).value for i in range(2, 336))
39     w.close()
40     (run/'workbook_qa/readonly_export_check.json').write_text(json.dumps(checks, ensure_ascii=False, indent=2), encoding='utf-8')
41     print(json.dumps(checks, ensure_ascii=False, indent=2))
42
43 if __name__ == '__main__':
44     verify(Path(sys.argv[1]).resolve())

```

D 问题三源程序

forecasting.py

```
1 """ 只使用目标日前已完成日期的预测与历史样本外残差场景。
2
3 候选模型是预先定义的历史均值/加权均值, 不使用全年数据调参。
4 枚举最近历史联合残差, 形成离散经验分布, 不额外随机重抽样。
5 """
6 from dataclasses import dataclass
7 from datetime import date, timedelta
8 from pathlib import Path
9 import csv
10 import hashlib
11
12 import numpy as np
13
14
15 def clock_text(minute):
16     if not isinstance(minute,int) or not 0<=minute<=1440:
17         raise ValueError("Minute must be an integer from 0 to 1440")
18     return f"{minute//60:02d}:{minute%60:02d}"
19
20
21 def interval_label(index):
22     if not isinstance(index,int) or not 0<=index<144:
23         raise ValueError("Interval index must be 0..143")
24     return clock_text(index*10)+"-"+clock_text((index+1)*10)
25
26
27 @dataclass
28 class Dataset:
29     dates: list
30     load_kw: np.ndarray
31     pv_kw: np.ndarray
32     prices: np.ndarray
33     baseline_load_kw: np.ndarray
34     baseline_pv_kw: np.ndarray
35     sources: dict
36
37
38 def read_dataset(processed):
39     processed=Path(processed)
40     actual_path=processed/"actual_points.csv"
41     baseline_path=processed/"baseline_points.csv"
42     with actual_path.open(encoding="utf-8-sig",newline="") as stream:
43         rows=list(csv.DictReader(stream))
44     expected=[date(2025,1,1)+timedelta(days=i) for i in range(365)]
45     if len(rows)!=365*144:
46         raise ValueError("Expected 365 complete days, each with 144 records")
47     load,pv=np.empty((365,144)),np.empty((365,144))
48     for row_id,row in enumerate(rows):
49         day,t=divmod(row_id,144)
50         if row["date"]!=expected[day].isoformat() or int(row["point_index"])!=t+1 or
51             ↪ row["time_label"]!=clock_text((t+1)*10):
52             raise ValueError(f"Unexpected date/time/point mapping at input row {row_id+2}")
53         load[day,t],pv[day,t]=float(row["load_kw"]),float(row["pv_actual_kw"])
54     with baseline_path.open(encoding="utf-8-sig",newline="") as stream:
55         baseline=list(csv.DictReader(stream))
56     if len(baseline)!=144:
57         raise ValueError("Invalid baseline length")
58     for t,row in enumerate(baseline):
59         if int(row["point_index"])!=t+1 or row["time_label"]!=clock_text((t+1)*10):
60             raise ValueError("Baseline time mapping differs from actual data")
61     p,bl,bg=[np.array([float(row[field]) for row in baseline]) for field in
62             ↪ ("price_yuan_per_kwh","load_kw","pv_forecast_kw")]
63     for arr in (load,pv,p,bl,bg):
64         if not np.isfinite(arr).all() or (arr<0).any():
65             raise ValueError("Nonfinite or negative input")
66     if (p<=0).any():
67         raise ValueError("Strictly positive electricity prices are required")
```

```

66     return Dataset(expected,load,pv,p,bl,bg,{x.name:hashlib.sha256(x.read_bytes()).hexdigest() for x in
    ↪ (actual_path,baseline_path)})
67
68
69 def point_forecast(history_load, history_pv, history_dates, target_date, model):
70     """ 历史只能包含 target_date 之前的完整日。模型参数预先指定。"""
71     load,pv=np.asarray(history_load,dtype=float),np.asarray(history_pv,dtype=float)
72     if load.ndim!=2 or load.shape!=pv.shape or len(load)!=len(history_dates) or len(load)==0:
73         raise ValueError("Invalid history shape")
74     if any(d>=target_date for d in history_dates) or any(b-a!=timedelta(days=1) for a,b in
    ↪ zip(history_dates,history_dates[1:])):
75         raise ValueError("History must contain only consecutive completed dates before the forecast origin")
76     if history_dates[-1]!=target_date-timedelta(days=1):
77         raise ValueError("Latest completed historical date is missing")
78     if not np.isfinite(load).all() or not np.isfinite(pv).all() or (load<0).any() or (pv<0).any():
79         raise ValueError("Invalid historical power")
80     if model=="mean7":
81         window=min(7,len(load)); weights=np.ones(window)
82     elif model=="weighted28":
83         window=min(28,len(load))
84         ages=np.arange(window,0,-1)
85         weights=np.exp(-np.log(2)*ages/7)
86         weights*=np.array([2. if d.weekday()==target_date.weekday() else 1. for d in history_dates[-window:]])
87     else:
88         raise ValueError("Unknown forecast model")
89     weights/=weights.sum()
90     return weights@load[-window:],weights@pv[-window:],{
91         "model":model,"training_start":history_dates[-window].isoformat(),
92         "training_end":history_dates[-1].isoformat(),"origin":target_date.isoformat(),
93         "history_days":window,"weights":weights.tolist()}
94
95
96 def causal_forecast_cache(dataset, model, minimum_history_days=7):
97     """ 离线重建每个历史日当时能得到的预测；调用时只传入其历史前缀。"""
98     if minimum_history_days<1 or minimum_history_days>=len(dataset.dates):
99         raise ValueError("Invalid historical warmup")
100     forecast_load=np.full_like(dataset.load_kw,np.nan,dtype=float)
101     forecast_pv=np.full_like(dataset.pv_kw,np.nan,dtype=float)
102     metadata={}
103     for day in range(minimum_history_days,len(dataset.dates)):
104         l,g,info=point_forecast(dataset.load_kw[:day],dataset.pv_kw[:day],dataset.dates[:day],dataset.dates[day],model)
    ↪ del
105         forecast_load[day],forecast_pv[day]=l,g
106         metadata[day]=info
107     return {"model":model,"load_kw":forecast_load,"pv_kw":forecast_pv,"metadata":metadata,
108           "minimum_history_days":minimum_history_days}
109
110
111 def make_scenarios(dataset, cache, day, *, residual_days=28, residual_scale=1., deterministic=False):
112     """ 整日负荷/光伏残差配对；截取 day 之前的样本外残差，转换为 kWh。
113
114 PV 支撑范围只由最近 28 个已观测日确定，完全未观测到发电的时段置零。
115 这不预知日出时间，也不能保证未来不会出现新的发电时段。
116 """
117     if day<cache["minimum_history_days"]+1 or day>=len(dataset.dates):
118         raise ValueError("Insufficient historical out-of-sample residuals")
119     if not isinstance(residual_days,int) or residual_days<1 or not np.isfinite(residual_scale) or residual_scale<0:
120         raise ValueError("Invalid residual configuration")
121     point_load=cache["load_kw"][day]
122     point_pv=cache["pv_kw"][day]
123     begin=max(cache["minimum_history_days"],day-residual_days)
124     ids=np.arange(begin,day)
125     if deterministic:
126         raw_load=point_load[None,:]
127         raw_pv=point_pv[None,:]
128     else:
129         # 只从已完成的历史日期减去其当时可用预测，不使用目标日误差。
130         residual_load=dataset.load_kw[ids]-cache["load_kw"][ids]
131         residual_pv=dataset.pv_kw[ids]-cache["pv_kw"][ids]
132         raw_load=point_load+residual_scale*residual_load
133         raw_pv=point_pv+residual_scale*residual_pv
134     support=dataset.pv_kw[max(0,day-28):day].max(axis=0)>0
135     scenario_load=np.maximum(raw_load,0)/6

```

```

136 scenario_pv=np.where(support,np.maximum(raw_pv,0),0)/6
137 if not np.isfinite(scenario_load).all() or not np.isfinite(scenario_pv).all():
138     raise ValueError("Nonfinite forecast/scenario; inspect warmup and history")
139 count=len(scenario_load)
140 audit={**cache["metadata"][day],"scenario_count":count,
141        "residual_start":None if deterministic else dataset.dates[begin].isoformat(),
142        "residual_end":None if deterministic else dataset.dates[day-1].isoformat(),
143        "scenario_construction":"point_forecast" if deterministic else
144        ↪ "enumerated_paired_daily_out_of_sample_residuals",
145        "residual_scale":residual_scale,"negative_load_values_clipped":int((raw_load<0).sum()),
146        "negative_pv_values_clipped":int((raw_pv<0).sum()),
147        "pv_values_masked_outside_recent_support":int(((raw_pv>0)&~support).sum())}
148 return scenario_load,scenario_pv,np.full(count,1/count),audit
149
150 def no_storage_purchase(load_scenarios,pv_scenarios,probabilities,quantile=.8):
151     net=load_scenarios-pv_scenarios
152     if not 0<quantile<1:
153         raise ValueError("Quantile must be in (0,1)")
154     order=np.argsort(net,axis=0)
155     sorted_net=np.take_along_axis(net,order,axis=0)
156     cdf=np.cumsum(np.asarray(probabilities)[order],axis=0)
157     index=np.argmax(cdf>=quantile-1e-12,axis=0)
158     return np.maximum(sorted_net[index,np.arange(net.shape[1])],0)

```

microgrid_core.py

```

1 """ 微网通用计算模块。所有调度流量为 kWh，功率上限为交流母线侧 kW。
2
3 函数不解释 Excel 时间标签、不推断初末条件、不选择预测模型。
4 第一问首尾相等由 solve_q1 显式实现；固定计划回放不重置跨日状态。
5 """
6 from dataclasses import asdict, dataclass
7 from pathlib import Path
8 import csv
9 from datetime import date
10 import math
11 import time
12
13 import numpy as np
14 from scipy.optimize import Bounds, LinearConstraint, linprog, milp
15 from scipy.sparse import lil_matrix
16
17
18 @dataclass(frozen=True)
19 class Battery:
20     eta_charge: float
21     eta_discharge: float
22     min_kwh: float = 1200.0
23     max_kwh: float = 10800.0
24     max_power_kw: float = 5000.0
25     step_hours: float = 1/6
26
27     def __post_init__(self):
28         if not all(math.isfinite(v) for v in asdict(self).values()):
29             raise ValueError("Battery parameters must be finite")
30         if not 0 < self.eta_charge <= 1 or not 0 < self.eta_discharge <= 1:
31             raise ValueError("Charge and discharge efficiencies must be in (0, 1]")
32         if not 0 <= self.min_kwh < self.max_kwh:
33             raise ValueError("Invalid battery capacity bounds")
34         if self.max_power_kw <= 0 or self.step_hours <= 0:
35             raise ValueError("Power and time step must be positive")
36
37     @property
38     def limit_kwh(self):
39         return self.max_power_kw * self.step_hours
40
41     def validate_initial(self, value):
42         # 跨日独立重算可产生 1e-10 级边界误差，允许数值容差，不重置或截断储量。
43         if not math.isfinite(value) or not self.min_kwh-1e-7 <= value <= self.max_kwh+1e-7:
44             raise ValueError("Initial stored energy is outside the permitted range")

```

```

45
46
47 def vector(values, name, length=None, nonnegative=True):
48     a = np.asarray(values, dtype=float)
49     if a.ndim != 1 or len(a) == 0 or (length is not None and len(a) != length):
50         raise ValueError(f"{name}: expected a nonempty vector of length {length}")
51     if not np.isfinite(a).all() or (nonnegative and (a < 0).any()):
52         raise ValueError(f"{name}: invalid numeric values")
53     return a
54
55
56 def dispatch_checks(charge, discharge, state, battery, initial):
57     """ 独立按物理公式计算残差, 输出单位为 kWh。"""
58     c = vector(charge, "charge", nonnegative=False)
59     d = vector(discharge, "discharge", len(c), nonnegative=False)
60     s = vector(state, "state", len(c)+1)
61     return {
62         "nonnegative_dispatch_violation_kwh": float(max(0, -c.min(), -d.min())),
63         "state_transition_residual_kwh":
64             ↪ float(np.abs(np.diff(s)-battery.eta_charge*c+d/battery.eta_discharge).max()),
65         "state_bound_violation_kwh": float(max(0, battery.min_kwh-s.min(), s.max()-battery.max_kwh)),
66         "power_bound_violation_kwh": float(max(0, c.max()-battery.limit_kwh, d.max()-battery.limit_kwh)),
67         "simultaneous_charge_discharge_kwh": float(np.minimum(np.maximum(c,0), np.maximum(d,0)).sum()),
68         "initial_state_difference_kwh": float(abs(s[0]-initial)),
69     }
70
71 def require_checks(checks, tolerance=1e-5):
72     failed = {k:v for k,v in checks.items() if not math.isfinite(v) or v > tolerance}
73     if failed:
74         raise RuntimeError(f"Independent verification failed: {failed}")
75
76
77 def solve_q1(price, load_kwh, pv_kwh, *, battery, initial_kwh,
78             method="milp", time_limit_s=60., mip_rel_gap=1e-9):
79     """ 求解第一问, 首尾储量均为 initial_kwh; LP 同时充放电另行标记。
80
81     失败或超时不会作为已验证最优结果返回。输入的日期/时间含义由调用方提供。
82     """
83     p = vector(price, "price")
84     n = len(p)
85     load, pv = vector(load_kwh, "load", n), vector(pv_kwh, "pv", n)
86     if (p <= 0).any():
87         raise ValueError("This implementation requires strictly positive prices")
88     if method not in ("lp", "milp"):
89         raise ValueError("method must be lp or milp")
90     if not math.isfinite(time_limit_s) or time_limit_s <= 0 or not 0 <= mip_rel_gap < 1:
91         raise ValueError("Invalid solver limits")
92     battery.validate_initial(initial_kwh)
93     integer = method == "milp"
94     size = (6 if integer else 5)*n+1
95     q,c,d,w = [np.arange(k*n,(k+1)*n) for k in range(4)]
96     s,z = np.arange(4*n,5*n+1), np.arange(5*n+1,6*n+1)
97     objective = np.zeros(size)
98     objective[q] = p
99     low, high = np.zeros(size), np.full(size,np.inf)
100     high[c] = high[d] = battery.limit_kwh
101     high[w] = pv
102     low[s], high[s] = battery.min_kwh,battery.max_kwh
103     low[s[0]] = high[s[0]] = initial_kwh
104     low[s[-1]] = high[s[-1]] = initial_kwh
105     eq = lil_matrix((2*n,size))
106     rhs = np.r_[load-pv,np.zeros(n)]
107     for t in range(n):
108         eq[t,q[t]],eq[t,c[t]],eq[t,d[t]],eq[t,w[t]]=1,-1,1,-1
109         eq[n+t,s[t+1]],eq[n+t,s[t]]=1,-1
110         eq[n+t,c[t]],eq[n+t,d[t]]=-battery.eta_charge,1/battery.eta_discharge
111     started = time.perf_counter()
112     if integer:
113         high[z] = 1
114         integrality = np.zeros(size)
115         integrality[z] = 1
116         ub = lil_matrix((2*n,size))

```

```

117         for t in range(n):
118             ub[t,c[t]],ub[t,z[t]]=1,-battery.limit_kwh
119             ub[n+t,d[t]],ub[n+t,z[t]]=1,battery.limit_kwh
120         result = milp(objective,integrality=integrality,bounds=Bounds(low,high),
121                     constraints=[LinearConstraint(eq.tocsr(),rhs,rhs),
122                                LinearConstraint(ub.tocsr(),-np.inf,np.r_[np.zeros(n),np.full(n,battery.limit_kwh)
123                                ↪ h)]),
124                     options={"time_limit":time_limit_s,"mip_rel_gap":mip_rel_gap})
125     else:
126         result = linprog(objective,A_eq=eq.tocsr(),b_eq=rhs,bounds=list(zip(low,high)),
127                         method="highs",options={"time_limit":time_limit_s})
128     runtime = time.perf_counter()-started
129     if not result.success:
130         raise RuntimeError(f"{method} not solved to requested tolerance; status={result.status}: {result.message}")
131     Q,C,D,W,S = (result.x[i] for i in (q,c,d,w,s))
132     # 浮点求解器可能返回极小负数, 保留原值并按容差检查, 不修改调度来通过检查。
133     negative = float(max(0,-result.x.min()))
134     if negative > 1e-5:
135         raise RuntimeError("Solver returned negative energy outside tolerance")
136     checks = {
137         "bus_balance_residual_kwh":float(np.abs(Q+pv+D-load-C-W).max()),
138         "state_transition_residual_kwh":float(np.abs(np.diff(S)-battery.eta_charge*C+D/battery.eta_discharge).max())
139         ↪ ),
140         "state_bound_violation_kwh":float(max(0,battery.min_kwh-S.min(),S.max()-battery.max_kwh)),
141         "power_bound_violation_kwh":float(max(0,C.max()-battery.limit_kwh,D.max()-battery.limit_kwh)),
142         "pv_spill_bound_violation_kwh":float(max(0,(W-pv).max(),-W.min())),
143         "nonnegative_violation_kwh":negative,
144         "initial_state_difference_kwh":float(abs(S[0]-initial_kwh)),
145         "terminal_state_difference_kwh":float(abs(S[-1]-initial_kwh)),
146         "objective_difference_yuan":float(abs(p@Q-result.fun)),
147     }
148     require_checks(checks)
149     simultaneous = float(np.minimum(np.maximum(C,0),np.maximum(D,0)).sum())
150     if integer:
151         require_checks({"simultaneous_charge_discharge_kwh":simultaneous})
152     return {
153         "method":method,"status":int(result.status),"message":str(result.message),
154         "runtime_s":runtime,"mip_gap":float(result.mip_gap) if integer else None,
155         "cost_yuan":float(p@Q),"purchase_kwh":float(Q.sum()),
156         "simultaneous_charge_discharge_kwh":simultaneous,
157         "physically_feasible":simultaneous <= 1e-5,"checks":checks,
158         "plan":{name:values.tolist() for name,values in zip(("Q","C","D","W","S"),(Q,C,D,W,S))},
159     }
160
161 def simulate_fixed_plan(price, load_kwh, pv_kwh, purchase_kwh, charge_kwh, discharge_kwh,
162                        *, battery, initial_kwh, emergency_multiplier):
163     """ 第二问固定电池计划的逐时回放; 实际曲线仅用于结算, 不改变日前计划。
164     总富余可能来自光伏、计划购电或电池, 不能全部称作已付费浪费。
165     调用方必须把返回的终值作为下一日初值; 本函数没有每日重置逻辑。
166     """
167     p = vector(price,"price")
168     n = len(p)
169     load,pv = vector(load_kwh,"load",n),vector(pv_kwh,"pv",n)
170     q,c,d = [vector(v,name,n,nonnegative=False) for v,name in zip(
171         (purchase_kwh,charge_kwh,discharge_kwh),("Q","C","D"))]
172     require_checks({"negative_plan_violation_kwh":float(max(0,-q.min(),-c.min(),-d.min()))})
173     battery.validate_initial(initial_kwh)
174     if (p <= 0).any() or not math.isfinite(emergency_multiplier) or emergency_multiplier <= 0:
175         raise ValueError("Prices and emergency multiplier must be positive")
176     s = np.r_[initial_kwh,initial_kwh+np.cumsum(battery.eta_charge*c-d/battery.eta_discharge)]
177     deficit = load+c-q-pv-d
178     e,u = np.maximum(deficit,0),np.maximum(-deficit,0)
179     checks = dispatch_checks(c,d,s,battery,initial_kwh)
180     checks["bus_balance_residual_kwh"] = float(np.abs(q+e+pv+d-load-c-u).max())
181     require_checks(checks)
182     planned,emergency = float(p@q),float(emergency_multiplier*p@e)
183     return {"policy":"fixed_day_ahead_battery","planned_cost_yuan":planned,
184           "emergency_cost_yuan":emergency,"cash_cost_yuan":planned+emergency,
185           "terminal_kwh":float(s[-1]),"checks":checks,
186           "S":s.tolist(),"E":e.tolist(),"U_total_unused":u.tolist()}
187

```

```

188
189 def load_baseline_csv(path, *, step_hours):
190     """ 读取已核查派生 CSV, 严格保留原始记录次序, 不推断区间起点。"""
191     with Path(path).open(encoding="utf-8-sig",newline="") as stream:
192         rows = list(csv.DictReader(stream))
193         if len(rows) != 144 or [int(r["point_index"]) for r in rows] != list(range(1,145)):
194             raise ValueError("Baseline must contain ordered point_index 1..144")
195         if not math.isfinite(step_hours) or step_hours <= 0:
196             raise ValueError("Invalid conversion time step")
197         p = vector([r["price_yuan_per_kwh"] for r in rows], "price", 144)
198         load = vector([r["load_kw"] for r in rows], "load_kw", 144)*step_hours
199         pv = vector([r["pv_forecast_kw"] for r in rows], "pv_kw", 144)*step_hours
200         return {"point_index": [int(r["point_index"]) for r in rows],
201                 "source_time_label": [r["time_label"] for r in rows],
202                 "price": p, "load_kwh": load, "pv_kwh": pv}
203
204
205 def history_before_date(path, decision_date):
206     """ 按源记录所属日期严格截取日前历史, 尚不解释 10 分钟标签。
207
208 返回值可供后续预测模块使用; 不进行补值、标准化或全年参数估计。
209 """
210     cutoff = date.fromisoformat(decision_date)
211     rows = []
212     keys = set()
213     with Path(path).open(encoding="utf-8-sig",newline="") as stream:
214         for row in csv.DictReader(stream):
215             day = date.fromisoformat(row["date"])
216             if day >= cutoff:
217                 continue
218             index = int(row["point_index"])
219             key = (day, index)
220             if not 1 <= index <= 144 or key in keys:
221                 raise ValueError("Invalid or duplicate historical date/point key")
222             keys.add(key)
223             values = vector([row["load_kw"], row["pv_actual_kw"]], "historical power", 2)
224             rows.append({**row, "point_index": index, "load_kw": float(values[0]), "pv_actual_kw": float(values[1])})
225     rows.sort(key=lambda r: (r["date"], r["point_index"]))
226     return rows

```

q3_forecasting.py

```

1 """ 附件 3 按发布时间对齐、因果偏差/尺度校准及成对历史标准残差场景。"""
2 from datetime import datetime, timedelta
3 from pathlib import Path
4 import csv
5 import hashlib
6 import numpy as np
7 from forecasting import point_forecast
8
9 MODES=('pooled', 'exponential', 'binned')
10 HOURS=np.array([0,6,12,18])
11
12
13 def read_official(path, dates):
14     out=np.full((len(dates),4,24),np.nan);seen=set()
15     with Path(path).open(encoding='utf-8-sig',newline='') as f:
16         for row in csv.DictReader(f):
17             issued=datetime.fromisoformat(row['issued_at']);lead=int(row['lead_hours'])
18             valid=datetime.fromisoformat(row['valid_at']);i=(issued.date()-dates[0]).days
19             if issued.minute or issued.second or issued.hour not in HOURS or not 0<=i<len(dates) or not 1<=lead<=24:
20                 raise ValueError('Invalid official forecast issue/lead')
21             if valid!=issued+timedelta(hours=lead):raise ValueError('Forecast valid time mismatch')
22             key=(i,issued.hour//6,lead-1)
23             if key in seen:raise ValueError('Duplicate official forecast')
24             seen.add(key);out[key]=float(row['pv_forecast_kw'])
25     if len(seen)!=len(dates)*4*24 or not np.isfinite(out).all() or (out<0).any():
26         raise ValueError('Incomplete/invalid official forecast grid')
27     return out
28
29

```

```

30 def integrate_hourly(hourly, anchor, issue_hour):
31     """ 整点值作瞬时节点，逐 10 分钟区间精确积分分段线性曲线。"""
32     hourly=np.asarray(hourly, dtype=float)
33     if hourly.shape!=(24,) or issue_hour not in HOURS or not np.isfinite(hourly).all() or np.any(hourly<0) or
    ↪ anchor<0:
34         raise ValueError('Invalid hourly forecast input')
35     n=(24-issue_hour)*6;x=np.arange(n+1)/6
36     values=np.interp(x,np.arange(25),np.r_[anchor,hourly])
37     return (values[:-1]+values[1:])/2
38
39
40 def masks(bin_hours=6):
41     if bin_hours not in (3,6):raise ValueError('Lead bins must be 3 or 6 hours')
42     g=np.broadcast_to(np.arange(144)//36,(4,144))
43     h=(np.arange(144)[None,:]+1)/6-HOURS[:,None]
44     ell=np.ceil(h/bin_hours).astype(int)-1
45     valid=h>0
46     groups={(a,b):(g==a)&(ell==b)&valid for a in range(4) for b in range(24//bin_hours) if
    ↪ np.any((g==a)&(ell==b)&valid)}
47     return g,h,ell,valid,groups
48
49
50 def day_moments(values,groups):
51     n=len(values);mean=np.full((n,4,max(b for _,b in groups)+1),np.nan);second=mean.copy()
52     for (g,l),mask in groups.items():
53         a=values[:,mask]
54         mean[:,g,l]=a.mean(axis=1);second[:,g,l]=(a*a).mean(axis=1)
55     return mean,second
56
57
58 def fit_scale(mean,second,start,end,floor,shrink,sref,groups,h):
59     if end<=start:
60         return np.full(mean.shape[1:],sref),np.full(4,sref),sref,0.,np.full(4,sref)
61     all_mean=mean[start:end];all_second=second[start:end]
62     pool_mean=float(np.nanmean(all_mean));pool_var=max(0,float(np.nanmean(all_second))-pool_mean**2)
63     pool_sigma=max(floor,np.sqrt(pool_var));bins=np.full(mean.shape[1:],pool_sigma)
64     n=end-start
65     for (g,l) in groups:
66         var=max(0,float(all_second[:,g,l].mean())-float(all_mean[:,g,l].mean())**2)
67         bins[g,l]=np.sqrt(max(floor**2,(n*var+shrink*pool_var)/(n+shrink)))
68     # 组固定截距、共享非负提前量斜率的加权最小二乘：每个分箱按独立日数加权。
69     xs={g:[ ] for g in range(4)};ys={g:[ ] for g in range(4)}
70     for (g,l),mask in groups.items():
71         xs[g].append(float(h[mask].mean()));ys[g].append(float(np.log(bins[g,l]/sref)))
72     numerator=denominator=0.
73     for g in range(4):
74         x=np.array(xs[g]);y=np.array(ys[g]);numerator+=n*float((x-x.mean())@(y-y.mean()));denominator+=n*float((x-x
    ↪ .mean())@(x-x.mean()))
75     beta=max(0,numerator/denominator) if denominator>0 else 0.
76     sigma0=np.array([sref*np.exp(np.mean(ys[g]))-beta*np.mean(xs[g])] for g in range(4))
77     group_constant=np.array([sref*np.exp(np.mean(ys[g])) for g in range(4)])
78     return bins,group_constant,pool_sigma,beta,sigma0
79
80
81 def build_cache(ds,official>window=28,shrink=7.,floor=25.,bin_hours=6,interpretation='linear_nodes'):
82     if window<7 or shrink<=0 or floor<=0:raise ValueError('Invalid calibration settings')
83     if interpretation not in ('linear_nodes','hourly_mean'):raise ValueError('Invalid forecast interpretation')
84     days=len(ds.dates);g,h,ell,valid,groups=masks(bin_hours)
85     raw=np.full((days,4,144),np.nan);mu_g=raw.copy();mu_l=np.zeros((days,144))
86     for i in range(days):
87         if i:
88             mu_l[i]=point_forecast(ds.load_kw[:,i],ds.pv_kw[:,i],ds.dates[:,i],ds.dates[i],'weighted28')[0]
89         for issue,k in enumerate(HOURS):
90             t=k*6;anchor=float(ds.pv_kw[i,t-1]) if t else (float(ds.pv_kw[i-1,-1]) if i else 0.)
91             raw[i,issue,t:]=(integrate_hourly(official[i,issue],anchor,int(k)) if interpretation=='linear_nodes'
    ↪ else np.repeat(official[i,issue,:24-k],6))
92     raw_error=ds.pv_kw[:,None,:]-raw
93     raw_mean,raw_second=day_moments(raw_error,groups)
94     cal_mean=np.full_like(raw_mean,np.nan);cal_second=np.full_like(raw_mean,np.nan)
95     scales_g={mode:np.full_like(raw,np.nan) for mode in MODES}
96     scales_l=np.full((days,144),floor);betas=np.zeros(days);sigma0s=np.zeros((days,4));srefs=np.zeros(days)
97     bias_all=np.zeros(raw_mean.shape);bins_all=np.zeros(raw_mean.shape)
98     for i in range(days):

```

```

100     start=max(0,i-window);n=i-start
101     sref=1000.
102     if i>=7:
103         # 评价期开始前固定参考：不倒灌到前七天的决策。
104         daymask=np.broadcast_to(ds.pv_kw[:7,None,:]>0,raw[:7].shape)&np.broadcast_to(valid,raw[:7].shape)
105         sref=max(floor,float(np.sqrt(np.mean(raw_error[:7][daymask]**2))))
106     srefs[i]=sref
107     pool=float(np.nanmean(raw_mean[start:i])) if n else 0.
108     for (group,lead),mask in groups.items():
109         bias=(n*float(raw_mean[start:i,group,lead].mean())+shrink*pool)/(n+shrink) if n else 0.
110         bias_all[i,group,lead]=bias
111         mu_g[i][mask]=np.maximum(0,raw[i][mask]+bias)
112     bins,constant,pool_sigma,beta,sigma0=fit_scale(cal_mean,cal_second,start,i,floor,shrink,sref,groups,h)
113     betas[i]=beta;sigma0s[i]=sigma0;bins_all[i]=bins
114     for (group,lead),mask in groups.items():
115         scales_g['pooled'][i][mask]=max(floor,pool_sigma)
116         scales_g['binned'][i][mask]=bins[group,lead]
117         scales_g['exponential'][i][mask]=sigma0[group]*np.exp(beta*h[mask])
118     # 当天校准后的误差进入档案：只在后续日期使用。
119     err=ds.pv_kw[i,None,:]-mu_g[i]
120     for (group,lead),mask in groups.items():
121         cal_mean[i,group,lead]=err[mask].mean();cal_second[i,group,lead]=(err[mask]**2).mean()
122     hist_start=max(1,start)
123     if i>hist_start:
124         error=ds.load_kw[hist_start:i]-mu_l[hist_start:i]
125         pool_var=float(error.var())
126         for group in range(4):
127             sl=slice(group*36,(group+1)*36);var=float(error[:,sl].var());count=len(error)
128             scales_l[i,sl]=np.sqrt(max(floor**2,(count*var+shrink*pool_var)/(count+shrink)))
129     else:scales_l[i]=sref
130 return {'raw_g':raw,'mu_g':mu_g,'mu_l':mu_l,'sigma_l':scales_l,
131         **{'sigma_g'+key:value for key,value in scales_g.items()},
132         **{'reliability_g'+key:reliability(value,srefs[:,None,None]) for key,value in scales_g.items()},
133         'window_days':np.array(window),'bin_hours':np.array(bin_hours),'shrink_days':np.array(shrink),
134         'interpretation':np.array(interpretation),
135         'beta':betas,'sigma0':sigma0s,'sref':srefs,'bias':bias_all,'binned_scale':bins_all}
136
137
138 def reliability(sigma,sref):
139     """PDF 式 (16)：无量纲尺度指标，不是正确概率。"""
140     return 1/(1+(np.asarray(sigma)/sref)**2)
141
142
143 def scale_from_reliability(r,sref):
144     return sref*np.sqrt(1/np.asarray(r)-1)
145
146
147 def make_scenarios(ds,cache,day,current_hour,source_hour,*,mode='binned',draws=64,window=None,seed=20260912,point=F
↪ else):
148     if current_hour not in HOURS or source_hour not in HOURS or source_hour>current_hour or mode not in MODES:
149         raise ValueError('Invalid forecast issue visibility or scale mode')
150     t=current_hour*6;issue=source_hour//6
151     window=int(cache.get('window_days',28)) if window is None else window
152     if day<0 or day>=len(ds.dates) or draws<1:raise ValueError('Invalid scenario date/count')
153     ml=cache['mu_l'][day,t];mg=cache['raw_g' if point else 'mu_g'][day,issue,t:]
154     history=np.arange(max(7,day-window),day)
155     if point or len(history)==0:
156         load=ml[None,:];pv=mg[None,:];prob=np.ones(1);sampled=[];unique=[]
157     else:
158         rng=np.random.default_rng(np.random.SeedSequence([seed,day,source_hour]))
159         sampled=rng.choice(history,size=draws,replace=True)
160         unique,counts=np.unique(sampled,return_counts=True);prob=counts/draws
161         z_l=(ds.load_kw[unique,t:]-cache['mu_l'][unique,t:])/cache['sigma_l'][unique,t:]
162         z_g=(ds.pv_kw[unique,t:]-cache['mu_g'][unique,issue,t:])/cache['sigma_g'+mode][unique,issue,t:]
163         load=np.maximum(0,ml+cache['sigma_l'][day,t:]*z_l)
164         pv=np.maximum(0,mg+cache['sigma_g'+mode][day,issue,t:]*z_g)
165     if not np.isfinite(load).all() or not np.isfinite(pv).all():raise ValueError('Nonfinite scenario')
166     audit={'date':ds.dates[day].isoformat(),'decision_hour':int(current_hour),'official_issue_hour':int(source_hour)
↪ },
167         'forecast_age_hours':int(current_hour-source_hour),'first_lead_hours':float(current_hour-source_hour+1/6)
↪ },
168         'last_lead_hours':float(24-source_hour),'calibration_end':ds.dates[day-1].isoformat() if day else None,
169         'available_independent_days':len(history),'unique_sampled_days':len(unique),'requested_draws':draws,

```

```

170         'sampled_day_indices':[int(x) for x in sampled], 'scale_mode':mode, 'point_forecast':point,
171         'window_days':int(window), 'bin_hours':int(cache.get('bin_hours',6)),
172         'shrink_days':float(cache.get('shrink_days',7))}
173     return load/6,pv/6,prob,audit
174
175
176 def probability_scores(scenarios,prob,y,delta=.1):
177     x=np.asarray(scenarios,dtype=float);p=np.asarray(prob,dtype=float);y=np.asarray(y,dtype=float)
178     if x.ndim!=2 or p.shape!=(len(x),) or y.shape!=(x.shape[1],):raise ValueError('Invalid score dimensions')
179     order=np.argsort(x,axis=0);values=np.take_along_axis(x,order,axis=0)
180     weights=np.broadcast_to(p[:,None],x.shape);weights=np.take_along_axis(weights,order,axis=0)
181     cumulative=weights.cumsum(axis=0)
182     quant=lambda a:values[np.argmax(cumulative>=a,axis=0),np.arange(x.shape[1])]
183     low,high=quant(delta/2),quant(1-delta/2)
184     crps=(p[:,None]*abs(x-y)).sum(axis=0)-(weights*values*(2*cumulative-weights-1)).sum(axis=0)
185     return {'crps':crps,'coverage':((y>=low)&(y<=high)).astype(float),'width':high-low,
186           'interval_score':high-low+2/delta*(np.maximum(low-y,0)+np.maximum(y-high,0)),
187           'mae':abs((p[:,None]*x).sum(axis=0)-y),
188           'squared_error':((p[:,None]*x).sum(axis=0)-y)**2}

```

q3_dispatch.py

```

1 """ 第三问：固定本次剩余时域计划的随机 MILP，支持主/备选结算和 CVaR. """
2 import math
3 import time
4 import numpy as np
5 from scipy.optimize import Bounds, LinearConstraint, milp
6 from scipy.sparse import coo_matrix
7 from microgrid_core import dispatch_checks, require_checks
8
9
10 def weighted_cvar(losses, probabilities, alpha):
11     loss=np.asarray(losses,dtype=float);p=np.asarray(probabilities,dtype=float)
12     if loss.shape!=p.shape or loss.ndim!=1 or len(loss)==0 or not np.isfinite(loss).all():
13         raise ValueError('Invalid CVaR sample')
14     if not 0<alpha<1 or (p<=0).any() or abs(p.sum()-1)>1e-9:
15         raise ValueError('Invalid CVaR probabilities or level')
16     order=np.argsort(loss)[::-1];remaining=1-alpha;value=0.
17     for index in order:
18         weight=min(float(p[index]),remaining)
19         value+=weight*float(loss[index]);remaining-=weight
20         if remaining<=1e-14:break
21     return value/(1-alpha)
22
23
24 def settlement(price,q,r,mode='net_refund'):
25     p,q,r=[np.asarray(x,dtype=float) for x in (price,q,r)]
26     if p.shape!=q.shape or p.shape!=r.shape or mode not in ('net_refund','no_refund_penalty'):
27         raise ValueError('Invalid settlement inputs')
28     up=np.maximum(r-q,0);down=np.maximum(q-r,0)
29     sign=-1 if mode=='net_refund' else 1
30     return p*q+1.5*p*up+sign*.5*p*down
31
32
33 def solve_roll(price,load,pv,prob,*,battery,initial_kwh,original_q=None,
34               risk_lambda=0.,alpha=.90,terminal_target=6000.,terminal_gamma=0.,
35               terminal_kwh=None,settlement_mode='net_refund',relax=False,time_limit=60.):
36     p=np.asarray(price,dtype=float);load=np.asarray(load,dtype=float);pv=np.asarray(pv,dtype=float)
37     prob=np.asarray(prob,dtype=float);n=len(p)
38     if p.ndim!=1 or n<1 or load.ndim!=2 or pv.shape!=load.shape or load.shape[1]!=n:
39         raise ValueError('Invalid horizon or scenario shapes')
40     if not all(np.isfinite(x).all() for x in [p,load,pv,prob]) or (p<=0).any() or (load<0).any() or (pv<0).any():
41         raise ValueError('Nonfinite or invalid price/scenario input')
42     k=len(load)
43     if prob.shape!=(k,) or (prob<=0).any() or abs(prob.sum()-1)>1e-9:
44         raise ValueError('Scenario probabilities must be positive and sum to one')
45     if not 0<alpha<1 or not math.isfinite(risk_lambda) or risk_lambda<0 or terminal_gamma<0 or not
46     ↪ math.isfinite(terminal_gamma):
47         raise ValueError('Invalid risk or terminal parameters')
48     if settlement_mode not in ('net_refund','no_refund_penalty') or time_limit<=0:
49         raise ValueError('Invalid settlement mode/solver limit')

```

```

49 battery.validate_initial(initial_kwh);battery.validate_initial(terminal_target)
50 if terminal_kwh is not None: battery.validate_initial(terminal_kwh)
51 q=None if original_q is None else np.asarray(original_q,dtype=float)
52 if q is not None and (q.shape!=(n,) or not np.isfinite(q).all() or q.min()<-1e-6):
53     raise ValueError('Invalid original commitment')
54 start=time.perf_counter();cursor=0
55 def alloc(length):
56     nonlocal cursor
57     result=np.arange(cursor,cursor+length);cursor+=length;return result
58 r,c,d,s,z=alloc(n),alloc(n),alloc(n),alloc(n+1),alloc(n)
59 e=alloc(k*n).reshape(k,n)
60 ap,am=(alloc(n),alloc(n)) if q is not None else (None,None)
61 b=alloc(1)[0] if terminal_gamma>0 else None
62 zeta,v=(alloc(1)[0],alloc(k)) if risk_lambda>0 else (None,None)
63 objective=np.zeros(cursor);lower=np.zeros(cursor);upper=np.full(cursor,np.inf);integer=np.zeros(cursor)
64 lower[s]=battery.min_kwh;upper[s]=battery.max_kwh
65 lower[s[0]]=upper[s[0]]=initial_kwh
66 if terminal_kwh is not None:lower[s[-1]]=upper[s[-1]]=terminal_kwh
67 upper[c]=upper[d]=battery.limit_kwh;upper[z]=1
68 if not relax:integer[z]=1
69 constant=0.
70 if q is None:objective[r]=p
71 else:
72     constant=float(p@q);objective[ap]=1.5*p
73     objective[am]=(-.5 if settlement_mode=='net_refund' else .5)*p
74 objective[e]=(prob[:,None]*5*p)
75 if b is not None:objective[b]=terminal_gamma
76 if zeta is not None:
77     lower[zeta]=-np.inf;objective[zeta]=risk_lambda;objective[v]=risk_lambda*prob/(1-alpha)
78 rows=[];cols=[];vals=[];lo=[];hi=[]
79 def add(ids,coeffs,l=-np.inf,u=np.inf):
80     row=len(lo);rows.extend([row]*len(ids));cols.extend(ids);vals.extend(coeffs);lo.append(l);hi.append(u)
81 for t in range(n):
82     add([s[t+1],s[t],c[t],d[t]],[1,-1,-battery.eta_charge,1/battery.eta_discharge],0,0)
83     add([c[t],z[t]],[1,-battery.limit_kwh],u=0)
84     add([d[t],z[t]],[1,battery.limit_kwh],u=battery.limit_kwh)
85     if q is not None:add([r[t],ap[t],am[t]],[1,-1,1],q[t],q[t])
86 # 消去 u>=0: r+e-c+d>=L-G; 正的期望紧急费使 e 取最小缺口。
87 net=load-pv
88 for omega in range(k):
89     for t in range(n):
90         add([r[t],e[omega,t],c[t],d[t]],[1,1,-1,1],l=net[omega,t])
91     if zeta is not None:
92         add(list(e[omega])+[zeta,v[omega]],list(5*p)+[-1,-1],u=0)
93 if b is not None:add([b,s[-1]],[1,1],l=terminal_target)
94 matrix=coo_matrix((vals,(rows,cols)),shape=(len(lo),cursor)).tocsr()
95 constraint=LinearConstraint(matrix,np.array(lo),np.array(hi))
96 result=milp(objective,integrality=integer,bounds=Bounds(lower,upper),constraints=constraint,
97     options={'time_limit':time_limit,'mip_rel_gap':1e-9})
98 route='milp';attempts=[{'route':route,'status':int(result.status),'message':str(result.message)}]
99 if not result.success and result.status==1 and not relax:
100     # 先检验 LP 下界能否直接给出满足充放电互斥的解。
101     # 只有该解同时是原 MILP 可行解, 才能用 LP 下界证明原问题最优。
102     linear=milp(objective,integrality=np.zeros(cursor),bounds=Bounds(lower,upper),constraints=constraint,
103         options={'time_limit':time_limit})
104     attempts.append({'route':'lp_certificate','status':int(linear.status),'message':str(linear.message)})
105     overlap=np.inf if not linear.success else
106     ⇨ float(np.minimum(np.maximum(linear.x[c],0),np.maximum(linear.x[d],0)).sum())
107 if linear.success and overlap<=1e-7:
108     linear.x[z]=(linear.x[c]>1e-7).astype(float)
109     linear['mip_dual_bound']=float(linear.fun);result=linear;route='lp_bound_with_exclusive_dispatch'
110 else:
111     result=milp(objective,integrality=integer,bounds=Bounds(lower,upper),constraints=constraint,
112         options={'time_limit':max(120.,time_limit),'mip_rel_gap':1e-9,'presolve':False})
113     route='milp_without_presolve';attempts.append({'route':route,'status':int(result.status),'message':str(
114         ⇨ result.message),'lp_overlap_kwh':float(overlap)})
115 if not result.success:raise RuntimeError(f'Q3 solve failed: {attempts}')
116 x=result.x;R,C,D,S=x[r],x[c],x[d],x[s]
117 deficit=net+C-D-R;E=np.maximum(deficit,0);U=np.maximum(-deficit,0)
118 loss=E@S(5*p)
119 normal=float(p@R) if q is None else float(settlement(p,q,R,settlement_mode).sum())
120 expected=float(prob@loss);tail=weighted_cvar(loss,prob,alpha)
121 terminal=terminal_gamma*max(terminal_target-S[-1],0)

```

```

120 total=normal+expected+risk_lambda*tail+terminal
121 dual=float(getattr(result, 'mip_dual_bound', result.fun))+constant
122 checks=dispatch_checks(C,D,S,battery,initial_kwh)
123 if relax:checks.pop('simultaneous_charge_discharge_kwh')
124 checks.update(objective_yuan=abs(total-(float(result.fun)+constant)),
125               recourse_kwh=float(abs(x[e]-E).max()),
126               bus_balance_kwh=float(abs(R+E+pv+D-load-C-U).max()))
127 if q is not None:checks['opposing_adjustments_kwh']=float(np.minimum(x[ap],x[am]).max())
128 if terminal_kwh is not None:checks['terminal_kwh']=abs(S[-1]-terminal_kwh)
129 require_checks(checks,1e-4)
130 if total-dual>1e-4+1e-8*abs(total):raise RuntimeError('Requested objective bound gap not attained')
131 return {'plan':{'R':R,'C':C,'D':D,'S':S,'Z':x[z]},'objective_yuan':float(total),
132         'normal_cost_yuan':normal,'expected_emergency_yuan':expected,'scenario_cvar_yuan':tail,
133         'terminal_penalty_yuan':float(terminal),'gap_yuan':float(max(0,total-dual)),
134         'runtime_s':time.perf_counter()-start,'checks':checks,'scenario_losses_yuan':loss,
135         'status':'optimal_to_tolerance','relaxed':relax,'solver_route':route,'attempts':attempts}

```

q3_workflow.py

```

1 """PDF 主模型 B3、B0—B4 对照与严格历史验证；独立进程只写各自实验目录。"""
2 from pathlib import Path
3 from concurrent.futures import ProcessPoolExecutor,as_completed
4 from itertools import product
5 from contextlib import redirect_stdout
6 import argparse,json,hashlib,shutil
7 import numpy as np
8 from run_q3 import ROOT,run_policy,write_json,write_csv,score_modes
9 from forecasting import read_dataset
10 from q3_forecasting import build_cache,read_official,MODES
11
12 GRID=[{'window':w,'shrink':s,'bin_hours':b,'floor':25.,'interpretation':'linear_nodes'}
13        for w,s,b in product([14,28],[3.,7.],[3,6])]
14
15 def config_id(c):
16     return f"w{c['window']}_s{c['shrink']}_g{c['bin_hours']}"+(' _mean' if c.get('interpretation')== 'hourly_mean'
17     ↪ else '')
18
19 def get_cache(run,c,ds=None):
20     path=Path(run)/'caches'/f'{config_id(c)}.npz'
21     if path.exists():return dict(np.load(path))
22     ds=read_dataset(ROOT/'data/processed') if ds is None else ds
23     official=read_official(ROOT/'data/processed/pv_forecasts.csv',ds.dates)
24     cache=build_cache(ds,official,**c);path.parent.mkdir(exist_ok=True)
25     np.savez_compressed(path,**cache);return cache
26
27 def validation_job(run,c,initial,gammas):
28     run=Path(run);name=config_id(c);folder=run/'validation'/name;folder.mkdir(parents=True,exist_ok=True)
29     ds=read_dataset(ROOT/'data/processed');cache=get_cache(run,c,ds)
30     scores=score_modes(ds,cache,20,31);write_csv(folder/'probability_scores.csv',scores)
31     crps={m:sum(r['crps']*r['points'] for r in scores if r['mode']==m and r['subset']=='daylight')/
32           sum(r['points'] for r in scores if r['mode']==m and r['subset']=='daylight') for m in MODES}
33     candidates=[]
34     with (folder/'progress.log').open('a',encoding='utf-8') as log,redirect_stdout(log):
35         for mode in MODES:
36             for j,gamma in enumerate(gammas):
37                 policy={'mode':mode,'gamma':gamma,'mask':7,'calibration':c}
38                 result=run_policy(ds,cache,folder/f'{mode}_gamma{j}',20,31,initial,policy)
39                 candidates.append({'config_id':name,'calibration':c,'mode':mode,'gamma':gamma,
40                                   'total_fee_yuan':result['total_fee_yuan'],'daylight_crps_kw':crps[mode],
41                                   'exponential_eligible':crps['exponential']<=min(crps['pooled'],crps['binned'])})
42     write_json(folder/'candidates.json',candidates);return candidates
43
44 def experiment_job(run,name,policy,initial,start=31,end=365):
45     run=Path(run);ds=read_dataset(ROOT/'data/processed');cache=get_cache(run,policy['calibration'],ds)
46     folder=run/('experiments' if start==31 else 'risk_validation')/name;folder.mkdir(parents=True,exist_ok=True)
47     with (folder/'progress.log').open('a',encoding='utf-8') as log,redirect_stdout(log):
48         return name,run_policy(ds,cache,folder,start,end,initial,policy)
49
50 def build_experiments(selection,gamma0):
51     selected=selection['selected']
52     base={k:selected[k] for k in ['mode','gamma','calibration']};base['mask']=7

```

```

52 experiments={'main':base,'point_official':{'**base','point':True},
53             'scale_pooled':{'**base','mode':'pooled'}}
54 experiments.update({f'info_mask{m}':{'**base','mask':m} for m in range(7)})
55 for mode in ('binned','exponential'):
56     if mode!=base['mode']:experiments['scale_'+mode]={'**base','mode':mode}
57 for alpha in (.90,.95):
58     for lam in (.05,.1):
59         name=f'cvar{round(alpha*100)}'+('lambda005' if lam==.05 else '')
60         experiments[name]={'**base','risk_lambda':lam,'alpha':alpha}
61 if base['gamma']!=0:experiments['terminal_zero']={'**base','gamma':0.}
62 else:experiments['terminal_soft']={'**base','gamma':gamma0}
63 experiments['terminal_hard']={'**base','gamma':0.,'daily_terminal':True}
64 experiments['settlement_alternative']={'**base','settlement':'no_refund_penalty'}
65 experiments['efficiency_roundtrip90']={'**base','eta':float(np.sqrt(.9))}
66 experiments['hourly_mean']={'**base','calibration':{'**base['calibration'],'interpretation':'hourly_mean'}}
67 for key in ['probability_selected','global_cash_selected']:
68     p={k:selection[key][k] for k in ['mode','gamma','calibration']};p['mask']=7
69     if p!=base:experiments[key]=p
70 return experiments
71
72 def run_jobs(jobs,workers,function):
73     with ProcessPoolExecutor(max_workers=workers) as pool:
74         pending={pool.submit(function,*args):name for name,args in jobs}
75         for future in as_completed(pending):
76             value=future.result();print('COMPLETED '+pending[future],flush=True);yield value
77
78 def main():
79     parser=argparse.ArgumentParser();parser.add_argument('--run',required=True)
80     parser.add_argument('--workers',type=int,default=3);parser.add_argument('--phase',choices=['all','calibrate','e']
    ↪ valuate'],default='all')
81     parser.add_argument('--only',help='Comma-separated evaluation experiments')
82     args=parser.parse_args();run=Path(args.run).resolve();run.mkdir(parents=True,exist_ok=True)
83     ds=read_dataset(ROOT/'data/processed');gamma0=float(ds.prices.min()/.9);gammas=[0.,gamma0,2*gamma0]
84     inputs={str(p.relative_to(ROOT)):hashlib.sha256(p.read_bytes()).hexdigest() for p in (ROOT/'data').rglob('*') if
    ↪ p.is_file() and p.suffix in ('.xlsx','.csv')}
85     numeric={n:hashlib.sha256((ROOT/'src'/n).read_bytes()).hexdigest() for n in
86         ['q3_dispatch.py','q3_forecasting.py','run_q3.py','q3_workflow.py','forecasting.py','microgrid_core.py']
    ↪ ''}
87     cfg={'input_sha256':inputs,'numerical_code_sha256':numeric,'calibration_grid':GRID,'gamma_candidates':gammas,
88         'validation_dates':['2025-01-21','2025-01-31'],'evaluation_dates':['2025-02-01','2025-12-31'],
89         'draws':64,'seed':20260912,'sigma_floor_kw':25.,'battery_eta_charge':.9,'battery_eta_discharge':.9,
90         'primary_model':'B3 conditional lead scale, risk neutral; B4 only adds CVaR to exactly this B3',
91         'selection_rule':'Reject exponential per configuration if its January daylight CRPS exceeds pooled or
    ↪ binned; select conditional model by January cash. Also report unrestricted cash selection and
    ↪ conditional CRPS selection.',
92         'risk_grid':{'alpha': [.9,.95],'lambda': [.05,.1]},
93         'warmup_policy':'Jan1 idle battery and zero Q; Jan2-31 predeclared binned window28 shrink7 bin6
    ↪ gamma=min_price/eta, causal continuous state.',
94         'evaluation_terminal_kwh':6000,'settlement':'p*q+1.5*p*positive(r-q)-0.5*p*positive(q-r)'}
95     configpath=run/'run_config.json'
96     if configpath.exists():
97         old=json.loads(configpath.read_text(encoding='utf-8'))
98         if old!=cfg:raise ValueError('Changed numerical configuration; use a new run directory')
99     else:write_json(configpath,cfg);shutil.copytree(ROOT/'src',run/'code_snapshot',dirs_exist_ok=True)
100     if args.phase in ('all','calibrate'):
101         warm_c=GRID[-1];cache=get_cache(run,warm_c,ds)
102         warm=run_policy(ds,cache,run/'warmup_jan2_31',1,31,6000.,{'mode':'binned','gamma':gamma0,'mask':7,'calibrat
    ↪ ion':warm_c},end_target=False)
103         write_json(run/'jan1_cold_start.json',{'initial_kwh':6000,'terminal_kwh':6000,'planned_kwh':0.,
104             'emergency_fee_yuan':float(np.maximum(ds.load_kw[0]-ds.pv_kw[0],0)/6/(5*ds.prices)),
105             'reason':'No pre-2025 observed load history was supplied.'})
106         initial=float(dict(np.load(run/'warmup_jan2_31/dispatch.npz'))['S'][-1])
107         candidates=[]
108         jobs=[(config_id(c),(str(run),c,initial,gammas)) for c in GRID]
109         for result in run_jobs(jobs,args.workers,validation_job):candidates.extend(result)
110         candidates.sort(key=lambda x:(x['config_id'],x['mode'],x['gamma']))
111         eligible=[x for x in candidates if x['mode']!='exponential' or x['exponential_eligible']]
112         conditional=[x for x in eligible if x['mode']!='pooled']
113         selected=min(conditional,key=lambda x:x['total_fee_yuan'])
114         # 同一 gamma 下比较概率评分选参和决策费用选参，避免混入终端偏好变化。
115         prob=min([x for x in conditional if x['gamma']==selected['gamma']],key=lambda
    ↪ x:(x['daylight_crps_kw'],x['config_id']))
116         selection={'selected':selected,'global_cash_selected':min(eligible,key=lambda x:x['total_fee_yuan']),

```

```

117         'probability_selected':prob,'all_candidates':candidates,'initial_validation_kwh':initial,
118         'feb1_initial_kwh':warm['terminal_kwh'],'selection_date':'2025-01-31 after 24:00'})
119     write_json(run/'selection.json',selection)
120     write_csv(run/'january_validation_grid.csv',[{k:v for k,v in x.items() if k!='calibration'} for x in
    ↪ candidates])
121     print('SELECTED '+json.dumps(selected),flush=True)
122     selection=json.loads((run/'selection.json').read_text(encoding='utf-8'))
123     experiments=build_experiments(selection,gamma0);write_json(run/'experiment_registry.json',experiments)
124     if args.phase in ('all','calibrate'):
125         jobs=[(name,(str(run),name,p,selection['initial_validation_kwh'],20,31)) for name,p in experiments.items()
    ↪ if name.startswith('cvar'))]
126         risk=[]
127         for name,result in run_jobs(jobs,args.workers,experiment_job):risk.append({'name':name,**result})
128         write_json(run/'january_risk_validation.json',risk)
129     if args.phase in ('all','evaluate'):
130         if args.only:
131             names=args.only.split(',')
132             if set(names)-set(experiments):raise ValueError('Unknown experiment')
133             experiments={n:experiments[n] for n in names}
134             # Build unique caches once before independent experiments start.
135             for p in experiments.values():get_cache(run,p['calibration'],ds)
136             main_cache=get_cache(run,selection['selected']['calibration'],ds)
137             np.savez_compressed(run/'forecast_cache.npz',**main_cache)
138             write_csv(run/'annual_probability_scores.csv',score_modes(ds,main_cache,31,365))
139             jobs=[(name,(str(run),name,p,selection['feb1_initial_kwh'])) for name,p in experiments.items())]
140             for name,result in run_jobs(jobs,args.workers,experiment_job):print(f"{name}: {result['total_fee_yuan']:.6f}
    ↪ yuan",flush=True)
141     print('Q3 WORKFLOW COMPLETED '+str(run),flush=True)
142
143 if __name__=='__main__':main()

```

prepare_q3.py

```

1 """ 从通过审计的最终执行轨迹生成模板载荷、论文表格和完整版本档案。"""
2 from pathlib import Path
3 import argparse,json
4 import numpy as np
5 from forecasting import read_dataset, interval_label, clock_text
6 from reporting import four_hour_rows,emergency_segments
7 from run_q3 import ROOT,write_json,write_csv
8 from audit_q3 import audit_run
9
10 def prepare(run):
11     run=Path(run);audit=audit_run(run)
12     if 'main' not in audit:raise RuntimeError('Main experiment has not completed')
13     ds=read_dataset(ROOT/'data/processed');a=dict(np.load(run/'experiments/main/dispatch.npz'))
14     out=run/'deliverables';out.mkdir(exist_ok=True)
15     Rows=[];Rrows=[];battery=[];emergency=[];daily=[];selected=[];timeline=[];versions=[];omitted=0.
16     for i,day in enumerate(a['day_indices']):
17         date=str(ds.dates[day]);q,r,c,d,e,u,s=[a[k][i] for k in ['Q','R','C','D','E','U','S']]
18         normal=float(a['normal_fee'][i].sum());ecost=float(e*(5*ds.prices));total=normal+ecost
19         Rows.append([date,*q.tolist(),float(q.sum()),float(q@ds.prices)])
20         Rrows.append([date,*r.tolist(),float(r.sum()),total])
21         four=four_hour_rows(c,d,s)
22         for j,row in enumerate(four):battery.append([date if j==0 else None,*row])
23         segments,small=emergency_segments(e);omitted+=small
24         for j,row in enumerate(segments):emergency.append([date if j==0 else
    ↪ None,row['interval'],row['energy_kwh']])
25         daily.append({'date':date,'planned_kwh':float(q.sum()),'adjusted_normal_kwh':float(r.sum()),
26                     'emergency_kwh':float(e.sum()),'original_plan_fee_yuan':float(q@ds.prices),
27                     'up_surcharge_yuan':float(1.5*ds.prices@np.maximum(r-q,0)),
28                     'down_refund_yuan':float(.5*ds.prices@np.maximum(q-r,0)),
29                     'final_normal_fee_yuan':normal,'emergency_fee_yuan':ecost,'total_fee_yuan':total,
30                     'initial_kwh':float(s[0]),'terminal_kwh':float(s[-1]),'unused_kwh':float(u.sum())})
31         if date in ['2025-03-20','2025-06-21','2025-09-23','2025-12-21']:
32             selected.append({'daily':-1,'purchases':[[interval_label(h*6),float(q[h*6]),float(r[h*6])] for h in
    ↪ [10,12,14,16,18,20]],
33                             'battery':four,'emergency_segments':segments})
34         for t in range(144):
35             timeline.append({'date':date,'interval':interval_label(t),'load_kw':float(ds.load_kw[day,t]),
36                             'pv_actual_kw':float(ds.pv_kw[day,t]),'price_yuan_per_kwh':float(ds.prices[t]),

```

```

37         **{k:float(a[k][i,t]) for k in ['Q','R','C','D','E','U','normal_fee']],
38         'S_start_kwh':float(s[t]), 'S_end_kwh':float(s[t+1])})
39     for j,h in enumerate([0,6,12,18]):
40         for t in range(h*6,144):
41             versions.append({'date':date, 'decision_hour':h, 'interval':interval_label(t),
42                             'is_executed_version':int(h*6<=t<h*6+36), 'original_Q_kwh':float(q[t]),
43                             **{k+'_kwh':float(a[k+'_versions'][i,j,t]) for k in ['R','C','D'])})
44     if omitted>1e-4:raise RuntimeError('Material emergency energy omitted')
45     payload={'headers':[ '日期\\时间',*[interval_label(t) for t in range(144)], '全天购电量', '全天购电费'],
46             'plan':Qrows, 'adjusted':Rrows, 'battery':battery, 'emergency':emergency,
47             'emergency_omitted_kwh':omitted, 'total_fee_yuan':sum(x['total_fee_yuan'] for x in daily)}
48     write_json(run/'q3_workbook_payload.json',payload);write_json(out/'指定日期结果.json',selected)
49     write_csv(out/'第三问每日费用与电量.csv',daily);write_csv(out/'第三问 10 分钟完整执行轨迹.csv',timeline)
50     write_csv(out/'第三问各时刻完整计划版本.csv',versions)
51     text=['# 第三问: 题目指定日期结果', '', '单位: 电量 kWh, 费用元。Q为0点承诺; R为该时段最终执行的正常购电量, 不是调整增量。
52     ↪ 全天总费用包括正常结算与紧急购电。', '']
53     for row in selected:
54         text.extend(['## {row['date']}','', '| 时间段 | 0 点计划 Q| 最终调整 R|', '|---|---|---|'])
55         text.extend(['|{t}|{q:.2f}|{r:.2f}|' for t,q,r in row['purchases']])
56         text.extend(['',f"全天 Q = {row['planned_kwh']:.2f}; R = {row['adjusted_normal_kwh']:.2f}; 紧急购电 =
57         ↪ {row['emergency_kwh']:.2f}。",
58                 f"原计划费 = {row['original_plan_fee_yuan']:.2f}; 最终正常结算 =
59                 ↪ {row['final_normal_fee_yuan']:.2f}; 紧急费 = {row['emergency_fee_yuan']:.2f}; 总费 =
60                 ↪ **{row['total_fee_yuan']:.2f}**。", ''],
61                 '| 时间段 | 充电量 | 放电量 |', '|---|---|---|'])
62         text.extend(['|{b[0]}|{b[1]:.2f}|{b[2]:.2f}|' for b in row['battery']])
63         text.extend(['',f"0:00储量 {row['initial_kwh']:.2f}; 24:00储量 {row['terminal_kwh']:.2f}。
64         ↪ ", '|紧急购电时间段|购电量|', '|---|---|'])
65         text.extend(['|{e['interval']}|{e['energy_kwh']:.2f}|' for e in row['emergency_segments']])
66         if not row['emergency_segments']:text.append('| 无 |0.00|')
67         text.append('')
68     (out/'题目指定日期表格.md').write_text('\n'.join(text),encoding='utf-8')
69     print(json.dumps({'days':len(daily), 'emergency_segments':len(emergency), 'version_rows':len(versions), 'total_fee':
70     ↪ 'payload['total_fee_yuan']}))
71
72 if __name__=='__main__':
73     parser=argparse.ArgumentParser();parser.add_argument('run');args=parser.parse_args();prepare(args.run)

```

run_q3.py

```

1 """ 第三问: 连续热身、1 月验证、2—12 月逐日滚动回测; 可断点恢复。"""
2 from pathlib import Path
3 from dataclasses import asdict
4 import argparse, csv, hashlib, json, shutil, time
5 import numpy as np
6 from forecasting import read_dataset
7 from microgrid_core import Battery, dispatch_checks, require_checks
8 from q3_dispatch import solve_roll, settlement, weighted_cvar
9 from q3_forecasting import read_official, build_cache, make_scenarios, probability_scores, MODES
10
11 ROOT=Path(__file__).resolve().parents[1]
12 BATTERY=Battery(.9,.9)
13
14 def write_json(path,value):
15     Path(path).write_text(json.dumps(value,ensure_ascii=False,indent=2,allow_nan=False),encoding='utf-8')
16
17 def write_csv(path,rows):
18     if not rows:return
19     with Path(path).open('w',encoding='utf-8-sig',newline='') as f:
20         writer=csv.DictWriter(f,fieldnames=list(rows[0]));writer.writeheader();writer.writerows(rows)
21
22 def run_policy(ds,cache,folder,start,end,initial,policy,*,end_target=True):
23     folder=Path(folder);folder.mkdir(parents=True,exist_ok=True)
24     eta=policy.get('eta',.9);battery=Battery(eta,eta)
25     if (folder/'summary.json').exists():
26         saved=json.loads((folder/'summary.json').read_text(encoding='utf-8'))
27         if saved['policy']!=policy or saved['start']!=str(ds.dates[start]) or saved['end']!=str(ds.dates[end-1]) or
28         ↪ abs(saved['initial_kwh']-initial)>1e-7:
29             raise ValueError('Existing result belongs to different settings; use a new run directory')
30     return saved
31     write_json(folder/'policy.json',policy)

```

```

31 days=end-start;n=144
32 arrays={k:np.zeros((days,n)) for k in ['Q','R','C','D','E','U','normal_fee']}
33 arrays['S']=np.zeros((days,145))
34 for k in ['R_versions','C_versions','D_versions']:arrays[k]=np.full((days,4,n),np.nan)
35 daily=[];updates=[];state=float(initial);started=time.perf_counter();completed=0
36 checkpoint=folder/'checkpoint.npz';metadata=folder/'checkpoint.json'
37 if checkpoint.exists() and metadata.exists():
38     saved=json.loads(metadata.read_text(encoding='utf-8'));completed=saved['completed_days']
39     if saved['policy']!=policy or saved['start']!=start or saved['end']!=end or saved['initial']!=initial:raise
    ↪ ValueError('Incompatible checkpoint')
40     arrays.update(dict(np.load(checkpoint)));daily=saved['daily'];updates=saved['updates'];state=saved['state']
41 for di,day in enumerate(range(start,end)):
42     if di<completed:continue
43     S=arrays['S'][di];S[0]=state
44     for issue,hour in enumerate([0,6,12,18]):
45         t=hour*6;sl=slice(t,t+36)
46         active=[0]+[h for j,h in enumerate([6,12,18]) if policy.get('mask',7)&(1<=j) and h<=hour]
47         source=max(active)
48         L,G,prob,audit=make_scenarios(ds,cache,day,hour,source,mode=policy['mode'],point=policy.get('point',False),
    ↪ se))
49         terminal=6000. if policy.get('daily_terminal',False) or (end_target and day==end-1) else None
50         out=solve_roll(ds.prices[t:],L,G,prob,battery=battery,initial_kwh=state,
51                         original_q=None if issue==0 else arrays['Q'][di,t:],
52                         risk_lambda=policy.get('risk_lambda',0.),alpha=policy.get('alpha',.9),
53                         terminal_gamma=policy['gamma'],terminal_kwh=terminal,
54                         settlement_mode=policy.get('settlement','net_refund'))
55         plan=out['plan']
56         if issue==0:arrays['Q'][di]=plan['R']
57         for k in ['R','C','D']:
58             arrays[k][di,sl]=plan[k][:36];arrays[k+'_versions'][di,issue,t]=plan[k]
59         # 独立按已执行充放电递推实际状态，下一轮接入这个状态。
60         c,d=arrays['C'][di,sl],arrays['D'][di,sl]
61         S[t+1:t+37]=state+np.cumsum(eta*c-d/eta);state=float(S[t+36])
62         deficit=(ds.load_kw[day,sl]-ds.pv_kw[day,sl])/6+c-d-arrays['R'][di,sl]
63         arrays['E'][di,sl]=np.maximum(deficit,0);arrays['U'][di,sl]=np.maximum(-deficit,0)
64         audit.update(objective_yuan=out['objective_yuan'],scenario_cvar_yuan=out['scenario_cvar_yuan'],
65                      terminal_penalty_yuan=out['terminal_penalty_yuan'],solver_gap_yuan=out['gap_yuan'],
66                      runtime_s=out['runtime_s'],max_model_residual=max(out['checks'].values()),
67                      initial_kwh=float(S[t]),executed_end_kwh=state,
68                      solver_route=out['solver_route'],solver_attempts=out['attempts'])
69         updates.append(audit)
70         Q,R,C,D,E,U=[arrays[k][di] for k in ['Q','R','C','D','E','U']]
71         fee=settlement(ds.prices,Q,R,policy.get('settlement','net_refund'));arrays['normal_fee'][di]=fee
72         checks=dispatch_checks(C,D,S,battery,float(S[0]))
73         if policy.get('daily_terminal'):checks['daily_terminal']=abs(S[-1]-6000)
74         checks['balance']=float(abs(R+E+ds.pv_kw[day]/6+D-ds.load_kw[day]/6-C-U).max())
75         checks['midnight_commitment']=float(abs(R[:36]-Q[:36]).max())
76         checks['negative_purchase']=float(max(0,-Q.min()),-R.min())
77         require_checks(checks,1e-4)
78         em=float(E@(5*ds.prices));qcost=float(Q@ds.prices)
79         daily.append({'date':ds.dates[day].isoformat(),'q_kwh':float(Q.sum()),'r_kwh':float(R.sum()),
80                     'q_fee_yuan':qcost,'up_fee_yuan':float(1.5*ds.prices@np.maximum(R-Q,0)),
81                     'down_half_fee_yuan':float(.5*ds.prices@np.maximum(Q-R,0)),
82                     'normal_fee_yuan':float(fee.sum()),'emergency_fee_yuan':em,'total_fee_yuan':float(fee.sum()+e
    ↪ m),
83                     'emergency_kwh':float(E.sum()),'unused_kwh':float(U.sum()),'charge_kwh':float(C.sum()),
84                     'discharge_kwh':float(D.sum()),'throughput_kwh':float(C.sum()+D.sum()),
85                     'adjust_down_kwh':float(np.maximum(R-Q,0).sum()),'adjust_down_kwh':float(np.maximum(Q-R,0).sum(
    ↪ )),
86                     'adjusted_intervals':int((abs(R-Q)>1e-6).sum()),
87                     'initial_kwh':float(S[0]),'terminal_kwh':state,
88                     'max_physical_residual':max(checks.values())})
89         if (di+1)%14==0 or day==end-1:
90             print(f'{folder.name}: {di+1}/{days}, {ds.dates[day]}, {time.perf_counter()-started:.1f}s',flush=True)
91             write_json(folder/'progress.json',{'completed_days':di+1,'days':days,'date':str(ds.dates[day])})
92             np.savez_compressed(checkpoint,**arrays)
93             write_json(metadata,{'completed_days':di+1,'policy':policy,'start':start,'end':end,'initial':initial,
94                                 'state':state,'daily':daily,'updates':updates})
95         np.savez_compressed(folder/'dispatch.npz',day_indices=np.arange(start,end),**arrays)
96         write_csv(folder/'daily.csv',daily);write_json(folder/'updates.json',updates)
97         losses=np.array([r['emergency_fee_yuan'] for r in daily]);total=sum(r['total_fee_yuan'] for r in daily)
98         summary={'policy':policy,'start':str(ds.dates[start]),'end':str(ds.dates[end-1]),'days':days,
99                 'total_fee_yuan':total,'normal_fee_yuan':sum(r['normal_fee_yuan'] for r in daily),

```

```

100         'emergency_fee_yuan':float(losses.sum()),'emergency_kwh':float(arrays['E'].sum()),
101         'unused_kwh':float(arrays['U'].sum()),'daily_emergency_cvar90_yuan':weighted_cvar(losses,np.ones(days)
↪ /days,.9),
102         'daily_emergency_cvar95_yuan':weighted_cvar(losses,np.ones(days)/days,.95),
103         'emergency_days':int(np.sum(arrays['E'].sum(axis=1)>1e-6)),
104         'initial_kwh':initial,'terminal_kwh':state,'runtime_s':time.perf_counter()-started,
105         'max_physical_residual':max(r['max_physical_residual'] for r in daily),
106         'max_solver_gap_yuan':max(u['solver_gap_yuan'] for u in updates)}
107     for key in
↪ ['charge_kwh','discharge_kwh','throughput_kwh','adjust_up_kwh','adjust_down_kwh','adjusted_intervals']:
108         summary[key]=sum(r[key] for r in daily)
109     summary['solver_runtime_total_s']=sum(u['runtime_s'] for u in updates)
110     summary['solver_runtime_max_s']=max(u['runtime_s'] for u in updates)
111     write_json(folder/'summary.json',summary);return summary
112
113 def score_modes(ds,cache,start,end):
114     rows=[]
115     for mode in MODES:
116         for hour in [0,6,12,18]:
117             bags={key:[] for key in ['crps','coverage','width','interval_score','mae','squared_error']};active=[]
118             for day in range(start,end):
119                 _,g,p,_=make_scenarios(ds,cache,day,hour,hour,mode=mode)
120                 actual=ds.pv_kw[day,hour*6:];scores=probability_scores(g*6,p,actual)
121                 active.append(actual>0)
122                 for key in bags:bags[key].append(scores[key])
123             mask=np.concatenate(active)
124             for subset,sel in [(('all',np.ones(len(mask),bool)),('daylight',mask))]:
125                 if not sel.any():continue
126                 row={'mode':mode,'issue_hour':hour,'subset':subset,'points':int(sel.sum()),
127                     **{key:float(np.concatenate(value)[sel].mean()) for key,value in bags.items()}}
128                 row['rmse']=float(np.sqrt(row.pop('squared_error')));rows.append(row)
129     return rows
130
131 if __name__=='__main__':
132     from q3_workflow import main
133     main()

```

q3_diagnostics.py

```

1 """PDF 第 4、5、12 节: R、分组点/概率误差和场景截断影响。评价不反馈给决策。"""
2 from pathlib import Path
3 from collections import defaultdict
4 import csv,json
5 import numpy as np
6 from forecasting import read_dataset
7 from run_q3 import ROOT,write_csv,write_json
8 from q3_forecasting import MODES,make_scenarios,probability_scores,masks,scale_from_reliability
9
10 def diagnostics(run):
11     run=Path(run);out=run/'deliverables';out.mkdir(exist_ok=True)
12     ds=read_dataset(ROOT/'data/processed');cache=dict(np.load(run/'forecast_cache.npz'))
13     selection=json.loads((run/'selection.json').read_text(encoding='utf-8'));selected=selection['selected']['mode']
14     totals={};moment_rows=[];trace=[];inverse_error=0.
15     for day in range(31,365):
16         for hour in (0,6,12,18):
17             t=hour*6;issue=hour//6;y=ds.pv_kw[day,t:];lead=(np.arange(t,144)+1)/6-hour
18             raw=cache['raw_g'][day,issue,t:];mu=cache['mu_g'][day,issue,t:]
19             for mode in MODES:
20                 le,ge,p,audit=make_scenarios(ds,cache,day,hour,hour,mode=mode);l,g=le*6,ge*6
21                 scores=probability_scores(g,p,y)
22                 arrays={**scores,'raw_mae':abs(raw-y),'raw_mse':(raw-y)**2,'center_mae':abs(mu-y),'center_mse':(mu-
↪ y)**2}
23                 for b in [0,*range(1,25-hour)]:
24                     leadmask=np.ones(len(y),bool) if b==0 else np.ceil(lead)==b
25                     for subset,mask in [(('all',leadmask),('daylight',leadmask&(y>0)),('zero_pv',leadmask&(y==0)))]:
26                         if not mask.any():continue
27                         key=(mode,hour,b,subset)
28                         if key not in totals:totals[key]={ 'points':0,'independent_days':0,**{k:0. for k in arrays}}
29                         acc=totals[key];acc['points']+=int(mask.sum());acc['independent_days']+=1
30                         for k,v in arrays.items():acc[k]+=float(v[mask].sum())
31     unique=np.unique(audit['sampled_day_indices'])

```

```

32     zg=(ds.pv_kw[unique,t:]-cache['mu_g_'][unique,issue,t:])/cache['sigma_g_'+mode][unique,issue,t:]
33     zl=(ds.load_kw[unique,t:]-cache['mu_l_'][unique,t:])/cache['sigma_l_'][unique,t:]
34     unclipped=mu+cache['sigma_g_'+mode][day,issue,t:]*zg
35     ml=p@l;mg=p@g;vl=p@((l-ml)**2);vg=p@((g-mg)**2);cov=p@((l-ml)*(g-mg))
36     corr=np.divide(cov,np.sqrt(vl*vg),out=np.full_like(vg,np.nan),where=vl*vg>1e-16)
37     raw_l=cache['mu_l_'][day,t:]+cache['sigma_l_'][day,t:]*zl
38     uml=p@raw_l;umg=p@unclipped;uvl=p@((raw_l-uml)**2);uvg=p@((unclipped-umg)**2)
39     ucov=p@((raw_l-uml)*(unclipped-umg))
40     ucorr=np.divide(ucov,np.sqrt(uvl*uvg),out=np.full_like(uvg,np.nan),where=uvl*uvg>1e-16)
41     for subset,mask in [(('all',np.ones(len(y),bool)),('daylight',y>0)),('zero_pv',y==0)]:
42         if not mask.any():continue
43         finite=mask&np.isfinite(corr)&np.isfinite(ucorr)
44         moment_rows.append({'date':str(ds.dates[day]),'mode':mode,'issue_hour':hour,'subset':subset,'po
↳ ints':int(mask.sum()),
45         'clipped_probability_mean':float((p@unclipped<0))[mask].mean()),
46         'center_to_scenario_mean_abs_kw':float(abs(mg-mu)[mask].mean()),
47         'scale_to_scenario_std_abs_kw':float(abs(np.sqrt(vg)-cache['sigma_g_'+mode][day,issue,t:])[
↳ mask].mean()),
48         'clipping_mean_increase_kw':float((mg-umg)[mask].mean()),
49         'clipping_variance_change_kw2':float((vg-uvg)[mask].mean()),
50         'joint_covariance_before_kw2':float(ucov[mask].mean()),'joint_covariance_after_kw2':float(c
↳ ov[mask].mean()),
51         'correlation_defined_points':int(finite.sum()),
52         'joint_correlation_before':float(ucorr[finite].mean()) if finite.any() else None,
53         'joint_correlation_after':float(corr[finite].mean()) if finite.any() else None})
54     for offset,h in enumerate(lead):
55         row={'date':str(ds.dates[day]),'issue_hour':hour,'valid_interval_end_hour':float((t+offset+1)/6),
56         'original_lead_hours':float(h),'raw_pv_kw':float(raw[offset]),'calibrated_center_kw':float(mu[
↳ offset]),
57         's_ref_kw':float(cache['sref'][day]),'beta_per_hour':float(cache['beta'][day]),
58         'historical_days':min(day,int(cache['window_days']))})
59     for mode in MODES:
60         s=float(cache['sigma_g_'+mode][day,issue,t+offset]);r=float(cache['reliability_g_'+mode][day,is
↳ sue,t+offset])
61         row['sigma_'+mode+'_kw']=s;row['R_'+mode]=r
62         inverse_error=max(inverse_error,abs(float(scale_from_reliability(r,row['s_ref_kw']))-s))
63         trace.append(row)
64     rows=[]
65     for (mode,hour,b,subset),acc in sorted(totals.items()):
66         n=acc['points'];row={'mode':mode,'issue_hour':hour,'lead_hour_bin':b,'subset':subset,
67         'points':n,'independent_days':acc['independent_days']}
68         for key in ('crps','coverage','width','interval_score','mae','raw_mae','center_mae'):row[key]=acc[key]/n
69         row.update(rmse=np.sqrt(acc['squared_error']/n),raw_rmse=np.sqrt(acc['raw_mse']/n),center_rmse=np.sqrt(acc[
↳ 'center_mse']/n))
70         rows.append(row)
71     write_csv(out/'提前量与发布时间分组误差.csv',rows)
72     write_csv(out/'非负截断与联合场景诊断.csv',moment_rows)
73     write_csv(out/'预测尺度与可信度 R 完整记录.csv',trace)
74     write_json(run/'probability_diagnostics_audit.json',{'R_to_sigma_max_error_kw':inverse_error,'reliability_rows'
↳ :len(trace),
75         'score_rows':len(rows),'scene_moment_rows':len(moment_rows),'evaluation_days':334,
76         'zero_pv_definition':'Actual PV equals zero; this post-hoc evaluation group includes night and potentially
↳ zero-generation daytime.',
77         'selected_mode':selected,'grouping':'lead_hour_bin=0 means all leads; scores in kW; repeated releases are
↳ not independent observations.',
78         'correlation_definition':'Scenario-probability-weighted L/G covariance and correlation at each time, then
↳ averaged over defined time points; not causal independence.')}
79     print('DIAGNOSTICS '+json.dumps({'R_inverse_error_kw':inverse_error,'rows':len(trace)}),flush=True)
80
81 if __name__=='__main__':
82     import sys
83     diagnostics(sys.argv[1])

```

q3_extra_figures.py

```

1 """ 保持现有科研图风格, 增加 PDF 消融链、终端对照和 R 可解释性。"""
2 import numpy as np
3 import pandas as pd
4 from scientific_style import plt,COLORS,panel,note,save_figure
5
6 def extra_figures(run,records,out,pdf):

```

```

7 qa=[]
8 fig,axs=plt.subplots(1,2,figsize=(9,4.4));fig.subplots_adjust(left=.10,right=.97,top=.86,bottom=.30,wspace=.35)
9 names=['info_mask0','point_official','scale_pooled','main','cvar95']
10 labels=['B0\n 仅 0 点预报','B1\n 原始点预测','B2\n 统一尺度','B3\n 条件尺度','B4\nCVaR95']
11 colors=[COLORS['muted'],COLORS['planned_cost'],COLORS['purchase'],COLORS['pv'],COLORS['state']]
12 for ax,metric,ylabel,title in [(axs[0],'total_fee_yuan','2—12 月总费用 (万元)','逐项模型对照'),
13                               (axs[1],'daily_emergency_cvar95_yuan','日紧急费用 CVaR95 (万元)','实现尾部风险')]:
14     values=[records[n][metric]/1e4 for n in names]
15     ax.bar(range(5),values,color=colors,width=.62)
16     ax.set_xticks(range(5),labels,fontsize=7);ax.set_ylabel(ylabel);ax.set_ylim(0,max(values)*1.18)
17     for j,v in enumerate(values):ax.text(j,v+max(values)*.025,f'{v:.2f}',ha='center',fontsize=7)
18     panel(ax,'a' if ax is axs[0] else 'b',title)
19 note(fig,'B2-B3 只改变误差尺度; B3-B4 只增加风险项 ( $\alpha=0.95$ ,  $\lambda=0.1$ )。B0 仍每日优化 4 次, 仅屏蔽日内新预报。')
20 qa.append(save_figure(fig,out,'05_B0 至 B4 模型对照',pdf=pdf))
21
22 cache=dict(np.load(run/'forecast_cache.npz'));day=171;reference=cache['sref'][day]
23 fig,axs=plt.subplots(1,2,figsize=(9,4.3));fig.subplots_adjust(left=.10,right=.97,top=.84,bottom=.28,wspace=.34)
24 for g,color in enumerate([COLORS['muted'],COLORS['purchase'],COLORS['pv'],COLORS['state']]):
25     # 图示式 14 函数: 仅画当天剩余时域能出现的提前量支持区间。
26     h=np.linspace(1/6,(g+1)*6,100);sigma=cache['sigma0'][day,g]*np.exp(cache['beta'][day]*h)
27     axs[0].plot(h,sigma,color=color,label=f'目标时刻 {6*g}—{6*g+6}h')
28     axs[1].plot(h,1/(1+(sigma/reference)**2),color=color)
29 for ax in axs:ax.set_xlabel('原始预报提前量 h (小时)');ax.set_xticks([0,6,12,18,24])
30 axs[0].set_ylabel('指数条件尺度  $\sigma$  (kW)');axs[0].legend(fontsize=7);panel(axs[0],'a','6 月 21 日历史拟合的指数候选')
31 axs[1].set_ylabel('可信度指标 R');axs[1].set_ylim(0,1.05);panel(axs[1],'b','同组内由尺度确定的 R')
32 note(fig,'R=1/[1+( $\sigma/s_{ref}$ )2], 不是正确概率。本图解释指数候选; 主方案按 1 月验证使用分箱尺度, 其 R 不强单调。')
33 qa.append(save_figure(fig,out,'06_ 提前量尺度与可信度解释',pdf=pdf))
34
35 fig,axs=plt.subplots(1,2,figsize=(9,4.3));fig.subplots_adjust(left=.11,right=.97,top=.84,bottom=.28,wspace=.39)
36 names=['terminal_zero' if 'terminal_zero' in records else 'main','main' if records['main']['gamma'] else
37         'terminal_soft','terminal_hard']
38 labels=['无软目标','正软目标','每日硬目标']
39 for j,name in enumerate(names):
40     row=records[name];axs[0].bar(j,row['total_fee_yuan']/1e4,color=[COLORS['muted'],COLORS['purchase'],COLORS['state']][j],width=.6)
41     axs[0].set_xticks(range(3),labels);axs[0].set_ylabel('2—12月总费用 (万元)');panel(axs[0],'a','相同预测与信息条件的终端对照')
42 for alpha,color,marker in [(0.9,COLORS['purchase'],'o'),(0.95,COLORS['state'],'s')]:
43     names=['main',f'cvar{round(100*alpha)}_lambda005',f'cvar{round(100*alpha)}']
44     x=[records[n]['total_fee_yuan']/1e4 for n in names];y=[records[n]['daily_emergency_cvar95_yuan']/1e4 for n
45         in names]
46     axs[1].plot(x,y,color=color,marker=marker,label=f' $\alpha={alpha:.2f}$ ')
47     for xx,yy,lam in zip(x[1:],y[1:],[0.05,.1]):axs[1].annotate(f' $\lambda={lam:g}$ ',(xx,yy),xytext=(4,5),textcoords='offset
48         points',fontsize=7)
49     axs[1].set_xlabel('2—12 月总费用 (万元)');axs[1].set_ylabel('日紧急费用 CVaR95 (万元)');axs[1].legend(fontsize=7)
50 panel(axs[1],'b','CVaR风险偏好网格');note(fig,'所有方案跨日连续并在年末统一6000 kWh。软目标仅进入优化目标, 不计入实际电费;
51      $\lambda=0$ 对应B3。')
52 qa.append(save_figure(fig,out,'07_ 终端条件与风险参数',pdf=pdf))
53 return qa

```

q3_report.py

```

1 """ 根据本次已完成实验生成论文可引用报告, 禁止硬编码旧版数值或测试数量。"""
2 import json,shutil,platform
3 import numpy as np
4 import pandas as pd
5 from audit_q3 import audit_run
6 from run_q3 import write_json,write_csv
7
8 CROSSWALK=[
9     ('式(1)-(32)','R+E+G+D=L+C+U','q3_dispatch.solve_roll; run_q3.run_policy','Q为0点原承诺, R替代Q; U为总富余, 无售电收益、
10     无U≤G约束。'),
11     ('式(2)-(8)',' $\Delta t=1/6h$ ,  $T=144$ ; h=有效时刻-原发布时刻','q3_forecasting.read_official / integrate_hourly /
12     make_scenarios','旧预报复用时h不归零; 整点节点线性积分为主, 小时均值解释另跑全年。'),
13     ('式(9)-(11)','分组偏差收缩:  $\mu G=\max(0, \text{原始预报}+b)$ ','q3_forecasting.day_moments / build_cache','按完整历史日等权;
14     删除旧版历史零支撑掩码, 保留晨昏误差可能性。'),
15     ('式(12)-(15)','历史因果残差方差收缩:  $\sigma(h)=\sigma_0 \exp(\beta h)$ ,  $\beta \geq 0$ ','q3_forecasting.fit_scale','目标钟点粗组与提前量分箱;
16     指数通过带组截距、共享斜率的日数加权拟合。分箱、统一尺度作为对照。'),
17     ('式(16)-(19)',' $R=1/[1+(\sigma/s_{ref})^2]$ ;  $\sigma=s_{ref}\sqrt{(1/R-1)}$ ','q3_forecasting.reliability /
18     scale_from_reliability','R全部导出并独立反演核。场景直接用等价 $\sigma$ , R不乘预测中心, 也不替代场景概率。'),

```

```

14 ('式(20)一(23)', '同日成对L/G标准残差块, 经当前 $\mu$ 、 $\sigma$ 还原, 截断为非负', 'q3_forecasting.make_scenarios', '只抽已完成日;
    ↳ 64次有放回抽样并按重复次数压缩权重; 补充截断前后矩、协方差诊断。'),
15 ('式(24)一(28)', 'pQ+1.5p(R-Q)+-0.5p(Q-R)+', 'q3_dispatch.settlement;
    ↳ audit_q3.audit_arrays', '每个时段最终执行版相对原始Q结算一次; 下调另罚备选口径单列, 不相加滚动目标。'),
16 ('式(29)一(38)', '场景紧急费用CvAR:  $\zeta$ 实数、 $v \geq Cw - \zeta$ 、 $v \geq 0$ ', 'q3_dispatch.solve_roll / weighted_cvar', 'B4复用B3的全部校准、场景、
    ↳  $\gamma$ 与信息, 只增加 $\alpha$ 、 $\lambda$ ; 另按实现日紧急费重算CvAR。'),
17 ('式(33)一(40)', '电池效率、界限、互斥与软终端 $\Phi = \gamma(6000 - S_{未}) +$ ', 'q3_dispatch.solve_roll;
    ↳ run_q3.run_policy', 'C/D交流侧 $\leq 5000/6\text{kWh}$ ;  $S \in [1200, 10800]$ 。无软目标、正软目标、每日硬6000均已比较。'),
18 ('第8—10节', '0/6/12/18h滚动; 只执行下一段36点; 跨日连续', 'run_q3.run_policy', '保留四次剩余计划版本。首次0点解得到Q;
    ↳ 6小时内电池按固定计划执行。'),
19 ('第11节', '8种信息组合与B0—B4消融', 'q3_workflow.build_experiments; summarize_q3.collect', '屏蔽新预报仍每天重算4次;
    ↳ 配对7日移动块bootstrap估计单年探索性区间。'),
20 ('第12节', '历史概率/费用选参、分组误差、风险与运行指标', 'q3_workflow; q3_diagnostics; q3_report', '72组1月候选;  $\alpha = .90/.95$ 、
    ↳  $\lambda = 0/.05/.1$ ; 提前量/发布时刻/实际光伏正或零分组MAE、RMSE与概率评分。')]
21
22 def report(run, records, marginal, maskdaily):
23     out=run/'deliverables'; selection=json.loads((run/'selection.json').read_text(encoding='utf-8'))
24     main=records['main']; selected=selection['selected']; c=selected['calibration']; audit=audit_run(run)
25     test=json.loads((run/'test_results.json').read_text(encoding='utf-8'))
26     diag=json.loads((run/'probability_diagnostics_audit.json').read_text(encoding='utf-8'))
27     cache=dict(np.load(run/'forecast_cache.npz'))
28     frame=pd.read_csv(out/'提前量与发布时间分组误差.csv')
29     daylight=frame[(frame['mode']==selected['mode']) & (frame['lead_hour_bin']==0) & (frame['subset']=='daylight')]
30     cover=float(np.average(daylight.coverage, weights=daylight.points)); crps=float(np.average(daylight.crps, weights=
    ↳ daylight.points))
31     maxerr=max([v['max_residual'] for v in audit.values()]); best=min(maskdaily, key=lambda m: maskdaily[m].sum())
32     registry=json.loads((run/'experiment_registry.json').read_text(encoding='utf-8'))
33     unique_policies=len([json.dumps(p, sort_keys=True) for p in registry.values()])
34     saving=float((maskdaily[0]-maskdaily[7]).sum()); allnames=[n for n in records if not n.startswith('info_mask')]
35     from summarize_q3 import label, mask_label
36     lines=['# 第三问模型核检与求解报告', '',
37           f"本次按建模手`model-q3.pdf`的条件尺度随机滚动模型重新计算, 正式主方案为 **B3: 分箱条件尺度、风险中性随机MPC**。  

    ↳ 2025年2月1日至12月31日共334天, 实际购电总费 **{main['total_fee_yuan']:, .2f} 元**, 正常结算  

    ↳ {main['normal_fee_yuan']:, .2f} 元, 紧急购电 {main['emergency_fee_yuan']:, .2f} 元。  

    ↳ 该费用是有限历史场景策略的回溯结果, 不是全年全信息最优值。",
38     '', '## 1. 这次修正了什么', '',
39     '删除旧版PDF之外的历史零光伏支撑掩码, 使预测中心严格按式(11)生成; 指数尺度按式(14)生成, 不再额外逐点抬高。  

    ↳ CvAR从原来的统一尺度模型改为叠加在同一个B3条件尺度模型上。补齐每日硬终端对照、参数历史验证、R完整序列、  

    ↳ 提前量误差分组和场景截断影响。代码、提交表、指定日期表与图表均由本次新轨迹生成。',
40     '', '## 2. 公式与代码对应', '', '|PDF 位置 | 数学含义 | 实现入口 | 口径与核检 |', '---|---|---|---|']
41     lines.extend('|'+ '|'.join(row)+'|' for row in CROSSWALK)
42     lines+=['', '## 3. 数据和论文必须写清的实现选择', '',
43           '1. 附件1提供重复使用的144点电价; 附件2提供365×144点实际负荷和光伏; 附件3提供365×4×24条官方预报; 附件5保留result3的四个工作表。  

    ↳ 本问不使用附件4的波动电价。',
44           '2. 00:10—24:00按前—10分钟区间终点解释, 模板改正时段标签, 数值不循环移动。小时官方预报主方案按未来整点节点线性积分;  

    ↳ j=0锚点只用刚结束的实测时段, 0点用前一天末值。小时均值解释另列敏感性结果。',
45           '3. 负荷中心采用过去完整28日加权均值, 属于PDF允许的历史均值分支。没有把它写成第二问version2已选中的随机森林;  

    ↳ 第三问所有消融固定这套负荷预测。若论文写负荷随机森林, 应另实现并重算, 不能仅改模型名称。',
46           f"4. 目标钟点仍为四个6小时粗组; 提前量分箱、历史窗口、偏差与方差共同收缩强度通过1月验证选择。本次为窗口{c['window']}日、  

    ↳ 收缩{c['shrink']}:g)日、提前量{c['bin_hours']}小时分箱。尺度数值下限25kW、抽样64次、随机种子20260912是登记数值设置,  

    ↳ 不宣称也做了全面优化。",
47           f"5. s_ref由首7日已实现的光伏正值时段原始预报RMSE确定, 1月8日起固定为{cache['sref'][31]:.6f} kW; 前7日采用1000kW临时参考。  

    ↳ R与 $\sigma$ 是一映射, R本身没有额外优化收益, R=0.8不表示80%正确率。分箱R可非单调;  $\beta \geq 0$ 仅约束指数候选的同组曲线。",
48           '6. 主效率为充电、放电各0.9; 两侧取 $\sqrt{0.9}$ 的往返效率90%解释单例。C、D为交流母线侧电量, S为内部储量, 容量12000kWh、安全区间1200—  

    ↳ 10800kWh。每6小时重算电池计划, 区间内没有额外实时电池反馈控制。',
49           f"7. 1月1日S=6000kWh; 缺少上年负荷历史, 采用PDF允许的零Q、空闲电池冷启动, 缺口紧急补足。1月2—31日按预先固定28/7/6分箱模型、  

    ↳  $\gamma$ =最低电价/0.9连续热身, 各方案共同从2月1日S={main['initial_kwh']:.9f}kWh出发。没有用1月末选参结果改写过去热身决策。",
50           '8. 每天不重置库存。主方案的6000kWh软目标是近似未来价值; 全年末统一6000kWh是公平比较的附加条件, 不能写成题目要求每天首尾相同。  

    ↳ 硬终端对照才逐日强制6000。软惩罚和CvAR项不计入实际现金账单。',
51     '', '## 4. 参数选择与 B0—B4 关系', '',
52     '1月21—31日为验证期。每个验证日只利用之前完整日校准, 并接入当时可见的官方预报。候选窗口{14, 28}日、收缩{3, 7}日、分箱{3, 6}小时、  

    ↳ 尺度{统一, 分箱, 指数}、 $\gamma \in \{0, p_{\min}/0.9, 2p_{\min}/0.9\}$ 共72组。所有候选从同一个1月20日末库存开始、验证期末6000,  

    ↳ 以排除库存耗尽造成的假节省。',
53     f"B3的结构预先固定为PDF的条件尺度主模型; 其参数按1月实际现金费选择, 指数在同一配置下若白天CRPS差于统一或分箱则淘汰。  

    ↳ 最终B3选中{selected['mode']},  $\gamma = \{selected['gamma']:.9f\}$ 元/kWh, 1月验证费用{selected['total_fee_yuan']:, .2f}元。  

    ↳ ",
54     f"允许统一尺度参与的全候选最低费用方案为{selection['global_cash_selected']['mode']}  

    ↳ ({selection['global_cash_selected']['config_id']}),  

    ↳ 验证费用{selection['global_cash_selected']['total_fee_yuan']:, .2f}元。因此不能声称B3已在历史验证中全面优于简单模型。  

    ↳ 统一尺度结果保留为B2与全候选费用选择对照。",

```

```

55     f"同一γ下按条件模型概率CRPS选参得到{selection['probability_selected']['config_id']},
    ↳ {selection['probability_selected']['mode']}。本次概率选参和B3费用选参{'恰好一致，因此共用一条正式回测轨迹，
    ↳ 不能虚构两者有改进差异' if selection['probability_selected']==selected else '不同，正式回测分别列出'}。
    ↳ 完整候选表附在'1月参数验证完整表.csv'。",
56     '', '| 标识 | 实验目录 | 变化 |', '---|---|---|',
57     '|B0|info_mask0| 仅 0 点新预报；仍每日 4 次优化，其他设置同 B3|',
58     '|B1|point_official| 使用原始官方光伏点预测，去掉偏差校准及概率场景 |',
59     '|B2|scale_pooled| 校准中心 + 统一误差尺度 + 成对日场景，λ=0|',
60     '|B3|main|B2 改为提前量条件尺度，其他参数和日块抽样一致 |',
61     '|B4|cvar90/cvar95 及 lambda005|B3 基础上仅加 CVar 风险项 |',
62     '', '## 5. 正式求解顺序', '',
63     '校验附件日期、电价、单位及官方预报发布时间 → 每日用过去完整日计算μ和σ → 用历史当时保存的因果μ、σ构建对标准残差日块 →
    ↳ 均匀有放回抽样64次并压缩重复日权重 → 0点解全天MILP形成Q → 6/12/18点依据最新允许预报和真实S重解剩余时域 →
    ↳ 只执行接下来36个时段 → 实测到达后独立重算E、U与费用 → 跨日承接S → 最后导出表格与论文图。',
64     '求解器采用SciPy/HiGHS的确定性等价MILP，保留充放电方向二进制变量。未强行改写为Benders；本次有限日场景规模可由直接MILP求解；
    ↳ PDF规定的是模型，并未强制分解算法。求解器超时只有在LP解满足电池互斥且有下界证书时才接受LP，否则重试MILP。',
65     '', '## 6. 费用、风险、终端与敏感性比较', '',
66     '下表均为 2—12 月实现现金费；CVar95 按 334 个实现日紧急费重新计算，分位点保留部分概率质量。',
67     '', '| 方案 | 总费/元 | 紧急费/元 | 日紧急费 CVar95/元 |', '---|---|---|'|
68 for name in allnames:
69     r=records[name];lines.append(f"|{label(name)}|{r['total_fee_yuan']:, .2f}|{r['emergency_fee_yuan']:, .2f}|{r['
    ↳ 'daily_emergency_cvar95_yuan']:, .2f}|")
70 lines+=['', 'α∈{.90, .95}、λ∈{0, .05, .1}预先登记，λ=0复用B3。四个正风险参数组合既做1月验证，也做正式回测，
    ↳ 见'1月风险参数验证.csv'。风险偏好没有按全年最低费用倒选；风险方案增加均值费用并不自动说明模型较差，应同时报告尾部变化。',
71     '小时均值、效率解释和备选下调结算属于模型口径敏感性，不混入B0—B4的算法归因。全年费用最小的事后方案不自动替换本次预先定义的主方案。
    ↳ ',
72     '', '## 7. 预测质量与 R 解释', '',
73     f"主方案光伏正值样本的合并CRPS为{crps:.6f}kW，90%经验区间覆盖率{100*cover:.2f}%。该覆盖率{'低于' if cover<.9 else
    ↳ '达到或高于'}标称值，应如实报告；不能由R或有限场景直接声称概率安全保证。同一目标被多次发布评价，不能把全部点数当独立天气样本数。
    ↳ ",
74     '提前量与发布时间分组误差.csv'给出原始点预测、校准中心和场景均值的MAE/RMSE，以及CRPS、PICP90、MPIW、IS0.1。
    ↳ lead_hour_bin=0表示该发布时刻的全部提前量；其它值表示1小时分箱。实际光伏为零的组包括夜间，也可能包括白天零发电，
    ↳ 只有实测评价时使用该分组，不用于未来预测遮罩。',
75     '非负截断与联合场景诊断.csv'记录截断概率、截断前后均值/方差变化、同一时段的场景加权负荷—光伏协方差与相关系数。
    ↳ 标准化日块保留历史联合路径，但有限抽样与非负截断可能改变矩和相关性，不保证场景均值恰好等于μ、标准差恰好等于σ。',
76     f"预测尺度与可信度R完整记录.csv`包含{diag['reliability_rows']:,}条发布—有效时段记录，每条给出三个尺度及R。
    ↳ 反演σ最大绝对误差为{diag['R_to_sigma_max_error_kw']:.3g}kW。直接用σ构造场景与通过R再反演数学等价，
    ↳ 不应把展示R本身写成新增优化模块。",
77     '', '## 8. 预报时刻价值', '',
78     f"全四次预报相对仅0点新预报累计节省{saving:, .2f}元。本年度事后最低费用组合为{mask_label(best)}。各组合仍每日重算4次，
    ↳ 复用旧预报时保留原始发布时刻和h=A+H。",
79     '', '| 可用预报时刻 | 总费用/元 | 相对仅 0 点节省/元 |', '---|---|---|'|
80 for m in range(8):lines.append(f"|{mask_label(m)}|{maskdaily[m].sum():, .2f}|{(maskdaily[0]-maskdaily[m]).sum():}
    ↳ +, .2f}|")
81 lines+=['', '新增某时刻相对于保留其余两次日内预报的边际区间:', '',
82     '| 新增预报 | 累计节省/元 | 平均日节省/元 | 探索性 95% 区间/元 |', '---|---|---|---|'|
83 for r in marginal:
84     if r['new_mask']==7:lines.append(f"|{r['added_hour']}|时|{r['total_saving_yuan']:, .2f}|{r['mean_daily_savin
    ↳ g_yuan']:, .2f}|{r['mean_daily_ci_low_yuan']:.2f}, {r['mean_daily_ci_high_yuan']:.2f}|")
85 lines+=['', '配对7日移动块bootstrap重复2000次，区间仅描述这一年、有季节变化的样本，不是跨年保证。附件设有3/9/15/21时新发布预报，
    ↳ 不能用重算旧预报代替额外气象信息，也不能声称已证明这些时刻购买新预报的收益。',
86     '', '## 9. 验证与写作边界', '',
87     f"完成{len(audit)}组334日实验，包含{unique_policies}种不同策略，共{sum(a['updates']) for a in
    ↳ audit.values():,}次正式滚动求解的独立轨迹审计。本次全候选费用选择与B2设置相同，重复运行用于核对，不算新模型。
    ↳ 最大残差{maxerr:.3g}，电量项单位kWh、账单项单位元，阈值1e-4。主方案最大求解上下界差{main['max_solver_gap_yuan']:.3g}元。
    ↳ 物理约束、跨日连续、硬终端、最终版本、原始承诺、结算等价式与可见信息时序均校验。",
88     f"{test['tests_run']}项自动测试通过，包含未来实测/未发布预报反泄漏、CVar离散分位质量、公式结算、小时积分、硬终端约束、
    ↳ B3/B4参数一致性、R反演与晨昏零历史反例。所有指标均来自本次数据，不沿用旧版数值。",
89     '另用独立代码按PDF式(10)一(16)重算334天、120240条发布—有效时段的偏差、中心、分箱/统一尺度和指数拟合，结果通过1e-6容差核验；
    ↳ 记录见independent_calibration_audit.json。主方案本年每天末库存恰好为6000kWh，这是软目标下的最优选择，
    ↳ 并非代码加入了每日等式约束。',
90     '数值MILP求解成功不等于预测准确或分布假设正确。本实现覆盖PDF的主体模型和上述对照；PDF中同日偏差动态更新、
    ↳ 更复杂负荷模型等可选扩展没有全部采用。论文应使用本报告中实际分箱尺度、历史均值负荷、固定6小时内电池计划的分支，
    ↳ 不写成必然指数衰减、随机森林负荷或10分钟自适应电池。',
91     '', '## 10. 论文手阅读顺序', '',
92     '先读`PDF与代码逐项对应.md`，再引用本报告的模型、参数和边界；结果用`题目指定日期表格.md`及`result3.xlsx`，图用`科研图表`。
    ↳ 每日账单、完整执行轨迹、各次剩余计划、验证网格和R/概率诊断均单列。原计划费列与实际总费列不能相加。',
93     '', '来源：用户提供的15页model-q3.pdf、C题题面及附件1/2/3/5。建模稿文献引用未另行全文核验，本报告不把算法组合宣称为原创理论。']
94 (out/'第三问模型核验与求解报告.md').write_text('\n'.join(lines)+'\n',encoding='utf-8')
95 cross=['# PDF 与代码逐项对应', '', '本次正式主方案为 B3 分箱条件尺度；B4 严格复用 B3 并加入 CVar。', '',
96     '|PDF 位置 | 含义 | 代码入口 | 实现说明 |', '---|---|---|---|',*['|'+'|'.join(r)+'|' for r in CROSSWALK],
97     '', '论文必须使用的口径：分箱尺度；28日历史加权负荷；6小时内固定电池计划；下调净退50%；跨日连续；年末6000作为比较条件。',

```

```

98     '', '指数尺度是验证后未采用的候选，不应把主方案写成“验证了指数可信度衰减规律”。主模型也不能写成“历史验证费用全候选最优”，
    ↪ 因为统一尺度候选验证费用略低。',
99     '', '详见《第三问模型校验与求解报告》的完整参数、验证和数值。']
100 (out/'PDF 与代码逐项对应.md').write_text('\n'.join(cross)+'\n',encoding='utf-8')
101 shutil.copy2(run/'january_validation_grid.csv',out/'1 月参数验证完整表.csv')
102 risk=json.loads((run/'january_risk_validation.json').read_text(encoding='utf-8'))
103 config=json.loads((run/'run_config.json').read_text(encoding='utf-8'))
104 gamma_index=config['gamma_candidates'].index(selected['gamma'])
105 selected_summary=json.loads((run/'validation'/selected['config_id']/f"{selected['mode']}_gamma{gamma_index}"/'s'
    ↪ ummary.json').read_text(encoding='utf-8'))
106 riskrows=[]
107 if selected_summary:riskrows.append({'name':'B3_lambda0','alpha':.9,'lambda':0.,**{k:selected_summary[k] for k
    ↪ in ['total_fee_yuan','emergency_fee_yuan','daily_emergency_cvar90_yuan','daily_emergency_cvar95_yuan']}})
108 for r in risk:riskrows.append({'name':r['name'],'alpha':r['policy']['alpha'],'lambda':r['policy']['risk_lambda']
    ↪ },**{k:r[k] for k in
    ↪ ['total_fee_yuan','emergency_fee_yuan','daily_emergency_cvar90_yuan','daily_emergency_cvar95_yuan']})
109 write_csv(out/'1 月风险参数验证.csv',riskrows)
110 daily=pd.read_csv(run/'experiments/main/daily.csv')
111 daily['month']=daily.date.str[:7]
112 fields=['total_fee_yuan','normal_fee_yuan','emergency_fee_yuan','emergency_kwh','unused_kwh','throughput_kwh',''
    ↪ adjust_up_kwh','adjust_down_kwh','adjusted_intervals']
113 monthly=daily.groupby('month')[fields].sum().reset_index()
114 write_csv(out/'第三问月度费用与运行指标.csv',monthly.to_dict('records'))
115 write_json(run/'runtime_versions.json',{'python':platform.python_version(),'numpy':np.__version__,'pandas':pd._
    ↪ _version_})

```

summarize_q3.py

```

1  """ 汇总第三问对照、预报增量价值、概率检验和论文图表。"""
2  from pathlib import Path
3  import argparse,json,hashlib,platform
4  import numpy as np
5  import pandas as pd
6  import scipy,matplotlib
7  from matplotlib.backends.backend_pdf import PdfPages
8  from forecasting import read_dataset
9  from run_q3 import ROOT,write_json,write_csv
10 from audit_q3 import audit_run
11 from scientific_style import apply_style,plt,COLORS,panel,time_axis,step,note,save_figure
12
13 def label(name):
14     return {'main':'B3 条件尺度（风险中性）','point_official':'B1 原始预报点预测','scale_pooled':'B2
    ↪ 统一尺度','scale_exponential':'指数尺度','scale_binned':'分箱尺度',
15             'cvar90':'CVaR 90%,  $\lambda=0.1$ ','cvar95':'CVaR 95%,  $\lambda=0.1$ ','terminal_zero':'无日末软目标',
16             'terminal_hard':'每日硬目标 6000','terminal_soft':'正软终端目标',
17             'cvar90_lambda005':'CVaR 90%,  $\lambda=0.05$ ','cvar95_lambda005':'CVaR 95%,  $\lambda=0.05$ ',
18             'hourly_mean':'小时均值解释','efficiency_roundtrip90':'往返效率 90%',
19             'probability_selected':'概率评分选参','global_cash_selected':'全候选费用选参',
20             'settlement_alternative':'备选结算：下调另罚'}.get(name,name)
21
22 def mask_label(mask):
23     selected=[str(h) for j,h in enumerate([6,12,18]) if mask&(1<<j)]
24     return '0'+(' ' + ' ', '.join(selected) if selected else '')+' 时'
25
26 def collect(run):
27     summaries=[]
28     for p in sorted((run/'experiments').glob('*summary.json')):
29         item=json.loads(p.read_text(encoding='utf-8'));policy=item.pop('policy')
30         summaries.append({'name':p.parent.name,**item,**policy})
31     write_json(run/'experiment_summary.json',summaries)
32     out=run/'deliverables';out.mkdir(exist_ok=True)
33     fields=['name','mode','gamma','total_fee_yuan','normal_fee_yuan','emergency_fee_yuan','emergency_kwh',
34             'daily_emergency_cvar90_yuan','daily_emergency_cvar95_yuan','unused_kwh','initial_kwh','terminal_kwh',''
    ↪ runtime_s',
35             'adjust_up_kwh','adjust_down_kwh','adjusted_intervals','throughput_kwh','solver_runtime_total_s','max_s'
    ↪ olver_gap_yuan']
36     write_csv(out/'第三问全部方案对比.csv',[[k:r[k] for k in fields] for r in summaries])
37     records={r['name']:r for r in summaries}
38     expected=set(json.loads((run/'experiment_registry.json').read_text(encoding='utf-8')))
39     if not expected.issubset(records):raise RuntimeError('Unfinished experiments:
    ↪ '+str(sorted(expected-set(records))))

```

```

40 rows=[];maskdaily={}
41 for mask in range(8):
42     name='main' if mask==7 else f'info_mask{mask}'
43     maskdaily[mask]=pd.read_csv(run/'experiments'/name/'daily.csv')['total_fee_yuan'].to_numpy()
44 # 配对 7 日移动块 bootstrap: 保留日差相关性, 区间仅作这一个年份的探索性描述。
45 rng=np.random.default_rng(20260912);n=334;block=7
46 starts=rng.integers(0,n-block+1,size=(2000,int(np.ceil(n/block))))
47 sample=(starts[:,None]+np.arange(block)[None,None,:]).reshape(2000,-1)[:,:n]
48 for mask in range(8):
49     for j,hour in enumerate([6,12,18]):
50         if mask&(1<j):continue
51         dest=mask|(1<j);delta=maskdaily[mask]-maskdaily[dest];sample_means=delta[sample].mean(axis=1)
52         lo,hi=np.quantile(sample_means,[.025,.975])
53         rows.append({'base_mask':mask,'added_hour':hour,'new_mask':dest,'total_saving_yuan':float(delta.sum()),
54                     'mean_daily_saving_yuan':float(delta.mean()),'mean_daily_ci_low_yuan':float(lo),
55                     'mean_daily_ci_high_yuan':float(hi),'days_with_saving':int((delta>1e-6).sum()),
56                     'bootstrap_block_days':7,'bootstrap_draws':2000})
57 write_csv(out/'预报时刻边际价值.csv',rows)
58 write_csv(out/'预报时刻组合对比.csv',[{ 'mask':m,'forecast_hours':mask_label(m),'total_fee_yuan':float(maskdaily[m]
↪ ].sum()),
59                                     'saving_vs_midnight_only_yuan':float((maskdaily[0]-maskdaily[m]).sum())}
↪ for m in range(8)])
60 return records,rows,maskdaily
61
62 def figures(run,records,marginal,maskdaily):
63 out=run/'deliverables/科研图表';out.mkdir(parents=True,exist_ok=True);apply_style();qa=[]
64 a=dict(np.load(run/'experiments/main/dispatch.npz'));ds=read_dataset(ROOT/'data/processed')
65 daily=pd.read_csv(run/'experiments/main/daily.csv');scores=pd.read_csv(run/'annual_probability_scores.csv')
66 with PdfPages(out/'第三问科研图表.pdf') as pdf:
67     fig,axs=plt.subplots(4,2,figsize=(9,10));fig.subplots_adjust(left=.10,right=.96,top=.925,bottom=.09,hspace=
↪ .58,wspace=.33)
68     for j,date in enumerate(['2025-03-20','2025-06-21','2025-09-23','2025-12-21']):
69         i=list(daily.date).index(date);ax=axs[j,0]
70         step(ax,a['Q'][i],color=COLORS['muted'],ls='--',label='0 点计划 Q')
71         step(ax,a['R'][i],color=COLORS['purchase'],label='最终正常购电 R')
72         step(ax,a['E'][i],color=COLORS['emergency'],lw=1,label='紧急购电 E')
73         panel(ax,chr(97+2*j),date+' 购电');ax.set_ylabel('电量 (kWh / 10 min)');time_axis(ax)
74         ax=axs[j,1];ax.plot(np.arange(145)/6,a['S'][i],color=COLORS['state'],label='电池内部储量 S')
75         ax.axhline(1200,color=COLORS['rule'],lw=.7,ls=':');ax.axhline(10800,color=COLORS['rule'],lw=.7,ls=':')
76         for h in [6,12,18]:ax.axvline(h,color=COLORS['rule'],lw=.6,ls='--')
77         ax.set_ylim(0,12000);ax.set_ylabel('储电量 (kWh)');time_axis(ax);panel(ax,chr(98+2*j),date+' 储能')
78         handles,names=axs[0,0].get_legend_handles_labels();h2,n2=axs[0,1].get_legend_handles_labels()
79         fig.legend(handles+h2,names+n2,ncol=4,fontsize=8,loc='upper center',bbox_to_anchor=(.53,.993))
80         note(fig,'主方案: B3 条件尺度、风险中性随机滚动调度。每 6 小时执行新计划;电量按 10 分钟区间展示,未作平滑。')
81         qa.append(save_figure(fig,out,'01_ 指定日期购电与储能',pdf=pdf))
82
83     fig,axs=plt.subplots(1,2,figsize=(9,3.8));fig.subplots_adjust(left=.09,right=.97,top=.85,bottom=.24,wspace=
↪ .36)
84     monthly=daily.groupby(pd.to_datetime(daily.date).dt.month)[['normal_fee_yuan','emergency_fee_yuan']].sum()/
↪ 1e4
85     ax=axs[0];ax.bar(monthly.index,monthly.normal_fee_yuan,color=COLORS['planned_cost'],width=.66,label='正常结
↪ 算')
86     ax.bar(monthly.index,monthly.emergency_fee_yuan,bottom=monthly.normal_fee_yuan,color=COLORS['emergency'],wi
↪ dth=.66,label='紧急费用')
87     ax.set_xticks(monthly.index);ax.set_xlabel('月份');ax.set_ylabel('费用
↪ (万元)');ax.legend(ncol=2,fontsize=7);panel(ax,'a','各月实际购电费用')
88     ax=axs[1]
89     for name,color,marker in
↪ [('main',COLORS['purchase'],'o'),('cvar90',COLORS['pv'],'s'),('cvar95',COLORS['state'],'D')]:
90         item=records[name];x=item['total_fee_yuan']/1e4;y=item['daily_emergency_cvar95_yuan']/1e4
91         ax.scatter(x,y,color=color,marker=marker,s=48,label=label(name))
92         ax.set_xlabel('2—12月总费用 (万元)');ax.set_ylabel('日紧急费用 CVar95 (万元)');ax.ticklabel_format(useOffset=F
↪
↪ else);ax.legend(fontsize=7,loc='best');panel(ax,'b','实际费用与尾部风险')
93         note(fig,'右图按 334 个已实现的日紧急费用重算 CVar95。该指标不等于各次优化目标内 CVaR 之和。',y=.025)
94         qa.append(save_figure(fig,out,'02_ 月度费用与风险权衡',pdf=pdf))
95
96     fig,axs=plt.subplots(1,2,figsize=(9,4.3));fig.subplots_adjust(left=.17,right=.97,top=.85,bottom=.25,wspace=
↪ .52)
97     ax=axs[0];values=np.array([maskdaily[m].sum() for m in range(8)])/1e4
98     ax.barh(np.arange(8),values,color=[COLORS['purchase'] if m==7 else COLORS['planned_cost'] for m in
↪ range(8)],height=.64)
99     ax.set_yticks(np.arange(8),[mask_label(m) for m in range(8)]);ax.invert_yaxis();ax.set_xlabel('2—12月总费用
↪ (万元)');ax.set_xlim(0,values.max()*1.18)

```

```

100     for j,v in enumerate(values):ax.text(v+4,j,f'{v:.2f}',va='center',fontsize=7)
101     panel(ax,'a','可用预报时刻的 8 种组合')
102     ax=axes[1];chosen=[next(r for r in marginal if r['new_mask']==7 and r['added_hour']==h) for h in [6,12,18]]
103     for j,row in enumerate(chosen):
104         mean=row['mean_daily_saving_yuan'];lo=row['mean_daily_ci_low_yuan'];hi=row['mean_daily_ci_high_yuan']
105         ax.errorbar(mean,j,xerr=[[max(0,mean-lo)], [max(0,hi-mean)]] ,fmt='o',color=[COLORS['pv'],COLORS['purchase']
106             ↪ e'],COLORS['state']][j],capsize=3)
107     ax.axvline(0,color=COLORS['rule'],lw=.8);ax.set_yticks(range(3),['增加6点预报','增加12点预报','增加18点预报']);a
108     ↪ x.invert_yaxis();ax.set_xlabel('平均每日节省费用 (元)');panel(ax,'b','保留其余预报时的增量价值')
109     note(fig,'各组合均每天重新优化 4 次, 仅屏蔽新预报。区间为配对 7 日移动块 bootstrap 95% 区间 (2000 次), 仅作探索性描述。')
110     qa.append(save_figure(fig,out,'03_ 预报信息增量价值',pdf=pdf))
111
112     fig,ax=plt.subplots(1,2,figsize=(9,3.8));fig.subplots_adjust(left=.09,right=.97,top=.85,bottom=.25,wspace=
113     ↪ .35)
114     for mode,color,marker,text in [('pooled',COLORS['purchase'],'o','合并尺度'),('exponential',COLORS['state'],'
115     ↪ s','指数尺度'),('binned',COLORS['pv'],'D','分箱尺度')]:
116         frame=scores[(scores['mode']==mode)&(scores['subset']=='daylight')].sort_values('issue_hour')
117         axes[0].plot(frame.issue_hour,frame.crips,marker=marker,color=color,label=text)
118         axes[1].plot(frame.issue_hour,frame.coverage*100,marker=marker,color=color,label=text)
119         for ax in axes:ax.set_xticks([0,6,12,18]);ax.set_xlabel('预报发布时间 (h)')
120         axes[0].set_ylabel('CRPS (kW), 越低越好');axes[0].legend(fontsize=7);panel(axes[0],'a','实际光伏为正时的概率评分')
121         axes[1].axhline(90,color=COLORS['muted'],ls='--',lw=.8,label='标称90%');axes[1].set_ylabel('90%区间实际覆盖率
122         ↪ (%)');axes[1].set_ylim(50,100);axes[1].legend(fontsize=7);panel(axes[1],'b','样本外区间覆盖检验')
123         note(fig,'每次只评价当天剩余时段; 不同发布时间对应的目标时段不同, 不能将横轴趋势直接解释为提前量的因果效果。')
124         qa.append(save_figure(fig,out,'04_ 概率预测样本外检验',pdf=pdf))
125         from q3_extra_figures import extra_figures
126         qa.extend(extra_figures(run,records,out,pdf))
127     write_json(run/'figure_qa.json',qa)
128
129     from q3_report import report
130
131     if __name__=='__main__':
132         parser=argparse.ArgumentParser();parser.add_argument('run');parser.add_argument('--no-figures',action='store_tr
133         ↪ ue');args=parser.parse_args()
134         run=Path(args.run);records,marginal,maskdaily=collect(run);report(run,records,marginal,maskdaily)
135         if not args.no_figures:figures(run,records,marginal,maskdaily)

```

finalize_q3_records.py

```

1  """ 逐项核验输入/数值源码哈希和完整实验注册表, 再封存本次真实记录。"""
2  import json,hashlib,shutil,sys
3  from pathlib import Path
4  from collections import Counter
5  from datetime import datetime,timezone
6  from run_q3 import ROOT,write_json
7  from audit_q3 import audit_run
8  from audit_q3_calibration import audit_calibration
9
10 run=Path(sys.argv[1]);j=lambda p:json.loads(p.read_text(encoding='utf-8'))
11 cfg=j(run/'run_config.json');registry=j(run/'experiment_registry.json')
12 for relative,expected in cfg['input_sha256'].items():
13     if hashlib.sha256((ROOT/relative).read_bytes()).hexdigest()!=expected:raise ValueError('Input changed:
14     ↪ '+relative)
15 for name,expected in cfg['numerical_code_sha256'].items():
16     if hashlib.sha256((ROOT/'src'/name).read_bytes()).hexdigest()!=expected:raise ValueError('Numerical source
17     ↪ changed: '+name)
18 audit=audit_run(run)
19 calibration_audit=audit_calibration(run)
20 if set(audit)!=set(registry):raise ValueError('Incomplete or unexpected experiments')
21 routes={};attempts=[];completed=0
22 for name,policy in registry.items():
23     folder=run/'experiments'/name
24     if j(folder/'policy.json')!=policy:raise ValueError('Policy mismatch: '+name)
25     updates=j(folder/'updates.json');completed+=len(updates)
26     routes[name]=dict(Counter(x['solver_route'] for x in updates))
27     for u in updates:
28         if u['solver_route']!='milp':attempts.append({'experiment':name,'date':u['date'],'hour':u['decision_hour'],
29         ↪ 'route':u['solver_route'],'attempts':u['solver_attempts']})
30 test=j(run/'test_results.json')
31 if not test['successful']:raise ValueError('Tests failed')
32 book=j(run/'workbook_qa/openpyxl_readonly_check.json')

```

```

30 if abs(book['total_fee_yuan']-j(run/'experiments/main/summary.json')['total_fee_yuan'])>1e-5:raise
    ↪ ValueError('Workbook fee mismatch')
31 figures=j(run/'figure_qa.json')
32 if len(figures)!=7 or any(x['glyph_check']!='passed' or x['text_bounds']!='passed' for x in figures):raise
    ↪ ValueError('Figure QA failed')
33 record={'input_hash_checks':len(cfg['input_sha256']),'numerical_code_sha256':cfg['numerical_code_sha256'],
34         'completed_annual_experiments':len(audit),'unique_annual_policies':len({json.dumps(p,sort_keys=True) for p in
    ↪ registry.values()}),
35         'completed_rolling_solves':completed,'january_calibration_candidates':len(j(run/'selection.json')['all_candidat
    ↪ es']),
36         'january_risk_comparators':len(j(run/'january_risk_validation.json')), 'solver_routes':routes, 'solver_fallbacks'
    ↪ :attempts,
37         'unit_tests':test, 'calibration_audit':calibration_audit, 'max_audit_residual':max(x['max_residual'] for x in
    ↪ audit.values()),
38         'workbook_total_fee_yuan':book['total_fee_yuan'],'figure_count':len(figures),'finalized_at_utc':datetime.now(ti
    ↪ mezone.utc).isoformat())
39 write_json(run/'completion_record.json',record)
40 shutil.copytree(ROOT/'src',run/'final_code_snapshot',dirs_exist_ok=True)
41 shutil.copytree(ROOT/'tests',run/'final_tests',dirs_exist_ok=True)
42 files=[p for p in (run/'deliverables').rglob('*') if p.is_file()]
43 write_json(run/'deliverable_manifest.json',[{'path':str(p.relative_to(run/'deliverables')), 'bytes':p.stat().st_size,
44         'sha256':hashlib.sha256(p.read_bytes()).hexdigest()} for p in files])
45 print(json.dumps({k:v for k,v in record.items() if k not in
    ↪ ('solver_routes','numerical_code_sha256','solver_fallbacks')},ensure_ascii=False))

```

audit_q3.py

```

1 """ 独立复算落地轨迹、账单与信息时序；不调用优化器。"""
2 import json
3 from pathlib import Path
4 import numpy as np
5 from forecasting import read_dataset
6 from microgrid_core import Battery, dispatch_checks, require_checks
7 from run_q3 import ROOT,write_json
8
9 def audit_arrays(ds,a,policy,summary,updates):
10     ids=a['day_indices'];n=len(ids)
11     if list(ids)!=list(range(31,365)):raise ValueError('Missing or reordered evaluation dates')
12     if len(updates)!=n*4:raise ValueError('Missing rolling decision audit')
13     for key in ['Q','R','C','D','E','U','normal_fee']:
14         if a[key].shape!=(n,144) or not np.isfinite(a[key]).all():raise ValueError('Invalid '+key)
15     checks={};eta=policy.get('eta',.9);battery=Battery(eta,eta)
16     for i,day in enumerate(ids):
17         physical=dispatch_checks(a['C'][i],a['D'][i],a['S'][i],battery,float(a['S'][i,0]))
18         for key,value in physical.items():checks[key]=max(checks.get(key,0),value)
19         for j,hour in enumerate([0,6,12,18]):
20             u=updates[i*4+j];t=hour*6
21             allowed=[0]+[h for bit,h in enumerate([6,12,18]) if policy.get('mask',7)&(1<<bit) and h<=hour]
22             if u['official_issue_hour']!=max(allowed) or u['date']!=str(ds.dates[day]) or u['decision_hour']!=hour:
23                 raise ValueError('Future or disallowed official forecast')
24             calibration=policy.get('calibration',{})
25             if (u['window_days']!=calibration.get('window',28) or u['bin_hours']!=calibration.get('bin_hours',6)
26                 or u['shrink_days']!=calibration.get('shrink',7.) or u['scale_mode']!=policy['mode']):
27                 raise ValueError('Calibration differs from registered policy')
28             if u['calibration_end']!=str(ds.dates[day-1]) or any(x>=day or x<max(7,day-u['window_days']) for x in
    ↪ u['sampled_day_indices']):
29                 raise ValueError('Historical sample look-ahead')
30             if abs(u['first_lead_hours']-(hour-u['official_issue_hour']+1/6))>1e-10:
31                 raise ValueError('Forecast lead reset')
32             for key in ['R','C','D']:
33                 v=a[key+'_versions'][i,j]
34                 if np.isfinite(v[t:t]).any() or not np.isfinite(v[t:t]).all():raise ValueError('Invalid remaining
    ↪ horizon archive')
35                 checks['executed_version']=max(checks.get('executed_version',0),float(abs(a[key][i,t:t+36]-v[t:t+36]
    ↪ ).max()))
36             checks['original_commitment']=max(checks.get('original_commitment',0),float(abs(a['Q'][i]-a['R_versions'][i]
    ↪ ,0)).max()))
37             if i:
38                 checks['previous_state_used']=max(checks.get('previous_state_used',0),abs(a['S'][i,0]-a['S'][i-1,-1]))
39 # 独立结算：主口径使用等价公式  $p \cdot r + 0.5 \cdot p \cdot \text{abs}(r - q)$ 。
40 p=ds.prices[None,:];q,r=a['Q'],a['R']

```

```

41 normal=p*r+.5*p*abs(r-q)
42 if policy.get('settlement')=='no_refund_penalty':normal+=p*np.maximum(q-r,0)
43 deficit=(ds.load_kw[ids]-ds.pv_kw[ids])/6+a['C']-a['D']-r
44 checks.update(fee_formula=float(abs(normal-a['normal_fee'])).max()),
45 emergency_replay=float(abs(np.maximum(deficit,0)-a['E'])).max()),
46 unused_replay=float(abs(np.maximum(-deficit,0)-a['U'])).max()),
47 annual_fee=abs(float((normal+5*p*a['E']).sum())-summary['total_fee_yuan']),
48 across_day=float(abs(a['S'][1:,0]-a['S'][:, -1, -1]).max()),
49 initial=abs(a['S'][0,0]-summary['initial_kwh']),year_end=abs(a['S'][:, -1, -1]-6000),
50 nonnegative_purchase=float(max(0,-q.min(),-r.min())),
51 first_six_hours=float(abs(q[:, :36]-r[:, :36]).max())
52 if policy.get('daily_terminal'):checks['daily_terminal']=float(abs(a['S'][:, -1]-6000).max())
53 require_checks(checks,1e-4)
54 return {'days':n,'updates':len(updates),'ten_minute_intervals':n*144,'checks':checks,
55         'max_residual':max(checks.values()),'scenario_draws':64,
56         'available_independent_days_min':min(u['available_independent_days'] for u in updates),
57         'available_independent_days_max':max(u['available_independent_days'] for u in updates),
58         'unique_sampled_days_min':min(u['unique_sampled_days'] for u in updates),
59         'unique_sampled_days_max':max(u['unique_sampled_days'] for u in updates)}
60
61 def audit_run(run):
62     run=Path(run);ds=read_dataset(ROOT/'data/processed');records={}
63     for folder in sorted((run/'experiments').iterdir()):
64         if not (folder/'summary.json').exists():continue
65         j=lambda name:json.loads((folder/name).read_text(encoding='utf-8'))
66         records[folder.name]=audit_arrays(ds,dict(np.load(folder/'dispatch.npz')),j('policy.json'),j('summary.json')
        ↪ ),j('updates.json'))
67     write_json(run/'independent_audit.json',records);return records
68
69 if __name__=='__main__':
70     import sys
71     result=audit_run(sys.argv[1]);print(json.dumps({k:v['max_residual'] for k,v in result.items()}))

```

audit_q3_calibration.py

```

1 """ 独立按 PDF 式 10—16 重算主缓存，不复用分组矩/尺度拟合函数。"""
2 from pathlib import Path
3 import json,sys
4 import numpy as np
5 from run_q3 import ROOT,write_json
6 from forecasting import read_dataset
7
8 def audit_calibration(run):
9     run=Path(run);c=dict(np.load(run/'forecast_cache.npz'));ds=read_dataset(ROOT/'data/processed')
10    window=int(c['window_days']);nu=float(c['shrink_days']);width=int(c['bin_hours'])
11    lead=(np.arange(144)[None,:]+1)/6-np.array([0,6,12,18])[ :,None]
12    clock=np.broadcast_to(np.arange(144)//36,lead.shape);bins=np.ceil(lead/width).astype(int)-1;valid=lead>0
13    grouping=[(g,b,(clock==g)&(bins==b)&valid) for g in range(4) for b in range(24//width) if
    ↪ np.any((clock==g)&(bins==b)&valid)]
14    maximum={'bias_kw':0.,'center_kw':0.,'binned_sigma_kw':0.,'pooled_sigma_kw':0.,'beta_per_hour':0.,'sigma0_kw':0}
    ↪ ., 'exponential_sigma_kw':0.}
15    count=0
16    for day in range(31,365):
17        start=max(0,day-window);n=day-start
18        rawerr=ds.pv_kw[start:day,None,:]-c['raw_g'][start:day]
19        err=ds.pv_kw[start:day,None,:]-c['mu_g'][start:day]
20        poolbias=rawerr[:,valid].mean(axis=1).mean()
21        poolmean=err[:,valid].mean(axis=1).mean();poolvar=max(0,(err[:,valid]**2).mean(axis=1).mean()-poolmean**2)
22        x=[];y=[];groups=[]
23        for g,b,mask in grouping:
24            bias=(n*rawerr[:,mask].mean(axis=1).mean()+nu*poolbias)/(n+nu)
25            maximum['bias_kw']=max(maximum['bias_kw'],abs(bias-c['bias'] [day,g,b]))
26            center=np.maximum(0,c['raw_g'] [day][mask]+bias)
27            maximum['center_kw']=max(maximum['center_kw'],float(abs(center-c['mu_g'] [day][mask]).max()))
28            sample=err[:,mask];v=max(0,(sample**2).mean(axis=1).mean()-sample.mean(axis=1).mean())**2
29            sigma=np.sqrt(max(25**2,(n*v+nu*poolvar)/(n+nu)))
30            maximum['binned_sigma_kw']=max(maximum['binned_sigma_kw'],float(abs(sigma-c['sigma_g_binned'] [day][mask]
    ↪ ))).max())
31            maximum['pooled_sigma_kw']=max(maximum['pooled_sigma_kw'],float(abs(max(25,np.sqrt(poolvar))-c['sigma_g]
    ↪ _pooled'] [day][mask]).max()))

```

```

32         x.append(float(lead[mask].mean()));y.append(np.log(sigma/c['sref'][day]));groups.append(g);count+=int(m_j
    ↪ ask.sum())
33     x=np.array(x);y=np.array(y);groups=np.array(groups)
34     design=np.c_[np.eye(4)[groups],x]
35     estimate=np.linalg.lstsq(np.sqrt(n)*design,np.sqrt(n)*y,rcond=None)[0]
36     beta=max(0,estimate[-1])
37     intercept=np.array([(y[groups==g]-beta*x[groups==g]).mean() for g in range(4)])
38     sigma0=c['sref'][day]*np.exp(intercept)
39     maximum['beta_per_hour']=max(maximum['beta_per_hour'],abs(beta-c['beta'][day]))
40     maximum['sigma0_kw']=max(maximum['sigma0_kw'],float(abs(sigma0-c['sigma0'][day]).max()))
41     expected=sigma0[clock]*np.exp(beta*lead)
42     maximum['exponential_sigma_kw']=max(maximum['exponential_sigma_kw'],float(abs(expected[valid]-c['sigma_g_ex
    ↪ ponential'][day][valid]).max()))
43     if max(maximum.values())>1e-6:raise ValueError(maximum)
44     result={'checked_evaluation_days':334,'checked_release_target_points':count,'independent_formula_max_errors':ma
    ↪ ximum,
45           'passed':True,'tolerance':1e-6,'variance_convention':'Equal complete-day weights, within-day population
    ↪ second moments.'}
46     write_json(run/'independent_calibration_audit.json',result);print(json.dumps(result))
47     return result
48
49 if __name__=='__main__':audit_calibration(sys.argv[1])

```

verify_q3_workbook.py

```

1 """ 只读检查导出的模板：逐值、逐日期及事件合计，不用 openpyxl 写文件。"""
2 from pathlib import Path
3 import sys,json
4 from datetime import datetime
5 import numpy as np
6 from openpyxl import load_workbook
7 run=Path(sys.argv[1]);p=json.loads((run/'q3_workbook_payload.json').read_text(encoding='utf-8'))
8 wb=load_workbook(run/'deliverables/result3.xlsx',data_only=True)
9 assert wb.sheetnames==['计划购电量','调整购电量','充放电量','紧急购电量']
10 checks={}
11 for sheet,key in [('计划购电量','plan'),('调整购电量','adjusted')]:
12     rows=list(wb[sheet].iter_rows(min_row=2,max_row=335,max_col=147,values_only=True))
13     for i,row in enumerate(rows):
14         assert row[0].date().isoformat()==p[key][i][0]
15         np.testing.assert_allclose(row[1:],p[key][i][1:],rtol=0,atol=1e-6)
16     assert wb[sheet]['B1'].value=='00:00-00:10';assert wb[sheet]['E01'].value=='23:50-24:00'
17     checks[sheet]={'dates':334,'ten_minute_values':334*144,'date_and_value_checks':'passed'}
18 for sheet,key,cols in [('充放电量','battery',6),('紧急购电量','emergency',3)]:
19     rows=list(wb[sheet].iter_rows(min_row=2,max_row=len(p[key])+1,max_col=cols,values_only=True))
20     for actual,expected in zip(rows,p[key]):
21         for j,(a,e) in enumerate(zip(actual,expected)):
22             if j==0 and e is not None:assert a.date().isoformat()==e
23             elif isinstance(e,(int,float)):assert a is not None and abs(a-e)<1e-6
24             else:assert a==e,(sheet,j,a,e)
25     checks[sheet]={'rows':len(rows),'values':'passed'}
26 total=sum(wb['调整购电量'].cell(i,147).value for i in range(2,336))
27 assert abs(total-p['total_fee_yuan'])<1e-5
28 for s in wb:
29     for row in s:
30         assert all(c.data_type!='e' for c in row)
31 checks['total_fee_yuan']=total
32 (run/'workbook_qa/openpyxl_readonly_check.json').write_text(json.dumps(checks,ensure_ascii=False,indent=2),encoding
    ↪ ='utf-8')
33 print(json.dumps(checks,ensure_ascii=False))

```

reporting.py

```

1 """ 由完整 10 分钟轨迹生成论文表格、模板填充值与独立核对材料。"""
2 from pathlib import Path
3 import json
4
5 import numpy as np
6

```

```

7 from forecasting import interval_label, clock_text
8
9
10 def emergency_segments(energy, threshold=1e-6):
11     e=np.asarray(energy, dtype=float)
12     if not np.isfinite(threshold) or threshold<0:
13         raise ValueError("Emergency reporting threshold must be finite and nonnegative")
14     if e.shape!=(144,) or not np.isfinite(e).all() or (e<0).any():
15         raise ValueError("Expected 144 nonnegative emergency energy values")
16     mask=e>threshold
17     segments=[]
18     t=0
19     while t<144:
20         if not mask[t]:
21             t+=1; continue
22         start=t
23         while t<144 and mask[t]:
24             t+=1
25         segments.append({"start_index":start, "end_index_exclusive":t,
26                         "interval":clock_text(start*10)+"-"+clock_text(t*10),
27                         "energy_kwh":float(e[start:t].sum())})
28     return segments, float(e[~mask].sum())
29
30
31 def four_hour_rows(charge, discharge, state):
32     c,d,s=np.asarray(charge), np.asarray(discharge), np.asarray(state)
33     if c.shape!=(144,) or d.shape!=(144,) or s.shape!=(145,):
34         raise ValueError("Four-hour aggregation requires 144 intervals and 145 states")
35     return [[clock_text(j*240)+"-"+clock_text((j+1)*240), float(c[j*24:(j+1)*24].sum()),
36             float(d[j*24:(j+1)*24].sum()), "00:00" if j==0 else "24:00" if j==1 else None,
37             float(s[0]) if j==0 else float(s[-1]) if j==1 else None] for j in range(6)]
38
39
40 def build_payload(run_path, dataset, config):
41     run=Path(run_path)
42     q1=json.loads((run/"q1.json").read_text(encoding="utf-8"))["milp"]
43     data=np.load(run/"main_weighted28/trajectories.npz")
44     ids=data["day_indices"].tolist()
45     if ids!=list(range(31,365)):
46         raise ValueError("Formal template payload requires all 334 dates")
47     dates=[dataset.dates[day].isoformat() for day in ids]
48     plan_rows=[]; battery_rows=[]; emergency_rows=[]; daily=[]; segments_by_day={}
49     excluded=0.
50     for i, day in enumerate(ids):
51         q,c,d,s,e,u=[data[key][i] for key in ("Q", "C", "D", "S", "E", "U")]
52         planned_cost=float(dataset.prices[q])
53         emergency_cost=float(5*dataset.prices[e])
54         total_cost=planned_cost+emergency_cost
55         plan_rows.append([dates[i], *q.tolist(), float(q.sum()), total_cost])
56         for j, row in enumerate(four_hour_rows(c,d,s)):
57             battery_rows.append([dates[i] if j==0 else None, *row])
58             segments, omitted=emergency_segments(e, config["emergency_reporting_threshold_kwh"])
59             excluded+=omitted; segments_by_day[dates[i]]=segments
60             for j, segment in enumerate(segments):
61                 emergency_rows.append([dates[i] if j==0 else None, segment["interval"], segment["energy_kwh"]])
62             daily.append({"date":dates[i], "planned_purchase_kwh":float(q.sum()), "emergency_purchase_kwh":float(e.sum()),
63                         "planned_plus_emergency_kwh":float(q.sum()+e.sum()), "planned_cost_yuan":planned_cost,
64                         "emergency_cost_yuan":emergency_cost, "actual_total_cost_yuan":total_cost,
65                         "initial_kwh":float(s[0]), "terminal_kwh":float(s[-1]), "total_unused_kwh":float(u.sum())})
66     if excluded>1e-4:
67         raise ValueError(f"Emergency reporting threshold suppresses material energy: {excluded} kWh")
68     selected=[]
69     for date_text in ("2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"):
70         i=dates.index(date_text)
71         selected.append({"daily[i], "purchases_at_requested_intervals": [[interval_label(hour*6), float(data["Q"][i, h]
72             ↳ our*6)]] for hour in (10,12,14,16,18,20)],
73                         "battery_four_hour": four_hour_rows(data["C"][i], data["D"][i], data["S"][i]),
74                         "emergency_segments": segments_by_day[date_text]})
75     a=q1["plan"]
76     payload={"q1":{"purchases": [[interval_label(t), a["Q"][t]] for t in range(144)],
77                             "battery": four_hour_rows(a["C"], a["D"], a["S"]),
78                             "cost_yuan": q1["cost_yuan"], "purchase_kwh": q1["purchase_kwh"],
79                             "requested_intervals": [[interval_label(hour*6), a["Q"][hour*6]] for hour in (10,12,14,16,18,20)]},

```

```

79         "q2":{"headers":[" 日期\\时间",*[interval_label(t) for t in range(144)]," 全天购电量"," 全天购电费"],
80             "purchases":plan_rows,"battery":battery_rows,"emergency":emergency_rows,
81             "daily":daily,"selected_dates":selected,"emergency_omitted_below_threshold_kwh":excluded},
82         "notes":{"time_mapping":" 源标签为区间终点; 输出模板改为 00:00-00:10 至 23:50-24:00, 点序和数值不循环平移。",
83             "q2_total_columns":"计划购电量表EP列为计划Q的日合计; EQ列为计划费用加实际紧急购电费用。紧急电量在单独工作表。
            ↩",
84             "energy_units":" 各电量 kWh, 费用元。C、D 为交流母线侧电量, S 为电池内部储量。",
85             "sources":"C 题题面、附件 1、附件 2、附件 5 及前两问修订模型。",
86             "method":" 零点固定全天电池计划; 28 日加权历史预测、最近历史样本外配对残差经验场景; 一月闲置、年末 6000。"}}
87     return payload

```

scientific_style.py

```

1  """ 科研图表的统一视觉约定; 颜色表达变量身份, 标记和线型提供第二种区分。"""
2  from pathlib import Path
3  import hashlib
4  import json
5  import warnings
6  import textwrap
7
8  import numpy as np
9  import matplotlib
10 matplotlib.use('Agg')
11 import matplotlib.pyplot as plt
12 from matplotlib import font_manager
13 from matplotlib.text import Text
14
15 STYLE_VERSION = 'academic_v2_20260912'
16 COLORS = {
17     'load': '#0072B2', 'pv': '#E69F00', 'purchase': '#009E73',
18     'charge': '#56B4E9', 'discharge': '#D55E00', 'state': '#8C6BB1',
19     'price': '#48424C', 'planned_cost': '#4477AA', 'emergency': '#D55E00',
20     'ink': '#202630', 'muted': '#62717D', 'rule': '#A6AFB7', 'light': '#EFF3F6',
21 }
22 METHOD_COLORS = ['#009E73', '#56B4E9', '#D55E00', '#8C6BB1']
23
24
25 def apply_style():
26     for name in ('arial.ttf', 'msyh.ttc'):
27         file = Path('C:/Windows/Fonts') / name
28         if file.exists():
29             font_manager.fontManager.addfont(str(file))
30     plt.rcParams.update({
31         'font.family': ['Arial', 'Microsoft YaHei', 'DejaVu Sans'],
32         'font.size': 9, 'axes.labelsize': 9, 'axes.titlesize': 9.5,
33         'xtick.labelsize': 8, 'ytick.labelsize': 8, 'legend.fontsize': 8,
34         'axes.unicode_minus': False, 'axes.spines.top': False,
35         'axes.spines.right': False, 'axes.linewidth': .7, 'axes.edgecolor': COLORS['ink'],
36         'axes.labelcolor': COLORS['ink'], 'text.color': COLORS['ink'],
37         'xtick.color': COLORS['ink'], 'ytick.color': COLORS['ink'],
38         'xtick.direction': 'out', 'ytick.direction': 'out',
39         'xtick.major.width': .7, 'ytick.major.width': .7,
40         'xtick.major.size': 3, 'ytick.major.size': 3, 'axes.grid': False,
41         'legend.frameon': False, 'legend.handlelength': 2.2,
42         'lines.linewidth': 1.25, 'figure.facecolor': 'white', 'axes.facecolor': 'white',
43         'savefig.facecolor': 'white', 'savefig.dpi': 450,
44         'svg.fonttype': 'none', 'pdf.fonttype': 42, 'ps.fonttype': 42,
45         'svg.hashsalt': STYLE_VERSION,
46     })
47
48
49 def step(ax, values, **kwargs):
50     """144 个区间值映射到 0—24 时, 保留真实阶梯, 不插值平滑。"""
51     values = np.asarray(values, dtype=float)
52     if values.shape != (144,) or not np.isfinite(values).all():
53         raise ValueError('A daily step series must have 144 finite values')
54     return ax.step(np.arange(145)/6, np.r_[values, values[-1]], where='post', **kwargs)
55
56
57 def time_axis(ax, *, label=True, ticks=4):
58     ax.set_xlim(0, 24)

```

```

59     ax.set_xticks(np.arange(0, 25, ticks))
60     if label:
61         ax.set_xlabel('时刻 (h)')
62     ax.margins(x=0)
63
64
65 def panel(ax, letter, title):
66     ax.set_title(title, loc='left', pad=9, fontweight='normal')
67     ax.text(-.105, 1.04, letter, transform=ax.transAxes, fontsize=11,
68            fontweight='bold', va='bottom', ha='left')
69
70
71 def note(fig, text, *, y=.025):
72     text='\n'.join(textwrap.fill(line,width=60,break_long_words=True) for line in text.splitlines())
73     fig.text(.12, y, text, fontsize=7.2, color=COLORS['muted'], va='bottom')
74
75
76 def save_figure(fig, folder, name, *, pdf=None, dpi=450):
77     folder = Path(folder)
78     folder.mkdir(parents=True, exist_ok=True)
79     # 保存时记录缺字及布局警告, 避免仅凭程序未报错便交付。
80     with warnings.catch_warnings(record=True) as caught:
81         warnings.simplefilter('always')
82         fig.canvas.draw()
83         renderer = fig.canvas.get_renderer()
84         outside = []
85         for obj in fig.findobj(match=Text):
86             if not obj.get_visible() or not obj.get_text() or obj.get_clip_on():
87                 continue
88             box = obj.get_window_extent(renderer)
89             if box.width and box.height and (box.x0 < -1 or box.y0 < -1 or
90                box.x1 > fig.bbox.width+1 or box.y1 > fig.bbox.height+1):
91                 outside.append(obj.get_text())
92         if outside:
93             raise ValueError(f'{name}: text extends outside figure: {outside}')
94         files = []
95         for suffix in ('png', 'svg'):
96             target = folder/f'{name}.{suffix}'
97             fig.savefig(target, dpi=dpi, facecolor='white')
98             files.append({'file': target.name, 'bytes': target.stat().st_size,
99                'sha256': hashlib.sha256(target.read_bytes()).hexdigest()})
100         if pdf is not None:
101             pdf.savefig(fig, dpi=dpi)
102         problems = sorted({str(w.message) for w in caught
103             if 'Glyph' in str(w.message) or 'layout' in str(w.message).lower()})
104         if problems:
105             raise ValueError(f'{name}: rendering warnings: {problems}')
106     result = {'name': name, 'width_mm': round(fig.get_figwidth()*25.4, 1),
107             'height_mm': round(fig.get_figheight()*25.4, 1), 'png_dpi': dpi,
108             'axes': len(fig.axes), 'text_bounds': 'passed', 'glyph_check': 'passed',
109             'files': files}
110     plt.close(fig)
111     return result

```

E 问题四源程序

forecasting.py

```

1 """ 只使用目标日前已完成日期的预测与历史样本外残差场景。
2
3 候选模型是预先定义的历史均值/加权均值, 不使用全年数据调参。
4 枚举最近历史联合残差, 形成离散经验分布, 不额外随机重抽样。
5 """
6 from dataclasses import dataclass
7 from datetime import date, timedelta
8 from pathlib import Path
9 import csv
10 import hashlib

```

```

11
12 import numpy as np
13
14
15 def clock_text(minute):
16     if not isinstance(minute,int) or not 0<=minute<=1440:
17         raise ValueError("Minute must be an integer from 0 to 1440")
18     return f"{minute//60:02d}:{minute%60:02d}"
19
20
21 def interval_label(index):
22     if not isinstance(index,int) or not 0<=index<144:
23         raise ValueError("Interval index must be 0..143")
24     return clock_text(index*10)+"-"+clock_text((index+1)*10)
25
26
27 @dataclass
28 class Dataset:
29     dates: list
30     load_kw: np.ndarray
31     pv_kw: np.ndarray
32     prices: np.ndarray
33     baseline_load_kw: np.ndarray
34     baseline_pv_kw: np.ndarray
35     sources: dict
36
37
38 def read_dataset(processed):
39     processed=Path(processed)
40     actual_path=processed/"actual_points.csv"
41     baseline_path=processed/"baseline_points.csv"
42     with actual_path.open(encoding="utf-8-sig",newline="") as stream:
43         rows=list(csv.DictReader(stream))
44         expected=[date(2025,1,1)+timedelta(days=i) for i in range(365)]
45         if len(rows)!=365*144:
46             raise ValueError("Expected 365 complete days, each with 144 records")
47         load,pv=np.empty((365,144)),np.empty((365,144))
48         for row_id,row in enumerate(rows):
49             day,t=divmod(row_id,144)
50             if row["date"]!=expected[day].isoformat() or int(row["point_index"])!=t+1 or
↳ row["time_label"]!=clock_text((t+1)*10):
51                 raise ValueError(f"Unexpected date/time/point mapping at input row {row_id+2}")
52             load[day,t],pv[day,t]=float(row["load_kw"]),float(row["pv_actual_kw"])
53     with baseline_path.open(encoding="utf-8-sig",newline="") as stream:
54         baseline=list(csv.DictReader(stream))
55         if len(baseline)!=144:
56             raise ValueError("Invalid baseline length")
57         for t,row in enumerate(baseline):
58             if int(row["point_index"])!=t+1 or row["time_label"]!=clock_text((t+1)*10):
59                 raise ValueError("Baseline time mapping differs from actual data")
60         p,bl,bg=[np.array([float(row[field]) for row in baseline]) for field in
↳ ("price_yuan_per_kwh","load_kw","pv_forecast_kw")]
61         for arr in (load,pv,p,bl,bg):
62             if not np.isfinite(arr).all() or (arr<0).any():
63                 raise ValueError("Nonfinite or negative input")
64         if (p<=0).any():
65             raise ValueError("Strictly positive electricity prices are required")
66     return Dataset(expected,load,pv,p,bl,bg,{x.name:hashlib.sha256(x.read_bytes()).hexdigest() for x in
↳ (actual_path,baseline_path)})
67
68
69 def point_forecast(history_load, history_pv, history_dates, target_date, model):
70     """ 历史只能包含 target_date 之前的完整日。模型参数预先指定。"""
71     load,pv=np.asarray(history_load,dtype=float),np.asarray(history_pv,dtype=float)
72     if load.ndim!=2 or load.shape!=pv.shape or len(load)!=len(history_dates) or len(load)==0:
73         raise ValueError("Invalid history shape")
74     if any(d>target_date for d in history_dates) or any(b-a!=timedelta(days=1) for a,b in
↳ zip(history_dates,history_dates[1:])):
75         raise ValueError("History must contain only consecutive completed dates before the forecast origin")
76     if history_dates[-1]!=target_date-timedelta(days=1):
77         raise ValueError("Latest completed historical date is missing")
78     if not np.isfinite(load).all() or not np.isfinite(pv).all() or (load<0).any() or (pv<0).any():
79         raise ValueError("Invalid historical power")

```

```

80     if model=="mean7":
81         window=min(7,len(load)); weights=np.ones(window)
82     elif model=="weighted28":
83         window=min(28,len(load))
84         ages=np.arange(window,0,-1)
85         weights=np.exp(-np.log(2)*ages/7)
86         weights=np.array([2. if d.weekday()==target_date.weekday() else 1. for d in history_dates[-window:]])
87     else:
88         raise ValueError("Unknown forecast model")
89     weights/=weights.sum()
90     return weights@load[-window:],weights@pv[-window:],{
91         "model":model,"training_start":history_dates[-window].isoformat(),
92         "training_end":history_dates[-1].isoformat(),"origin":target_date.isoformat(),
93         "history_days":window,"weights":weights.tolist()}
94
95
96 def causal_forecast_cache(dataset, model, minimum_history_days=7):
97     """ 离线重建每个历史日当时能得到的预测; 调用时只传入其历史前缀。"""
98     if minimum_history_days<1 or minimum_history_days>=len(dataset.dates):
99         raise ValueError("Invalid historical warmup")
100     forecast_load=np.full_like(dataset.load_kw,np.nan,dtype=float)
101     forecast_pv=np.full_like(dataset.pv_kw,np.nan,dtype=float)
102     metadata={}
103     for day in range(minimum_history_days,len(dataset.dates)):
104         l,g,info=point_forecast(dataset.load_kw[:day],dataset.pv_kw[:day],dataset.dates[:day],dataset.dates[day],mo)
105         ↵ del)
106         forecast_load[day],forecast_pv[day]=l,g
107         metadata[day]=info
108     return {"model":model,"load_kw":forecast_load,"pv_kw":forecast_pv,"metadata":metadata,
109             "minimum_history_days":minimum_history_days}
110
111 def make_scenarios(dataset, cache, day, *, residual_days=28, residual_scale=1., deterministic=False):
112     """ 整日负荷/光伏残差配对; 截取 day 之前的样本外残差, 转换为 kWh。
113
114     PV 支撑范围只由最近 28 个已观测日确定, 完全未观测到发电的时段置零。
115     这不预知日出时间, 也不能保证未来不会出现新的发电时段。
116     """
117     if day<cache["minimum_history_days"]+1 or day>=len(dataset.dates):
118         raise ValueError("Insufficient historical out-of-sample residuals")
119     if not isinstance(residual_days,int) or residual_days<1 or not np.isfinite(residual_scale) or residual_scale<0:
120         raise ValueError("Invalid residual configuration")
121     point_load=cache["load_kw"][day]
122     point_pv=cache["pv_kw"][day]
123     begin=max(cache["minimum_history_days"],day-residual_days)
124     ids=np.arange(begin,day)
125     if deterministic:
126         raw_load=point_load[None,:]
127         raw_pv=point_pv[None,:]
128     else:
129         # 只从已完成的历史日期减去其当时可用预测, 不使用目标日误差。
130         residual_load=dataset.load_kw[ids]-cache["load_kw"][ids]
131         residual_pv=dataset.pv_kw[ids]-cache["pv_kw"][ids]
132         raw_load=point_load+residual_scale*residual_load
133         raw_pv=point_pv+residual_scale*residual_pv
134     support=dataset.pv_kw[max(0,day-28):day].max(axis=0)>0
135     scenario_load=np.maximum(raw_load,0)/6
136     scenario_pv=np.where(support,np.maximum(raw_pv,0),0)/6
137     if not np.isfinite(scenario_load).all() or not np.isfinite(scenario_pv).all():
138         raise ValueError("Nonfinite forecast/scenario; inspect warmup and history")
139     count=len(scenario_load)
140     audit={**cache["metadata"][day],"scenario_count":count,
141            "residual_start":None if deterministic else dataset.dates[begin].isoformat(),
142            "residual_end":None if deterministic else dataset.dates[day-1].isoformat(),
143            "scenario_construction":"point_forecast" if deterministic else
144            ↵ "enumerated_paired_daily_out_of_sample_residuals",
145            "residual_scale":residual_scale,"negative_load_values_clipped":int((raw_load<0).sum()),
146            "negative_pv_values_clipped":int((raw_pv<0).sum()),
147            "pv_values_masked_outside_recent_support":int(((raw_pv>0)&~support).sum())}
148     return scenario_load,scenario_pv,np.full(count,1/count),audit
149
150 def no_storage_purchase(load_scenarios,pv_scenarios,probabilities,quantile=.8):

```

```

151 net=load_scenarios-pv_scenarios
152 if not 0<quantile<1:
153     raise ValueError("Quantile must be in (0,1)")
154 order=np.argsort(net,axis=0)
155 sorted_net=np.take_along_axis(net,order,axis=0)
156 cdf=np.cumsum(np.asarray(probabilities)[order],axis=0)
157 index=np.argmax(cdf>=quantile-1e-12,axis=0)
158 return np.maximum(sorted_net[index,np.arange(net.shape[1])],0)

```

forecasting_v2.py

```

1 """PDF §3.2-3.3: 因果预测、配对整日残差、有放回抽样。
2
3 P2 使用日周期傅里叶回归 + ARIMA(1,0,0) 残差; P3 使用随机森林。
4 所有拟合函数只接收已完成日期, 禁止把目标日实际值作为特征。
5 """
6 from datetime import timedelta
7 import warnings
8 import numpy as np
9 from sklearn.ensemble import RandomForestRegressor
10 from statsmodels.tsa.arima.model import ARIMA
11 from forecasting import point_forecast
12
13 MODELS = ('P1_weighted28', 'P2_fourier_arima', 'P3_random_forest')
14
15
16 def fourier(t, harmonics=8):
17     t = np.asarray(t)
18     angle = 2 * np.pi * t[:, None] * np.arange(1, harmonics + 1) / 144
19     return np.c_[np.ones(len(t)), np.sin(angle), np.cos(angle)]
20
21
22 def arima_forecast(history):
23     """ 先估计确定日周期, 再对残差拟合低阶平稳 ARIMA; 不做全样本调参。"""
24     y = np.asarray(history[-28:], dtype=float).ravel()
25     design = fourier(np.arange(len(y)))
26     coef = np.linalg.lstsq(design, y, rcond=None)[0]
27     residual = y - design @ coef
28     if np.std(residual) < 1e-8:
29         phi, correction, diagnostics = 0., np.zeros(144), []
30     else:
31         with warnings.catch_warnings(record=True) as captured:
32             warnings.simplefilter('always')
33             fit = ARIMA(residual, order=(1, 0, 0), trend='n').fit(method='burg')
34             phi = float(fit.arparams[0])
35             correction = np.asarray(fit.forecast(144))
36             diagnostics = [str(w.message) for w in captured]
37     prediction = fourier(np.arange(len(y), len(y) + 144)) @ coef + correction
38     return np.maximum(prediction, 0), {'ar1': phi, 'harmonics': 8,
39         'fit': 'Fourier OLS then ARIMA(1,0,0), Burg estimation', 'warnings': diagnostics}
40
41
42 def tree_features(history, day, target_date):
43     """day 可等于 len(history); 只索引 day-1 及更早的整日曲线。"""
44     if day < 7 or day > len(history):
45         raise ValueError('Tree features require seven completed days')
46     previous = history[day-1]
47     period = np.arange(144)
48     clock = fourier(period, 3)[: , 1:]
49     calendar = np.tile([np.sin(2*np.pi*target_date.weekday()/7),
50         np.cos(2*np.pi*target_date.weekday()/7),
51         np.sin(2*np.pi*(target_date.month-1)/12),
52         np.cos(2*np.pi*(target_date.month-1)/12)], (144, 1))
53     return np.c_[clock, calendar, previous, history[day-7], history[day-7:day].mean(axis=0),
54         np.roll(previous, 1), np.roll(previous, -1)]
55
56
57 def tree_forecast(history, history_dates, target_date, seed):
58     end = len(history)
59     begin = max(7, end-28)
60     x = np.vstack([tree_features(history, j, history_dates[j]) for j in range(begin, end)])

```

```

61 y = history[begin:end].ravel()
62 reg = RandomForestRegressor(n_estimators=64, max_depth=10, min_samples_leaf=8,
63                             max_features=1., random_state=seed, n_jobs=1)
64 reg.fit(x, y)
65 return np.maximum(reg.predict(tree_features(history, end, target_date)), 0)
66
67
68 def predict(history_load, history_pv, history_dates, target_date, model, prior_load, prior_pv, seed):
69     n = len(history_dates)
70     if model not in MODELS:
71         raise ValueError('Unknown candidate')
72     if n:
73         if history_dates[-1] != target_date-timedelta(days=1) or any(d >= target_date for d in history_dates):
74             raise ValueError('Future observations or missing latest completed day')
75         if any(b-a != timedelta(days=1) for a, b in zip(history_dates, history_dates[1:])):
76             raise ValueError('History must be consecutive')
77     info = {'model': model, 'origin': target_date.isoformat(),
78            'training_end': history_dates[-1].isoformat() if n else None,
79            'training_start': history_dates[max(0,n-28)].isoformat() if n else None,
80            'history_days': min(n, 28)}
81     if n == 0:
82         info['implementation'] = 'attachment1_prior_PDF_3.9.4'
83         return np.asarray(prior_load).copy(), np.asarray(prior_pv).copy(), info
84     if model == MODELS[0] or n < (7 if model == MODELS[1] else 8):
85         l, g, meta = point_forecast(history_load, history_pv, history_dates, target_date, 'weighted28')
86         info['implementation'] = 'weighted28' if model == MODELS[0] else 'short_history_weighted28_fallback'
87         info['weights'] = meta['weights']
88         return l, g, info
89     if model == MODELS[1]:
90         l, il = arima_forecast(history_load)
91         g, ig = arima_forecast(history_pv)
92         info.update(implementation='Fourier_plus_ARIMA_errors', load_fit=il, pv_fit=ig)
93     else:
94         l = tree_forecast(history_load, history_dates, target_date, seed)
95         g = tree_forecast(history_pv, history_dates, target_date, seed)
96         # 最早训练行还引用其前 7 天，因此记录的原始观测边界再向前 7 天。
97         info['training_start'] = history_dates[max(0, n-35)].isoformat()
98         info.update(implementation='separate_random_forests', n_estimators=64,
99                   max_depth=10, min_samples_leaf=8, training_target_days=min(n-7,28))
100     if not np.isfinite(l).all() or not np.isfinite(g).all():
101         raise ValueError('Nonfinite forecast')
102     return l, g, info
103
104
105 def build_cache(dataset, model, seed=20260912, stop=365, progress=None):
106     l, g = np.full_like(dataset.load_kw, np.nan), np.full_like(dataset.pv_kw, np.nan)
107     metadata = {}
108     for day in range(stop):
109         l[day], g[day], metadata[day] = predict(dataset.load_kw[:day], dataset.pv_kw[:day],
110         dataset.dates[:day], dataset.dates[day], model, dataset.baseline_load_kw,
111         dataset.baseline_pv_kw, seed)
112         if progress and (day == 0 or (day+1) % 60 == 0 or day+1 == stop):
113             progress(f'{model}: forecasts {day+1}/{stop}')
114     return {'model': model, 'load_kw': l, 'pv_kw': g, 'metadata': metadata}
115
116
117 def scenarios(dataset, cache, day, *, seed=20260912, draws=256, residual_days=28,
118             residual_scale=1., mode='bootstrap'):
119     """ 同一抽样日索引同时作用于 144 点负荷和 144 点光伏。
120
121     重复抽到同一天时合并变量块，概率为抽中次数/draws，与未压缩模型严格等价。
122     仅按式 (25) 截断负值，不叠加历史支撑范围遮罩。
123     """
124     if mode not in ('bootstrap', 'enumerate', 'point'):
125         raise ValueError('Unknown scenario rule')
126     if day < 0 or day >= len(dataset.dates) or draws < 1 or residual_days < 1 or residual_scale < 0:
127         raise ValueError('Invalid scenario configuration')
128     pl, pg = cache['load_kw'][day], cache['pv_kw'][day]
129     audit = {'**cache['metadata']': cache['metadata'], 'scenario_method': mode, 'seed': seed,
130            'draw_count': draws if mode == 'bootstrap' and day else 0,
131            'residual_scale': residual_scale}
132     if day == 0 or mode == 'point':
133         ids, counts = np.array([], dtype=int), np.array([1])

```

```

134     raw_l, raw_g = pl[None, :], pg[None, :]
135     prob = np.array([1.])
136     else:
137         pool = np.arange(max(0, day-residual_days), day)
138         if mode == 'bootstrap':
139             rng = np.random.default_rng(np.random.SeedSequence([seed, day]))
140             ids, counts = np.unique(rng.choice(pool, size=draws, replace=True), return_counts=True)
141         else:
142             ids, counts = pool, np.ones(len(pool), dtype=int)
143         raw_l = pl + residual_scale*(dataset.load_kw[ids]-cache['load_kw'][ids])
144         raw_g = pg + residual_scale*(dataset.pv_kw[ids]-cache['pv_kw'][ids])
145         prob = counts/counts.sum()
146     audit.update(residual_dates=[dataset.dates[j].isoformat() for j in ids],
147                 residual_counts=counts.tolist(), scenario_count=len(prob),
148                 negative_load_clipped=int((raw_l < 0).sum()), negative_pv_clipped=int((raw_g < 0).sum()))
149     l, g = np.maximum(raw_l, 0)/6, np.maximum(raw_g, 0)/6
150     if not np.isfinite(l).all() or not np.isfinite(g).all():
151         raise ValueError('Forecast cache is incomplete')
152     return l, g, prob, audit

```

q3_forecasting.py

```

1 """ 附件 3 按发布时间对齐、因果偏差/尺度校准及成对历史标准残差场景。"""
2 from datetime import datetime, timedelta
3 from pathlib import Path
4 import csv
5 import hashlib
6 import numpy as np
7 from forecasting import point_forecast
8
9 MODES=('pooled', 'exponential', 'binned')
10 HOURS=np.array([0,6,12,18])
11
12
13 def read_official(path,dates):
14     out=np.full((len(dates),4,24),np.nan);seen=set()
15     with Path(path).open(encoding='utf-8-sig',newline='') as f:
16         for row in csv.DictReader(f):
17             issued=datetime.fromisoformat(row['issued_at']);lead=int(row['lead_hours'])
18             valid=datetime.fromisoformat(row['valid_at']);i=(issued.date()-dates[0]).days
19             if issued.minute or issued.second or issued.hour not in HOURS or not 0<=i<len(dates) or not 1<=lead<=24:
20                 raise ValueError('Invalid official forecast issue/lead')
21             if valid!=issued+timedelta(hours=lead):raise ValueError('Forecast valid time mismatch')
22             key=(i,issued.hour//6,lead-1)
23             if key in seen:raise ValueError('Duplicate official forecast')
24             seen.add(key);out[key]=float(row['pv_forecast_kw'])
25     if len(seen)!=len(dates)*4*24 or not np.isfinite(out).all() or (out<0).any():
26         raise ValueError('Incomplete/invalid official forecast grid')
27     return out
28
29
30 def integrate_hourly(hourly,anchor,issue_hour):
31     """ 整点值作瞬时节点，逐 10 分钟区间精确积分分段线性曲线。"""
32     hourly=np.asarray(hourly,dtype=float)
33     if hourly.shape!=(24,) or issue_hour not in HOURS or not np.isfinite(hourly).all() or np.any(hourly<0) or
34        ↪ anchor<0:
35         raise ValueError('Invalid hourly forecast input')
36     n=(24-issue_hour)*6;x=np.arange(n+1)/6
37     values=np.interp(x,np.arange(25),np.r_[anchor,hourly])
38     return (values[:-1]+values[1:])/2
39
40 def masks(bin_hours=6):
41     if bin_hours not in (3,6):raise ValueError('Lead bins must be 3 or 6 hours')
42     g=np.broadcast_to(np.arange(144)//36,(4,144))
43     h=(np.arange(144)[None,:]+1)/6-HOURS[:,None]
44     ell=np.ceil(h/bin_hours).astype(int)-1
45     valid=h>0
46     groups={(a,b):(g==a)&(ell==b)&valid for a in range(4) for b in range(24//bin_hours) if
47        ↪ np.any((g==a)&(ell==b)&valid)}

```

```

48
49
50 def day_moments(values, groups):
51     n=len(values); mean=np.full((n,4,max(b for _,b in groups)+1),np.nan); second=mean.copy()
52     for (g,l),mask in groups.items():
53         a=values[:,mask]
54         mean[:,g,l]=a.mean(axis=1); second[:,g,l]=(a*a).mean(axis=1)
55     return mean, second
56
57
58 def fit_scale(mean, second, start, end, floor, shrink, sref, groups, h):
59     if end<=start:
60         return np.full(mean.shape[1:], sref), np.full(4, sref), sref, 0., np.full(4, sref)
61     all_mean=mean[start:end]; all_second=second[start:end]
62     pool_mean=float(np.nanmean(all_mean)); pool_var=max(0, float(np.nanmean(all_second))-pool_mean**2)
63     pool_sigma=max(floor, np.sqrt(pool_var)); bins=np.full(mean.shape[1:], pool_sigma)
64     n=end-start
65     for (g,l) in groups:
66         var=max(0, float(all_second[:,g,l].mean())-float(all_mean[:,g,l].mean())**2)
67         bins[g,l]=np.sqrt(max(floor**2, (n*var+shrink*pool_var)/(n+shrink)))
68     # 组固定截距、共享非负提前量斜率的加权最小二乘：每个分箱按独立日数加权。
69     xs={g:[ ] for g in range(4)}; ys={g:[ ] for g in range(4)}
70     for (g,l),mask in groups.items():
71         xs[g].append(float(h[mask].mean())); ys[g].append(float(np.log(bins[g,l]/sref)))
72     numerator=denominator=0.
73     for g in range(4):
74         x=np.array(xs[g]); y=np.array(ys[g]); numerator+=n*float((x-x.mean())*(y-y.mean())); denominator+=n*float((x-x.
75         ↪ mean())*(x-x.mean()))
76     beta=max(0, numerator/denominator) if denominator>0 else 0.
77     sigma0=np.array([sref*np.exp(np.mean(ys[g]))-beta*np.mean(xs[g])) for g in range(4)])
78     group_constant=np.array([sref*np.exp(np.mean(ys[g])) for g in range(4)])
79     return bins, group_constant, pool_sigma, beta, sigma0
80
81 def build_cache(ds, official, window=28, shrink=7., floor=25., bin_hours=6, interpretation='linear_nodes'):
82     if window<7 or shrink<=0 or floor<=0: raise ValueError('Invalid calibration settings')
83     if interpretation not in ('linear_nodes', 'hourly_mean'): raise ValueError('Invalid forecast interpretation')
84     days=len(ds.dates); g,h,ell,valid,groups=masks(bin_hours)
85     raw=np.full((days,4,144), np.nan); mu_g=raw.copy(); mu_l=np.zeros((days,144))
86     for i in range(days):
87         if i:
88             mu_l[i]=point_forecast(ds.load_kw[:,i], ds.pv_kw[:,i], ds.dates[:,i], ds.dates[i], 'weighted28')[0]
89             for issue,k in enumerate(HOURS):
90                 t=k*6; anchor=float(ds.pv_kw[i,t-1]) if t else (float(ds.pv_kw[i-1,-1]) if i else 0.)
91                 raw[i,issue,t]=(integrate_hourly(official[i,issue], anchor, int(k)) if interpretation=='linear_nodes'
92                 else np.repeat(official[i,issue,:24-k],6))
93     raw_error=ds.pv_kw[:,None,:]-raw
94     raw_mean, raw_second=day_moments(raw_error, groups)
95     cal_mean=np.full_like(raw_mean, np.nan); cal_second=np.full_like(raw_mean, np.nan)
96     scales_g={mode:np.full_like(raw, np.nan) for mode in MODES}
97     scales_l=np.full((days,144), floor); betas=np.zeros(days); sigma0s=np.zeros((days,4)); srefs=np.zeros(days)
98     bias_all=np.zeros(raw_mean.shape); bins_all=np.zeros(raw_mean.shape)
99     for i in range(days):
100         start=max(0,i-window); n=i-start
101         sref=1000.
102         if i>=7:
103             # 评价期开始前固定参考：不倒灌到前七天的决策。
104             daymask=np.broadcast_to(ds.pv_kw[:,7,None,:], >0, raw[:,7].shape)&np.broadcast_to(valid, raw[:,7].shape)
105             sref=max(floor, float(np.sqrt(np.mean(raw_error[:,7][daymask]**2))))
106             srefs[i]=sref
107             pool=float(np.nanmean(raw_mean[start:i])) if n else 0.
108             for (group,lead),mask in groups.items():
109                 bias=(n*float(raw_mean[start:i,group,lead].mean())+shrink*pool)/(n+shrink) if n else 0.
110                 bias_all[i,group,lead]=bias
111                 mu_g[i][mask]=np.maximum(0, raw[i][mask]+bias)
112             bins, constant, pool_sigma, beta, sigma0=fit_scale(cal_mean, cal_second, start, i, floor, shrink, sref, groups, h)
113             betas[i]=beta; sigma0s[i]=sigma0; bins_all[i]=bins
114             for (group,lead),mask in groups.items():
115                 scales_g['pooled'][i][mask]=max(floor, pool_sigma)
116                 scales_g['binned'][i][mask]=bins[group,lead]
117                 scales_g['exponential'][i][mask]=sigma0[group]*np.exp(beta*h[mask])
118             # 当天校准后的误差进入档案：只在后续日期使用。
119             err=ds.pv_kw[i,None,:]-mu_g[i]

```

```

120     for (group,lead),mask in groups.items():
121         cal_mean[i,group,lead]=err[mask].mean();cal_second[i,group,lead]=(err[mask]**2).mean()
122     hist_start=max(1,start)
123     if i>hist_start:
124         error=ds.load_kw[hist_start:i]-mu_l[hist_start:i]
125         pool_var=float(error.var())
126         for group in range(4):
127             sl=slice(group*36,(group+1)*36);var=float(error[:,sl].var());count=len(error)
128             scales_l[i,sl]=np.sqrt(max(floor**2,(count*var+shrink*pool_var)/(count+shrink)))
129     else:scales_l[i]=sref
130 return {'raw_g':raw,'mu_g':mu_g,'mu_l':mu_l,'sigma_l':scales_l,
131        **{'sigma_g_'+key:value for key,value in scales_g.items()},
132        **{'reliability_g_'+key:reliability(value,srefs[:,None,None]) for key,value in scales_g.items()},
133        'window_days':np.array(window),'bin_hours':np.array(bin_hours),'shrink_days':np.array(shrink),
134        'interpretation':np.array(interpretation),
135        'beta':betas,'sigma0':sigma0s,'sref':srefs,'bias':bias_all,'binned_scale':bins_all}
136
137
138 def reliability(sigma,sref):
139     """PDF 式 (16): 无量纲尺度指标, 不是正确概率."""
140     return 1/(1+(np.asarray(sigma)/sref)**2)
141
142
143 def scale_from_reliability(r,sref):
144     return sref*np.sqrt(1/np.asarray(r)-1)
145
146
147 def make_scenarios(ds,cache,day,current_hour,source_hour,*,mode='binned',draws=64,window=None,seed=20260912,point=F
↪ else):
148     if current_hour not in HOURS or source_hour not in HOURS or source_hour>current_hour or mode not in MODES:
149         raise ValueError('Invalid forecast issue visibility or scale mode')
150     t=current_hour*6;issue=source_hour//6
151     window=int(cache.get('window_days',28)) if window is None else window
152     if day<0 or day>=len(ds.dates) or draws<1:raise ValueError('Invalid scenario date/count')
153     ml=cache['mu_l'][day,t:];mg=cache['raw_g'] if point else 'mu_g'][day,issue,t:]
154     history=np.arange(max(7,day-window),day)
155     if point or len(history)==0:
156         load=ml[None,:];pv=mg[None,:];prob=np.ones(1);sampled=[];unique=[]
157     else:
158         rng=np.random.default_rng(np.random.SeedSequence([seed,day,source_hour]))
159         sampled=rng.choice(history,size=draws,replace=True)
160         unique,counts=np.unique(sampled,return_counts=True);prob=counts/draws
161         z_l=(ds.load_kw[unique,t:]-cache['mu_l'][unique,t:])/cache['sigma_l'][unique,t:]
162         z_g=(ds.pv_kw[unique,t:]-cache['mu_g'][unique,issue,t:])/cache['sigma_g_'+mode][unique,issue,t:]
163         load=np.maximum(0,ml+cache['sigma_l'][day,t:]*z_l)
164         pv=np.maximum(0,mg+cache['sigma_g_'+mode][day,issue,t:]*z_g)
165     if not np.isfinite(load).all() or not np.isfinite(pv).all():raise ValueError('Nonfinite scenario')
166     audit={'date':ds.dates[day].isoformat(),'decision_hour':int(current_hour),'official_issue_hour':int(source_hour)
↪ },
167     'forecast_age_hours':int(current_hour-source_hour),'first_lead_hours':float(current_hour-source_hour+1/6)
↪ },
168     'last_lead_hours':float(24-source_hour),'calibration_end':ds.dates[day-1].isoformat() if day else None,
169     'available_independent_days':len(history),'unique_sampled_days':len(unique),'requested_draws':draws,
170     'sampled_day_indices':[int(x) for x in sampled],'scale_mode':mode,'point_forecast':point,
171     'window_days':int(window),'bin_hours':int(cache.get('bin_hours',6)),
172     'shrink_days':float(cache.get('shrink_days',7))}
173     return load/6,pv/6,prob,audit
174
175
176 def probability_scores(scenarios,prob,y,delta=.1):
177     x=np.asarray(scenarios,dtype=float);p=np.asarray(prob,dtype=float);y=np.asarray(y,dtype=float)
178     if x.ndim!=2 or p.shape!=(len(x),) or y.shape!=(x.shape[1],):raise ValueError('Invalid score dimensions')
179     order=np.argsort(x,axis=0);values=np.take_along_axis(x,order,axis=0)
180     weights=np.broadcast_to(p[:,None],x.shape);weights=np.take_along_axis(weights,order,axis=0)
181     cumulative=weights.cumsum(axis=0)
182     quant=lambda a:values[np.argmax(cumulative>=a,axis=0),np.arange(x.shape[1])]
183     low,high=quant(delta/2),quant(1-delta/2)
184     crps=(p[:,None]*abs(x-y)).sum(axis=0)-(weights*values*(2*cumulative-weights-1)).sum(axis=0)
185     return {'crps':crps,'coverage':((y>=low)&(y<=high)).astype(float),'width':high-low,
186           'interval_score':high-low+2/delta*(np.maximum(low-y,0)+np.maximum(y-high,0)),
187           'mae':abs((p[:,None]*x).sum(axis=0)-y),
188           'squared_error':((p[:,None]*x).sum(axis=0)-y)**2}

```

q4_forecasting.py

```

1 """Causal price/net-load forecasts and next-release signal scenarios."""
2 from pathlib import Path
3 import csv
4 import hashlib
5 import json
6 import numpy as np
7 from sklearn.cluster import KMeans
8 from forecasting import read_dataset, clock_text
9 from forecasting_v2 import predict
10 from q3_forecasting import read_official, integrate_hourly
11
12
13 def read_inputs(root):
14     root=Path(root)
15     ds=read_dataset(root)
16     rows=list(csv.DictReader((root/'price_points.csv').open(encoding='utf-8-sig',newline='')))
17     if len(rows)!=365*144:raise ValueError('Incomplete price grid')
18     prices=np.empty((365,144))
19     for ix,row in enumerate(rows):
20         day,t=divmod(ix,144)
21         if row['date']!=ds.dates[day].isoformat() or int(row['point_index'])!=t+1 or
22             ↪ row['time_label']!=clock_text((t+1)*10):
23             raise ValueError('Price date/time mapping mismatch')
24         prices[day,t]=float(row['price_yuan_per_kwh'])
25     if not np.isfinite(prices).all() or (prices<=0).any():raise ValueError('Positive finite actual prices required')
26     official=read_official(root/'pv_forecasts.csv',ds.dates)
27     ds.sources.update({name:hashlib.sha256((root/name).read_bytes()).hexdigest() for name in
28         ↪ ('price_points.csv','pv_forecasts.csv')})
29     return ds,prices,official
30
31 def price_predictions(prices,prior,method,alpha=0.,decay=.1):
32     """The day-zero price prior is attachment 1; all other inputs are past."""
33     prices=np.asarray(prices,float);days=len(prices);logp=np.log(prices)
34     result=np.full((days,4,144),np.nan)
35     for i in range(days):
36         if not i:base=np.log(prior)
37         elif method=='yesterday':base=logp[i-1].copy()
38         else:
39             window=int(method.split('_')[1]);begin=max(0,i-window)
40             if method.startswith('mean_'):
41                 base=np.log(prices[begin:i].mean(axis=0))
42             else:
43                 weights=2.*(-np.arange(i-begin-1,-1,-1)/7.);weights/=weights.sum()
44                 base=weights@logp[begin:i]
45             delta=0.
46             for issue in range(4):
47                 t=issue*36
48                 if issue:
49                     for j in range(t-36,t):delta=(1-alpha)*delta+alpha*(logp[i,j]-base[j])
50                     lead=(np.arange(t,144)+1)/6-issue*6
51                     result[i,issue,t]=base[t]+np.exp(-decay*lead)*delta
52     return result
53
54 def select_price(prices,prior,start=20,end=31):
55     methods=['yesterday','mean_7','mean_14','mean_28','weighted_7','weighted_14','weighted_28']
56     candidates=[]
57     for method in methods:
58         forecast=price_predictions(prices[:end],prior,method)
59         error=np.exp(forecast[start:end,0])-prices[start:end]
60         candidates.append({'method':method,'mae':float(np.abs(error).mean()),'rmse':float(np.sqrt(np.mean(error**2))
61             ↪ )})
62     chosen=min(candidates,key=lambda x:(x['mae'],methods.index(x['method'])))['method']
63     corrections=[]
64     for alpha in (0.,.2,.5):
65         for decay in ((0.,) if alpha==0 else (0.,.1,.5)):
66             forecast=price_predictions(prices[:end],prior,chosen,alpha,decay)
67             errors=np.concatenate([(np.exp(forecast[start:end,k,k*36:])-prices[start:end,k*36:]).ravel() for k in
68                 ↪ (1,2,3)])
69             corrections.append({'alpha':alpha,'decay_per_hour':decay,'mae':float(np.abs(errors).mean()),

```

```

68         'rmse':float(np.sqrt(np.mean(errors**2))))
69     best=min(corrections,key=lambda x:(x['mae'],x['alpha'],x['decay_per_hour']))
70     return {'method':chosen,**best,'price_candidates':candidates,'correction_candidates':corrections,
71            'validation_start':'2025-01-21','validation_end':'2025-01-31','criterion':'historical MAE',
72            'formal_frozen_at':'2025-02-01T00:00'}
73
74
75 def supply_predictions(ds,official,stop=365,progress=None):
76     load=np.empty((stop,144));pv=np.empty_like(load)
77     p1_load=np.empty_like(load);p1_pv=np.empty_like(load)
78     raw=np.full((stop,4,144),np.nan)
79     for i in range(stop):
80         for model,l,g in [('P3_random_forest',load,pv),('P1_weighted28',p1_load,p1_pv)]:
81             l[i],g[i],_=predict(ds.load_kw[:i],ds.pv_kw[:i],ds.dates[:i],ds.dates[i],model,
82                                ds.baseline_load_kw,ds.baseline_pv_kw,20260912)
83         for issue in range(4):
84             t=issue*36
85             anchor=ds.pv_kw[i,t-1] if t else (ds.pv_kw[i-1,-1] if i else 0.)
86             raw[i,issue,t]=integrate_hourly(official[i,issue],float(anchor),issue*6)/6
87             if progress and (i==0 or (i+1)%60==0):progress(f'supply forecasts {i+1}/{stop}')
88     return {'load':load/6,'pv':pv/6,'p1_load':p1_load/6,'p1_pv':p1_pv/6,'official':raw}
89
90
91 def make_cache(ds,prices,supply,selection,*,warmup=False):
92     # During actual January warmup the method is preregistered, not selected on January's future.
93     days=len(supply['load'])
94     method='weighted_7' if warmup else selection['method']
95     alpha=0. if warmup else selection['alpha'];decay=0. if warmup else selection['decay_per_hour']
96     y=price_predictions(prices[:days],ds.prices,method,alpha,decay)
97     y0=price_predictions(prices[:days],ds.prices,method,0.,0.)
98     load=supply['p1_load' if warmup else 'load']
99     pv=supply['p1_pv' if warmup else 'pv']
100    return {'**supply','active_load':load,'net_q42':load-pv,'net_q43':load[:,None,:]-supply['official'],
101           'log_price':y,'log_price_uncorrected':y0}
102
103
104 def information_groups(signals,k,minimum_days=5,seed=20260913):
105     signals=np.asarray(signals,float)
106     sigma=signals.std(axis=0)
107     active=sigma>1e-10
108     metadata={'requested_k':int(k),'minimum_distinct_days':int(minimum_days),
109             'scale':sigma.tolist(),'active_dimensions':active.tolist(),'centers':[]}
110     if k==1 or len(signals)<k*minimum_days or not active.any():
111         metadata.update(actual_k=1,reason='requested one group or insufficient signal samples')
112         return np.zeros(len(signals),int),metadata
113     z=signals[:,active]/sigma[active]
114     model=KMeans(n_clusters=k,random_state=seed,n_init=10).fit(z)
115     count=np.bincount(model.labels_,minlength=k)
116     if count.min()<minimum_days:
117         metadata.update(actual_k=1,reason='group has fewer than five distinct historical days')
118         return np.zeros(len(signals),int),metadata
119     metadata.update(actual_k=int(k),centers=model.cluster_centers_.tolist(),counts=count.tolist(),reason='historical
120     ↪ paired signals')
121     return model.labels_,metadata
122
123 def classify_signal(observed,metadata):
124     if metadata['actual_k']==1:return 0
125     active=np.array(metadata['active_dimensions'],bool)
126     z=np.asarray(observed)[active]/np.asarray(metadata['scale'])[active]
127     return int(np.argmax(((np.asarray(metadata['centers'])-z)**2).sum(axis=1))))
128
129
130 def scenarios(ds,prices,cache,day,issue,*,mode='q42',window=28,point=False,price_correction=True,
131             groups=1,price_mean_only=False):
132     """Only whole historical days enter residuals and future-release scenarios."""
133     if mode=='q42' and issue!=0:raise ValueError('Q4-2 only plans at midnight')
134     t=issue*36
135     predicted_net=cache['net_q42'][day] if mode=='q42' else cache['net_q43'][day,issue]
136     log_forecast=cache['log_price' if price_correction else 'log_price_uncorrected']
137     historical=np.arange(max(7,day-window),day)
138     if point or not len(historical):
139         net=predicted_net[None,t:].copy();p=np.exp(log_forecast[day,issue,t:])[None,:]
```

```

140     ids=np.array([],int);prob=np.ones(1)
141     else:
142         ids=historical
143         old_prediction=cache['net_q42'][ids,t:] if mode=='q42' else cache['net_q43'][ids,issue,t:]
144         actual_net=(ds.load_kw[ids,t:]-ds.pv_kw[ids,t:])/6
145         net=predicted_net[None,t:]+actual_net-old_prediction
146         price_error=np.log(prices[ids,t:]-log_forecast[ids,issue,t:])
147         p=np.exp(log_forecast[day,issue,t:]+price_error)
148         prob=np.full(len(ids),1/len(ids))
149     if price_mean_only:p=np.broadcast_to(prob@p,p.shape).copy()
150     if not np.isfinite(net).all() or not np.isfinite(p).all() or (p<=0).any():raise ValueError('Invalid generated
↪ scenario')
151     grouping=None
152     meta={'day':ds.dates[day].isoformat(),'hour':int(issue*6),'history_days':ids.tolist(),
153           'latest_history':ds.dates[day-1].isoformat() if day else None,'point':bool(point),
154           'price_prefix_end_index':t,'official_visible_issue':int(issue*6) if mode=='q43' else None,
155           'scenario_count':len(prob),'negative_net_values':int((net<0).sum())}
156     if mode=='q43' and issue<3:
157         b=t+36
158         if len(ids):
159             # Simulated next release is current official forecast + a historical paired revision.
160             now=cache['official'][day,issue,b:b+36]
161             revision=cache['official'][ids,issue+1,b:b+36]-cache['official'][ids,issue,b:b+36]
162             hypothetical=np.maximum(0,now+revision)
163             zg=(hypothetical-now).sum(axis=1)
164             zp=(np.log(p[:30:36])-log_forecast[day,issue,b-6:b]).mean(axis=1)
165             signal=np.c_[zg,zp]
166             grouping,group_meta=information_groups(signal,groups)
167             group_meta['next_forecast_negative_clipped']=int((now+revision<0).sum())
168             group_meta['signal_scenarios']=signal.tolist()
169         else:
170             grouping=np.zeros(1,int);group_meta={'requested_k':groups,'actual_k':1,'reason':'deterministic cold
↪ start'}
171     meta['grouping']=group_meta
172     return net,p,prob,grouping,meta
173
174
175 def realized_signal(ds,prices,cache,day,issue,price_correction=True):
176     """Called at the NEXT update only, after its observations are released."""
177     b=(issue+1)*36
178     if issue>=3:raise ValueError('No within-day next update')
179     zg=float((cache['official'][day,issue+1,b:b+36]-cache['official'][day,issue,b:b+36]).sum())
180     y=cache['log_price' if price_correction else 'log_price_uncorrected']
181     zp=float((np.log(prices[day,b-6:b])-y[day,issue,b-6:b]).mean())
182     return np.array([zg,zp])

```

q4_dispatch.py

```

1 """model-q4.tex: paired/shrunk recourse and one-update grouped lookahead.
2
3 All flow variables are AC-side kWh; stock is internal kWh. The solver only
4 receives current forecasts/scenarios, never future observations.
5 """
6 from dataclasses import dataclass
7 import time
8 import numpy as np
9 from scipy.optimize import linprog, milp, Bounds, LinearConstraint
10 from scipy.sparse import coo_matrix
11
12
13 @dataclass(frozen=True)
14 class Battery:
15     eta_c: float = .9
16     eta_d: float = .9
17     minimum: float = 1200.
18     maximum: float = 10800.
19     limit: float = 5000/6
20     reference: float = 6000.
21
22
23 class LinearModel:

```

```

24 def __init__(self):
25     self.obj, self.lo, self.hi, self.integer = [], [], [], []
26     self.eq, self.rhs_eq, self.ub, self.rhs_ub = [], [], [], []
27
28 def variables(self, size, *, cost=0., lo=0., hi=np.inf, integer=False):
29     start = len(self.obj)
30     for target, value in ((self.obj,cost),(self.lo,lo),(self.hi,hi),(self.integer,int(integer))):
31         target.extend(np.broadcast_to(value,(size,)).tolist())
32     return np.arange(start,start+size)
33
34 def row(self, pairs, rhs, equality=False):
35     (self.eq if equality else self.ub).append(dict(pairs))
36     (self.rhs_eq if equality else self.rhs_ub).append(float(rhs))
37
38 def matrix(self, rows):
39     ri, ci, data = [], [], []
40     for i,row in enumerate(rows):
41         for j,v in row.items():
42             if v:
43                 ri.append(i); ci.append(j); data.append(v)
44     return coo_matrix((data,(ri,ci)),shape=(len(rows),len(self.obj))).tocsr()
45
46 def solve(self, method):
47     ae, au = self.matrix(self.eq), self.matrix(self.ub)
48     if method == 'milp':
49         constraints = [LinearConstraint(ae,self.rhs_eq,self.rhs_eq),
50                        LinearConstraint(au,np.full(len(self.ub),-np.inf),self.rhs_ub)]
51         result = milp(np.array(self.obj),integrality=np.array(self.integer),
52                       bounds=Bounds(self.lo,self.hi),constraints=constraints,
53                       options={'time_limit':120.,'mip_rel_gap':1e-9})
54     else:
55         result = linprog(self.obj,A_ub=au,b_ub=self.rhs_ub,A_eq=ae,b_eq=self.rhs_eq,
56                           bounds=list(zip(self.lo,self.hi)),method='highs',
57                           options={'time_limit':120.,'dual_feasibility_tolerance':1e-8,
58                                    'primal_feasibility_tolerance':1e-8})
59     if not result.success:
60         raise RuntimeError(f'{method} solver failed: {result.message}')
61     return result
62
63
64 def purify(charge, discharge, battery):
65     """The exact stock-preserving LP purification in model-q4 §14."""
66     c, d = np.array(charge,copy=True), np.array(discharge,copy=True)
67     epsilon = np.minimum(c,d/(battery.eta_c*battery.eta_d))
68     return c-epsilon,d-battery.eta_c*battery.eta_d*epsilon
69
70
71 def bill(price, net, q, r, c, d, *, adjustment=False, settlement='refund'):
72     e = np.maximum(net+c-d-r,0.)
73     u = np.maximum(r+d-c-net,0.)
74     up, down = np.maximum(r-q,0.), np.maximum(q-r,0.)
75     planned = float(price*q)
76     adjustment_cost = float(price@(1.5*up + (-.5 if settlement=='refund' else .5)*down)) if adjustment else 0.
77     emergency = float(5*price*e)
78     return dict(E=e,U=u,planned_cost=planned,adjustment_cost=adjustment_cost,
79                emergency_cost=emergency,cash_cost=planned+adjustment_cost+emergency,
80                adjustment_up=up,adjustment_down=down)
81
82
83 def physical_checks(net, r, c, d, s, initial, battery, terminal_min=None):
84     e, u = np.maximum(net+c-d-r,0.),np.maximum(r+d-c-net,0.)
85     return {
86         'stock_transition':float(np.max(np.abs(np.diff(s)-battery.eta_c*c+d/battery.eta_d))),
87         'stock_bounds':max(0.,float(battery.minimum-s.min()),float(s.max()-battery.maximum)),
88         'power_bounds':max(0.,float(c.max()-battery.limit),float(d.max()-battery.limit)),
89         'nonnegative':max(0.,float(-min(r.min(),c.min(),d.min()))),
90         'simultaneous':float(np.minimum(c,d).max()),
91         'initial_stock':abs(float(s[0])-initial),
92         'balance':float(np.max(np.abs(r+e+d-c-u-net))),
93         'terminal_min':max(0.,float(terminal_min-s[-1])) if terminal_min is not None else 0.
94     }
95
96

```

```

97 def solve_dispatch(net, price, prob, initial, *, mode='q42', original_q=None,
98                   groups=None, prefix=36, rho=1., gamma=.5, terminal_min=None,
99                   battery=Battery(), method='lp', settlement='refund'):
100     """Common prefix, one conditional tail per group; groups=None is baseline.
101
102     original_q=None means midnight: all q are jointly optimized. Q4-2 forbids
103     adjustments. For Q4-3 at midnight the executed prefix has r=q. At later
104     updates original_q is fixed. Terminal conditions apply to EVERY branch.
105     """
106     start = time.perf_counter()
107     net, price, prob = np.asarray(net,float),np.asarray(price,float),np.asarray(prob,float)
108     if net.ndim!=2 or net.shape!=price.shape or not np.isfinite(net).all() or not np.isfinite(price).all():
109         raise ValueError('Finite matching scenario arrays required')
110     scenarios, n = net.shape
111     if prob.shape!=(scenarios,) or (prob<=0).any() or abs(prob.sum()-1)>1e-10 or (price<=0).any():
112         raise ValueError('Positive prices and normalized positive probabilities required')
113     if mode not in ('q42','q43') or method not in ('lp','milp') or not 0<=rho<=1 or gamma<0:
114         raise ValueError('Invalid model option')
115     if not battery.minimum-1e-6 <= initial <= battery.maximum+1e-6:
116         raise ValueError('Initial stock outside bounds')
117     if mode=='q43' and rho!=1:
118         raise ValueError('Q4-3 main model does not combine dependency shrinkage with lookahead')
119     if settlement not in ('refund','penalty'):
120         raise ValueError('Unknown adjustment settlement')
121     midnight = original_q is None
122     if not midnight:
123         original_q = np.asarray(original_q,float)
124         if original_q.shape!=(n,) or not np.isfinite(original_q).all() or (original_q < -1e-7).any():
125             raise ValueError('Invalid original commitment')
126     branched = groups is not None and prefix<n and mode=='q43'
127     if branched:
128         labels, group = np.unique(np.asarray(groups),return_inverse=True)
129         if group.shape!=(scenarios,) or not 0<prefix<n:
130             raise ValueError('Invalid information groups')
131     else:
132         labels, group, prefix = np.array([0]),np.zeros(scenarios,dtype=int),n
133     weights = prob[:,None]*price
134     mean_price = weights.sum(axis=0)
135     recourse_weights = prob[:,None]*(rho*price+(1-rho)*mean_price)
136     model = LinearModel()
137     q = model.variables(n,lo=0. if midnight else original_q,hi=np.inf if midnight else original_q)
138     nodes = []
139     node_map = np.zeros((scenarios,n),dtype=int)
140     definitions = [(0,prefix,np.arange(scenarios))]
141     if branched:
142         for m in range(len(labels)):
143             ids = np.flatnonzero(group==m)
144             definitions.append((prefix,n,ids)); node_map[ids,prefix:]=m+1
145     for number,(a,b,ids) in enumerate(definitions):
146         length = b-a
147         wp = weights[ids,a:b].sum(axis=0)
148         pm = float(prob[ids].sum())
149         r = model.variables(length,cost=wp)
150         c, d = model.variables(length,hi=battery.limit),model.variables(length,hi=battery.limit)
151         stock = model.variables(length+1,lo=battery.minimum,hi=battery.maximum)
152         if number==0:
153             model.row([(stock[0],1)],initial,True)
154         else:
155             model.row([(stock[0],1),(nodes[0]['s'][-1],-1)],0,True)
156         if b==n:
157             v = model.variables(1,cost=gamma*pm)[0]
158             model.row([(stock[-1],-1),(v,-1)],-battery.reference)
159             if terminal_min is not None:
160                 model.row([(stock[-1],-1)],-terminal_min)
161         fixed = mode=='q42' or (midnight and number==0)
162         if fixed:
163             for t in range(length):model.row([(r[t],1),(q[a+t],-1)],0,True)
164         else:
165             up = model.variables(length,cost=.5*wp)
166             down = model.variables(length,cost=(.5 if settlement=='refund' else 1.5)*wp)
167             for t in range(length):
168                 model.row([(r[t],1),(q[a+t],-1),(up[t],-1),(down[t],1)],0,True)
169     for t in range(length):

```

```

170     model.row([(stock[t+1],1),(stock[t],-1),(c[t],-battery.eta_c),(d[t],1/battery.eta_d)],0,True)
171     if method=='milp':
172         z=model.variables(length,hi=1,integer=True)
173         for t in range(length):
174             model.row([(c[t],1),(z[t],-battery.limit)],0)
175             model.row([(d[t],1),(z[t],battery.limit)],battery.limit)
176         nodes.append(dict(a=a,b=b,ids=ids,prob=pm,r=r,c=c,d=d,s=stock))
177     emergency=model.variables(scenarios*n,cost=(5*recourse_weights).ravel()).reshape(scenarios,n)
178     for w in range(scenarios):
179         for t in range(n):
180             node=nodes[node_map[w,t]]; j=t-node['a']
181             model.row([(node['c'][j],1),(node['d'][j],-1),(node['r'][j],-1),(emergency[w,t],-1)],-net[w,t])
182     solved=model.solve(method)
183     x=solved.x; planned=np.maximum(x[q],0.)
184     action_nodes=[]; maxima={}; purification_reduction=0.
185     for node in nodes:
186         c0,d0=x[node['c']],x[node['d']]
187         c,d=purify(np.maximum(c0,0.),np.maximum(d0,0.),battery)
188         r=np.maximum(x[node['r']],0.); stock=x[node['s']]
189         check=physical_checks(net[node['ids'][0],node['a']]:node['b'],r,c,d,stock,float(stock[0]),battery,
190                               terminal_min if node['b']==n else None)
191         check['purification_stock']=float(np.max(np.abs(battery.eta_c*c0-d0/battery.eta_d-battery.eta_c*c+d/battery_
192 ↪ eta_d)))
193         for k,v in check.items():maxima[k]=max(maxima.get(k,0.),v)
194         purification_reduction+=float((c0-c).sum())
195         action_nodes.append(dict(start=node['a'],stop=node['b'],ids=node['ids'].tolist(),prob=node['prob'],
196 ↪ R=r,C=c,D=d,S=stock))
197     r_all=np.empty_like(net);c_all=np.empty_like(net);d_all=np.empty_like(net)
198     terminal=0.
199     for node in action_nodes:
200         ids=np.array(node['ids']);a,b=node['start'],node['stop']
201         r_all[ids,a:b]=node['R'];c_all[ids,a:b]=node['C'];d_all[ids,a:b]=node['D']
202         if b==n:terminal+=gamma*node['prob']*max(battery.reference-node['S'][-1],0.)
203     e_all=np.maximum(net+c_all-d_all-r_all,0.)
204     up=np.maximum(r_all-planned,0.);down=np.maximum(planned-r_all,0.)
205     normal=r_all if mode=='q42' else planned+1.5*up+(-.5 if settlement=='refund' else .5)*down
206     cash=float((weights*normal).sum()+5*(recourse_weights*e_all).sum())
207     objective=cash+terminal
208     maxima['objective_recomputed']=abs(objective-float(solved.fun))
209     maxima['bridge']=max([abs(float(node['S'][0])-float(action_nodes[0]['S'][-1])) for node in action_nodes[1:]] or
210 ↪ [0.])
211     maxima['initial']=abs(float(action_nodes[0]['S'][0])-initial)
212     if max(maxima.values())>2e-5:
213         raise RuntimeError(f'Dispatch verification failed: {maxima}')
214     return dict(Q=planned,nodes=action_nodes,groups=group,objective=objective,expected_cash=cash,
215 ↪ terminal_cost=terminal,solver_objective=float(solved.fun),checks=maxima,
216 ↪ runtime_s=time.perf_counter()-start,method=method,
217 ↪ mip_gap=float(getattr(solved,'mip_gap',0.) or 0.),
218 ↪ purified_charge_kwh=purification_reduction,scenarios=scenarios,branches=len(labels) if branched else
219 ↪ 1)

```

prepare_q4.py

```

1 from pathlib import Path
2 import json
3 import hashlib
4 import numpy as np
5 from q4_forecasting import read_inputs, select_price, supply_predictions, make_cache
6
7 ROOT=Path(__file__).resolve().parents[1]
8
9
10 def main():
11     folder=ROOT/'outputs/prepared'
12     folder.mkdir(parents=True,exist_ok=True)
13     ds,prices,official=read_inputs(ROOT/'data/processed')
14     selection=select_price(prices,ds.prices)
15     source_hashes={p.name:hashlib.sha256(p.read_bytes()).hexdigest() for p in (ROOT/'src').glob('*.*py')}
16     if p.name in ('q4_forecasting.py','forecasting.py','forecasting_v2.py','q3_forecasting.py')}
17     signature={'inputs':ds.sources,'prediction_sources':source_hashes,'seed':20260912}
18     if (folder/'manifest.json').exists():

```

```

19         if json.loads((folder/'manifest.json').read_text('utf-8'))!=signature:
20             raise ValueError('Prediction source/input changed; use a new prepared directory')
21         (folder/'manifest.json').write_text(json.dumps(signature,ensure_ascii=False,indent=2),encoding='utf-8')
22     if (folder/'supply.npz').exists():
23         with np.load(folder/'supply.npz') as z:supply={k:z[k] for k in z.files}
24     else:
25         supply=supply_predictions(ds,official,progress=lambda s:print(s,flush=True))
26         np.savez_compressed(folder/'supply.npz',**supply)
27     for key,warmup in [ ('formal',False), ('warmup',True)]:
28         cache=make_cache(ds,prices,supply,selection,warmup=warmup)
29         np.savez_compressed(folder/f'{key}.npz',**cache)
30     (folder/'price_selection.json').write_text(json.dumps(selection,ensure_ascii=False,indent=2),encoding='utf-8')
31     print(json.dumps(selection,ensure_ascii=False,indent=2),flush=True)
32
33
34 if __name__=='__main__':main()

```

run_q4.py

```

1 """Preregistered historical selection, continuous replay, and Q4 ablations."""
2 from pathlib import Path
3 from dataclasses import asdict
4 import argparse
5 import csv
6 import hashlib
7 import json
8 import time
9 import numpy as np
10 from q4_dispatch import Battery, solve_dispatch, bill, physical_checks
11 from q4_forecasting import read_inputs, scenarios, realized_signal, classify_signal
12
13 ROOT=Path(__file__).resolve().parents[1]
14
15
16 def write_json(path,value):
17     Path(path).parent.mkdir(parents=True,exist_ok=True)
18     def default(obj):
19         if isinstance(obj,np.ndarray):return obj.tolist()
20         if isinstance(obj,np.generic):return obj.item()
21         raise TypeError(type(obj).__name__)
22     Path(path).write_text(json.dumps(value,ensure_ascii=False,indent=2,default=default)+'\n',encoding='utf-8')
23
24
25 def write_csv(path,rows):
26     with Path(path).open('w',encoding='utf-8-sig',newline='') as f:
27         writer=csv.DictWriter(f,fieldnames=list(rows[0]));writer.writeheader();writer.writerows(rows)
28
29
30 def load_cache(name):
31     with np.load(ROOT/f'outputs/prepared/{name}.npz') as data:return {k:data[k] for k in data.files}
32
33
34 def prediction_score(values,prob,actual):
35     means=prob@values
36     order=np.argsort(values,axis=0);v=np.take_along_axis(values,order,axis=0)
37     w=np.take_along_axis(np.broadcast_to(prob[:,None],values.shape),order,axis=0)
38     cw=np.cumsum(w,axis=0)
39     crps=(prob@np.abs(values-actual)-(w*v*(2*cw-w-1)).sum(axis=0))
40     low=np.take_along_axis(v,(cw>=.1).argmax(axis=0)[None,:],axis=0)[0]
41     high=np.take_along_axis(v,(cw>=.9).argmax(axis=0)[None,:],axis=0)[0]
42     return {'abs_error':float(np.abs(means-actual).sum()),'squared_error':float(((means-actual)**2).sum()),
43           'crps':float(crps.sum()),'coverage80':int(((actual>=low)&(actual<=high)).sum()),'points':len(actual)}
44
45
46 def experiment(ds,prices,cache,folder,days,initial,*,mode,rho=1.,gamma=.5,k=1,
47               point=False,price_correction=True,lookahead=False,terminal=False,
48               settlement='refund',method='lp',progress=True):
49     folder=Path(folder);folder.mkdir(parents=True,exist_ok=True)
50     days=list(days)
51     config={'mode':mode,'rho':rho,'gamma':gamma,'k':k,'point':point,'price_correction':price_correction,

```

```

52         'lookahead':lookahead,'days':days,'initial':initial,'terminal':terminal,'settlement':settlement,'method'
↪         ':method')
53     if (folder/'summary.json').exists():
54         if json.loads((folder/'config.json').read_text('utf-8'))!=config:
55             raise ValueError('Refusing to resume an experiment with different settings')
56         return json.loads((folder/'summary.json').read_text('utf-8'))
57     write_json(folder/'config.json',config)
58     arrays={key:np.zeros((len(days),145 if key=='S' else 144)) for key in
↪     ('Q','R','C','D','S','E','U','forecast_price')}
59     arrays['day_indices']=np.array(days)
60     rows=[];logs=[];maxima={};state=float(initial);battery=Battery()
61     total_runtime=0.;scores=dict(abs_error=0.,squared_error=0.,crps=0.,coverage80=0,points=0)
62     groups_count={};started=time.perf_counter();optimizations=0
63     for ix,day in enumerate(days):
64         q=None;before=None;day_initial=state;planned_cost=adjustment_cost=emergency_cost=0.
65         arrays['S'][ix,0]=state
66         issues=(0,) if mode=='q42' else (0,1,2,3)
67         day_solver=0.
68         for issue in issues:
69             t=issue*36;count=144 if mode=='q42' else 36
70             observed_group=None
71             if before is not None and 'grouping' in before and 'centers' in before['grouping']:
72                 observed_group=classify_signal(realized_signal(ds,prices,cache,day,issue-1,price_correction),before)
↪                 ['grouping'])
73             net,price,prob,group,meta=scenarios(ds,prices,cache,day,issue,mode=mode,point=point,
74                 price_correction=price_correction,groups=k if lookahead else 1)
75             final_min=6000. if terminal and day==days[-1] else None
76             solved=solve_dispatch(net,price,prob,state,mode=mode,original_q=None if q is None else q[t:],
77                 groups=group if lookahead else None,prefix=36,rho=rho,gamma=gamma,
78                 terminal_min=final_min,battery=battery,method=method,settlement=settlement)
79             if q is None:q=solved['Q'].copy();arrays['Q'][ix]=q
80             node=solved['nodes'][0]
81             r,c,d=[node[key][:count] for key in ('R','C','D')]
82             s=np.r_[state,state+np.cumsum(battery.eta_c*d/battery.eta_d)]
83             actual_net=(ds.load_kw[day,t:t+count]-ds.pv_kw[day,t:t+count])/6
84             actual_price=prices[day,t:t+count]
85             paid=bill(actual_price,actual_net,q[t:t+count],r,c,d,adjustment=mode=='q43',settlement=settlement)
86             for key,values in
↪             [(('R',r),('C',c),('D',d),('E',paid['E']),('U',paid['U'])),('forecast_price',prob@price[:,count]))]:
87                 arrays[key][ix,t:t+count]=values
88             arrays['S'][ix,t:t+count+1]=s
89             check=physical_checks(actual_net,r,c,d,s,state,battery,final_min if t+count==144 else None)
90             check['planned_execution_state']=float(np.max(np.abs(s-node['S'][:count+1])))
91             for key,value in (**solved['checks'],**check).items():maxima[key]=max(maxima.get(key,0.),value)
92             if max(check.values())>2e-5:raise RuntimeError(f'Physical replay violation: {check}')
93             block_score=prediction_score(price[:,count],prob,actual_price)
94             for key in scores:scores[key]+=block_score[key]
95             state=float(s[-1]);planned_cost+=paid['planned_cost'];adjustment_cost+=paid['adjustment_cost'];emergenc
↪             y_cost+=paid['emergency_cost']
96             day_solver+=solved['runtime_s'];optimizations+=1
97             groups_count[str(solved['branches'])]=groups_count.get(str(solved['branches']),0)+1
98             # Future group actions are recorded for audit, never executed at this update.
99             logs.append({'*meta','observed_previous_group':observed_group,'initial_stock':float(s[0]),
100                 'executed_terminal_stock':state,'objective':solved['objective'],'expected_cash':solved['expected_ca
↪                 sh'],
101                 'terminal_penalty':solved['terminal_cost'],'solver_runtime_s':solved['runtime_s'],
102                 'solver_method':method,'mip_gap':solved['mip_gap'],'purified_charge_kwh':solved['purified_charge_kw
↪                 h'],
103                 'branch_nodes':[{key:n[key] for key in ('start','stop','ids','prob','R','C','D','S')} for n in
↪                 solved['nodes']],
104                 'checks':solved['checks']})
105             before=meta
106         total_runtime+=day_solver
107         rows.append({'date':ds.dates[day].isoformat(),'planned_kwh':float(q.sum()),'executed_normal_kwh':float(arr
↪         ys['R'][ix].sum()),
108             'planned_cost_yuan':planned_cost,'adjustment_cost_yuan':adjustment_cost,'emergency_cost_yuan':emergency
↪             _cost,
109             'cash_cost_yuan':planned_cost+adjustment_cost+emergency_cost,'emergency_kwh':float(arrays['E'][ix].sum(
↪             _)),
110             'unused_kwh':float(arrays['U'][ix].sum()),'charge_kwh':float(arrays['C'][ix].sum()),'discharge_kwh':flo
↪             at(arrays['D'][ix].sum()),
111             'up_kwh':float(np.maximum(arrays['R'][ix]-q,0.).sum()),'down_kwh':float(np.maximum(q-arrays['R'][ix],0.
↪             ).sum()),

```

```

112     'initial_kwh':day_initial,'terminal_kwh':state,'solver_runtime_s':day_solver})
113     if progress and (ix==0 or (ix+1)%30==0 or ix+1==len(days)):
114         print(f'{folder.name}: {ix+1}/{len(days)}, cash={sum(r["cash_cost_yuan"] for r in rows):.2f},
115             ↳ {time.perf_counter()-started:.1f}s', flush=True)
116 np.savez_compressed(folder/'trajectory.npz',**arrays)
117 write_csv(folder/'daily.csv',rows)
118 write_json(folder/'planning_log.json',logs)
119 summary={'name':folder.name,'days':len(days),'initial_kwh':initial,'terminal_kwh':state,**config,
120         'cash_cost_yuan':sum(r['cash_cost_yuan'] for r in rows),
121         **{key:sum(r[key] for r in rows) for key in ('planned_cost_yuan','adjustment_cost_yuan','emergency_cost_yua
122             ↳ n','emergency_kwh','unused_kwh','charge_kwh','discharge_kwh','up_kwh','down_kwh')},
123         'price_mae':scores['abs_error']/scores['points'],'price_rmse':np.sqrt(scores['squared_error']/scores['point
124             ↳ s'])},
125         'price_crps':scores['crps']/scores['points'],'price_coverage80':scores['coverage80']/scores['points'],
126         'optimization_count':optimizations,'solver_runtime_s':total_runtime,'wall_runtime_s':time.perf_counter()-st
127             ↳ arted,
128         'checks_maxima':maxima,'group_counts':groups_count,
129         'end_at_min':sum(abs(r['terminal_kwh']-1200)<1e-4 for r in rows),
130         'end_at_max':sum(abs(r['terminal_kwh']-10800)<1e-4 for r in rows))
131 # Config contains the exact day indices; expose the count separately.
132 summary['days']=len(days)
133 write_json(folder/'summary.json',summary)
134 return summary
135
136 def select_parameters(ds,prices,folder):
137     warmcache,cache=load_cache('warmup'),load_cache('formal')
138     warms={}
139     for mode in ('q42','q43'):
140         warms[mode]=experiment(ds,prices,warmcache,folder/('warmup_'+mode),range(31),6000.,
141                               mode=mode,rho=0. if mode=='q42' else 1.,gamma=.5)
142     choices={'selection_cutoff':'2025-01-31T24:00','relative_tie_tolerance':1e-4,'warmup':warms}
143     validation=[]
144     for mode in ('q42','q43'):
145         with np.load(folder/('warmup_'+mode)/'trajectory.npz') as z:initial=float(z['S'][19,-1])
146         terminal_tests=[]
147         for gamma in (0.,.25,.5,1.):
148             name=f'validate_{mode}_gamma_{gamma:g}'
149             s=experiment(ds,prices,cache,folder/name,range(20,31),initial,mode=mode,gamma=gamma,terminal=True)
150             terminal_tests.append(s);validation.append(s)
151             gamma=min(terminal_tests,key=lambda x:(x['cash_cost_yuan'],x['gamma']))['gamma']
152         candidates=[]
153         for value in ((0.,.25,.5,.75,1.) if mode=='q42' else (1,2,3)):
154             key='rho' if mode=='q42' else 'k'
155             kw={'rho':value} if mode=='q42' else {'k':value,'lookahead':True}
156             s=experiment(ds,prices,cache,folder/f'validate_{mode}_{key}_{value:g}',range(20,31),initial,
157                         mode=mode,gamma=gamma,terminal=True,**kw)
158             candidates.append(s);validation.append(s)
159         minimum=min(s['cash_cost_yuan'] for s in candidates)
160         key='rho' if mode=='q42' else 'k'
161         best=min((s for s in candidates if s['cash_cost_yuan']<=minimum+abs(minimum)*1e-4),key=lambda s:s[key])
162         choices[mode]={'gamma':gamma,key:best[key],'validation_cash_yuan':best['cash_cost_yuan'],
163                       'validation_initial_kwh':initial,'formal_initial_kwh':warms[mode]['terminal_kwh']}
164     choices['validation']=validation
165     choices['q42_main']='q42_selected'
166     choices['q43_main']='q43_common' if choices['q43']['k']==1 else f'q43_lookahead_k{choices["q43"]["k"]}'
167     write_json(folder/'selection.json',choices)
168     return choices
169
170 def definitions(selection):
171     g2,g3=selection['q42']['gamma'],selection['q43']['gamma']
172     return {
173         'q42_point':dict(mode='q42',gamma=g2,point=True,rho=0.),
174         'q42_pair':dict(mode='q42',gamma=g2,rho=1.),
175         'q42_independent':dict(mode='q42',gamma=g2,rho=0.),
176         'q42_selected':dict(mode='q42',gamma=g2,rho=selection['q42']['rho']),
177         'q42_gamma0':dict(mode='q42',gamma=0.,rho=selection['q42']['rho']),
178         'q43_point':dict(mode='q43',gamma=g3,point=True),
179         'q43_no_price_correction':dict(mode='q43',gamma=g3,price_correction=False),
180         'q43_common':dict(mode='q43',gamma=g3),
181         'q43_lookahead_k2':dict(mode='q43',gamma=g3,k=2,lookahead=True),
182         'q43_lookahead_k3':dict(mode='q43',gamma=g3,k=3,lookahead=True),

```

```

181         'q43_gamma0':dict(mode='q43',gamma=0.,k=selection['q43']['k'],lookahead=selection['q43']['k']>1),
182     }
183
184
185 def main():
186     parser=argparse.ArgumentParser();parser.add_argument('--run',default='full_q4_20260913')
187     parser.add_argument('--selection-only',action='store_true');parser.add_argument('--experiment')
188     parser.add_argument('--pilot',action='store_true');args=parser.parse_args()
189     folder=ROOT/'outputs'/args.run;folder.mkdir(exist_ok=True)
190     ds,prices,official=read_inputs(ROOT/'data/processed')
191     manifest={'inputs':ds.sources,'model':hashlib.sha256((ROOT/'model/model-q4.tex').read_bytes()).hexdigest(),
192              'numeric_code':{n:hashlib.sha256((ROOT/'src'/n).read_bytes()).hexdigest() for n in
193                  ↪ ('q4_dispatch.py','q4_forecasting.py','run_q4.py')},
194              'pilot':args.pilot,'risk_weight':0,'settlement':'delivery_real_price_original_commitment_once',
195              'battery':asdict(Battery()),'residual_window':28,'residual_first_day_index':7,
196              'gamma_candidates':[0,.25,.5,1.],'rho_candidates':[0,.25,.5,.75,1.],'K_candidates':[1,2,3]}
197     if (folder/'manifest.json').exists() and json.loads((folder/'manifest.json').read_text('utf-8'))!=manifest:
198         raise ValueError('Run source or inputs changed; start a new run name')
199     write_json(folder/'manifest.json',manifest)
200     selection=json.loads((folder/'selection.json').read_text('utf-8')) if (folder/'selection.json').exists() else
201     ↪ select_parameters(ds,prices,folder)
202     if args.selection_only:return
203     cache=load_cache('formal');experiments=definitions(selection)
204     selected=[args.experiment] if args.experiment else list(experiments)
205     for name in selected:
206         spec=experiments[name];mode=spec['mode']
207         experiment(ds,prices,cache,folder/name,range(31,34 if args.pilot else 365),
208                 selection[mode]['formal_initial_kwh'],terminal=not args.pilot,**spec)
209     print('COMPLETE',args.run,', '.join(selected),flush=True)
210
211 if __name__=='__main__':main()

```

run_all_q4.py

```

1 """Bounded independent processes; each experiment has its own output directory."""
2 from concurrent.futures import ThreadPoolExecutor,as_completed
3 from pathlib import Path
4 import argparse,json,os,subprocess,sys
5 from run_q4 import definitions
6
7 def main():
8     p=argparse.ArgumentParser();p.add_argument('--run',default='full_q4_20260913');p.add_argument('--workers',type=
9     ↪ int,default=3)
10    a=p.parse_args();root=Path(__file__).resolve().parents[1];run=root/'outputs'/a.run
11    subprocess.run([sys.executable,str(root/'src/run_q4.py'),'--run',a.run,'--selection-only'],check=True)
12    selection=json.loads((run/'selection.json').read_text('utf-8'))
13    env=os.environ.copy()
14    for key in ('OPENBLAS_NUM_THREADS','OMP_NUM_THREADS','MKL_NUM_THREADS'):env[key]='1'
15    env['PYTHONIOENCODING']='utf-8';env['PYTHONDONTWRITEBYTECODE']='1'
16    def execute(name):
17        with (run/(name+'.log')).open('w',encoding='utf-8') as f:
18            subprocess.run([sys.executable,str(root/'src/run_q4.py'),'--run',a.run,'--experiment',name],
19                            cwd=root,env=env,stdout=f,stderr=subprocess.STDOUT,check=True)
20        return name
21    with ThreadPoolExecutor(max_workers=a.workers) as pool:
22        for job in as_completed([pool.submit(execute,name) for name in definitions(selection)]):
23            print('FINISHED',job.result(),flush=True)
24
25 if __name__=='__main__':main()

```

plot_q4.py

```

1 """Publication figures from audited data; 450 dpi PNG, editable SVG and PDF."""
2 from pathlib import Path
3 import argparse, csv, json
4 from datetime import date
5 import numpy as np

```

```

6 import matplotlib.pyplot as plt
7 from matplotlib.backends.backend_pdf import PdfPages
8 from matplotlib.ticker import MaxNLocator
9 from scientific_style import apply_style,save_figure,step,time_axis
10 from q4_forecasting import read_inputs,scenarios
11 from run_q4 import ROOT,load_cache,write_json,write_csv
12 from structural_checks import DATES
13
14 C2='#0072B2';C3='#009E73';RED='#D55E00';PURPLE='#8C6BB1';GRAY='#63717D'
15 LABELS={'q42_point':'点预测','q42_pair':'成对场景 p=1','q42_independent':'独立边际 p=0 (主方案)',
16         'q42_gamma0':'主方案 y=0','q43_point':'点预测','q43_no_price_correction':'不修正日内电价',
17         'q43_common':'共同动作 K=1 (主方案)','q43_lookahead_k2':'分组前瞻 K=2',
18         'q43_lookahead_k3':'分组前瞻 K=3','q43_gamma0':'主方案 y=0'}
19
20 def main():
21     parser=argparse.ArgumentParser();parser.add_argument('--run',default='full_q4_20260913');arg=parser.parse_args()
22     run=ROOT/'outputs'/arg.run;out=run/'deliverables/论文用图';out.mkdir(parents=True,exist_ok=True)
23     selection=json.loads((run/'selection.json').read_text('utf-8'))
24     summaries={p.parent.name:json.loads(p.read_text('utf-8')) for p in run.glob('q4*/summary.json')}
25     daily={name:list(csv.DictReader((run/name/'daily.csv').open(encoding='utf-8-sig')) for name in summaries)
26           ds,prices,_=read_inputs(ROOT/'data/processed');cache=load_cache('formal');qa=[];captions=[]
27     arrays={}
28     for mode in ('q42','q43'):
29         with np.load(run/selection[mode+'_main']/'trajectory.npz') as z:arrays[mode]={k:z[k] for k in z.files}
30     months=np.array([d.month for d in ds.dates[31:]]);x=np.arange(2,13)
31     def values(name,key):return np.array([float(r[key]) for r in daily[name]])
32     def axstyle(ax,title,ylabel=None):
33         ax.set_title(title,loc='left',pad=10);ax.grid(axis='y',color='#DDE3E8',lw=.5,zorder=0)
34         ax.set_axisbelow(True)
35         if ylabel:ax.set_ylabel(ylabel)
36     def header(fig,title,subtitle):
37         fig.text(.09,.974,title,fontsize=13,fontweight='bold',va='top')
38         fig.text(.09,.974-23/(fig.get_figheight()*72),subtitle,fontsize=8,color=GRAY,va='top')
39     def save(fig,name,caption,pdf):
40         qa.append(save_figure(fig,out,name,pdf=pdf));captions.append((name,caption))
41     apply_style()
42     with PdfPages(out/'第四问图集.pdf') as pdf:
43         # 1. Price information, units and uncertainty.
44         fig,axs=plt.subplots(1,2,figsize=(8.1,3.8));fig.subplots_adjust(left=.09,right=.975,bottom=.18,top=.78,wspace
45         ↪ ce=.3)
46         header(fig,'电价预测与信息更新','2025 年 2—12 月评价: 右侧按已执行时段汇总, 不重复计算滚动预测的重叠部分。')
47         day=[str(d) for d in ds.dates].index('2025-06-21');row=day-31
48         step(axs[0],prices[day],color='#242C33',label='实际电价',linewidth=1.)
49         step(axs[0],arrays['q42']['forecast_price'][row],color=C2,linestyle='--',label='零点场景均价')
50         step(axs[0],arrays['q43']['forecast_price'][row],color=C3,label='滚动场景均价')
51         lo=[];hi=[]
52         for issue in range(4):
53             _,p,prob,_=scenarios(ds,prices,cache,day,issue,mode='q43')
54             lo.extend(np.quantile(p[:, :36],.1,axis=0,method='inverted_cdf'));hi.extend(np.quantile(p[:, :36],.9,axis
55             ↪ =0,method='inverted_cdf'))
56         axs[0].fill_between(np.arange(145)/6,np.r_[lo,lo[-1]],np.r_[hi,hi[-1]],step='post',color=C3,alpha=.12,label
57         ↪ ='滚动场景80%区间')
58         for h in (6,12,18):axs[0].axvline(h,color=GRAY,lw=.6,ls=':')
59         time_axis(axs[0]);axstyle(axs[0],a 6月21日: 日内更新','电价 (元/kWh)');axs[0].legend(fontsize=6.8,loc='upper
60         ↪ left')
61         for mode,color,label in [('q42',C2,'零点预测'),('q43',C3,'滚动预测')]:
62             error=np.abs(arrays[mode]['forecast_price']-prices[31:]).mean(axis=1)
63             axs[1].plot(x,[error[months==m].mean() for m in x],marker='o' if mode=='q42' else
64             ↪ 's',ms=3,color=color,label=label)
65         axstyle(axs[1],b 各月已执行时段预测误差','电价 MAE (元/kWh)
66         ↪ ');axs[1].set_xticks(x);axs[1].set_xlabel('月份');axs[1].legend()
67         save(fig,'01_电价预测与信息更新','实线为实际价格与滚动场景均价, 虚线为零点场景均价; 淡绿色为历史残差场景的10%—90%区间,
68         ↪ 不能解释为已校准的概率保证。',pdf)
69         # 2. Annual total bills, zero-baseline bars and exact stacked components.
70         fig,axs=plt.subplots(1,2,figsize=(9.0,4.4));fig.subplots_adjust(left=.23,right=.975,bottom=.16,top=.79,wspace
71         ↪ ce=1.0)
72         header(fig,'全年实际购电费对照','评价期为 2 月 1 日至 12 月 31 日; 1 月已冻结主方案, 全年对照结果不用于倒选参数。')
73         for ax,mode,names in [(axs[0],'Q4-2',[ 'q42_point','q42_pair','q42_independent','q42_gamma0']),
74                               (axs[1],'Q4-3',[ 'q43_point','q43_no_price_correction','q43_common','q43_lookahead_k2','
75                               ↪ q43_lookahead_k3','q43_gamma0'])]:
76             normal=np.array([summaries[n]['planned_cost_yuan']+summaries[n]['adjustment_cost_yuan'] for n in
77             ↪ names])/1e4
78             emerg=np.array([summaries[n]['emergency_cost_yuan'] for n in names])/1e4;y=np.arange(len(names))

```

```

69     ax.barh(y,normal,color=C2 if mode=='Q4-2' else C3,height=.6,label='正常结算')
70     ax.barh(y,emerg,left=normal,color=RED,alpha=.8,height=.6,label='紧急购电')
71     for j,v in enumerate(normal+emerg):ax.text(v+18,j,f'{v:.2f}',va='center',fontsize=7)
72     ax.set_yticks(y,[LABELS[n] for n in names],fontsize=7);ax.invert_yaxis();ax.set_xlim(0,1930)
73     axstyle(ax,mode);ax.grid(False);ax.set_xlabel('实际总费用 (万元)')
    ↪ ' ');ax.xaxis.set_major_locator(MaxNLocator(4));ax.legend(loc='lower
    ↪ left',bbox_to_anchor=(-.1,-.25),ncol=2,fontsize=7)
74 save(fig,'02_全年实际费用对照','所有条形从零开始。正常结算包含相对原计划的上调费用与下调退款,紧急购电单列;
    ↪ 预测目标中的终端惩罚不计入实际账单。',pdf)
75 # 3. Seasonal behavior and emergency amount.
76 fig,ax=plt.subplots(2,1,figsize=(7.5,5.6),sharex=True);fig.subplots_adjust(left=.12,right=.97,bottom=.13,t
    ↪ op=.83,hspace=.4)
77 header(fig,'月度费用与紧急购电','两条主方案均从 1 月 1 日 6000 kWh 开始热身,正式评价持续继承真实库存。')
78 monthly=[]
79 for mode,color,offset in [('q42',C2,-.18),('q43',C3,.18)]:
80     name=selection[mode+'_main']
81     for ax,key,scale in [(axs[0],'cash_cost_yuan',1e4),(axs[1],'emergency_kwh',1e3)]:
82         val=values(name,key);totals=np.array([val[months==m].sum()/scale for m in x])
83         ax.bar(x+offset,totals,width=.34,color=color,label='Q4-'+mode[-1])
84         for m in x:monthly.append({'mode':mode,'month':m,'cash_cost_yuan':float(values(name,'cash_cost_yuan')[m]
    ↪ onths==m).sum()),'emergency_kwh':float(values(name,'emergency_kwh')[months==m].sum())})
85 axstyle(axs[0],'a 每月实际购电费','费用 (万元)');axstyle(axs[1],'b 每月紧急购电量','电量 (MWh)')
86 axs[0].legend(ncol=2,loc='upper left');axs[1].set_xticks(x);axs[1].set_xlabel('月份')
87 write_csv(out/'月度图源数据.csv',monthly)
88 save(fig,'03_月度费用与紧急购电','费用与紧急电量分别使用独立纵轴:用于区分季节性购电规模和预测失配带来的补电需求。',pdf)
89 # 4. Frozen historical choices versus later out-of-sample evidence.
90 fig,ax=plt.subplots(2,2,figsize=(7.8,6.1));fig.subplots_adjust(left=.12,right=.97,bottom=.12,top=.82,hspac
    ↪ e=.55,wspace=.35)
91 header(fig,'参数选择与样本外检验','上排和左下仅使用1月21—31日账单;右下展示2—12月结果,不改变2月前已冻结的选择。')
92 for mode,color in [('q42',C2),('q43',C3)]:
93     rows=[r for r in selection['validation'] if r['name'].startswith('validate_'+mode+'_gamma')]
94     g=[r['gamma'] for r in rows];v=np.array([r['cash_cost_yuan'] for r in rows])/1e4
95     axs[0,0].plot(g,v,marker='o',ms=4,color=color,label='Q4-'+mode[-1])
96 axstyle(axs[0,0],'a 终端软惩罚','1 月验证费用 (万元)');axs[0,0].set_xlabel('γ (元/kWh)');axs[0,0].legend()
97 rr=[r for r in selection['validation'] if
    ↪ r['name'].startswith('validate_q42_rho')];v=np.array([r['cash_cost_yuan'] for r in rr]);xx=[r['rho'] for
    ↪ r in rr]
98 axs[0,1].plot(xx,v-v.min(),color=C2,marker='o',ms=4);axs[0,1].set_xticks(xx);axs[0,1].set_xlabel('依赖收缩参数
    ↪ ρ')
99 axstyle(axs[0,1],'b Q4-2: 历史选择 p=0','相对验证最低费用 (元)')
100 rr=[r for r in selection['validation'] if
    ↪ r['name'].startswith('validate_q43_k')];v=np.array([r['cash_cost_yuan'] for r in rr]);xx=[r['k'] for r
    ↪ in rr]
101 axs[1,0].bar(xx,v-v.min(),color=[C3,GRAY,GRAY],width=.55);axs[1,0].set_xticks(xx);axs[1,0].set_xlabel('候选
    ↪ 信息组数 K')
102 axstyle(axs[1,0],'c Q4-3: 历史选择 K=1','相对验证最低费用 (元)')
103 baseline=summaries['q43_common']['cash_cost_yuan'];diff=[summaries[n]['cash_cost_yuan']-baseline for n in
    ↪ ['q43_common','q43_lookahead_k2','q43_lookahead_k3']]
104 axs[1,1].bar([1,2,3],diff,color=[C3,PURPLE,PURPLE],width=.55);axs[1,1].axhline(0,color=GRAY,lw=.7)
105 axs[1,1].set_xticks([1,2,3]);axs[1,1].set_xlabel('候选信息组数 K');axstyle(axs[1,1],'d 全年分组前瞻对照','相对
    ↪ K=1 的费用变化 (元)')
106 save(fig,'04_参数选择与样本外检验','主方案由1月历史验证冻结为γ=0.5、Q4-2的p=0及Q4-3的K=1。全年K=2、3略有改善,
    ↪ 但不能据此回头更改年初参数。K表示请求组数,样本不足时按模型回退为1。',pdf)
107 # 5. A recorded nonanticipative branch plan, distinct from actual execution.
108 logs=json.loads((run/'q43_lookahead_k2/planning_log.json').read_text('utf-8'))
109 eligible=[r for r in logs if r['day']=='2025-06-21' and len(r['branch_nodes'])>2]
110 record=eligible[0] if eligible else next(r for r in logs if len(r['branch_nodes'])>2)
111 day=[str(d) for d in ds.dates].index(record['day']);hour=record['hour'];group=record['grouping'];nodes=reco
    ↪ rd['branch_nodes'];root=nodes[0]
112 fig,ax=plt.subplots(1,2,figsize=(8,4.1));fig.subplots_adjust(left=.12,right=.97,bottom=.19,top=.79,wspace=
    ↪ .33)
113 header(fig,'一次更新的分组前瞻结构','f' 示例: {record['day']} {hour:02d}:00: 实线为共同前缀,虚线为当时的条件尾部计划。")
114 sig=np.array(group['signal_scenarios']);colors=[C2,PURPLE]
115 for j,node in enumerate(nodes[1:]):
116     ids=np.array(node['ids']);axs[0].scatter(sig[ids,0]/1000,sig[ids,1],s=22,color=colors[j],alpha=.8,label
    ↪ =f'组{j+1},{len(ids)}个历史日')
117     hours=hour+np.arange(node['start'],node['stop']+1)/6
118     axs[1].plot(hours,np.array(node['S'])/1000,ls='--',color=colors[j],label=f'组{j+1} 条件计划')
119     axs[0].axhline(0,color=GRAY,lw=.5);axs[0].axvline(0,color=GRAY,lw=.5)
120 axstyle(axs[0],'a 过去整日样本配对的信息信号','电价对数误差信号');axs[0].set_xlabel('下一次光伏预报修订量 (MWh)
    ↪ ');axs[0].legend(fontsize=7)
121 axs[1].plot(hour+np.arange(len(root['S']))/6,np.array(root['S'])/1000,color='#242C33',lw=1.8,label='共同前缀
    ↪ ')

```

```

122     bridge=hour+6;axs[1].axvline(bridge,color=GRAY,lw=.8,ls=':');axs[1].scatter([bridge],[root['S'][-1]/1000],s_j
    ↪ =35,color='#242C33',zorder=5)
123     for value in (1.2,10.8):axs[1].axhline(value,color=GRAY,lw=.6,ls=':')
124     axs[1].set_ylim(.5,11.5);axs[1].set_xlim(hour,24);axs[1].set_xticks(np.arange(hour,25,6));axs[1].set_xlabel_j
    ↪ ('时刻(h)')
125     axstyle(axs[1], 'b 各尾部共享同一桥接库存','电池内部储量 (MWh)');axs[1].legend(fontsize=7,loc='best')
126     write_json(out/'前瞻示例图源数据.json',record)
127     save(fig,'05_分组信号与桥接库存','分组同时使用光伏预报修订和电价已结束时段误差信号。所有未来分支从同一共同前缀末库存出发:
    ↪ 只执行实践前缀,下一次更新时重新求解,虚线不是实际轨迹。示例优先选6月21日首个有效双分组时刻。',pdf)
128     # 6-7. Four required dates, exact intervals and separate energy/power units.
129     for number,mode,color in [(6,'q42',C2),(7,'q43',C3)]:
130         a=arrays[mode];fig,axs=plt.subplots(4,2,figsize=(7.8,9.5));fig.subplots_adjust(left=.10,right=.97,botto_j
    ↪ m=.08,top=.86,hspace=.60,wspace=.29)
131         header(fig,'Q4-'+mode[-1]+' : 题目指定四日调度','左列为交流母线侧平均功率,右列为电池内部储量:
    ↪ 所有曲线保留原始10分钟分辨率。')
132         handles=None
133         for j,daytext in enumerate(DATES):
134             day=[str(d) for d in ds.dates].index(daytext);ix=day-31;left,right=axs[j]
135             net=(ds.load_kw[day]-ds.pv_kw[day])/1000
136             step(left,net,color=GRAY,lw=1.,label='实际净负荷')
137             if mode=='q43':step(left,a['Q'][ix]*6/1000,color=C2,ls='--',lw=.9,label='零点原计划')
138             step(left,a['R'][ix]*6/1000,color=color,lw=1.2,label='最终正常购电')
139             left.fill_between(np.arange(145)/6,np.r_[a['E'][ix],a['E'][ix,-1]]*6/1000,step='post',color=RED,alp_j
    ↪ ha=.30,label='紧急购电')
140             left.axhline(0,color=GRAY,lw=.5);axstyle(left,daytext+' 购电','平均功率 (MW)')
141             right.plot(np.arange(145)/6,a['S'][ix]/1000,color=PURPLE,lw=1.3)
142             for value in (1.2,10.8):right.axhline(value,color=GRAY,lw=.6,ls=':')
143             if mode=='q43':
144                 for hour in (6,12,18):right.axvline(hour,color=GRAY,lw=.5,ls=':')
145             right.set_ylim(.5,11.5);right.set_yticks([1.2,6.,10.8]);axstyle(right,'库存轨迹','储量 (MWh)')
146             cash=values(selection[mode+'_main'],'cash_cost_yuan')[ix]
147             right.set_title(f'库存轨迹: 日总费 {cash/1e4:.3f} 万元',loc='left',pad=10,fontsize=8.5)
148             for ax in (left,right):time_axis(ax,label=j==3,ticks=6)
149             if j==0:handles=left.get_legend_handles_labels()
150             fig.legend(*handles,loc='upper left',bbox_to_anchor=(.09,.911),ncol=4,fontsize=7)
151             save(fig,f'{number:02d}_{mode.upper()}指定日期购电与储能','题目指定3月20日、6月21日、9月23日、12月21日。
    ↪ 左列电量除以1/6小时并换算成MW; 负净负荷表示光伏超过负荷。紧急补电与多余能量由实际净负荷结算,
    ↪ 不直接改变预先决定的电池路径。',pdf)
152     write_json(out/'图表校验.json',qa)
153     (out/'图注与使用顺序.md').write_text('# 第四问图表使用说明\n\n建议按照预测信息、实际账单、季节分解、参数选择、前瞻结构、
    ↪ 指定日期的顺序组织论文。PNG为450 dpi, SVG保留可编辑文字, PDF为同版图集.\n\n'+'\n\n'.join(f'## {name}\n\n{caption}'
    ↪ for name,caption in captions)+'\n',encoding='utf-8')
154     print('Created',len(qa),'figures',flush=True)
155
156 if __name__=='__main__':main()

```

report_q4.py

```

1  """Source-grounded final report, model mapping and reproducibility metadata."""
2  import argparse, csv, json, platform
3  import numpy as np
4  import scipy, sklearn, statsmodels, matplotlib, openpyxl
5  from run_q4 import ROOT, write_json, write_csv
6
7  def main():
8      p=argparse.ArgumentParser();p.add_argument('--run', default='full_q4_20260913');a=p.parse_args()
9      run=ROOT/'outputs'/a.run;out=run/'deliverables'
10     selection=json.loads((run/'selection.json').read_text('utf-8'))
11     price=json.loads((ROOT/'outputs/prepared/price_selection.json').read_text('utf-8'))
12     audit=json.loads((run/'audit.json').read_text('utf-8'));struct=json.loads((run/'structural_checks.json').read_t_j
    ↪ ext('utf-8'))
13     excel=json.loads((run/'workbook_verification.json').read_text('utf-8'))
14     summaries={p.parent.name:json.loads(p.read_text('utf-8')) for p in run.glob('q4*/summary.json')}
15     s2=summaries[selection['q42_main']];s3=summaries[selection['q43_main']]
16     names=['q42_point', 'q42_pair', 'q42_independent', 'q42_selected', 'q42_gamma0', 'q43_point', 'q43_no_price_correctio_j
    ↪ n', 'q43_common', 'q43_lookahead_k2', 'q43_lookahead_k3', 'q43_gamma0']
17     labels=['Q4-2 点预测', 'Q4-2 成对场景 p=1', 'Q4-2 独立边际 p=0', 'Q4-2 历史选择 (主)', 'Q4-2 主方案 y=0',
18             'Q4-3 点预测', 'Q4-3 不修正日内电价', 'Q4-3 共同动作 K=1 (主)', 'Q4-3 分组前瞻 K=2', 'Q4-3 分组前瞻 K=3', 'Q4-3
    ↪ 主方案 y=0']
19     totals=[]

```

```

20 for name,label in zip(names,labels):
21     s=summaries[name]
22     totals.append({'name':name,'label':label,**{k:s[k] for k in
    ↳ ('cash_cost_yuan','planned_cost_yuan','adjustment_cost_yuan','emergency_cost_yuan','emergency_kwh','unu
    ↳ sed_kwh','charge_kwh','discharge_kwh','up_kwh','down_kwh','price_mae','price_rmse','price_crps','price_
    ↳ coverage80','optimization_count','solver_runtime_s'}),
23     'normal_cost_yuan':s['planned_cost_yuan']+s['adjustment_cost_yuan']})
24 write_csv(out/'全年方案汇总.csv',totals)
25 # Descriptive uncertainty of paired daily bill differences. Does not affect any decisions.
26 pairs=[('q42_pair','q42_selected'),('q43_common','q43_lookahead_k2'),('q43_common','q43_lookahead_k3'),('q43_no
    ↳ _price_correction','q43_common')]
27 rng=np.random.default_rng(20260913);comparisons=[]
28 for old,new in pairs:
29     def cash(name):return np.array([float(r['cash_cost_yuan']) for r in
    ↳ csv.DictReader((run/name/'daily.csv').open(encoding='utf-8-sig'))])
30     delta=cash(old)-cash(new);n=len(delta);starts=rng.integers(0,n,size=(2000,(n+6)//7))
31     ids=((starts[:,None]+np.arange(7))%n).reshape(2000,-1)[:,:n]
32     boot=delta[ids].sum(axis=1)
33     comparisons.append({'baseline':old,'candidate':new,'saving_yuan':float(delta.sum()),'saving_pct':float(100*
    ↳ delta.sum()/cash(old).sum()),
34     'positive_saving_days':int((delta>1e-5).sum()),'negative_saving_days':int((delta < -1e-5).sum()),
35     'circular_block_days':7,'bootstrap_draws':2000,'seed':20260913,
36     'descriptive_95_interval_yuan':np.quantile(boot,[.025,.975]).tolist())}
37 write_json(run/'paired_comparisons.json',comparisons)
38 environment={'python':platform.python_version(),**{m.__name__:m.__version__ for m in
    ↳ (np,scipy,sklearn,statsmodels,matplotlib,openpyxl)},
39     'numerical_threads':1,'random_forest_seed':20260912,'grouping_seed':20260913}
40 write_json(run/'environment.json',environment)
41 lines=['# 第四问计算结果与合理性复核','
42     '模型依据为建模手提供的修订版 `model-q4.tex`、配套PDF、修订说明及假设清单。核心变量、结算公式、信息边界、
    ↳ 依赖收缩和条件前瞻结构均按该版本实现。',
43     'f"主结果: Q4-2为 **{s2['cash_cost_yuan']:.2f}元**, Q4-3为 **{s3['cash_cost_yuan']:.2f}元**.'
    ↳ 评价期为2025年2月1日至12月31日,共334天。两个表分别对应 `result4-2.xlsx` 与 `result4-3.xlsx`。",'',
44     '## 1. 主方案与实际账单','
45     '| 指标 | Q4-2|Q4-3|'+'|---|---|'+'|']
46 for label,key in [('原计划购电费(元)','planned_cost_yuan'),('调整净费用(元)','adjustment_cost_yuan'),('紧急购电费
    ↳ (元)','emergency_cost_yuan'),('实际总费(元)','cash_cost_yuan'),('紧急购电量(kWh)','emergency_kwh'),('多余电量
    ↳ (kWh)','unused_kwh'),('充电量(kWh)','charge_kwh'),('放电量(kWh)','discharge_kwh'),('上调量(kWh)
    ↳ ','up_kwh'),('下调量(kWh)','down_kwh')]:
47     lines.append(f'|{label}|{s2[key]:.2f}|{s3[key]:.2f}|')
48 lines.extend(['f"Q4-3主方案比Q4-2主方案少支出 **{s2['cash_cost_yuan']-s3['cash_cost_yuan']:.2f}元
    ↳ ({100*(1-s3['cash_cost_yuan']/s2['cash_cost_yuan']):.3f}%) **。两者同时存在光伏信息与调整权限差异,
    ↳ 因此该差額は方案整体比较,不能单独归因于控制频率。",'',
49     'Q4-2逐时实际费用为 `p_real * (Q + 5E)`。Q4-3逐时实际费用为 `p_real * [Q + 1.5(R-Q)正部 - 0.5(Q-R)正部 + 5E]`。
    ↳ 日内每次更新都保留当天零点Q; 每个最终执行时段只结算一次。负的调整净费用表示下调退款超过上调费用,最终正常购电成本仍非负。',
50     '','## 2. 实现选择与因果性','
51     '- 时间与单位: 每天144个10分钟区间, 145个库存时刻。附件标签00:10解释为00:00—00:10的终点;模板时段文字按这一约定纠正,
    ↳ 原始数据顺序不移动。交流母线侧功率除以6得到kWh。',
52     '- 电池: 充、放效率各 0.9, 库存 1200—10800 kWh, 交流侧充、放电功率各不超过 5000 kW。同一 10 分钟时段不同时充放电。',
53     '- 冷启动: 1月1日库存6000 kWh, 采用附件1典型曲线; 负荷和历史光伏预测在1月热身阶段使用预先固定的P1加权历史方法,
    ↳ 电价采用7日对数加权均值且不作日内修正。Q4-3可使用当时已发布的附件3预报。Q4-2热身p=0、Q4-3热身p=1, γ均为0.5。',
54     '- 残差档案: 跳过最初7日的短历史残差,从1月8日对应的预测误差开始积累。无已完成残差时用确定性场景,此后最多枚举最近28个历史日,
    ↳ 等概率使用全部独立日期。首次单残差场景出现在1月9日;这不是“至少有7个残差才启用”。',
55     '- 正式负荷预测继承第二问P3随机森林: 64棵树、最大深度10、叶节点最小样本8、最多28个训练目标日,种子20260912;
    ↳ 滞后特征最多追溯35日原始观测。每天只用此已结束的数据重新训练。Q4-2历史光伏同法预测,
    ↳ Q4-3使用当前已发布的官方整节点作分段线性积分,发布点锚定最新可见实测光伏。',
56     f"- 电价: 1月21—31日比较前日、7/14/28日均值和加权曲线,选中 **{price['method']}**; 按剩余时段MAE选择日内修正
    ↳ **α={price['alpha']}、衰减率={price['decay_per_hour']} h-1**。日内修正只吸收已结束的10分钟价格,
    ↳ 零点基准冻结至当天结束。价格场景通过对数残差指数变换得到,严格为正。优化中的均价为场景均价。',
57     '- 场景: 同一个已结束历史日期的净负荷残差与价格对数残差配对。净负荷允许为负,不另行截零。Q4-2主信息口径不使用附件3官方光伏预报。
    ↳ ',
58     '- 参数验证: 1月21—31日逐日回放,初始库存取统一热身轨迹1月20日末值;各候选在验证段末统一要求库存不低于6000 kWh。
    ↳ 先比较γ∈{0,0.25,0.5,1},再比较p或K。p/K费用相差不超过最低费的0.01%时取较小值。2月1日前冻结参数,不用2—12月结果调参。',
59     f"- 历史选择结果: Q4-2 **γ={selection['q42']['gamma']}、p={selection['q42']['rho']}**; Q4-3
    ↳ **γ={selection['q43']['gamma']}、K={selection['q43']['k']}**。同一题内各正式对照继承相同预先规定的1月热身库存;
    ↳ 2月1日两题实际衔接值均为6000 kWh,来自热身结果,并未手动重置。',
60     '- 前瞻实现: 在0/6/12点只展开下一次更新,共同前缀36步,未来组内动作共享,各分支必须从相同桥接库存出发。
    ↳ 信息信号严格使用下一次光伏预报修订量和更新前一小时价格对数误差。K-means最多3组,标准差归一化,至少每组5个不同历史日,
    ↳ 样本不足回退K=1。18点直接求解剩余时段。',
61     '- 终端: 常规日使用γ·max(6000-S末,0); 12月31日所有未来分支要求S末≥6000。没有强制每日回到6000。终端软惩罚用于决策,
    ↳ 不计入实际支付费用。',
62     '','## 3. 全年对照','| 方案 | 实际总费(元) | 正常结算(元) | 紧急费(元) |','|---|---|---|---|']

```

```

63     for row in totals:lines.append(f"|{row['label']}|{|row['cash_cost_yuan']:.2f}|{|row['normal_cost_yuan']:.2f}|{|x}
    ↳ ow['emergency_cost_yuan']:.2f}|")
64     lines.extend(['','f"相对各自点预测对照, Q4-2主方案节约
    ↳ **{100*(1-s2['cash_cost_yuan']/summaries['q42_point']['cash_cost_yuan']):.3f}%**, Q4-3主方案节约
    ↳ **{100*(1-s3['cash_cost_yuan']/summaries['q43_point']['cash_cost_yuan']):.3f}%**.
    ↳ 保留残差场景主要减少了高价紧急补电。",'',
65     f"Q4-2选择p=0, 比p=1全年少支出 {summaries['q42_pair']['cash_cost_yuan']-s2['cash_cost_yuan']:.2f}元, 幅度约
    ↳ {100*(1-s2['cash_cost_yuan']/summaries['q42_pair']['cash_cost_yuan']):.3f}%.
    ↳ p=0只表示在决策目标中对小样本价值依赖作完全收缩, 不能据此断言真实价格与净负荷独立。Q4-3仍保留成对价量场景p=1。",'',
66     f"Q4-3的K=2、3全年分别比K=1少支出
    ↳ {s3['cash_cost_yuan']-summaries['q43_lookahead_k2']['cash_cost_yuan']:.2f}元和
    ↳ {s3['cash_cost_yuan']-summaries['q43_lookahead_k3']['cash_cost_yuan']:.2f}元; 但1月验证均劣于K=1,
    ↳ 故提交表维持历史选出的K=1。该结果符合模型的收益边界: 前瞻能降低同信息、同初值下的优化目标, 但不保证任意样本本期实际账单降低。
    ↳ ','',
67     f"全年可分组的更新共有1002次(0/6/12点)。请求K=2时,
    ↳ 有{summaries['q43_lookahead_k2']['group_counts'].get('2',0)}次形成2组; 请求K=3时,
    ↳ 有{summaries['q43_lookahead_k3']['group_counts'].get('3',0)}次形成3组。其余回退为1;
    ↳ 18点334次天然没有下一次日内分叉, 不能计作分组失败。",'',
68     '按7日循环块、2000次重抽样描述成对日费用差的波动, 结果如下。该区间只描述当前单年样本对分块重抽样的敏感性, 不证明跨年份收益。','',
69     '| 基准 | 候选 | 实际节约(元) | 描述性 95% 区间(元) |','|---|---|---|')
70     for r in comparisons:
71         lo,hi=r['descriptive_95_interval_yuan'];lines.append(f"|{r['baseline']} →
    ↳ {r['candidate']}|{|r['saving_yuan']:.2f}|{|lo:.2f}, {hi:.2f}|")
72     lines.extend(['','## 4. 预测与数值复核','','| 指标 |Q4-2 主方案 |Q4-3 主方案 |','|---|---|---|')
73     for label,key in [('电价MAE(元/kWh)','price_mae'),('电价RMSE(元/kWh)','price_rmse'),('电价CRPS(元/kWh)
    ↳ ','price_crps'),('场景80%区间实际覆盖率','price_coverage80')]:
74         lines.append(f"|{label}|{|s2[key]:.6f}|{|s3[key]:.6f}|")
75     lines.extend(['','f"- 全部18项机制单元检查通过。独立审计共覆盖{audit['experiment_count']}组计算(含热身与验证)、
    ↳ {audit['days_checked']}个策略日、{audit['optimizations_checked']}次求解, 逐项重算原计划费、调整费、紧急费、场景目标、
    ↳ 分叉概率、能量平衡、互斥、库存边界与跨日衔接。最大绝对误差 **{audit['maximum_error']:.3e}**, 容差2×10-5。
    ↳ 不同检查分别采用其对应的元或kWh单位。",'',
76     f"- 在四个指定日期及各更新时刻完成{struct['lp_milp_cases']}组真实数据LP/MILP对照, 含分组模型; 最大目标差
    ↳ **{struct['maximum_lp_milp_error']:.3e}元**。同输入K=1与共同动作基准目标一致, K=2的优化目标不高于基准。",'',
77     '- 全年采用模型第13节给出的LP净化定理求解: 删除可同时出现的充放电循环, 严格保留库存路径, 使净购电需求不减。模型的正电价、
    ↳ 免费弃能、无限制补电等前提全部保持; 不是放松互斥后直接把不合物理的解交付。','
78     f"- 两份保存后的XLSX由独立只读程序检查, 共比对{sum(r['numeric_cells_checked'] for r in excel):.3e}个数值单元格, 最大差
    ↳ {max(r['maximum_cell_error'] for r in excel):.3e}; 逐日合计公式与全年实际总费均已核对。所有工作表均完成渲染检查。",'',
79     '','## 5. 为什么结果在模型下合理','','
80     '1. 储能守恒: 正式期两端库存相同, 全年放电量/充电量等于0.81, 与两次0.9效率一致。每个10分钟时段充放电互斥;
    ↳ 4小时汇总表中间一行同时出现充、放电数值, 表示该4小时内不同10分钟时段分别执行, 符合模型。','
81     f"2. 终端行为: Q4-2有{s2['end_at_min']}天、Q4-3有{s3['end_at_min']}天日未落在1200 kWh下界, 两者均无日未落在10800
    ↳ kWh上界的日期。日未状态可以变化; 12月31日均为6000 kWh, 未通过年末耗尽电池人为降低费用。",'',
82     '3. 购电功率可超过5 MW: 5000 kW约束作用于电池充放电, 而非无上限的正常购电通道。图中的购电尖峰与电池充电时段一致,
    ↳ 不能将其误判为越过电池功率限制。','
83     '4. 调度中的跳变来自10分钟电价与线性目标, 原模型没有额外的爬坡、启停或平滑成本。图保留真实阶梯,
    ↳ 未为了视觉效果更改策略或增加这些约束。','
84     '5. 多余能量 U 按原模型免费弃置, 同时包含光伏过剩和正常购电冗余; 不能直接把全年 U 称为纯弃光量。净负荷为负的时段正常保留。','
85     '6. 每次有限场景优化已达到求解器要求, 并不意味着已经得到未知未来条件下的全年全局最优策略。参数只做了一次1月验证冻结,
    ↳ 依赖收缩和信息分组的增益较小, 应使用上述如实对照表述。','
86     '','## 6. 文件使用与论文写法','','
87     '- `result4-2.xlsx`: “计划购电量”EQ 列为含紧急购电的日实际总费。','
88     '- `result4-3.xlsx`: “计划购电量”EQ列仅为原计划承诺费; “调整购电量”存最终正常购电R, EQ列为含紧急购电的日实际总费。
    ↳ 两页费用不能相加。','
89     '- `题目指定日期表格.md`: 四个指定日期的六个 10 分钟购电值、4 小时充放电量、首末库存与紧急购电区段。','
90     '- `q42/q43_10 分钟完整轨迹.csv`: 完整 Q、R、C、D、S、E、U 与实际价格, 供复核和重绘; `全年方案汇总.csv`收齐 11 组对照。','
91     '- `论文用图`: 7 张 450 dpi PNG、对应可编辑 SVG 与 7 页 PDF 图集, 另含图注、图源数据与排版校验记录。','
92     '- `模型与代码逐项对应.md`: 每个数学定义对应的函数和检查。求解日志、历史选参、源文件摘要与独立复核记录保存在计算工程中。','
93     '','可用于论文的结果段: ','',
94     f"> 基于价格—净负荷成对残差, 本文构建带小样本依赖收缩的日计划模型与前瞻下一次信息更新的条件调整模型。参数经1月历史验证后固定,
    ↳ 对2—12月334天进行连续回放。
    ↳ Q4-2与Q4-3主方案实际购电总费分别为{s2['cash_cost_yuan']/1e4:.4f}万元和{s3['cash_cost_yuan']/1e4:.4f}万元,
    ↳ 较各自点预测对照下降{100*(1-s2['cash_cost_yuan']/summaries['q42_point']['cash_cost_yuan']):.3f}%和{100*(1-
    ↳ s3['cash_cost_yuan']/summaries['q43_point']['cash_cost_yuan']):.3f}%。依赖收缩在历史验证中选择p=0;
    ↳ 分组前瞻在全年对照中有小幅收益, 但历史验证选择K=1, 故提交结果保留较简模型。能量平衡、跨日库存、
    ↳ 调整结算及分组非预见性均通过独立重算, LP净化与物理MILP在指定数据案例上保持目标一致。",'',
95     'CVar、额外负荷日内校正、锁定远期电价、Q4-2官方光伏预报扩展、全信息下界及下调额外罚金均未加入主结果。
    ↳ 代码保留下调额外罚金的结算入口, 但本次未声称已完成该敏感性回溯。']])
96     (out/'第四问结果与合理性复核.md').write_text('\n'.join(lines)+'\n',encoding='utf-8')
97     mapping='''# 模型与代码逐项对应
98
99 依据为随包保存的建模手修订版 model-q4.tex; 下表的章节和公式标签均来自该文档。Q4-2 与 Q4-3 共用物理层, 不另建额外模型。
100

```

101 | 模型位置 | 本次实现 | 核对证据 |

102 | ---|---|---|

103 | 第2节信息集|q4_forecasting.price_predictions、scenarios只使用已结束历史日、当前已结束价格前缀及当前官方发布；
 ↳ run_q4.experiment顺时执行|两项未来数据扰动单元检查；planning_log中的history_days与更新时间|

104 | 第3节 eq:bill2、eq:b、eq:bill3|q4_dispatch.bill；audit_q4 独立重写实际结算 | 每天与全年费用逐项重算；Q/R 原计划不变 |

105 | 第4节 eq:balance、eq:recourse| $e=\max(N+c-d-r,0)$ ， $u=\max(r+d-c-N,0)$ |audit_q4 能量平衡与补救定义 |

106 | 第4节 eq:stock、eq:capacity、eq:mode|Battery与solve_dispatch: 0.9双效率、1200—10800、5000/6、互斥|全轨迹与全部条件分支检查|

107 | 第4节 eq:terminal|gamma 正部软惩罚；年末每条分支 ≥ 6000 ；真实库存跨日传递 |audit_q4 跨日与末端检查 |

108 | 第5节 eq:correction|price_predictions: 零点均值基线，已结束对数误差递推，按小时指数衰减|测试当前/未来价格扰动不进入当前预测；
 ↳ price_selection.json|

109 | 第6节成对残差|scenarios 按同一历史日配对净负荷残差与价格对数残差，至多 28 天 | 配对检查、负净负荷保留检查 |

110 | 第7—8节共同动作基准|solve_dispatch(groups=None)，Q4-2固定Q，Q4-3日内固定零点Q调整R|run_q4的q42_pair与q43_common|

111 | 第9节 eq:H、eq:weights| 适时场景紧急费用的等价线性上图形式，没有加入跨时场景动作 | 场景路径重排不改变共同动作目标的单元检查 |

112 | 第10节 eq:shrink-H、eq:q42-new|recourse_weights= $\pi[pp+(1-p)$ 均价]；正常购电仍按均价；只有Q4-2可用 $p=1$ | $p=0$ 、 $p=1$ 、
 ↳ $p=0.25$ 笛卡尔混合等价检查|

113 | 第11节信息信号与分组|scenarios中的ZG/Zp与information_groups；下一次预报由当前预报+历史修订形成并按模型截断非负|未来预报扰动检查；
 ↳ 每组 ≥ 5 不同历史日；冻结分组映射记录|

114 | 第11节 eq:lookahead-main|共同36步前缀+每组一个剩余动作尾部；零点联合选择原计划Q|solve_dispatch分支节点及期望目标独立重算|

115 | 第11节 eq:branch-bridge|每个尾部初始库存与共同前缀末库存设等式|全部分支bridge检查；真实案例K=1退化与K=2可行集包含检查|

116 | 第11节执行与收益边界|仅执行根节点前36步；真实发布后重新求解；旧分支仅作参考计划档案|run_q4记录executed_terminal_stock，
 ↳ audit_q4逐项比对执行前缀|

117 | 第12节 eq:q80、eq:band| 仅用于解释和单元检查，不用分位数公式替代储能优化 | 价格加权 80% 与 70%—90% 不调整区间测试 |

118 | 第13节LP净化证明|purify按 $\varepsilon=\min(c,d/(\eta c \eta d))$ 消除同时充放电；
 ↳ method=milp可直接启用二进制物理模型|18项单元检查及32组真实数据LP/MILP对照|

119 | 第16节对照与验收|run_q4.select_parameters、definitions；audit_q4；structural_checks|11组全年对照、18组热身/验证、
 ↳ 完整实际账单和复核记录|

120

121 ## 实现层选择

122

123 模型允许少量参数由历史验证确定。本次候选网格、冷启动规则、验证日期、回退规则及终端处理均在《第四问结果与合理性复核》第2节列明。
 ↳ 没有把未定义的风险项、平滑成本或未来真实信息加到模型中。

124

125 forecasting.py、forecasting_v2.py为前问的兼容预测模块，Q4仅调用数据读取、时间标签、P1冷启动及P3随机森林。
 ↳ q3_forecasting.py仅调用read_official和integrate_hourly两个函数，未调用第三问的指数尺度场景模型。
 ↳ 第四问的场景分布与优化全部在q4_forecasting.py及q4_dispatch.py中定义。

126 '''

127 (out/'模型与代码逐项对应.md').write_text(mapping,encoding='utf-8')

128 print('Wrote result report, model mapping and numerical environment',flush=True)

129

130 if __name__=='__main__':main()

prepare_deliverables.py

```
1 """Tables and template payloads derived only from audited executed trajectories."""
2 import argparse, csv, json
3 import numpy as np
4 from forecasting import interval_label, clock_text
5 from q4_forecasting import read_inputs
6 from run_q4 import ROOT, write_json, write_csv
7 from structural_checks import DATES
8
9 def segments(e):
10     mask=e>1e-6; rows=[]; t=0
11     while t<144:
12         if not mask[t]:t+=1; continue
13         begin=t
14         while t<144 and mask[t]:t+=1
15         rows.append([clock_text(10*begin)+'-'+clock_text(10*t), float(e[begin:t].sum())])
16     return rows, float(e[~mask].sum())
17
18 def prepare(run):
19     selection=json.loads((run/'selection.json').read_text('utf-8')); audit=json.loads((run/'audit.json').read_text('
  ↳ utf-8'))
20     audited={r['experiment']:r for r in audit['experiments']}
21     ds, prices, _=read_inputs(ROOT/'data/processed'); out=run/'deliverables'; out.mkdir(exist_ok=True)
22     selected=[]
23     for mode in ('q42', 'q43'):
24         name=selection[mode+'__main']; assert name in audited
25         folder=run/name
26         with np.load(folder/'trajectory.npz') as z:a={k:z[k] for k in z.files}
27         assert a['day_indices'].tolist()==list(range(31, 365))
```

```

28 daily=list(csv.DictReader((folder/'daily.csv').open(encoding='utf-8-sig')))
29 payload={'headers': ['日期\\时间', *[interval_label(t) for t in range(144)], '全天购电量', '全天购电费'],
30          'plan': [], 'adjusted': [], 'battery': [], 'emergency': [], 'mode': mode, 'experiment': name}
31 timeline=[];omitted=0.
32 for i, day in enumerate(a['day_indices']):
33     date=str(ds.dates[day]);q,r,c,d,e,u,s=[a[k][i] for k in ('Q', 'R', 'C', 'D', 'E', 'U', 'S')];p=prices[day]
34     cash=float(daily[i]['cash_cost_yuan']);plan=float(p@q)
35     payload['plan'].append([date,*q.tolist(),float(q.sum()),cash if mode=='q42' else plan])
36     payload['adjusted'].append([date,*r.tolist(),float(r.sum()),cash])
37     battery=[[clock_text(240*j)+'-'+clock_text(240*(j+1)),float(c[j*24:(j+1)*24].sum()),
38              float(d[j*24:(j+1)*24].sum()),'00:00' if j==0 else '24:00' if j==1 else None,
39              float(s[0]) if j==0 else float(s[-1]) if j==1 else None] for j in range(6)]
40     payload['battery'].extend([[date if j==0 else None,*row] for j,row in enumerate(battery)])
41     seg,small=segments(e);omitted+=small
42     payload['emergency'].extend([[date if j==0 else None,*row] for j,row in enumerate(seg or [['无',0.]])])
43     if date in DATES:
44         selected.append({'mode':mode,**{k:v if k=='date' else float(v) for k,v in daily[i].items()},
45                          'purchases':[[interval_label(6*h),float(q[6*h]),float(r[6*h])] for h in (10,12,14,16,18,20)],
46                          'battery':battery, 'emergency_segments':seg})
47     for t in range(144):
48         normal=float(p[t]*(q[t]+1.5*max(r[t]-q[t],0)-.5*max(q[t]-r[t],0))) if mode=='q43' else
49         ↪ float(p[t]*q[t])
50         timeline.append({'date':date, 'interval':interval_label(t), 'load_kw':float(ds.load_kw[day,t]),
51                          'pv_kw':float(ds.pv_kw[day,t]), 'price_yuan_per_kwh':float(p[t]),
52                          **{k+'_kwh':float(a[k][i,t]) for k in ('Q', 'R', 'C', 'D', 'E', 'U')},
53                          'S_start_kwh':float(s[t]), 'S_end_kwh':float(s[t+1]), 'normal_cost_yuan':normal,
54                          'emergency_cost_yuan':float(5*p[t]*e[t]), 'cash_cost_yuan':normal+float(5*p[t]*e[t])})
55     assert omitted<1e-4
56     payload['emergency_omitted_kwh']=omitted
57     write_json(run/(mode+'_workbook_payload.json'),payload)
58     write_csv(out/(mode+'_ 每日账单.csv'),daily)
59     write_csv(out/(mode+'_10 分钟完整轨迹.csv'),timeline)
60 write_json(out/'指定日期结果.json',selected)
61 text=['# 第四问题指定日期表格',' ','Q 为零点原计划, R 为最终正常购电量; R 不是调整增量。电量单位 kWh, 费用单位元.','']
62 for row in selected:
63     text.extend([f"## {row['mode']}.upper() • {row['date']}"',','|时间段|零点计划 Q|最终正常购电
64     ↪ R|','|---|---|---|'])
65     text.extend(f'|t|{q:.2f}|{r:.2f}|' for t,q,r in row['purchases'])
66     text.extend([' ',f"全天实际总费 **{row['cash_cost_yuan']:.2f}**; 原计划费 {row['planned_cost_yuan']:.2f},
67     ↪ 调整净费用 {row['adjustment_cost_yuan']:.2f}, 紧急费 {row['emergency_cost_yuan']:.2f}。",
68     f"计划购电 {row['planned_kwh']:.2f}; 最终正常购电 {row['executed_normal_kwh']:.2f}; 紧急购电
69     ↪ {row['emergency_kwh']:.2f}。",
70     f"0:00 库存 {row['initial_kwh']:.2f}; 24:00 库存 {row['terminal_kwh']:.2f}。",' ',
71     '| 时间段 | 充电量 | 放电量 |','|---|---|---|'])
72     text.extend(f'|b[0]|{b[1]:.2f}|{b[2]:.2f}|' for b in row['battery'])
73     text.extend([' ','| 紧急购电时间段 | 购电量 |','|---|---|'])
74     text.extend(f'|t|{e:.2f}|' for t,e in (row['emergency_segments'] or [['无',0.]));text.append('')
75 (out/'题目指定日期表格.md').write_text('\n'.join(text)+'\n',encoding='utf-8')
76 print('Prepared two workbook payloads and full trajectories',flush=True)
77
78 if __name__=='__main__':
79     p=argparse.ArgumentParser();p.add_argument('--run',default='full_q4_20260913');a=p.parse_args();prepare(ROOT/'o')
80     ↪ utputs/'a.run)

```

audit_q4.py

```

1 """Independent algebraic replay of stored decisions and all conditional plans.
2
3 Does not call optimizer validation or billing helpers. Scenario generators are
4 reused only to reconstruct the exact archived optimization information set.
5 """
6 from pathlib import Path
7 import argparse, csv, json, hashlib
8 import numpy as np
9 from q4_forecasting import read_inputs, scenarios
10 from run_q4 import ROOT, load_cache, write_json
11
12 TOL=2e-5
13
14 def audit_experiment(folder, ds, prices, cache):
15     config=json.loads((folder/'config.json').read_text('utf-8'))

```

```

16 summary=json.loads((folder/'summary.json').read_text('utf-8'))
17 with np.load(folder/'trajectory.npz') as z:a={k:z[k] for k in z.files}
18 daily=list(csv.DictReader((folder/'daily.csv').open(encoding='utf-8-sig'))))
19 logs=json.loads((folder/'planning_log.json').read_text('utf-8'))
20 days=a['day_indices'];mode=config['mode'];maxima={}
21 def check(key,error):
22     error=float(np.max(np.abs(error)))
23     maxima[key]=max(maxima.get(key,0.),error)
24     if error>TOL:raise AssertionError(f'{folder.name} {key}: {error}')
25 assert days.tolist()==config['days'] and len(daily)==len(days)
26 for key in ('Q','R','C','D','E','U'):assert a[key].shape==(len(days),144) and np.isfinite(a[key]).all()
27 assert a['S'].shape==(len(days),145) and np.isfinite(a['S']).all()
28 check('cross_day_stock',a['S'][1:,0]-a['S'][:-1,-1] if len(days)>1 else 0)
29 check('initial_stock',a['S'][0,0]-config['initial'])
30 check('stock_recurrence',np.diff(a['S'],axis=1)-.9*a['C']+a['D']/.9)
31 check('stock_bounds',np.maximum(np.maximum(1200-a['S'],a['S']-10800),0))
32 check('power_bounds',np.maximum(np.maximum(a['C'],a['D'])-5000/6,0))
33 check('nonnegative',np.maximum(-np.stack([a[k] for k in ('Q','R','C','D','E','U')]),0))
34 check('charge_discharge_exclusive',np.minimum(a['C'],a['D']))
35 if config['terminal']:check('year_end_stock',max(6000-a['S'][-1,-1],0))
36 net=(ds.load_kw[days]-ds.pv_kw[days])/6;p=prices[days]
37 emergency=np.maximum(net+a['C']-a['D']-a['R'],0)
38 unused=np.maximum(a['R']+a['D']-a['C']-net,0)
39 check('emergency_definition',a['E']-emergency);check('unused_definition',a['U']-unused)
40 check('power_balance',a['R']+a['E']+a['D']-a['C']-a['U']-net)
41 up=np.maximum(a['R']-a['Q'],0);down=np.maximum(a['Q']-a['R'],0)
42 if mode=='q42':check('fixed_daily_commitment',a['R']-a['Q'])
43 else:check('midnight_executed_commitment',a['R'][:,36]-a['Q'][:,36])
44 adjustment=np.zeros_like(p) if mode=='q42' else p*(1.5*up+(-.5 if config['settlement']=='refund' else .5)*down)
45 costs={'planned_cost_yuan':(p*a['Q']).sum(axis=1),'adjustment_cost_yuan':adjustment.sum(axis=1),
46        'emergency_cost_yuan':(5*p*emergency).sum(axis=1)}
47 costs['cash_cost_yuan']=sum(costs.values())
48 for key,v in costs.items():
49     check('daily_'+key,v-np.array([float(r[key]) for r in daily]))
50     check('total_'+key,v.sum()-summary[key])
51 for key,values in [('emergency_kwh',emergency),('unused_kwh',unused),('charge_kwh',a['C']),('discharge_kwh',a['D']),('up_kwh',up),('down_kwh',down)]:
52     check('total_'+key,values.sum()-summary[key])
53 expected_issues=1 if mode=='q42' else 4
54 assert len(logs)==expected_issues*len(days)
55 branch_count=0;scenario_points=0
56 for j,log in enumerate(logs):
57     row=j//expected_issues;issue=j%expected_issues;day=int(days[row]);t=36*issue;n=144-t
58     count=144 if mode=='q42' else 36
59     assert log['day']==ds.dates[day].isoformat() and log['hour']==6*issue
60     assert all(h<day for h in log['history_days']) and len(set(log['history_days']))==len(log['history_days'])
61     assert log['price_prefix_end_index']==t and log['official_visible_issue']==(6*issue if mode=='q43' else
62     ↪ None)
63 nn,pp,prob,groups,meta=scenarios(ds,prices,cache,day,issue,mode=mode,point=config['point'],
64     price_correction=config['price_correction'],groups=config['k'] if config['lookahead'] else 1)
65 assert meta['history_days']==log['history_days'] and meta['scenario_count']==log['scenario_count']
66 if 'grouping' in meta:assert meta['grouping']==log['grouping']
67 shape=nn.shape;rr=np.zeros(shape);cc=np.zeros(shape);dd=np.zeros(shape);assigned=np.zeros(shape,int)
68 branch_terminal=0;root=log['branch_nodes'][0]
69 check('recorded_initial',root['S'][0]-a['S'][row,t])
70 for key in ('R','C','D'):check('executed_'+key,np.asarray(root[key])[:count]-a[key][row,t:t+count])
71 check('executed_stock',np.asarray(root['S'])[:count+1]-a['S'][row,t:t+count+1])
72 for k,node in enumerate(log['branch_nodes']):
73     ids=np.array(node['ids'],int);b,e=node['start'],node['stop'];s=np.asarray(node['S'])
74     r,c,d=np.asarray(node[key]) for key in ('R','C','D')
75     assert len(s)==e-b+1 and len(r)==e-b
76     check('branch_stock_recurrence',np.diff(s)-.9*c+d/.9)
77     check('branch_stock_bounds',np.maximum(np.maximum(1200-s,s-10800),0))
78     check('branch_power_bounds',np.maximum(np.maximum(c,d)-5000/6,0))
79     check('branch_exclusivity',np.minimum(c,d))
80     check('branch_nonnegative',np.maximum(-np.r_[r,c,d],0))
81     check('branch_probability',prob[ids].sum()-node['prob'])
82     if k:
83         check('shared_bridge',s[0]-root['S'][-1])
84         assert groups is not None and len(set(groups[ids]).tolist())==1
85         assert len(ids)>=5 or len(set(groups.tolist()))==1
86         rr[ids,b:e]=r;cc[ids,b:e]=c;dd[ids,b:e]=d;assigned[ids,b:e]+=1
87     if e==n:

```

```

87         branch_terminal+=config['gamma']*prob[ids].sum()*max(6000-s[-1],0)
88         if config['terminal'] and row==len(days)-1:check('branch_final_minimum',max(6000-s[-1],0))
89         branch_count+=1
90         assert (assigned==1).all()
91         q=a['Q'][row,t:]
92         if mode=='q42':check('plan_fixed_q',rr-q)
93         normal=rr if mode=='q42' else q+1.5*np.maximum(rr-q,0)+(-.5 if config['settlement']=='refund' else
↪ .5)*np.maximum(q-rr,0)
94         weighted=prob[:,None]*(config['rho']*pp+(1-config['rho'])*(prob@pp))
95         expected=float((prob[:,None]*pp*normal).sum()+5*(weighted*np.maximum(nn+cc-dd-rr,0)).sum())
96         check('expected_cash_objective',expected-log['expected_cash'])
97         check('terminal_penalty_objective',branch_terminal-log['terminal_penalty'])
98         check('total_objective',expected+branch_terminal-log['objective'])
99         scenario_points+=nn.size
100     return {'experiment':folder.name,'days':len(days),'optimizations':len(logs),'conditional_nodes':branch_count,
101            'scenario_time_pairs':scenario_points,'tolerance':TOL,'maximum_errors':maxima,
102            'maximum_error':max(maxima.values()),'cash_cost_yuan':float(costs['cash_cost_yuan'].sum()),
103            'trajectory_sha256':hashlib.sha256((folder/'trajectory.npz').read_bytes()).hexdigest()}
104
105 def main():
106     parser=argparse.ArgumentParser();parser.add_argument('--run',default='full_q4_20260913');a=parser.parse_args()
107     run=ROOT/'outputs'/a.run;ds,prices,_=read_inputs(ROOT/'data/processed')
108     caches={name:load_cache(name) for name in ('formal','warmup')};results=[]
109     for folder in sorted(run.iterdir()):
110         if not (folder/'summary.json').is_file():continue
111         result=audit_experiment(folder,ds,prices,caches['warmup' if folder.name.startswith('warmup_') else
↪ 'formal'])
112         results.append(result);print(folder.name,result['maximum_error'],flush=True)
113     write_json(run/'audit.json',{'experiments':results,'experiment_count':len(results),
114        'days_checked':sum(r['days'] for r in results),'optimizations_checked':sum(r['optimizations'] for r in
↪ results),
115        'maximum_error':max(r['maximum_error'] for r in results),'tolerance':TOL})
116
117 if __name__=='__main__':main()

```

structural_checks.py

```

1  """Real-data LP/MILP equivalence and same-information lookahead containment."""
2  import argparse,json
3  import numpy as np
4  from q4_dispatch import solve_dispatch
5  from q4_forecasting import read_inputs,scenarios
6  from run_q4 import ROOT,load_cache,write_json
7
8  DATES=('2025-03-20','2025-06-21','2025-09-23','2025-12-21')
9
10 def main():
11     p=argparse.ArgumentParser();p.add_argument('--run',default='full_q4_20260913');arg=p.parse_args()
12     run=ROOT/'outputs'/arg.run;selection=json.loads((run/'selection.json').read_text('utf-8'))
13     ds,prices,_=read_inputs(ROOT/'data/processed');cache=load_cache('formal');rows=[]
14     for mode in ('q42','q43'):
15         with np.load(run/selection[mode+'__main__']/'trajectory.npz') as z:a={k:z[k] for k in z.files}
16         for date in DATES:
17             day=[str(d) for d in ds.dates].index(date);row=day-31
18             for issue in ((0,) if mode=='q42' else (0,1,2,3)):
19                 t=36*issue;initial=float(a['S'][row,t]);original=None if not issue else a['Q'][row,t:]
20                 net,price,prob,groups,_=scenarios(ds,prices,cache,day,issue,mode=mode,groups=2)
21                 kw=dict(mode=mode,original_q=original,gamma=selection[mode]['gamma'],rho=selection[mode].get('rho',)
↪ 1.))
22                 lp=solve_dispatch(net,price,prob,initial,**kw)
23                 # Physical MILP checks the same ungrouped model at all four issue times.
24                 mi=solve_dispatch(net,price,prob,initial,method='milp',**kw)
25                 delta=abs(lp['objective']-mi['objective'])
26                 if delta>2e-5:raise AssertionError(('LP/MILP',date,issue,delta))
27                 result={'mode':mode,'date':date,'hour':6*issue,'lp_objective':lp['objective'],
28                    'milp_objective':mi['objective'],'lp_milp_error':delta,'milp_gap':mi['mip_gap']}
29                 if mode=='q43' and issue<3:
30                     one=solve_dispatch(net,price,prob,initial,groups=np.zeros(len(prob),int),**kw)
31                     two=solve_dispatch(net,price,prob,initial,groups=groups,**kw)
32                     if abs(one['objective']-lp['objective'])>2e-5 or two['objective']>lp['objective']+2e-5:
33                         raise AssertionError(('containment',date,issue))

```

```

34         # Also verify the actual branched physical formulation, not just its baseline.
35         two_mi=solve_dispatch(net,price,prob,initial,groups=groups,method='milp',**kw)
36         if abs(two_mi['objective']-two['objective'])>2e-5:raise AssertionError('branched LP/MILP')
37         result.update(k1_error=abs(one['objective']-lp['objective']),k2_groups=two['branches'],
38                     k2_objective=two['objective'],k2_predicted_reduction=lp['objective']-two['objective'],
39                     k2_milp_error=abs(two_mi['objective']-two['objective']))
40         rows.append(result);print(mode,date,6*issue,delta,flush=True)
41     write_json(run/'structural_checks.json',{'cases':rows,'tolerance':2e-5,
42         'lp_milp_cases':len(rows)+sum('k2_milp_error' in r for r in rows),
43         'maximum_lp_milp_error':max(max(r['lp_milp_error'],r.get('k2_milp_error',0)) for r in rows)})
44
45 if __name__=='__main__':main()

```

verify_delivery.py

```

1 """Verify the slim delivery without archived prediction caches or solver logs."""
2 from pathlib import Path
3 import csv,hashlib,io,json,zipfile
4 from forecasting import interval_label
5 from q4_forecasting import read_inputs
6
7 PROJECT=Path(__file__).resolve().parents[1]
8 PACKAGE=PROJECT.parent
9 RESULT=PACKAGE/'01_ 论文成果'
10 REVIEW=PACKAGE/'03_ 复核记录'
11
12 def sha(p):return hashlib.sha256(p.read_bytes()).hexdigest()
13
14 def main():
15     manifest=json.loads((REVIEW/'manifest.json').read_text('utf-8'))
16     prediction=json.loads((REVIEW/'预测来源 manifest.json').read_text('utf-8'))
17     for name,digest in manifest['inputs'].items():assert sha(PROJECT/'data/processed'/name)==digest,name
18     for name,digest in {'**manifest':['numeric_code'],'**prediction':['prediction_sources']}.items():
19         assert sha(PROJECT/'src'/name)==digest,name
20     assert sha(PROJECT/'model/model-q4.tex')==manifest['model']
21     saved=json.loads((REVIEW/'workbook_verification.json').read_text('utf-8'))
22     for r in saved:assert sha(RESULT/r['file'])==r['sha256'],r['file']
23     index=PACKAGE/'交付校验清单.json'
24     if index.exists():
25         for r in json.loads(index.read_text('utf-8'))['files']:assert sha(PACKAGE/r['path'])==r['sha256'],r['path']
26     ds,prices,_=read_inputs(PROJECT/'data/processed')
27     cases=[]
28     with zipfile.ZipFile(RESULT/'10 分钟完整轨迹.zip') as z:
29         assert z.testzip() is None
30         for mode in ('q42','q43'):
31             maxima={};cash=0.;last=None;count=0
32             def check(key,value):
33                 value=abs(float(value));maxima[key]=max(maxima.get(key,0.),value)
34                 assert value<2e-5,(mode,key,value)
35             rows=csv.DictReader(io.TextIOWrapper(z.open(mode+'10 分钟完整轨迹.csv'),encoding='utf-8-sig'))
36             for number,row in enumerate(rows):
37                 day,t=divmod(number,144);day+=31
38                 assert row['date']==str(ds.dates[day]) and row['interval']==interval_label(t)
39                 q,r,c,d,e,u,s,sn,p=[float(row[k]) for k in ('Q_kwh','R_kwh','C_kwh','D_kwh','E_kwh','U_kwh','S_star',
40                     ↪ 't_kwh','S_end_kwh','price_yuan_per_kwh')]
41                 check('price',p-prices[day,t]);net=(ds.load_kw[day,t]-ds.pv_kw[day,t])/6
42                 check('state',sn-s-.9*c+d/.9)
43                 check('state_bounds',max(0,1200-s,1200-sn,s-10800,sn-10800))
44                 check('flow_bounds',max(0,c-5000/6,d-5000/6,-min(q,r,c,d,e,u)))
45                 check('exclusive',min(c,d))
46                 check('emergency',e-max(net+c-d-r,0));check('unused',u-max(r+d-c-net,0))
47                 check('continuity',s-(6000 if last is None else last));last=sn
48                 if mode=='q42' or t<36:check('fixed_commitment',q-r)
49                 normal=p*q if mode=='q42' else p*(q+1.5*max(r-q,0)-.5*max(q-r,0))
50                 actual=normal+5*p*e
51                 check('normal_bill',normal-float(row['normal_cost_yuan']))
52                 check('emergency_bill',5*p*e-float(row['emergency_cost_yuan']))
53                 check('actual_bill',actual-float(row['cash_cost_yuan']))
54                 cash+=actual;count+=1
55             assert count==334*144
56             check('year_end',max(6000-last,0))

```

```

56         expected=next(r['cash_cost_yuan'] for r in saved if r['file']==('result4-2.xlsx' if mode=='q42' else
↪ 'result4-3.xlsx'))
57         check('annual_bill',cash-expected)
58         cases.append({'mode':mode,'intervals_checked':count,'cash_cost_yuan':expected,
59                       'maximum_error':max(maxima.values()),'checks':maxima})
60     result={'inputs_match':True,'numerical_sources_match':True,'workbooks_unchanged':True,
61            'csv_zip_crc':'passed','executed_trajectories':cases,'tolerance':2e-5}
62     (REVIEW/'精简验证.json').write_text(json.dumps(result,ensure_ascii=False,indent=2)+'\n',encoding='utf-8')
63     print(json.dumps(result,ensure_ascii=False),flush=True)
64
65 if __name__=='__main__':main()

```

verify_workbooks.py

```

1 """Read-only saved-XLSX reconciliation, independent of Artifact Tool exporter."""
2 import argparse,json,hashlib
3 from datetime import datetime
4 from openpyxl import load_workbook
5 from run_q4 import ROOT,write_json
6
7 def main():
8     p=argparse.ArgumentParser();p.add_argument('--run',default='full_q4_20260913');a=p.parse_args()
9     run=ROOT/'outputs'/a.run;results=[]
10    for mode,file in [(('q42','result4-2.xlsx'),('q43','result4-3.xlsx')):
11        payload=json.loads((run/(mode+'_workbook_payload.json')).read_text('utf-8'))
12        path=run/'deliverables'/file;wb=load_workbook(path,data_only=True)
13        formulas=load_workbook(path,data_only=False,read_only=True);maximum=0.;numbers=0
14        pairs=[('计划购电量','plan')]+[(('调整购电量','adjusted')) if mode=='q43' else
↪      []]+[(('充放电量','battery'),('紧急购电量','emergency'))]
15        assert wb.sheetnames==[name for name,_ in pairs]
16        for name,key in pairs:
17            s=wb[name];rows=payload[key]
18            for i,row in enumerate(rows,2):
19                for j,value in enumerate(row,1):
20                    actual=s.cell(i,j).value
21                    if j==1 and value is not None:
22                        assert isinstance(actual,datetime) and
↪          actual.strftime('%Y-%m-%d')==value,(file,name,i,j,actual,value)
23                    elif isinstance(value,(int,float)):
24                        assert isinstance(actual,(int,float)),(file,name,i,j,actual)
25                        error=abs(actual-value);maximum=max(maximum,error);numbers+=1
26                        assert error<2e-5,(file,name,i,j,actual,value)
27                    else:assert actual==value,(file,name,i,j,actual,value)
28            if key in ('plan','adjusted'):
29                assert [s.cell(1,j+2).value for j in range(144)]==payload['headers'][1:145]
30                for i in (2,168,335):assert formulas[name].cell(i,146).value==f'SUM(B{i}:E0{i})'
31            for row in s.iter_rows():
32                for cell in row:assert cell.data_type!='e',(file,name,cell.coordinate,cell.value)
33        cash_sheet='计划购电量' if mode=='q42' else '调整购电量'
34        cash=sum(wb[cash_sheet].cell(i,147).value for i in range(2,336))
35        expected=json.loads((run/payload['experiment']/'summary.json').read_text('utf-8'))['cash_cost_yuan']
36        assert abs(cash-expected)<2e-5
37        results.append({'file':file,'sheets':wb.sheetnames,'numeric_cells_checked':numbers,'maximum_cell_error':max
↪      imum,
38                       'cash_cost_yuan':cash,'cash_difference_yuan':cash-expected,'sha256':hashlib.sha256(path.read_bytes()).h
↪      exdigest()})
39        print(file,numbers,maximum,cash,flush=True)
40    write_json(run/'workbook_verification.json',results)
41
42 if __name__=='__main__':main()

```

scientific_style.py

```

1 """ 科研图表的统一视觉约定：颜色表达变量身份，标记和线型提供第二种区分。"""
2 from pathlib import Path
3 import hashlib
4 import json
5 import warnings

```

```

6 import textwrap
7
8 import numpy as np
9 import matplotlib
10 matplotlib.use('Agg')
11 import matplotlib.pyplot as plt
12 from matplotlib import font_manager
13 from matplotlib.text import Text
14
15 STYLE_VERSION = 'academic_v2_20260912'
16 COLORS = {
17     'load': '#0072B2', 'pv': '#E69F00', 'purchase': '#009E73',
18     'charge': '#56B4E9', 'discharge': '#D55E00', 'state': '#8C6BB1',
19     'price': '#48424C', 'planned_cost': '#4477AA', 'emergency': '#D55E00',
20     'ink': '#202630', 'muted': '#62717D', 'rule': '#A6AFB7', 'light': '#EFF3F6',
21 }
22 METHOD_COLORS = ['#009E73', '#56B4E9', '#D55E00', '#8C6BB1']
23
24
25 def apply_style():
26     for name in ('arial.ttf', 'msyh.ttc'):
27         file = Path('C:/Windows/Fonts') / name
28         if file.exists():
29             font_manager.fontManager.addfont(str(file))
30     plt.rcParams.update({
31         'font.family': ['Arial', 'Microsoft YaHei', 'DejaVu Sans'],
32         'font.size': 9, 'axes.labelsize': 9, 'axes.titlesize': 9.5,
33         'xtick.labelsize': 8, 'ytick.labelsize': 8, 'legend.fontsize': 8,
34         'axes.unicode_minus': False, 'axes.spines.top': False,
35         'axes.spines.right': False, 'axes.linewidth': .7, 'axes.edgecolor': COLORS['ink'],
36         'axes.labelcolor': COLORS['ink'], 'text.color': COLORS['ink'],
37         'xtick.color': COLORS['ink'], 'ytick.color': COLORS['ink'],
38         'xtick.direction': 'out', 'ytick.direction': 'out',
39         'xtick.major.width': .7, 'ytick.major.width': .7,
40         'xtick.major.size': 3, 'ytick.major.size': 3, 'axes.grid': False,
41         'legend.frameon': False, 'legend.handlelength': 2.2,
42         'lines.linewidth': 1.25, 'figure.facecolor': 'white', 'axes.facecolor': 'white',
43         'savefig.facecolor': 'white', 'savefig.dpi': 450,
44         'svg.fonttype': 'none', 'pdf.fonttype': 42, 'ps.fonttype': 42,
45         'svg.hashsalt': STYLE_VERSION,
46     })
47
48
49 def step(ax, values, **kwargs):
50     """144 个区间值映射到 0—24 时，保留真实阶梯，不插值平滑。"""
51     values = np.asarray(values, dtype=float)
52     if values.shape != (144,) or not np.isfinite(values).all():
53         raise ValueError('A daily step series must have 144 finite values')
54     return ax.step(np.arange(145)/6, np.r_[values, values[-1]], where='post', **kwargs)
55
56
57 def time_axis(ax, *, label=True, ticks=4):
58     ax.set_xlim(0, 24)
59     ax.set_xticks(np.arange(0, 25, ticks))
60     if label:
61         ax.set_xlabel('时刻 (h)')
62     ax.margins(x=0)
63
64
65 def panel(ax, letter, title):
66     ax.set_title(title, loc='left', pad=9, fontweight='normal')
67     ax.text(-.105, 1.04, letter, transform=ax.transAxes, fontsize=11,
68           fontweight='bold', va='bottom', ha='left')
69
70
71 def note(fig, text, *, y=.025):
72     text = '\n'.join(textwrap.fill(line, width=60, break_long_words=True) for line in text.splitlines())
73     fig.text(.12, y, text, fontsize=7.2, color=COLORS['muted'], va='bottom')
74
75
76 def save_figure(fig, folder, name, *, pdf=None, dpi=450):
77     folder = Path(folder)
78     folder.mkdir(parents=True, exist_ok=True)

```

```

79 # 保存时记录缺字及布局警告，避免仅凭程序未报错便交付。
80 with warnings.catch_warnings(record=True) as caught:
81     warnings.simplefilter('always')
82     fig.canvas.draw()
83     renderer = fig.canvas.get_renderer()
84     outside = []
85     for obj in fig.findobj(match=Text):
86         if not obj.get_visible() or not obj.get_text() or obj.get_clip_on():
87             continue
88         box = obj.get_window_extent(renderer)
89         if box.width and box.height and (box.x0 < -1 or box.y0 < -1 or
90             box.x1 > fig.bbox.width+1 or box.y1 > fig.bbox.height+1):
91             outside.append(obj.get_text())
92     if outside:
93         raise ValueError(f'{name}: text extends outside figure: {outside}')
94     files = []
95     for suffix in ('png', 'svg'):
96         target = folder/f'{name}.{suffix}'
97         fig.savefig(target, dpi=dpi, facecolor='white')
98         files.append({'file': target.name, 'bytes': target.stat().st_size,
99             'sha256': hashlib.sha256(target.read_bytes()).hexdigest()})
100     if pdf is not None:
101         pdf.savefig(fig, dpi=dpi)
102     problems = sorted({str(w.message) for w in caught
103         if 'Glyph' in str(w.message) or 'layout' in str(w.message).lower()})
104     if problems:
105         raise ValueError(f'{name}: rendering warnings: {problems}')
106     result = {'name': name, 'width_mm': round(fig.get_figwidth()*25.4, 1),
107         'height_mm': round(fig.get_figheight()*25.4, 1), 'png_dpi': dpi,
108         'axes': len(fig.axes), 'text_bounds': 'passed', 'glyph_check': 'passed',
109         'files': files}
110     plt.close(fig)
111     return result

```

F 机制测试源程序

问题二 test_v2.py

```

1 from datetime import date, timedelta
2 from dataclasses import replace
3 from pathlib import Path
4 import sys
5 import unittest
6 import numpy as np
7 sys.path.insert(0, str(Path(__file__).resolve().parents[1]/'src'))
8 from forecasting import Dataset
9 from forecasting_v2 import MODELS, build_cache, scenarios, predict, arima_forecast, tree_features
10 from microgrid_core import Battery
11 from stochastic_dispatch import solve_q2_fixed
12
13
14 def example():
15     rng = np.random.default_rng(8)
16     load = rng.uniform(200, 600, (35,144))
17     pv = rng.uniform(0, 180, (35,144))
18     return Dataset([date(2025,1,1)+timedelta(days=i) for i in range(35)], load, pv,
19         np.ones(144), np.full(144, 400.), np.full(144, 50.), {})
20
21
22 class ForecastTests(unittest.TestCase):
23     def test_candidate_forecasts_use_history_prefix(self):
24         d = example()
25         changed = replace(d, load_kw=d.load_kw.copy(), pv_kw=d.pv_kw.copy())
26         changed.load_kw[10:] = 1e7
27         changed.pv_kw[10:] = 1e6
28         for m in MODELS:
29             a = build_cache(d, m, stop=11)
30             b = build_cache(changed, m, stop=11)

```

```

31         np.testing.assert_array_equal(a['load_kw'][:11], b['load_kw'][:11])
32         np.testing.assert_array_equal(a['pv_kw'][:11], b['pv_kw'][:11])
33
34     def test_tree_features_no_same_day_target(self):
35         d = example()
36         before = tree_features(d.load_kw, 12, d.dates[12])
37         d.load_kw[12:] = 1e9
38         np.testing.assert_array_equal(before, tree_features(d.load_kw, 12, d.dates[12]))
39
40     def test_reject_future_history(self):
41         d = example()
42         with self.assertRaises(ValueError):
43             predict(d.load_kw[:3], d.pv_kw[:3], d.dates[:3], d.dates[2], MODELS[0],
44                     d.baseline_load_kw, d.baseline_pv_kw, 42)
45
46     def test_arma_reproduces_exact_periodic_signal(self):
47         shape = 400 + 80*np.sin(np.arange(144)*2*np.pi/144)
48         forecast, info = arima_forecast(np.tile(shape, (10,1)))
49         np.testing.assert_allclose(forecast, shape, atol=1e-9)
50
51     def test_bootstrap_pairing_clipping_and_compression(self):
52         d = example()
53         c = build_cache(d, MODELS[0], stop=32)
54         l, g, prob, audit = scenarios(d, c, 31, seed=42, draws=256)
55         pool = np.arange(3,31)
56         sampled = np.random.default_rng(np.random.SeedSequence([42,31])).choice(pool,256,replace=True)
57         ids, counts = np.unique(sampled, return_counts=True)
58         np.testing.assert_array_equal(audit['residual_counts'], counts)
59         np.testing.assert_allclose(prob, counts/256)
60         np.testing.assert_allclose(l, np.maximum(c['load_kw'][31]+d.load_kw[ids]-c['load_kw'][ids],0)/6)
61         np.testing.assert_allclose(g, np.maximum(c['pv_kw'][31]+d.pv_kw[ids]-c['pv_kw'][ids],0)/6)
62         raw_net = (np.maximum(c['load_kw'][31]+d.load_kw[sampled]-c['load_kw'][sampled],0)
63                   -np.maximum(c['pv_kw'][31]+d.pv_kw[sampled]-c['pv_kw'][sampled],0))/6
64         q = np.full(144,50.)
65         self.assertAlmostEqual(float(prob@(np.maximum(l-g-q,0).sum(axis=1))),
66                                float(np.maximum(raw_net-q,0).sum(axis=1).mean()), places=10)
67
68     def test_bootstrap_future_actual_changes_have_no_effect(self):
69         d = example()
70         c = build_cache(d, MODELS[0], stop=32)
71         original = scenarios(d,c,31)
72         d.load_kw[31:] = 1e9
73         d.pv_kw[31:] = 2e9
74         changed = scenarios(d,c,31)
75         for a,b in zip(original[:3],changed[:3]):
76             np.testing.assert_array_equal(a,b)
77
78     def test_cold_start_uses_only_provided_prior(self):
79         d = example()
80         c = build_cache(d, MODELS[0], stop=1)
81         l,g,p,a = scenarios(d,c,0)
82         np.testing.assert_allclose(l[0], d.baseline_load_kw/6)
83         self.assertEqual(a['training_end'], None)
84         self.assertEqual(a['residual_dates'], [])
85
86     def test_compressed_and_expanded_optimization_equivalent(self):
87         b = Battery(.9,.9,min_kwh=0,max_kwh=10,max_power_kw=12)
88         l = np.array([[1.,4.,2.],[3.,1.,4.]])
89         g = np.zeros_like(l)
90         params = dict(battery=b,initial_kwh=5.,terminal_kwh=5.,terminal_cost_per_kwh=-.1)
91         a = solve_q2_fixed([1.,2.,1.], l,g,[.75,.25]**params)
92         ids = [0,0,0,1]
93         c = solve_q2_fixed([1.,2.,1.], l[ids],g[ids],[.25]*4,**params)
94         self.assertAlmostEqual(a['objective_yuan'],c['objective_yuan'],places=7)
95
96
97 if __name__ == '__main__':
98     unittest.main()

```

问题三 test_q3.py

```
1 from pathlib import Path
```

```

2 from datetime import date, timedelta
3 from dataclasses import replace
4 import sys
5 import unittest
6 from unittest.mock import patch
7 from scipy.optimize import OptimizeResult
8 import numpy as np
9 sys.path.insert(0, str(Path(__file__).resolve().parents[1] / 'src'))
10 from microgrid_core import Battery
11 from forecasting import Dataset
12 from q3_dispatch import solve_roll, settlement, weighted_cvar
13 from q3_forecasting import integrate_hourly, build_cache, make_scenarios, probability_scores
14
15
16 class Q3Tests(unittest.TestCase):
17     def test_timeout_requires_feasible_lp_bound_certificate(self):
18         import q3_dispatch
19         original=q3_dispatch.milp; calls=[]
20         def timeout_once(*args, **kwargs):
21             calls.append(1)
22             if len(calls)==1: return OptimizeResult(success=False, status=1, message='Controlled test timeout')
23             return original(*args, **kwargs)
24         with patch('q3_dispatch.milp', side_effect=timeout_once):
25             out=solve_roll([.4, 1], [ [100., 300.], [200., 400.] ], [[0., 0.], [0., 0.]], [.5, .5],
26                               battery=Battery(.9, .9), initial_kwh=6000, terminal_kwh=6000)
27             self.assertEqual(out['solver_route'], 'lp_bound_with_exclusive_dispatch')
28             self.assertLess(out['gap_yuan'], 1e-5)
29             self.assertLess(max(out['checks'].values()), 1e-5)
30
31     def test_settlement_examples_and_alternative(self):
32         np.testing.assert_allclose(settlement([2, 2], [10, 10], [8, 12]), [18, 26])
33         np.testing.assert_allclose(settlement([2, 2], [10, 10], [8, 12], 'no_refund_penalty'), [22, 26])
34
35     def test_only_final_version_is_paid(self):
36         final=float(settlement([1], [10], [8])[0])
37         self.assertEqual(final, 9)
38         self.assertNotEqual(final, float(settlement([1], [10], [12])[0]+settlement([1], [12], [8])[0]))
39
40     def test_discrete_cvar_fractional_quantile_mass(self):
41         self.assertAlmostEqual(weighted_cvar([0, 10, 100], [.8, .15, .05], .9), 55)
42         self.assertAlmostEqual(weighted_cvar([0, 10, 100], [.8, .15, .05], .95), 100)
43
44     def test_hourly_linear_integral(self):
45         power=integrate_hourly(np.arange(1, 25)*60, 0, 0)
46         np.testing.assert_allclose(power[:6], [5, 15, 25, 35, 45, 55])
47         self.assertAlmostEqual(power[:6].sum()/6, 30)
48         self.assertEqual(len(integrate_hourly(np.ones(24)*600, 600, 18)), 36)
49
50     def test_cvar_milp_matches_direct_tail(self):
51         battery=Battery(.9, .9)
52         load=np.array([[4, 7, 2], [9, 3, 8], [15, 8, 10]], float)*100
53         pv=np.array([[0, 6, 0], [0, 8, 0], [0, 3, 0]], float)*100
54         for risk in [0, .2]:
55             out=solve_roll([.4, 1, .8], load, pv, [.5, .3, .2], battery=battery, initial_kwh=6000,
56                               terminal_kwh=6000, terminal_gamma=.4, risk_lambda=risk)
57             self.assertAlmostEqual(out['scenario_cvar_yuan'], weighted_cvar(out['scenario_losses_yuan'], [.5, .3, .2], .9)
58                                   ↪ 9))
59             self.assertLess(max(out['checks'].values()), 1e-4)
60
61     def test_single_period_adjustment_quantiles(self):
62         battery=Battery(.9, .9); load=np.arange(10, dtype=float)[: , None]; pv=np.zeros_like(load)
63         for q, low, high in [(1, 6, 7), (7.5, 7.5, 7.5), (12, 8, 9)]:
64             out=solve_roll([1], load, pv, np.full(10, .1), battery=battery, initial_kwh=6000,
65                               original_q=q, terminal_kwh=6000)
66             self.assertGreaterEqual(out['plan']['R'][0], low-1e-6)
67             self.assertLessEqual(out['plan']['R'][0], high+1e-6)
68
69     def test_weighted_crps_matches_pairwise_definition(self):
70         x=np.array([[0, 1], [5, 3], [10, 7]], float); p=np.array([.2, .5, .3]); y=np.array([6, 5])
71         expected=(p[: , None]*abs(x-y)).sum(axis=0) -.5*np.sum(p[: , None, None]*p[None, :, None]*abs(x[: , None, :]-x[None, :, :])
72                               ↪ :)], axis=(0, 1))
73         np.testing.assert_allclose(probability_scores(x, p, y)['crps'], expected)
74

```

```

73     @staticmethod
74     def dataset():
75         days=16;t=np.arange(144)/6;pv=np.maximum(np.sin((t-6)/12*np.pi),0)*4000
76         load=np.array([3000+i*30+200*np.cos(t) for i in range(days)])
77         pvs=np.array([pv*(1+i*.01) for i in range(days)])
78         ds=Dataset([date(2025,1,1)+timedelta(days=i) for i in range(days)],load,pvs,np.ones(144),load[0],pvs[0],{})
79         official=np.empty((days,4,24))
80         for i in range(days):
81             for k in range(4):official[i,k]=np.maximum(np.sin(((np.arange(1,25)+k*6)%24-6)/12*np.pi),0)*3800
82         return ds,official
83
84     def test_future_actuals_and_unreleased_forecasts_do_not_leak(self):
85         ds,official=self.dataset();day=12
86         changed=replace(ds,load_kw=ds.load_kw.copy(),pv_kw=ds.pv_kw.copy())
87         changed.load_kw[day:]=99999;changed.pv_kw[day:]=77777
88         later=official.copy();later[day,1:]=88888;later[day+1:]=55555
89         before=build_cache(ds,official);after=build_cache(changed,later)
90         a=make_scenarios(ds,before,day,0,0)
91         b=make_scenarios(changed,after,day,0,0)
92         for left,right in zip(a[:3],b[:3]):np.testing.assert_array_equal(left,right)
93
94     def test_old_forecast_retains_original_lead(self):
95         ds,official=self.dataset();cache=build_cache(ds,official)
96         _,_,_,audit=make_scenarios(ds,cache,12,12,0)
97         self.assertEqual(audit['first_lead_hours'],12+1/6)
98         self.assertEqual(audit['official_issue_hour'],0)
99         self.assertTrue(all(i<12 for i in audit['sampled_day_indices']))
100        with self.assertRaises(ValueError):make_scenarios(ds,cache,12,6,12)
101
102    def test_shared_days_preserve_joint_standard_residuals(self):
103        ds,official=self.dataset();cache=build_cache(ds,official)
104        l,g,p,audit=make_scenarios(ds,cache,12,6,6,draws=64)
105        self.assertEqual(len(l),audit['available_independent_days'])
106        self.assertEqual(len(audit['sampled_day_indices']),64)
107        self.assertEqual(p.sum(),1)
108        unique=np.unique(audit['sampled_day_indices']);t=36
109        zl=(ds.load_kw[unique,t:]-cache['mu_l'][unique,t:])/cache['sigma_l'][unique,t:]
110        np.testing.assert_allclose(1*6,np.maximum(0,cache['mu_l'][12,t:]+cache['sigma_l'][12,t:]*zl))
111
112
113 if __name__=='__main__':unittest.main()

```

问题三 test_q3_alignment.py

```

1  """ 针对 PDF 差异的反例测试；避免仅检查程序能够运行。"""
2  import sys,unittest
3  from pathlib import Path
4  from dataclasses import replace
5  import numpy as np
6  sys.path.insert(0,str(Path(__file__).resolve().parents[1]/'src'))
7  import test_q3
8  from q3_forecasting import build_cache,masks,make_scenarios,scale_from_reliability,probability_scores
9  from q3_workflow import build_experiments
10 from q3_dispatch import solve_roll
11 from microgrid_core import Battery
12
13 class AlignmentTests(unittest.TestCase):
14     def test_pdf_bias_formula_retains_dawn_without_historical_support(self):
15         ds,official=test_q3.Q3Tests.dataset()
16         # 目标第一时段过去永远为零，但同组其它点有正偏差；式 11 仍允许正中心。
17         ds=replace(ds,pv_kw=np.full_like(ds.pv_kw,1000.));ds.pv_kw[:,0]=0
18         official[:]=0;cache=build_cache(ds,official)
19         self.assertGreater(cache['mu_g'][12,0,0],0)
20         self.assertEqual(cache['mu_g'][12,0,0],max(0,cache['raw_g'][12,0,0]+cache['bias'][12,0,0]))
21         _,scenes,_,_=make_scenarios(ds,cache,12,0,0)
22         self.assertTrue(np.isfinite(scenes).all())
23
24     def test_three_hour_bins_cover_each_valid_point_once(self):
25         for width,expected in [(3,20),(6,10)]:
26             _,_,_,valid,groups=masks(width)
27             self.assertEqual(len(groups),expected)

```

```

28         np.testing.assert_array_equal(sum(groups.values()), valid.astype(int))
29         ds, official = test_q3.Q3Tests.dataset(); c = build_cache(ds, official, bin_hours=3)
30         self.assertEqual(c['bias'].shape, (16, 4, 8))
31
32     def test_exponential_and_reliability_inverse_including_low_scale(self):
33         ds, o = test_q3.Q3Tests.dataset(); c = build_cache(ds, o, bin_hours=3); g, h, _, valid, _ = masks(3)
34         for day in (0, 7, 12):
35             sigma = c['sigma_g_exponential'][day]
36             np.testing.assert_allclose(sigma[valid], (c['sigma0'][day][g] * np.exp(c['beta'][day] * h))[valid])
37             for mode in ('pooled', 'binned', 'exponential'):
38                 r = c['reliability_g_' + mode][day][valid]
39                 self.assertTrue((r > 0) & (r <= 1)).all()
40                 np.testing.assert_allclose(scale_from_reliability(r, c['sref'][day]), c['sigma_g_' + mode][day][valid],
41                 ↪ rtol=1e-10)
42             # 同一目标组跨发布时刻, h 升序对应 R 非增。
43             for group in range(4):
44                 sel = valid & (g == group); order = np.argsort(h[sel]); r = c['reliability_g_exponential'][day][sel][order]
45                 self.assertLessEqual(float(np.diff(r).max(initial=0)), 1e-12)
46
47     def test_scenario_window_follows_selected_cache(self):
48         ds, o = test_q3.Q3Tests.dataset(); c = build_cache(ds, o, window=7, bin_hours=3, shrink=3)
49         *_a, a = make_scenarios(ds, c, 15, 12, 6)
50         self.assertEqual(a['available_independent_days'], 7)
51         self.assertTrue(all(8 <= j < 15 for j in a['sampled_day_indices']))
52
53     def test_b4_changes_only_risk_relative_to_b3(self):
54         selected = {'mode': 'binned', 'gamma': .5, 'calibration': {'window': 14, 'shrink': 3, 'bin_hours': 3, 'floor': 25, 'inte
55         ↪ rpretation': 'linear_nodes'}}
56         e = build_experiments({'selected': selected, 'probability_selected': selected, 'global_cash_selected': selected}, .
57         ↪ 5)
58         for name, p in e.items():
59             if name.startswith('cvar'): self.assertEqual({k: v for k, v in p.items() if k not in
60             ↪ ('risk_lambda', 'alpha')}, e['main'])
61         self.assertEqual(e['scale_pooled'], {'**e['main'], 'mode': 'pooled'})
62         self.assertTrue(e['terminal_hard']['daily_terminal'])
63
64     def test_hard_terminal_really_changes_dispatch(self):
65         kwargs = dict(battery=Battery(.9, .9), initial_kwh=6000)
66         free = solve_roll([1], [[300]], [[0]], [1], **kwargs)
67         hard = solve_roll([1], [[300]], [[0]], [1], terminal_kwh=6000, **kwargs)
68         self.assertAlmostEqual(hard['plan']['S'][-1], 6000)
69         self.assertLess(free['plan']['S'][-1], 6000)
70         self.assertGreater(hard['objective_yuan'], free['objective_yuan'])
71
72     def test_hourly_mean_alternative_has_correct_energy(self):
73         ds, o = test_q3.Q3Tests.dataset(); c = build_cache(ds, o, interpretation='hourly_mean')
74         np.testing.assert_allclose(c['raw_g'][12, 1, 36:42], o[12, 1, 0])
75         self.assertAlmostEqual(c['raw_g'][12, 1, 36:42].sum() / 6, o[12, 1, 0])
76
77     def test_rmse_uses_squared_errors_not_mean_absolute_error(self):
78         scores = probability_scores([[0, 0]], [1], [0, 4])
79         self.assertAlmostEqual(np.sqrt(scores['squared_error'].mean()), np.sqrt(8))
80         self.assertEqual(scores['mae'].mean(), 2)
81
82     def test_midday_actuals_after_decision_and_later_release_do_not_leak(self):
83         ds, o = test_q3.Q3Tests.dataset(); changed = replace(ds, load_kw=ds.load_kw.copy(), pv_kw=ds.pv_kw.copy())
84         day = 12; changed.load_kw[day, 72:] = 90000; changed.pv_kw[day, 72:] = 80000
85         later = o.copy(); later[day, 3] = 70000
86         before = build_cache(ds, o, window=14, shrink=3, bin_hours=3)
87         after = build_cache(changed, later, window=14, shrink=3, bin_hours=3)
88         for a, b in zip(make_scenarios(ds, before, day, 12, 12)[:3], make_scenarios(changed, after, day, 12, 12)[:3]):
89             np.testing.assert_array_equal(a, b)
90
91 if __name__ == '__main__': unittest.main()

```

问题四 test_q4_forecasting.py

```

1 from pathlib import Path
2 from types import SimpleNamespace
3 from datetime import date, timedelta
4 import sys

```

```

5 import unittest
6 import numpy as np
7 sys.path.insert(0, str(Path(__file__).resolve().parents[1] / 'src'))
8 from q4_forecasting import price_predictions, scenarios, information_groups, classify_signal
9 from q3_forecasting import integrate_hourly
10
11
12 class ForecastTests(unittest.TestCase):
13     def setup_data(self):
14         rng=np.random.default_rng(3)
15         ds=SimpleNamespace(dates=[date(2025,1,1)+timedelta(days=i) for i in range(40)],
16                             load_kw=rng.uniform(2500,5000,(40,144)),pv_kw=rng.uniform(0,6000,(40,144)))
17         p=rng.uniform(.3,1.,(40,144));raw=rng.uniform(0,1000,(40,4,144))
18         cache={'net_q42':np.full((40,144),100.),'net_q43':np.full((40,4,144),80.),'official':raw,
19               'log_price':price_predictions(p,np.ones(144)*.5,'weighted_7',.2,.1)}
20         cache['log_price_uncorrected']=price_predictions(p,np.ones(144)*.5,'weighted_7')
21         return ds,p,cache
22
23     def test_current_and_future_prices_do_not_enter_prediction(self):
24         _,p,_=self.setup_data();changed=p.copy();changed[30,36:]=9.;changed[31:]=10.
25         a=price_predictions(p,np.ones(144)*.5,'weighted_7',.2,.1)
26         b=price_predictions(changed,np.ones(144)*.5,'weighted_7',.2,.1)
27         np.testing.assert_array_equal(a[30,1,36:],b[30,1,36:])
28
29     def test_scenarios_ignore_future_observations_and_unreleased_official(self):
30         ds,p,cache=self.setup_data()
31         baseline=scenarios(ds,p,cache,30,1,mode='q43',groups=2)
32         changed=SimpleNamespace(dates=ds.dates,load_kw=ds.load_kw.copy(),pv_kw=ds.pv_kw.copy())
33         changed.load_kw[30:,36:]=1e5;changed.pv_kw[30:,36:]=0
34         p2=p.copy();p2[30,36:]=8.;p2[31:]=10.
35         c2={k:v.copy() for k,v in cache.items()};c2['official'][30,2:]=1e6
36         c2['net_q43'][30,2:]=1e6
37         result=scenarios(changed,p2,c2,30,1,mode='q43',groups=2)
38         for a,b in zip(baseline[:4],result[:4]):np.testing.assert_array_equal(a,b)
39         self.assertLess(max(result[4]['history_days']),30)
40
41     def test_price_net_residuals_keep_same_day_pairing(self):
42         ds,p,cache=self.setup_data()
43         net,price,prob,_,meta=scenarios(ds,p,cache,30,0>window=1)
44         np.testing.assert_allclose(net[0],100+(ds.load_kw[29]-ds.pv_kw[29])/6-cache['net_q42'][29])
45         np.testing.assert_allclose(np.log(price[0]),cache['log_price'][30,0]+np.log(p[29])-cache['log_price'][29,0])
46         self.assertEqual(meta['history_days'],[29]);self.assertEqual(prob.tolist(),[1.])
47
48     def test_groups_require_five_distinct_days_and_mapping_is_executable(self):
49         labels,meta=information_groups(np.array([[0,0],[1,1],[2,2.]]),2)
50         self.assertEqual(meta['actual_k'],1)
51         signal=np.r_[np.tile([-1.,0.],(10,1)),np.tile([1.,2.],(10,1))]
52         labels,meta=information_groups(signal,2)
53         self.assertEqual(meta['actual_k'],2)
54         for ix in range(20):self.assertEqual(classify_signal(signal[ix],meta),labels[ix])
55
56     def test_negative_net_load_is_not_clipped(self):
57         ds,p,cache=self.setup_data();cache['net_q42'][30]=-10000.
58         net,*_=scenarios(ds,p,cache,30,0)
59         self.assertTrue((net<0).all())
60
61     def test_official_linear_interpolation_energy(self):
62         forecast=integrate_hourly(np.full(24,1200.),0.,0)
63         self.assertAlmostEqual(float(forecast[:6].sum())/6,600.)
64         self.assertAlmostEqual(float(forecast[6:12].sum())/6,1200.)
65
66
67 if __name__=='__main__':unittest.main()

```

问题四 test_q4_dispatch.py

```

1 from pathlib import Path
2 import sys
3 import unittest
4 import numpy as np
5 sys.path.insert(0, str(Path(__file__).resolve().parents[1] / 'src'))

```

```

6 from q4_dispatch import Battery, bill, purify, solve_dispatch
7
8
9 class DispatchTests(unittest.TestCase):
10     def setUp(self):
11         self.net=np.array([[40.,200.,30.,250.,20.,140.],[-80.,100.,170.,300.,160.,30.],
12                             [250.,10.,190.,50.,350.,230.],[130.,270.,80.,210.,50.,300.]])
13         self.price=np.array([[.3,.5,1.2,.8,1.1,.4],[.5,.4,1.5,.7,1.3,.6],
14                               [.2,.6,1.,1.2,.9,.3],[.4,.3,1.3,.9,1.4,.5]])
15         self.prob=np.array([.1,.2,.3,.4])
16         self.kw=dict(initial=6000.,gamma=.5,terminal_min=6000.)
17
18     def test_weighted_eighty_percent_quantile(self):
19         b=Battery(minimum=0,maximum=1,limit=0,reference=0)
20         r=solve_dispatch(np.array([[100.],[200.],[300.]]) ,np.array([[1.],[1.],[4.]]),
21                         np.array([.6,.3,.1]),0.,battery=b,gamma=0)
22         self.assertAlmostEqual(r['Q'][0],300.)
23
24     def test_shrink_rho_zero_matches_constant_price(self):
25         r=solve_dispatch(self.net,self.price,self.prob,rho=0.,**self.kw)
26         means=np.broadcast_to(self.prob@self.price,self.price.shape)
27         s=solve_dispatch(self.net,means,self.prob,rho=1.,**self.kw)
28         self.assertAlmostEqual(r['objective'],s['objective'],places=6)
29
30     def test_shrink_matches_independent_cartesian_mixture(self):
31         rho=.25
32         ns=[self.net];ps=[self.price];weights=[rho*self.prob]
33         for j in range(4):
34             ns.append(self.net);ps.append(np.broadcast_to(self.price[j],self.price.shape))
35             weights.append((1-rho)*self.prob[j]*self.prob)
36         a=solve_dispatch(self.net,self.price,self.prob,rho=rho,**self.kw)
37         b=solve_dispatch(np.concatenate(ns),np.concatenate(ps),np.concatenate(weights),**self.kw)
38         self.assertAlmostEqual(a['objective'],b['objective'],places=6)
39
40     def test_path_repairing_preserves_common_objective(self):
41         net=self.net.copy();p=self.price.copy();prob=np.ones(4)/4
42         for t in range(6):
43             order=np.roll(np.arange(4),t);net[:,t]=self.net[order,t];p[:,t]=self.price[order,t]
44         a=solve_dispatch(self.net,self.price,self.prob,**self.kw)
45         b=solve_dispatch(net,p,prob,**self.kw)
46         self.assertAlmostEqual(a['objective'],b['objective'],places=6)
47
48     def test_one_group_equals_old_midnight(self):
49         a=solve_dispatch(self.net,self.price,self.prob,mode='q43',**self.kw)
50         b=solve_dispatch(self.net,self.price,self.prob,mode='q43',groups=np.zeros(4),prefix=2,**self.kw)
51         self.assertAlmostEqual(a['objective'],b['objective'],places=6)
52
53     def test_one_group_equals_old_update(self):
54         q=np.full(6,100.)
55         a=solve_dispatch(self.net,self.price,self.prob,mode='q43',original_q=q,**self.kw)
56         b=solve_dispatch(self.net,self.price,self.prob,mode='q43',original_q=q,groups=np.zeros(4),prefix=2,**self.kw)
57         self.assertAlmostEqual(a['objective'],b['objective'],places=6)
58
59     def test_lookahead_includes_common_policy_and_shared_bridge(self):
60         a=solve_dispatch(self.net,self.price,self.prob,mode='q43',**self.kw)
61         b=solve_dispatch(self.net,self.price,self.prob,mode='q43',groups=[0,0,1,1],prefix=2,**self.kw)
62         self.assertLessEqual(b['objective'],a['objective']+1e-6)
63         for node in b['nodes'][1:]:self.assertAlmostEqual(node['S'][0],b['nodes'][0]['S'][-1])
64         np.testing.assert_allclose(b['nodes'][0]['R'],b['Q'][:2],atol=1e-6)
65
66     def test_lp_purification_equals_milp(self):
67         for mode,groups in (('q42',None),('q43',[0,0,1,1])):
68             a=solve_dispatch(self.net,self.price,self.prob,mode=mode,groups=groups,prefix=2,**self.kw)
69             b=solve_dispatch(self.net,self.price,self.prob,mode=mode,groups=groups,prefix=2,method='milp',**self.kw)
70             self.assertAlmostEqual(a['objective'],b['objective'],places=5)
71             self.assertLess(a['checks']['simultaneous'],1e-7)
72
73     def test_purification_preserves_stock_and_reduces_deficit(self):
74         b=Battery();c=np.array([100.,50.]);d=np.array([40.,100.]);cp,dp=purify(c,d,b)
75         np.testing.assert_allclose(b.eta_c*c-d/b.eta_d,b.eta_c*cp-dp/b.eta_d)
76         self.assertTrue(np.all(cp-dp<=c-d+1e-9));self.assertEqual(np.minimum(cp,dp).max(),0)
77

```

```

78 def test_adjustment_seventy_ninety_band(self):
79     b=Battery(minimum=0,maximum=1,limit=0,reference=0)
80     net=np.arange(1.,101.)[:,None];price=np.ones_like(net);prob=np.ones(100)/100
81     for q,expected in [(40.,70.), (80.,80.), (98.,90.)]:
82         r=solve_dispatch(net,price,prob,0.,mode='q43',original_q=[q],battery=b,gamma=0.)
83         self.assertEqual(abs(r['nodes'][0]['R'][0]-expected),1.000001)
84
85 def test_real_bill_uses_actual_price_and_original_commitment(self):
86     a=bill(np.array([2.,3.]),np.array([150.,50.]),np.array([100.,100.]),np.array([120.,60.]),
87           np.zeros(2),np.zeros(2),adjustment=True)
88     self.assertAlmostEqual(a['cash_cost'],500+60-60+300)
89     self.assertAlmostEqual(a['E'].sum(),30.)
90
91 def test_invalid_price_or_mixed_q43_shrink_rejected(self):
92     with self.assertRaises(ValueError):solve_dispatch(self.net,-self.price,self.prob,**self.kw)
93     with self.assertRaises(ValueError):solve_dispatch(self.net,self.price,self.prob,mode='q43',rho=.5,**self.kw)
94
95
96 if __name__=='__main__':unittest.main()

```